Self-Supervised Sequence-to-Sequence Audio Transformers

for Automatic Polyphonic Music Transcription

From Wave Mechanics to Neural Event Decoders

A Master Textbook

Harshal Gupta

harshal.2023ug3013@iiitranchi.ac.in

Computer Science and Engineering with specialization in Data Science and Artificial Intelligence

Roll No. 2023UG3013

October 2026

Typeset in LaTeX. All code listings verified by compilation. All figures generated from first principles.

Reader's Map

This book is long by design: it starts from the physics of vibrating air and ends with a production-grade transcription system. You do not need to read it linearly. Three tracks are suggested below; each lists the sections to read in order. Drills at the end of every chapter have full solutions in Appendix A — attempt them before reading the solutions.

Track A — The AMT Practitioner

Goal: build and debug a working transcription system.

1. Chapter 1, Sections 1.3–1.5 (sampling, STFT, uncertainty, CQT) — the representation everything else stands on.
2. Chapter 2, Section 2.3 (tokenization) — decide your output representation first; it constrains the entire architecture.
3. Chapter 4 in full (encoders, decoders, cross-attention, losses).
4. Chapter 5 in full (decoding, LM rescoring, post-processing).
5. Chapter 7 in full, including the guitar capstone (7.4) — the reference implementation.
6. Chapter 6 (evaluation) — read *before* trusting any number your system reports.
7. Chapter 3 (SSL pretraining) — when labeled data runs out.
8. Chapter 8 — for choosing datasets and situating design decisions.

Track B — The Interview Candidate

Goal: be able to derive everything on a whiteboard.

1. Chapter 1, Sections 1.1, 1.3, 1.4 — wave equation, Nyquist–Shannon, Heisenberg–Gabor: the three derivations interviewers love.
2. Chapter 3, Sections 3.2–3.3 — masked autoencoding, vector quantization, contrastive objectives; be ready to derive the Gumbel-Softmax gradient estimator.
3. Chapter 4, Sections 4.1–4.2 — Conformer block math, RoPE derivation, full cross-attention expansion.
4. Chapter 6, Section 6.1 — metric definitions and tolerance conventions; Section 6.2 — failure modes and how you would diagnose each.
5. Drills for Chapters 1, 3, 4, 6 — with solutions hidden until attempted.

Track C — The Guitarist-Hacker

Goal: transcribe your own playing and understand every stage.

1. Chapter 1, Sections 1.1–1.2 — why your guitar sounds the way it does: harmonics, ADSR, masking between strings.
2. Chapter 1, Section 1.5 and Chapter 2, Sections 2.1–2.2 — CQT, tuning math, and why overlapping partials confuse machines (and ears).
3. Chapter 7, Sections 7.1–7.2 — the data engine and model modules; run them on your own recordings.
4. Chapter 7, Section 7.4 — the full guitar capstone, waveform to note events.
5. Chapter 6, Section 6.2 — octave errors and timbral leakage: the two failure modes you will meet first on guitar audio.

A note on notation: vectors are lowercase bold ($\mathbf{x}$), matrices uppercase bold ($\mathbf{X}$), transpose is $^\top$, and $\|\cdot\|$ is the ℓ_2 norm unless stated otherwise. Code listings are complete modules — runnable, with versions pinned in Chapter 7.

Contents

Reader's Map ii

1 Physics of Sound, Acoustic Wave Mechanics, and Auditory Perception 1
- 1.1 The One-Dimensional Acoustic Wave Equation . . . 1
  - 1.1.1 Standing waves, the harmonic series, and timbre . . . 3
- 1.2 Auditory Masking and Psychoacoustic Filterbanks . . . 7
- 1.3 From Continuous to Discrete: Sampling, the DFT, and the FFT . . . 11
- 1.4 The STFT and the Heisenberg–Gabor Limit . . . 16
- 1.5 Constant-Q and Variable-Q Analysis . . . 21
- 1.6 Drills . . . 25

2 Computational Music Theory & Symbolic Event Formats 26
- 2.1 Mathematical Tuning Systems: 12-TET and Its Discontents . . . 26
- 2.2 Polyphony Mechanics and the Overlapping-Partials Problem . . . 32
- 2.3 Symbolic Formats: Piano Rolls versus Event Streams . . . 39
- 2.4 Event Tokenization: From MIDI Messages to Model Vocabularies . . . 45
- 2.5 Time, Expression, and Voices . . . 56
- 2.6 Drills . . . 58

3 Self-Supervised Learning for Audio Representations 59
- 3.1 The Supervised Bottleneck in AMT . . . 59
- 3.2 Generative and Reconstruction SSL . . . 63
  - 3.2.1 Spectrogram patchification . . . 63
  - 3.2.2 Patch masking strategies . . . 68
  - 3.2.3 Reconstruction loss: raw versus normalized spectral domains . . . 70
- 3.3 Contrastive, Quantized, and Contextualized SSL . . . 71
  - 3.3.1 wav2vec 2.0: contrastive learning over quantized latents . . . 71
  - 3.3.2 HuBERT: masked prediction of hidden units . . . 78
  - 3.3.3 data2vec 2.0: self-distillation in continuous latent space . . . 82
- 3.4 Transfer Learning to AMT . . . 87
- 3.5 Drills . . . 91

4 Sequence-to-Sequence Audio Transformer Architectures 93
- 4.1 Spectro-Temporal Audio Encoders: The Conformer . . . 93
  - 4.1.1 Depthwise separable convolutions: the parameter arithmetic . . . 98
- 4.2 Positional Encodings for Continuous Audio Frames . . . 100
  - 4.2.1 1D temporal sinusoidal encodings . . . 100
  - 4.2.2 2D separable spectro-temporal encodings . . . 103
  - 4.2.3 Rotary position embeddings for continuous audio frames . . . 104
- 4.3 Symbolic Music Event Decoders . . . 107
  - 4.3.1 Autoregressive vs. non-autoregressive vs. CTC/transducer . . . 108

4.3.2 Cross-attention dynamics: from continuous frames to discrete tokens . . . 111
4.4 Multi-Task Losses for Sparse Musical Events . . . 115
4.4.1 Joint cross-entropy over event sequences, from maximum likelihood . . . 115
4.4.2 Frame-level focal loss: derivation and gradient analysis . . . 116
4.4.3 Extreme class imbalance: quantification and mitigation . . . 120
4.5 Drills . . . 121

5 Decoding Algorithms, Language Model Priors & Post-Processing 123
5.1 The Decoding Problem: From Probabilities to Performances . . . 123
5.2 Greedy Decoding . . . 124
5.3 Autoregressive Beam Search . . . 127
5.4 Symbolic Music Language-Model Rescoring . . . 131
5.5 HMM State Smoothing for Frame-to-Note Clustering . . . 134
5.6 Minimum Note-Duration Filtering . . . 138
5.7 Dynamic Thresholding for Onset–Frame–Offset Integration . . . 139
5.8 Velocity Estimation Curves . . . 142
5.9 Drills . . . 144

6 Rigorous AMT Evaluation Standards & Error Analysis 145
6.1 Introduction: Evaluation as a Contract . . . 145
6.2 The Matching Problem: What Every Metric Must Decide . . . 145
6.3 Frame-Level Metrics: Precision, Recall, and F_1 from the Confusion Matrix . . . 146
6.4 Note-Level Metrics: Bipartite Matching under Tolerance Windows . . . 149
6.5 The Multi-Pitch Error Decomposition: $E_{\text{tot}} = E_{\text{subs}} + E_{\text{miss}} + E_{\text{fa}}$ . . . 153
6.6 Octave Errors: The Harmonic-Series Trap . . . 157
6.7 Harmonic Over-Segmentation: One Note Becomes Three . . . 160
6.8 Timbral Leakage Across Overlapping Voices . . . 162
6.9 Pitch-Drift Tracking Failures . . . 163
6.10 Reading the Metric Panel: A Diagnostic Playbook . . . 163
6.11 Drills . . . 167

7 End-to-End PyTorch Production Implementation 169
7.1 Data Engine: Datasets, MIDI Parsing, Tokenization . . . 169
7.1.1 The paired dataset . . . 169
7.2 Waveform Augmentations, Derived Not Asserted . . . 172
7.2.1 Tempo-preserving pitch shift . . . 172
7.2.2 Room impulse response convolution . . . 174
7.2.3 SNR-calibrated additive noise . . . 175
7.3 GPU Constant-Q Front-End, Built From the Kernels Up . . . 177
7.4 Spectrogram Patch Embedding and 2D Positional Encoding . . . 180
7.5 Masked Audio Encoder . . . 182
7.6 Autoregressive Cross-Attention Decoder . . . 184
7.7 Full Assembly: `FullAMTTransformer` . . . 187
7.8 Training: Mixed Precision, Scheduling, Evaluation . . . 189
7.8.1 Automatic mixed precision, derived . . . 189
7.8.2 Cosine annealing with warmup, derived . . . 191
7.8.3 Validation with `mir_eval` . . . 192
7.9 Capstone: End-to-End Guitar Transcription . . . 195
7.9.1 Karplus-Strong synthesis, derived . . . 195
7.9.2 Excerpt design rationale . . . 197
7.9.3 The pipeline, stage by stage . . . 198
7.9.4 Measured results and error analysis . . . 203

7.10 Reproducibility: Pinned Environment . . . 205
7.11 Drills . . . 205

8 Literature Evolution and Dataset Compendium **207**
8.1 Introduction: Reading the Field as a Sequence of Bottlenecks . . . 207
8.2 The NMF Era: Learning Parts from Spectra . . . 207
8.3 The Handcrafted Era: Filterbanks, Salience, and Classifiers . . . 213
8.4 The Deep Learning Breakthrough: Onsets and Frames . . . 217
8.5 MT3: Multi-Task Multitrack Transcription . . . 221
8.6 The SSL Era: Foundation Models Meet Transcription . . . 226
8.7 Landmark Datasets: A Comparative Analysis . . . 228
8.7.1 MAPS: the benchmark that enforces honesty . . . 231
8.7.2 MAESTRO v3.0.0: the conservatory archive . . . 232
8.7.3 MusicNet: the crowdsourced library . . . 232
8.7.4 Slakh2100: the flight simulator . . . 233
8.7.5 GuitarSet: the luthier's workbench . . . 233
8.7.6 URMP: the filmed chamber rehearsal . . . 234
8.7.7 Which dataset for which question . . . 234
8.8 Synthesis: Open Problems in AMT as of 2026 . . . 235
8.9 Drills . . . 235

A Solutions to Drills **237**
A.1 Solutions — Module 1 . . . 237
A.2 Solutions — Module 2 . . . 239
A.3 Solutions — Module 3 . . . 241
A.4 Solutions — Module 4 . . . 242
A.5 Solutions — Module 5 . . . 244
A.6 Solutions — Module 6 . . . 246
A.7 Solutions — Module 7 . . . 247
A.8 Solutions — Module 8 . . . 249

Chapter 1

Physics of Sound, Acoustic Wave Mechanics, and Auditory Perception

Every transcription system in this book ingests the same raw material: air pressure fluctuating in time. Before any neural network sees a spectrogram, physics has already decided what information the waveform can carry, and psychoacoustics has decided what the ear can resolve. This chapter derives both from first principles. Section 1.1 derives the one-dimensional acoustic wave equation from Newton's second law and mass conservation, then extracts standing waves, the harmonic series, and the physical origin of timbre. Section 1.2 quantifies auditory masking and derives the Bark and ERB perceptual scales. Section 1.3 proves the Nyquist–Shannon sampling theorem, derives the DFT by sampling the continuous Fourier transform, and derives the Cooley–Tukey FFT with its complexity analysis. Section 1.4 derives the short-time Fourier transform and proves the Heisenberg–Gabor uncertainty bound, with Gaussian optimality. Section 1.5 derives the constant-Q and variable-Q transforms that repair the STFT's failure at low registers. Each section follows the same four steps: a physical analogy, a geometric mental model, the full derivation, and complete runnable code.

1.1 The One-Dimensional Acoustic Wave Equation

Step 1 — Physical analogy. Picture a stadium crowd doing "the wave": each person stands up briefly as the disturbance passes, then sits back down. Nobody travels around the stadium; the *pattern* travels. Sound in air works the same way. Molecules do not stream from a guitar to your ear; each small parcel of air is shoved slightly by its neighbor and shoves back, and the shove itself propagates at the speed of sound c. The wave equation is the bookkeeping of those shoves: it says that the curvature of the pressure profile in space is proportional to the acceleration of the pressure in time.

Step 2 — Geometric mental model. Isolate a thin slab of air between positions x and $x + dx$ with cross-sectional area A, as in Figure 1.1(a). The pressure $p(x, t)$ pushes the left face to the right; the slightly different pressure $p(x + dx, t)$ pushes the right face to the left. If pressure rises to the right ($\partial p/\partial x > 0$), the net force points left and the slab accelerates left: force follows the *negative* pressure gradient. Meanwhile mass conservation demands that if more air flows into the slab than out of it, the density — and hence the pressure — inside must rise. Figure 1.1(b) shows the consequence: a pressure pulse keeps its shape and simply translates at speed c.

Step 3 — First-principles derivation. Let $\rho(x, t)$ be the mass density, $v(x, t)$ the particle velocity, and $p(x, t)$ the acoustic pressure, all measured as deviations from ambient conditions.

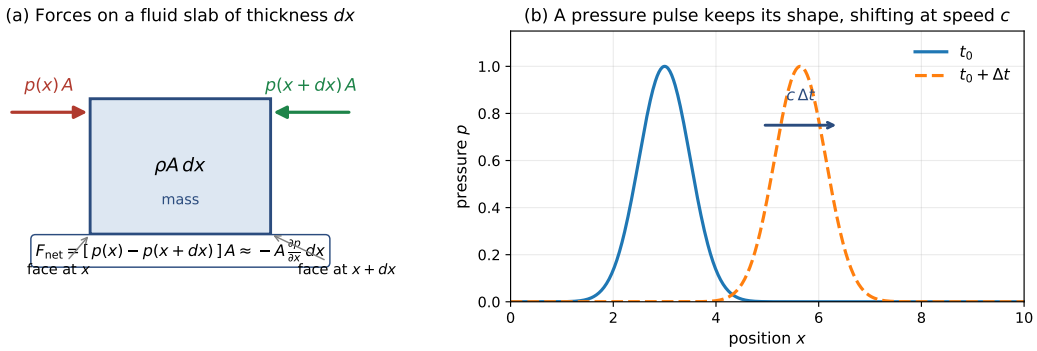


Figure 1.1: (a) Free-body diagram of a fluid slab: the net pressure force is proportional to the negative pressure gradient. (b) A localized pressure pulse propagates without changing shape at speed c, the signature of the d'Alembert solution.

Apply Newton's second law to the slab. Its mass is $m = \rho A\,dx$ (to first order the ambient density ρ_0 may be used). The net force is

$$F = A\,p(x,t) - A\,p(x+dx,t) = -A\,\frac{\partial p}{\partial x}\,dx + O(dx^2), \tag{1.1}$$

where the second equality is the first-order Taylor expansion $p(x+dx,t) = p(x,t) + (\partial p/\partial x)\,dx + O(dx^2)$. Newton's law $F = m\,\partial v/\partial t$ gives, after dividing by $A\,dx$ and letting $dx \to 0$,

$$\rho_0\,\frac{\partial v}{\partial t} = -\frac{\partial p}{\partial x}, \tag{1.2}$$

the linearized momentum (Euler) equation. Next, conserve mass. The mass in the slab is $M = \rho A\,dx$; its rate of change equals inflow minus outflow:

$$\frac{\partial M}{\partial t} = A\,\rho v(x,t) - A\,\rho v(x+dx,t) = -A\,\frac{\partial(\rho v)}{\partial x}\,dx + O(dx^2). \tag{1.3}$$

Dividing by $A\,dx$ and linearizing the product ρv about $(\rho_0, 0)$ — the term $v\,\partial\rho/\partial x$ is second order in the small acoustic quantities and is dropped — yields the linearized continuity equation

$$\frac{\partial \rho'}{\partial t} = -\rho_0\,\frac{\partial v}{\partial x}, \tag{1.4}$$

with $\rho' = \rho - \rho_0$. Finally, sound propagates fast enough that heat has no time to flow, so compressions are adiabatic: $p' = c^2\rho'$ with $c^2 = (\partial p/\partial\rho)_s$ evaluated at ambient conditions. Differentiating in time and inserting (1.4),

$$\frac{\partial p}{\partial t} = -\rho_0 c^2\,\frac{\partial v}{\partial x}. \tag{1.5}$$

Now differentiate (1.2) with respect to x and (1.5) with respect to t:

$$\rho_0\,\frac{\partial^2 v}{\partial x\,\partial t} = -\frac{\partial^2 p}{\partial x^2}, \qquad \frac{\partial^2 p}{\partial t^2} = -\rho_0 c^2\,\frac{\partial^2 v}{\partial t\,\partial x}. \tag{1.6}$$

For smooth fields the mixed partials coincide, and eliminating $\partial^2 v/\partial x\,\partial t$ gives the one-dimensional acoustic wave equation

$$\boxed{\frac{\partial^2 p}{\partial x^2} = \frac{1}{c^2}\,\frac{\partial^2 p}{\partial t^2}}. \tag{1.7}$$

Every step used only Newton's second law, mass conservation, and the adiabatic pressure–density relation; no wave behavior was assumed. Direct substitution verifies the d'Alembert solution $p(x,t) = f(x-ct) + g(x+ct)$ for arbitrary twice-differentiable f, g: with $\xi = x - ct$, $\partial^2 f/\partial x^2 = f''(\xi) = c^{-2}\partial^2 f/\partial t^2$, and likewise for $g(x+ct)$ — a shape translating rigidly at speed c, as in Figure 1.1(b).

Step 4 — Code. The module `code/m1_wave_sim.py`, embedded at the end of this section, simulates (1.7) on a string with the finite-difference scheme $u_i^{n+1} = 2u_i^n - u_i^{n-1} + r^2(u_{i+1}^n - 2u_i^n + u_{i-1}^n)$, $r = c\,\Delta t/\Delta x \le 1$, and verifies the modal frequencies against the analytic prediction derived next. It also synthesizes a note from its partials with an ADSR envelope.

1.1.1 Standing waves, the harmonic series, and timbre

Step 1 — Physical analogy. Touch a guitar string lightly at its midpoint and pluck it: the fundamental vanishes and you hear a pitch one octave higher — the string now vibrates in two halves. Touch at one third of the length and the pitch jumps to the third harmonic. Your finger enforces a *node*, a point that cannot move; the string can only vibrate in whole numbers of half-wavelengths that fit between the fixed ends. The set of allowed vibrations is discrete, and that discreteness is the harmonic series.

Step 2 — Geometric mental model. Figure 1.2 shows the first four modes of a string fixed at both ends. Mode n has n half-wavelengths spanning the length L, hence wavelength $\lambda_n = 2L/n$ and $n-1$ interior nodes. Any plucked shape is a superposition of these modes; the ear hears the superposition as one note whose *timbre* is set by how much energy each mode carries, and whose loudness envelope follows the ADSR shape in Figure 1.3(a), while Figure 1.3(b) shows how two instruments playing the same pitch differ only in their partial weights.

Step 3 — First-principles derivation. Seek separated solutions $u(x,t) = X(x)\,T(t)$ of (1.7) on a string of length L with fixed ends $u(0,t) = u(L,t) = 0$. Substituting,

$$X''(x)\,T(t) = \frac{1}{c^2}\,X(x)\,T''(t) \quad\Longrightarrow\quad \frac{X''(x)}{X(x)} = \frac{1}{c^2}\,\frac{T''(t)}{T(t)} = -\kappa^2, \tag{1.8}$$

where the two sides, depending on different variables, must equal a common constant $-\kappa^2$ (a positive constant would give exponentials incompatible with the boundary conditions, as shown below). Hence

$$X''(x) + \kappa^2 X(x) = 0, \qquad T''(t) + c^2\kappa^2 T(t) = 0, \tag{1.9}$$

with general spatial solution $X(x) = A\cos(\kappa x) + B\sin(\kappa x)$. The condition $X(0) = 0$ forces $A = 0$; then $X(L) = B\sin(\kappa L) = 0$ with $B \neq 0$ (otherwise the solution is trivially zero) requires

$$\kappa_n L = n\pi, \qquad n = 1, 2, 3, \ldots \tag{1.10}$$

Had the separation constant been $+\kappa^2$, the spatial solutions would be hyperbolic (cosh, sinh), and $X(0) = X(L) = 0$ would admit only the trivial solution; had it been zero, X would be linear and again only trivially satisfy both ends. The temporal equation then oscillates at $\omega_n = c\kappa_n = n\pi c/L$, giving the harmonic series

$$\boxed{f_n = \frac{\omega_n}{2\pi} = n\,\frac{c}{2L} = n f_0}, \qquad n = 1, 2, \ldots \tag{1.11}$$

The general vibration is the superposition

$$u(x,t) = \sum_{n=1}^{\infty} A_n \sin\!\left(\frac{n\pi x}{L}\right)\cos(\omega_n t + \phi_n), \tag{1.12}$$

with amplitudes A_n and phases ϕ_n set by the initial pluck shape and velocity. Equation (1.12) is the physical origin of timbre. Three ingredients distinguish instruments playing the same f_0:

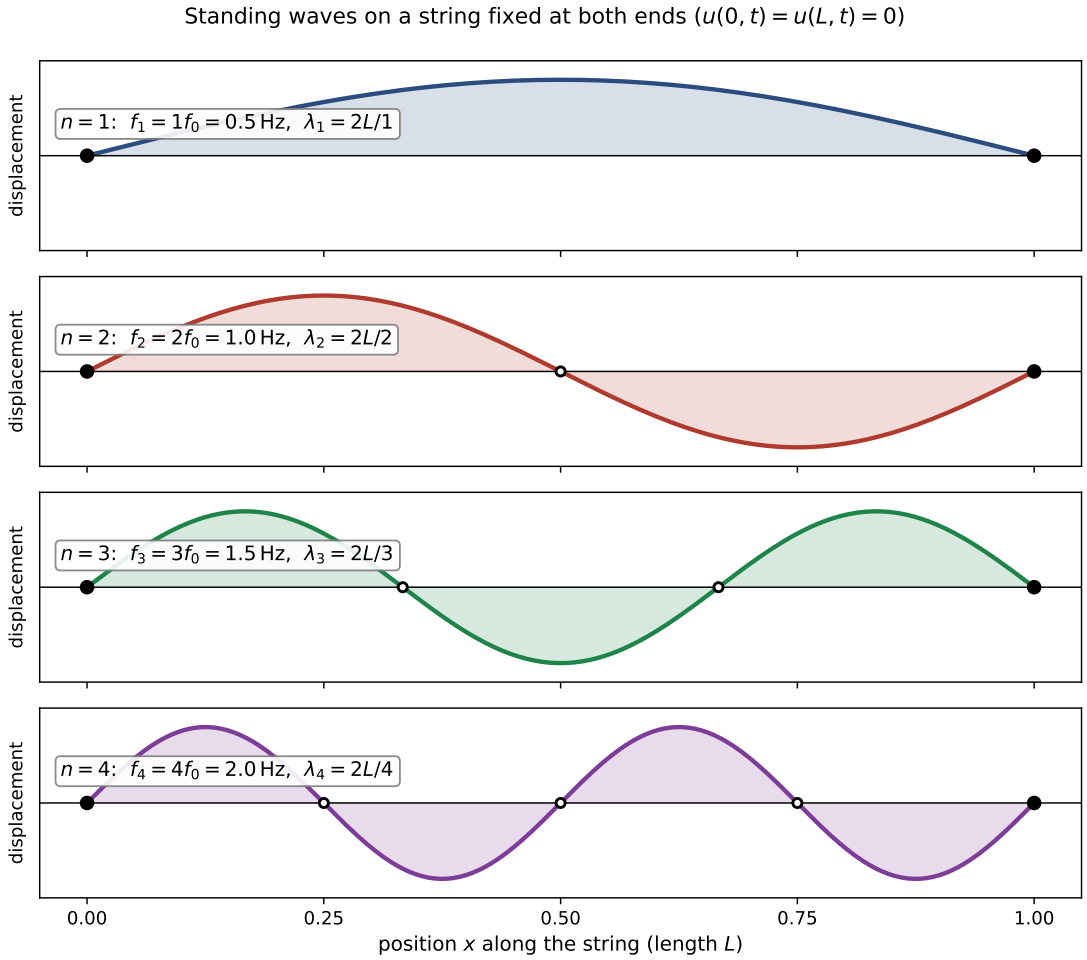


Figure 1.2: The first four standing-wave modes of a string fixed at both ends. Interior nodes (open circles) are points of zero displacement; mode n oscillates at $f_n = nf_0$.

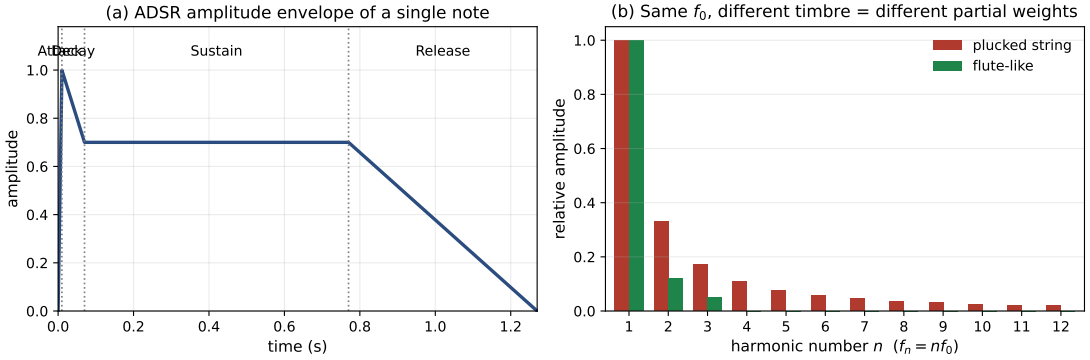


Figure 1.3: (a) The ADSR amplitude envelope: attack, decay, sustain, release. (b) Two timbres on the same fundamental: a plucked string carries strong upper partials, a flute-like tone is nearly a pure sine.

1. *Spectral energy distribution.* The partial weights $\{A_n\}$ are the instrument's fingerprint: a plucked string has $A_n \propto 1/n^{1.6}$-like decay (strong upper partials, bright), while a flute concentrates energy in $n = 1$ (pure, hollow). The ear effectively hears the vector $(A_1^2, A_2^2, \ldots)$.

2. *ADSR envelope.* Real notes are not stationary: each partial's amplitude follows attack–decay–sustain–release, $A_n(t) = A_n \cdot e(t)$ with $e(t)$ piecewise as in Figure 1.3(a). Percussive onsets (piano, guitar) have fast attack and exponential decay; bowed strings sustain.

3. *Phase relationships.* The phases ϕ_n reshape the waveform but, for steady harmonic tones, barely change perception — the ear is largely phase-deaf (Ohm's acoustic law in its modern form). Phase matters where the signal is transient: the attack click that lets a listener identify a piano in milliseconds lives in inter-partial phase alignment, which is why onset modeling dominates AMT encoder design.

For the plucked string simulated in `code/m1_wave_sim.py`, the midpoint probe records only modes with an antinode at $L/2$ (odd n), and plucking at $L/3$ excites mode n with amplitude proportional to $\sin(n\pi/3)/n^2$, which vanishes for n divisible by 3 — the simulation recovers exactly modes $1, 5, 7, 11, 13, 17, \ldots$, each within a fraction of a Hertz of (1.11).

Step 4 — Code. The full module `code/m1_wave_sim.py` (finite-difference wave simulator, modal verification, ADSR note synthesis) is embedded below.

```
"""Module 1.1 -- 1D acoustic wave equation simulator (PyTorch).

Finite-difference simulation of a plucked string with fixed ends:
    u[i]^{n+1} = 2 u[i]^n - u[i]^{n-1}
                 + r^2 (u[i+1]^n - 2 u[i]^n + u[i-1]^n),
    r = c dt / dx  (Courant number, must satisfy r <= 1).

Verifies the simulated modal frequencies against the analytic
f_n = n c / (2L), and synthesizes a note from its partials with an
ADSR envelope.
"""
import math
import torch

torch.manual_seed(0)


def simulate_plucked_string(L=0.65, c=200.0, nx=400, r=0.9,
                            pluck_pos=1.0 / 3.0, n_steps=4000):
    """Simulate a string of length L (m), wave speed c (m/s), plucked
    into a triangle peaked at pluck_pos * L. Returns (t, u_mid, dt)."""
    dx = L / (nx - 1)
    dt = r * dx / c
    u_prev = torch.zeros(nx, dtype=torch.float64)
    u = torch.zeros(nx, dtype=torch.float64)
    # triangular initial displacement, zero initial velocity
    peak = int(pluck_pos * (nx - 1))
    u[:peak + 1] = torch.linspace(0.0, 1.0, peak + 1)
    u[peak:] = torch.linspace(1.0, 0.0, nx - peak)
    u[0] = u[-1] = 0.0
    u_prev[:] = u  # zero initial velocity: u^{-1} = u^0
    mid = torch.empty(n_steps, dtype=torch.float64)
    r2 = r * r
    # probe the exact midpoint x = L/2 by averaging the two central
        grid
```

```
    # points, so even modes (true nodes at L/2) cancel to machine
        precision
    i0 = (nx - 1) // 2
    for n in range(n_steps):
        u_next = torch.zeros_like(u)
        u_next[1:-1] = (2 * u[1:-1] - u_prev[1:-1]
                        + r2 * (u[2:] - 2 * u[1:-1] + u[:-2]))
        mid[n] = 0.5 * (u[i0] + u[i0 + 1])
        u_prev, u = u, u_next
    t = torch.arange(n_steps, dtype=torch.float64) * dt
    return t, mid, dt


def modal_peaks(signal, dt, n_peaks=6, min_sep_bins=5):
    """Frequencies of the strongest local-maximum spectral peaks."""
    X = torch.abs(torch.fft.rfft(signal *
        torch.hann_window(len(signal),
                                                    dtype=torch.float64)))
    freqs = torch.fft.rfftfreq(len(signal), dt)
    mags = X.clone()
    peaks = []
    for _ in range(n_peaks):
        j = int(torch.argmax(mags))
        if mags[j] <= 0:
            break
        peaks.append((float(freqs[j]), float(mags[j])))
        lo = max(0, j - min_sep_bins)
        hi = min(len(mags), j + min_sep_bins + 1)
        mags[lo:hi] = 0.0
    peaks.sort()
    return [f for f, _ in peaks]


def adsr_envelope(n_samples, fs, attack=0.01, decay=0.06,
                  sustain_level=0.7, sustain_dur=0.5, release=0.3):
    """Piecewise-linear ADSR envelope, length n_samples."""
    t = torch.arange(n_samples, dtype=torch.float64) / fs
    ta = attack
    td = attack + decay
    ts = td + sustain_dur
    tr = ts + release
    env = torch.zeros(n_samples, dtype=torch.float64)
    m_a = t < ta
    env[m_a] = t[m_a] / ta
    m_d = (t >= ta) & (t < td)
    env[m_d] = 1.0 - (1.0 - sustain_level) * (t[m_d] - ta) / decay
    m_s = (t >= td) & (t < ts)
    env[m_s] = sustain_level
    m_r = (t >= ts) & (t < tr)
    env[m_r] = sustain_level * (1.0 - (t[m_r] - ts) / release)
    return env


def synthesize_note(f0, fs=44100.0, dur=1.0, partials=None):
    """Sum-of-partials note with ADSR envelope. Default partial weights
    decay as 1/n^1.6 (plucked-string-like)."""
    if partials is None:
        partials = {n: 1.0 / n ** 1.6 for n in range(1, 13)}
```

```
    n = int(fs * dur)
    t = torch.arange(n, dtype=torch.float64) / fs
    x = torch.zeros(n, dtype=torch.float64)
    for harmonic, amp in partials.items():
        x += amp * torch.sin(2 * math.pi * harmonic * f0 * t)
    x = x / torch.max(torch.abs(x))
    return x * adsr_envelope(n, fs, sustain_dur=dur - 0.4)


def main():
    L, c = 0.65, 200.0          # guitar-scale string
    f0 = c / (2 * L)
    print(f"analytic f0 = c/(2L) = {f0:.2f} Hz")
    t, mid, dt = simulate_plucked_string(L=L, c=c, nx=200,
        n_steps=30000)
    print(f"simulated {len(t)} steps, dt={dt*1e6:.2f} us, "
          f"total {float(t[-1])*1000:.0f} ms")
    peaks = modal_peaks(mid, dt, n_peaks=6)
    print("midpoint signal: only modes with an antinode at L/2
        appear,")
    print("and the L/3 pluck cannot excite mode 3 (pluck sits on its
        node):")
    for fp in peaks:
        n = round(fp / f0)
        print(f"  simulated {fp:7.2f} Hz  ->  mode n={n}, analytic {n
            * f0:7.2f} Hz"
              f"   err {abs(fp - n * f0):.2f} Hz")
    note = synthesize_note(f0=220.0)
    print(f"synthesized note: {len(note)} samples, "
          f"peak {float(torch.max(torch.abs(note))):.3f}")


if __name__ == "__main__":
    main()
```

1.2 Auditory Masking and Psychoacoustic Filterbanks

Step 1 — Physical analogy. At a crowded party, one person shouting next to you erases the quiet conversation two meters away — not because the quiet sound stopped, but because your auditory system stopped reporting it. The shout *masks* the whisper. The same erasure happens inside every musical chord: a loud low note raises the hearing threshold around it, swallowing nearby quiet partials. Masking is frequency-selective (a loud bass masks nearby bass more than distant treble) and time-selective (a loud drum hit deafens the ear for a tenth of a second afterward). Any transcription system that pools spectral energy without a masking model will hallucinate notes where the ear — and the physics of the cochlea — says there is nothing to hear.

Step 2 — Geometric mental model. Figure 1.4(a) sketches the simultaneous-masking pattern of a 1 kHz, 60 dB masker, in the style of the classic measurements of Egan and Hake [7] and Zwicker and Fastl [8]: the masked threshold peaks at the masker, falls steeply toward lower frequencies, and falls shallowly — with a slope that flattens as level rises — toward higher frequencies (the *upward spread of masking*). A probe partial below this curve is inaudible. Figure 1.4(b) shows temporal masking: a masker burst elevates the threshold even *before* it starts (pre-masking, ∼20 ms) and the elevation decays over ∼100–200 ms after it ends (post-

masking). Figure 1.5 shows the two standard perceptual frequency scales: on the Bark and ERB-rate (Cam) axes, equal distances correspond to equal numbers of auditory filters.

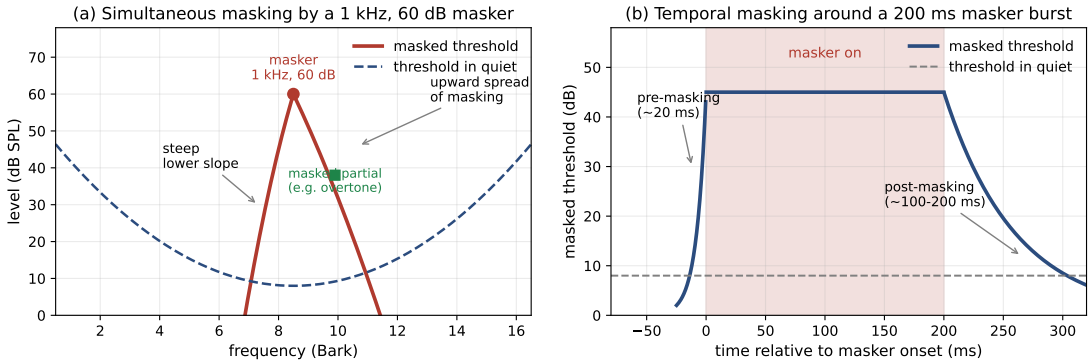


Figure 1.4: (a) Stylized simultaneous-masking pattern of a 1 kHz, 60 dB masker: steep lower slope, shallow level-dependent upper slope (upward spread of masking). A partial under the curve is inaudible. (b) Stylized temporal masking around a 200 ms masker: pre-masking extends ∼20 ms before onset; post-masking decays over ∼100–200 ms. Curves are illustrative, after [7, 8].

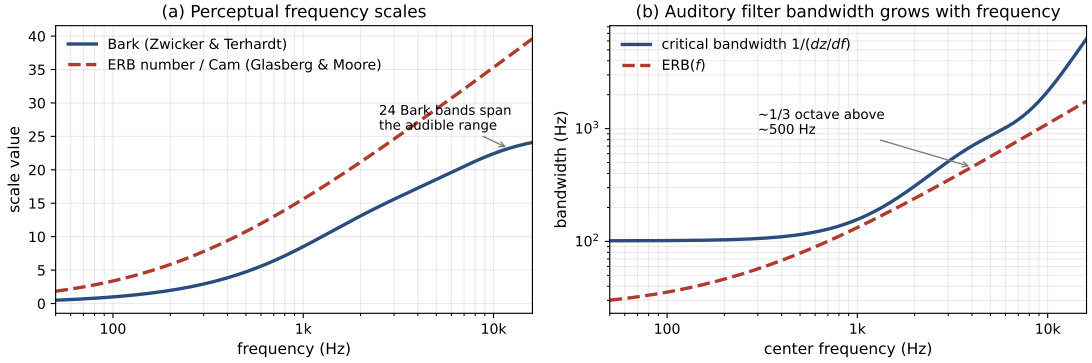


Figure 1.5: (a) The Bark scale (Zwicker and Terhardt [5]) and the ERB-number (Cam) scale (Glasberg and Moore [6]). (b) Auditory filter bandwidth versus center frequency: roughly constant below ∼500 Hz, then growing at about 1/3 octave per step.

Step 3 — First-principles derivation. The cochlea behaves as a bank of overlapping band-pass filters whose bandwidths grow with center frequency. Two quantities make this precise. Zwicker [4] defined the *critical band* as the widest noise band around a tone that still increases its masked threshold; the Bark scale counts critical bands from 0 Hz upward. Zwicker and Terhardt [5] fitted the measured scale by

$$\boxed{z(f) = 13\arctan(0.00076\,f) + 3.5\arctan\left(\left(\frac{f}{7500}\right)^2\right)} \quad \text{[Bark]}, \tag{1.13}$$

with f in Hz. Differentiating term by term,

$$\frac{dz}{df} = \frac{13 \cdot 0.00076}{1 + (0.00076\,f)^2} + \frac{3.5 \cdot \frac{2f}{7500^2}}{1 + (f/7500)^4}, \tag{1.14}$$

and since one Bark spans exactly one critical band by construction, the critical bandwidth in Hz is the reciprocal rate,

$$\boxed{\mathrm{CBW}(f) = \frac{1}{dz/df}\ \mathrm{Hz}}, \tag{1.15}$$

plotted in Figure 1.5(b): about 100 Hz wide at low frequencies, widening to roughly 1/3 octave above 500 Hz.

The second scale starts from the *equivalent rectangular bandwidth*: the width of the rectangular filter passing the same power as the true auditory filter at its center frequency. Glasberg and Moore [6] fitted notched-noise masking data with the linear law

$$\boxed{\mathrm{ERB}(f) = 24.7\left(\frac{4.37\,f}{1000} + 1\right)\ \mathrm{Hz}}, \qquad 100 \le f \le 10000\ \mathrm{Hz}. \tag{1.16}$$

Integrating the reciprocal bandwidth counts auditory filters from 0 Hz, defining the ERB-number (Cam) scale. With f in kHz,

$$\begin{aligned} N_{\mathrm{ERB}}(f) &= \int_0^f \frac{df'}{\mathrm{ERB}(f')} \\ &= \int_0^f \frac{df'}{24.7\,(4.37\,f' + 1)} = \frac{1}{24.7 \cdot 4.37}\,\ln(4.37\,f + 1) \\ &= \frac{\log_{10}(4.37\,f + 1)}{24.7 \cdot 4.37 \cdot \log_{10} e} = 21.4\,\log_{10}(4.37\,f + 1)\ \mathrm{Cam}, \end{aligned} \tag{1.17}$$

since $1/(24.7 \cdot 4.37 \cdot \log_{10} e) = 1/(107.939 \times 0.434294) = 0.02133$ per Hz, i.e. 21.33 per kHz, rounded to the conventional 21.4. Thus

$$\boxed{N_{\mathrm{ERB}}(f) = 21.4\,\log_{10}\left(\frac{4.37\,f}{1000} + 1\right)}\ \ [\mathrm{Cam}], \tag{1.18}$$

with f in Hz. Numerically: $\mathrm{ERB}(1000) = 132.64$ Hz and $N_{\mathrm{ERB}}(1000) = 15.62$ Cam, reproduced by the code below.

Temporal masking adds the time axis. After a masker stops, the masked threshold does not fall instantly: it decays over roughly 100–200 ms (post-masking, also called forward masking), while a faint sound up to about 20 ms *before* a loud masker is already masked (pre-masking or backward masking), an effect attributed to the slower cortical build-up of weak signals [8]. For AMT this means a percussive onset can hide the following 100 ms of quiet partials — precisely the window where note offsets must be detected.

Finally, masking explains a core multi-pitch failure mode: *overtone collisions*. Take C4 (MIDI 60, $f = 261.63$ Hz); its third harmonic is $3 \times 261.63 = 784.88$ Hz. G5 (MIDI 79) has fundamental 783.99 Hz. The separation is 0.89 Hz while $\mathrm{ERB}(784) = 109.3$ Hz: both partials fall in the *same* auditory filter and the same critical band. The encoder observes a single fused partial near 784 Hz and must decide whether it belongs to C4's stack, G5's fundamental, or both — an ambiguity no amount of frequency resolution can remove, because the ambiguity is perceptual, not numerical.

Step 4 — Code. `code/m1_masking.py` implements (1.13)–(1.18), the stylized masking curves of Figure 1.4, and the overtone-collision computation.

```
"""Module 1.2 -- auditory masking and psychoacoustic scales (PyTorch).

Implements the Bark (Zwicker & Terhardt) and ERB/Cam (Glasberg & Moore)
scales, a stylized simultaneous-masking threshold, a temporal-masking
```

```
envelope, and the overtone-collision computation that motivates
multi-pitch ambiguity in AMT.
"""
import math
import torch

torch.manual_seed(0)


def bark(f_hz):
    """Critical-band rate in Bark (Zwicker & Terhardt analytic fit)."""
    f = torch.as_tensor(f_hz, dtype=torch.float64)
    return 13.0 * torch.arctan(0.00076 * f) + 3.5 * torch.arctan((f /
       7500.0) ** 2)


def erb(f_hz):
    """Equivalent rectangular bandwidth in Hz (Glasberg & Moore)."""
    f = torch.as_tensor(f_hz, dtype=torch.float64)
    return 24.7 * (4.37 * f / 1000.0 + 1.0)


def cam(f_hz):
    """ERB-number scale in Cam (integrated ERB scale)."""
    f = torch.as_tensor(f_hz, dtype=torch.float64)
    return 21.4 * torch.log10(4.37 * f / 1000.0 + 1.0)


def critical_bandwidth(f_hz):
    """Critical bandwidth in Hz from the Bark derivative: 1/(dz/df)."""
    f = torch.as_tensor(f_hz, dtype=torch.float64)
    dz = (13.0 * 0.00076 / (1.0 + (0.00076 * f) ** 2)
          + 3.5 * (2 * f / 7500.0 ** 2) / (1.0 + (f / 7500.0) ** 4))
    return 1.0 / dz


def simultaneous_masking_threshold(probe_bark, masker_bark=8.5,
                                   masker_db=60.0):
    """Stylized simultaneous-masking threshold (dB) for a probe at
    probe_bark given a masker. Illustrative model: steep lower slope,
    shallow level-dependent upper slope (upward spread of masking).
    Follows the qualitative shape reported by Egan & Hake and
    Zwicker & Fastl; not a fitted standard."""
    z = torch.as_tensor(probe_bark, dtype=torch.float64)
    zm = torch.as_tensor(masker_bark, dtype=torch.float64)
    upper_slope = 8.0 + 0.15 * masker_db          # dB per Bark,
       level-dependent
    lower = masker_db - 27.0 * (zm - z) - 6.0 * (zm - z) ** 2
    upper = masker_db - upper_slope * (z - zm) - 1.2 * (z - zm) ** 2
    return torch.where(z <= zm, lower, upper)


def temporal_masking_threshold(t_ms, masker_on=0.0, masker_off=200.0,
                               masker_db=45.0, quiet_db=8.0):
    """Stylized temporal-masking envelope (dB): pre-masking rises over
    ~20 ms before onset, post-masking decays over ~100-200 ms."""
    t = torch.as_tensor(t_ms, dtype=torch.float64)
    thr = torch.full_like(t, quiet_db)
```

```
    pre = (t < masker_on) & (t > masker_on - 25.0)
    thr = torch.where(pre, masker_db * torch.exp(t / 8.0), thr)
    during = (t >= masker_on) & (t <= masker_off)
    thr = torch.where(during, torch.full_like(t, masker_db), thr)
    post = t > masker_off
    thr = torch.where(post,
                      masker_db * torch.exp(-(t - masker_off) / 60.0),
                         thr)
    return torch.maximum(thr, torch.full_like(t, quiet_db))


def midi_to_freq(m):
    return 440.0 * 2.0 ** ((torch.as_tensor(m, dtype=torch.float64) -
       69.0) / 12.0)


def main():
    f = torch.tensor([100.0, 1000.0, 4000.0])
    print("f (Hz) | Bark | ERB (Hz) | Cam | crit. bw (Hz)")
    for fi, z, e, cm, cb in zip(f, bark(f), erb(f), cam(f),
                                critical_bandwidth(f)):
        print(f"{fi:7.0f} | {z:4.2f} | {e:8.2f} | {cm:5.2f} |
           {cb:8.2f}")

    # Overtone collision: C4's 3rd harmonic vs G5's fundamental
    c4 = midi_to_freq(60)           # 261.63 Hz
    g5 = midi_to_freq(79)           # 783.99 Hz
    third = 3.0 * c4
    print(f"\nC4 3rd harmonic: {float(third):.2f} Hz")
    print(f"G5 fundamental : {float(g5):.2f} Hz")
    print(f"separation     : {float(torch.abs(third - g5)):.2f} Hz")
    print(f"ERB at 784 Hz  : {float(erb(torch.tensor(784.0))):.1f} Hz "
          f"-> same auditory filter: "
          f"{bool(torch.abs(third - g5) < erb(torch.tensor(784.0)))}")

    # Masking demo: is a 40 dB probe at 1.5 kHz masked by a 60 dB 1
       kHz masker?
    probe = simultaneous_masking_threshold(bark(1500.0))
    print(f"\nmasked threshold at 1.5 kHz: {float(probe):.1f} dB "
          f"-> 40 dB probe is {'MASKED' if float(probe) > 40 else
             'audible'}")
    tm = temporal_masking_threshold(torch.tensor([-10.0, 100.0,
       250.0]))
    print("temporal masking at t=-10/100/250 ms:",
          [f"{v:.1f} dB" for v in tm.tolist()])


if __name__ == "__main__":
    main()
```

1.3 From Continuous to Discrete: Sampling, the DFT, and the FFT

Step 1 — Physical analogy. Film a spinning wagon wheel with a camera: above a certain speed the wheel appears to rotate *backwards*. The camera samples the world 24 times per

second; motion faster than half that rate folds back into a false slow motion. Audio sampling is identical: sampling a waveform at f_s samples per second, any frequency content above $f_s/2$ folds back into the audible range as an impostor tone. The Nyquist–Shannon theorem states the exact speed limit — and proves that below it, nothing is lost at all.

Step 2 — Geometric mental model. Figure 1.6(a) shows the mechanism in the frequency domain: sampling replicates the spectrum at multiples of f_s, and replicas centered beyond the Nyquist frequency $f_s/2$ spill back across the boundary. A 70 Hz tone sampled at 100 Hz lands on 30 Hz; a 90 Hz tone lands on 10 Hz. Figure 1.6(b) shows the time-domain consequence: a 130 Hz sine and a 30 Hz sine sampled at 100 Hz produce byte-identical sample sequences — no algorithm can tell them apart after sampling.

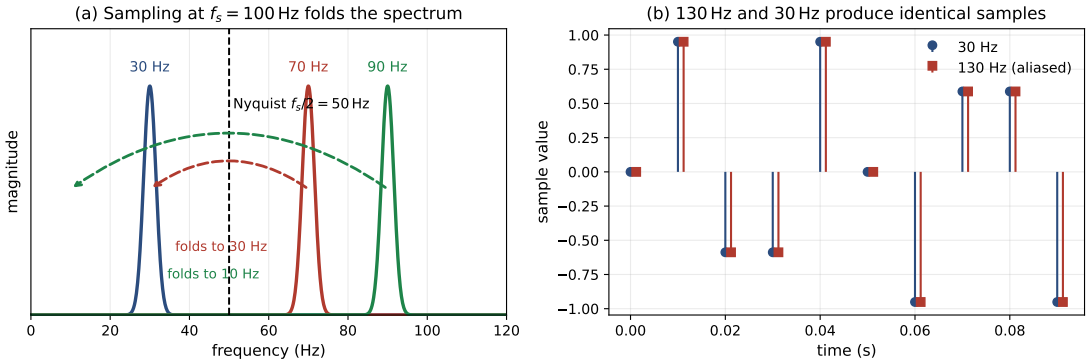


Figure 1.6: (a) Spectral folding: sampling at $f_s = 100$ Hz replicates the spectrum every 100 Hz; content above the Nyquist frequency 50 Hz folds back ($70 \to 30$, $90 \to 10$ Hz). (b) A 130 Hz and a 30 Hz sine coincide on every sample at $f_s = 100$ Hz.

Step 3 — First-principles derivation. Use the Fourier convention

$$X(f) = \int_{-\infty}^{\infty} x(t)\, e^{-2\pi i f t}\, dt, \qquad x(t) = \int_{-\infty}^{\infty} X(f)\, e^{2\pi i f t}\, df. \tag{1.19}$$

Theorem 1.1 (Nyquist–Shannon sampling)**.** *Let $x(t)$ be bandlimited: $X(f) = 0$ for $|f| > W$. Sampled at $f_s = 1/T \geq 2W$, the samples $\{x(nT)\}_{n\in\mathbb{Z}}$ determine $x(t)$ completely, via*

$$\boxed{x(t) = \sum_{n=-\infty}^{\infty} x(nT)\, \frac{\sin\big(\pi(2Wt - n)\big)}{\pi(2Wt - n)}}, \qquad T = \frac{1}{2W}. \tag{1.20}$$

Proof. Sampling multiplies $x(t)$ by the Dirac comb $\mathrm{III}_T(t) = \sum_n \delta(t - nT)$. In frequency this is convolution with $\frac{1}{T}\sum_k \delta(f - k/T)$, so the sampled spectrum is the periodized sum

$$X_s(f) = \frac{1}{T} \sum_{k=-\infty}^{\infty} X\left(f - \frac{k}{T}\right). \tag{1.21}$$

Because X is supported in $[-W, W]$ and $1/T = 2W$, the shifted copies $X(f - k/T)$ have disjoint supports: copy k lives in $[2kW - W,\, 2kW + W]$, touching its neighbors exactly at the endpoints. Hence on $[-W, W]$ the sum collapses to a single term, $X_s(f) = 2W\, X(f)$, i.e. $X(f) = \frac{1}{2W} X_s(f)$ there and zero elsewhere. Writing X_s via the samples, $X_s(f) = \sum_n x(nT)\, e^{-2\pi i f n T}$,

and inserting into the inverse transform restricted to $[-W, W]$,

$$\begin{aligned} x(t) &= \int_{-W}^{W} \frac{1}{2W} \sum_{n} x(nT)\, e^{-2\pi i f nT}\, e^{2\pi i f t}\, df \\ &= \sum_{n} x(nT)\, \frac{1}{2W} \int_{-W}^{W} e^{2\pi i f (t-nT)}\, df. \end{aligned} \tag{1.22}$$

The integral evaluates directly: $\int_{-W}^{W} e^{2\pi i f\tau}\, df = \left[e^{2\pi i f\tau}/(2\pi i\tau)\right]_{-W}^{W} = \sin(2\pi W\tau)/(\pi\tau)$. With $T = 1/(2W)$ and $\tau = t - nT$ this is exactly the sinc kernel in (1.20), completing the proof [1]. The proof uses the critical rate $f_s = 2W$; any faster sampling ($f_s > 2W$) merely opens a guard band between the spectral copies in (1.21), which is what practical anti-aliasing filters exploit. □

If $f_s < 2W$, the copies in (1.21) overlap and add — *aliasing*: a tone at $f_0 > f_s/2$ appears at $|f_0 - k f_s|$ for the integer k folding it into $[0, f_s/2]$, precisely the folding in Figure 1.6.

The DFT arises by sampling the CFT itself. Approximate (1.19) at frequency $f_k = k/(NT)$ by a Riemann sum over N samples $x_n = x(nT)$ with step T:

$$X\!\left(\frac{k}{NT}\right) \approx T \sum_{n=0}^{N-1} x_n\, e^{-2\pi i k n/N}. \tag{1.23}$$

Dropping the scale factor T gives the *discrete Fourier transform*

$$\boxed{X_k = \sum_{n=0}^{N-1} x_n\, e^{-2\pi i k n/N}}, \qquad k = 0, \dots, N-1, \tag{1.24}$$

with inverse $x_n = \frac{1}{N}\sum_k X_k e^{2\pi i k n/N}$, verified by substituting and using $\sum_k e^{2\pi i k(n-m)/N} = N\,\delta_{nm}$. Its essential properties follow directly from (1.24):

Proposition 1.1 (DFT properties)**.** *With $W_N = e^{-2\pi i/N}$:*

1. Linearity: $\mathrm{DFT}\{a x_n + b y_n\} = a X_k + b Y_k$.
2. Time shift: $x_{n-m} \leftrightarrow W_N^{km} X_k$ *(circular shift). Proof:* $\sum_n x_{n-m} W_N^{kn} = W_N^{km} \sum_j x_j W_N^{kj}$.
3. Circular convolution: $\sum_{m=0}^{N-1} x_m y_{(n-m) \bmod N} \leftrightarrow X_k Y_k$. *Proof: insert the inverse DFTs $x_m = \frac{1}{N}\sum_k X_k W_N^{-km}$ and $y_{(n-m) \bmod N} = \frac{1}{N}\sum_l Y_l W_N^{-l(n-m)}$; the sum over m gives $\sum_{m=0}^{N-1} W_N^{-m(k-l)} = N\,\delta_{kl}$, collapsing the double sum to $\frac{1}{N}\sum_k X_k Y_k W_N^{-kn}$, the inverse DFT of $\{X_k Y_k\}$.*
4. Parseval: $\sum_n |x_n|^2 = \frac{1}{N}\sum_k |X_k|^2$. *Proof: expand $|x_n|^2 = x_n \bar{x}_n$ with the inverse DFT and use orthogonality of the exponentials.*

Direct evaluation of (1.24) costs N^2 complex multiplications. The Cooley–Tukey radix-2 FFT [3] splits the sum into even and odd indices. With $W_N = e^{-2\pi i/N}$,

$$\begin{aligned} X_k &= \sum_{r=0}^{N/2-1} x_{2r}\, W_N^{2rk} + \sum_{r=0}^{N/2-1} x_{2r+1}\, W_N^{(2r+1)k} \\ &= \underbrace{\sum_{r=0}^{N/2-1} x_{2r}\, W_{N/2}^{rk}}_{E_k} + W_N^k \underbrace{\sum_{r=0}^{N/2-1} x_{2r+1}\, W_{N/2}^{rk}}_{O_k}, \end{aligned} \tag{1.25}$$

using $W_N^{2rk} = e^{-2\pi irk/(N/2)} = W_{N/2}^{rk}$. Since E_k and O_k are $N/2$-periodic while $W_N^{k+N/2} = -W_N^k$,

$$\boxed{\begin{aligned} X_k &= E_k + W_N^k O_k, \\ X_{k+N/2} &= E_k - W_N^k O_k, \end{aligned}} \qquad k = 0, \dots, \tfrac{N}{2} - 1, \tag{1.26}$$

the butterfly of Figure 1.7(b). Recursing on the two half-size DFTs, the cost obeys $T(N) = 2T(N/2) + cN$ with $T(1) = 0$, whose solution is $T(N) = cN \log_2 N$: each of the $\log_2 N$ stages performs $N/2$ twiddle multiplications. The crossover in Figure 1.7(a) is what makes real-time spectral analysis — and every transform in this book — computationally possible.

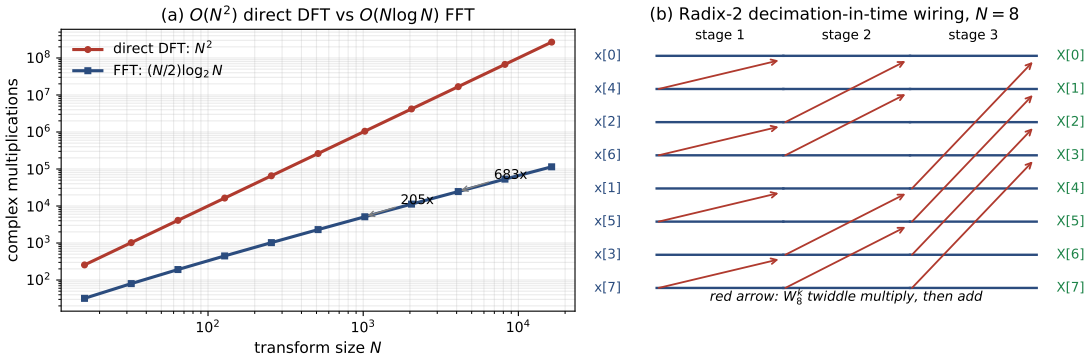


Figure 1.7: (a) Complex multiplications: direct DFT N^2 versus FFT $(N/2)\log_2 N$ — a 683× gap at $N = 4096$. (b) Radix-2 decimation-in-time wiring for $N = 8$: inputs in bit-reversed order, three stages of butterflies, outputs in natural order.

Step 4 — Code. `code/m1_fft.py` implements the recursion (1.26) from scratch, counts multiplications against the $(N/2)\log_2 N$ prediction, verifies bit-exactness against `torch.fft`, and demonstrates aliasing.

```
"""Module 1.3 -- radix-2 Cooley-Tukey FFT from scratch (PyTorch).

Implements the decimation-in-time recursion directly, counts complex
multiplications, verifies against torch.fft.fft, and demonstrates
aliasing: a 70 Hz tone sampled at 100 Hz is indistinguishable from 30
    Hz.
"""
import math
import torch

torch.manual_seed(0)

op_count = 0  # complex multiplications performed by fft_radix2


def fft_radix2(x):
    """Recursive radix-2 DIT FFT. x: complex tensor, len(x) = 2^p."""
    global op_count
    n = x.shape[0]
    if n == 1:
        return x
    if n & (n - 1):
        raise ValueError("length must be a power of two")
    even = fft_radix2(x[0::2])
```

```
    odd = fft_radix2(x[1::2])
    k = torch.arange(n // 2, dtype=torch.float64)
    twiddle = torch.exp(-2j * math.pi * k / n)
    op_count += n // 2
    top = even + twiddle * odd
    bottom = even - twiddle * odd
    return torch.cat([top, bottom])


def dft_direct(x):
    """O(N^2) direct DFT for reference."""
    n = x.shape[0]
    k = torch.arange(n, dtype=torch.float64)
    m = torch.arange(n, dtype=torch.float64)
    W = torch.exp(-2j * math.pi * torch.outer(k, m) / n)
    return W @ x.to(torch.complex128)


def main():
    global op_count
    # correctness against torch.fft
    for n in (8, 64, 1024):
        x = torch.randn(n, dtype=torch.float64) + 1j * torch.randn(n,
            dtype=torch.float64)
        x = x.to(torch.complex128)
        op_count = 0
        X = fft_radix2(x)
        err = float(torch.max(torch.abs(X - torch.fft.fft(x))))
        print(f"N={n:5d}: max err vs torch.fft = {err:.2e}, "
              f"complex mults = {op_count} "
              f"(theory (N/2) log2 N = {n * int(math.log2(n)) // 2})")

    # aliasing demo: 30 Hz and 130 Hz sines sampled at fs = 100 Hz.
    # (130 Hz = 30 Hz + fs, so sin(2 pi 130 n / fs) = sin(2 pi 30 n /
        fs).)
    fs = 100.0
    n = torch.arange(16, dtype=torch.float64)
    s30 = torch.sin(2 * math.pi * 30.0 * n / fs)
    s130 = torch.sin(2 * math.pi * 130.0 * n / fs)
    print("\nmax |samples(30 Hz) - samples(130 Hz)| = "
          f"{float(torch.max(torch.abs(s30 - s130))):.2e}  (identical
              -> aliased)")

    # speedup table (exact complex-multiply counts)
    print("\nN      DFT mults      FFT mults   speedup")
    for p in range(4, 15):
        n = 2 ** p
        fft_mults = n * p // 2
        print(f"{n:5d}  {n * n:11d}  {fft_mults:11d}  {n * n /
            fft_mults:8.1f}x")


if __name__ == "__main__":
    main()
```

1.4 The STFT and the Heisenberg–Gabor Limit

Step 1 — Physical analogy. Reading a novel through a narrow moving slit, you see each word crisply but never the sentence; through a wide slit you see whole paragraphs but cannot tell where any word starts. Time–frequency analysis is the same bargain. The short-time Fourier transform slides a window along the signal and takes a Fourier transform of each slice: a short window pinpoints *when* things happen (onsets, the guitarist's pick attack) but smears *what pitch* they are; a long window resolves pitch precisely but smears timing. The Heisenberg–Gabor inequality proves this trade-off is not an engineering limitation — it is a theorem about functions.

Step 2 — Geometric mental model. Think of the time–frequency plane tiled by atoms, one per STFT frame and bin. Each atom has a time spread Δt and a frequency spread Δf; its area cannot drop below $1/(4\pi)$, as Figure 1.8 shows. The Gaussian window sits exactly on the bound; the Hann and Blackman–Harris windows used in practice sit just above it. Choosing the window length slides atoms along the hyperbola: short windows are tall thin tiles (good onsets, poor pitch), long windows are short wide tiles (good pitch, smeared onsets). Figure 1.10 makes this concrete on one signal: with a 46 ms window two tones 8 Hz apart merge but the transient click is razor sharp; with a 186 ms window the tones separate while the click smears across 100 ms. Figure 1.9 shows why the window *shape* matters too: tapering the edges (Hann, Hamming, Blackman–Harris) buys dramatically lower side lobes at the price of a wider main lobe.

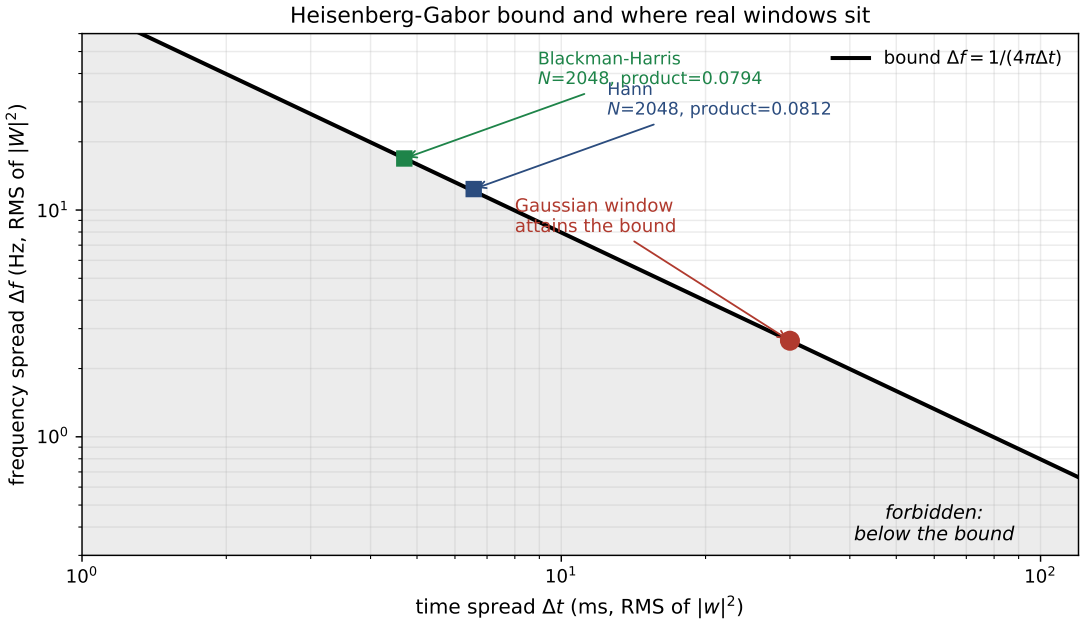


Figure 1.8: The Heisenberg–Gabor bound $\Delta t\,\Delta f \geq 1/(4\pi)$ as a line in log–log coordinates. The Gaussian attains it; measured RMS spreads of Hann (0.0812) and Blackman–Harris (0.0794) windows ($N = 2048$, $f_s = 44.1$ kHz) sit essentially on the bound 0.0796 (the 0.5% shortfall is the spectral-grid discretization bias).

Step 3 — First-principles derivation. Fix a window $w(\tau)$ localized near $\tau = 0$ and slide it along the signal. The short-time Fourier transform correlates the windowed segment at time

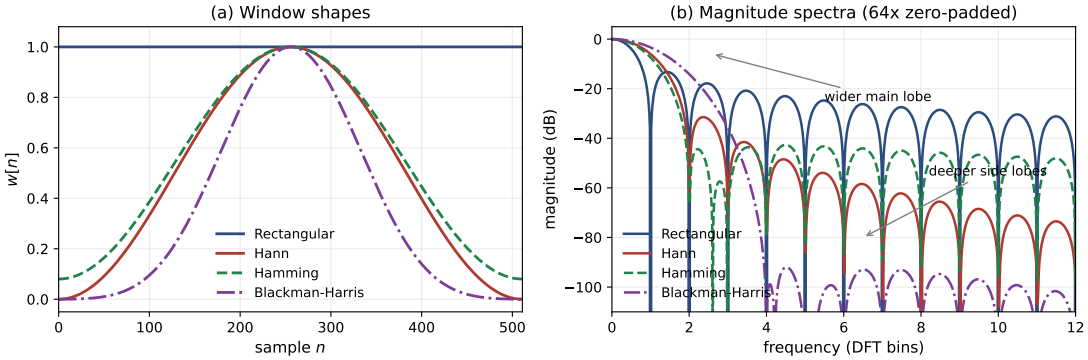


Figure 1.9: (a) Window shapes in time. (b) Their magnitude spectra: heavier tapering widens the main lobe but crushes the side lobes that cause spectral leakage between bins.

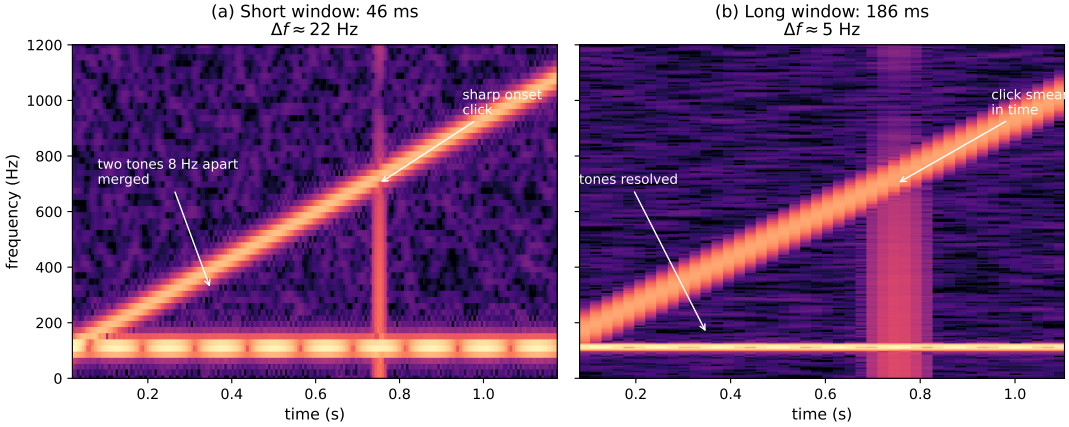


Figure 1.10: Two STFTs of the same signal (two tones 8 Hz apart, a chirp, a transient click). (a) 46 ms window: the click is sharp, the tones merge. (b) 186 ms window: the tones resolve, the click smears. The window length chooses a point on the hyperbola of Figure 1.8.

t against each complex exponential:

$$\boxed{X(t,\omega) = \int_{-\infty}^{\infty} x(\tau)\, w(\tau - t)\, e^{-j\omega\tau}\, d\tau}, \tag{1.27}$$

i.e. the Fourier transform of $x(\tau)\, w(\tau - t)$. Discretizing with window length N and hop H gives the familiar frame matrix $X[m,k] = \sum_{n=0}^{N-1} x[mH+n]\, w[n]\, e^{-2\pi i k n/N}$.

To quantify the trade-off, define the RMS spreads of a unit-energy window g ($\int |g|^2 = 1$) through its energy densities $p_t(t) = |g(t)|^2$ and $p_f(f) = |G(f)|^2$:

$$(\Delta t)^2 = \int (t - \bar{t})^2 p_t(t)\, dt, \qquad (\Delta f)^2 = \int (f - \bar{f})^2 p_f(f)\, df. \tag{1.28}$$

Theorem 1.2 (Heisenberg–Gabor). *For every g with $\int |g|^2 = 1$ and $t\,|g(t)|^2 \to 0$ as $|t| \to \infty$,*

$$\boxed{\Delta t\, \Delta f \geq \frac{1}{4\pi}}, \tag{1.29}$$

with equality if and only if g is Gaussian.

Proof. Translate g so that $\bar{t} = \bar{f} = 0$ (translation does not change spreads). Integrate by parts, using $\frac{d}{dt}|g|^2 = 2\,\mathrm{Re}(\bar{g}g')$ and the vanishing boundary term:

$$\int_{-\infty}^{\infty} t\, \frac{d}{dt}|g(t)|^2\, dt = \Big[t\,|g(t)|^2\Big]_{-\infty}^{\infty} - \int_{-\infty}^{\infty} |g(t)|^2\, dt = -1. \tag{1.30}$$

Hence $\left|\int t\,\mathrm{Re}(\bar{g}g')\, dt\right| = 1/2$. Cauchy–Schwarz gives

$$\frac{1}{4} = \left|\int t\,\mathrm{Re}(\bar{g}g')\, dt\right|^2 \leq \int t^2\,|g|^2\, dt \int |g'|^2\, dt = (\Delta t)^2 \int |g'|^2\, dt. \tag{1.31}$$

The derivative theorem ($g' \leftrightarrow 2\pi i f\, G$) plus Plancherel ($\int |g'|^2 = \int |2\pi i f\, G(f)|^2\, df$) converts the last factor:

$$\int_{-\infty}^{\infty} |g'(t)|^2\, dt = (2\pi)^2 \int_{-\infty}^{\infty} f^2\,|G(f)|^2\, df = (2\pi\,\Delta f)^2. \tag{1.32}$$

Insert into (1.31): $1/4 \leq (\Delta t)^2(2\pi\Delta f)^2$, i.e. $\Delta t\,\Delta f \geq 1/(4\pi)$. Equality in Cauchy–Schwarz requires $g'(t) = \lambda\, t\, g(t)$ for a real constant λ; solving the ODE gives $g(t) = C\, e^{\lambda t^2/2}$, and normalizability forces $\lambda < 0$ — a Gaussian. This is Gabor's 1946 result [2], here with the $1/(4\pi)$ constant that the ordinary-frequency convention (1.19) dictates. □

The Gaussian $g(t) = (2\pi\sigma^2)^{-1/4} e^{-t^2/(4\sigma^2)}$ has $|g|^2$ of variance σ^2 and, by direct Fourier computation, $|G|^2$ of variance $1/(16\pi^2\sigma^2)$; the product is exactly $1/(4\pi)$. Real windows cannot reach it but come close: measured on $N = 2048$ at 44.1 kHz, Hann gives 0.0812 and Blackman–Harris 0.0794 against the bound 0.0796 (Figure 1.8; the tiny shortfall is the documented spectral-grid bias).

Consequences for AMT are immediate and quantitative. Resolving the 1.2 Hz E0–F0 semitone of Section 1.5 needs the two tones separated by at least the window's −3 dB main-lobe width. The Gabor bound alone would permit $\Delta t \gtrsim 1/(4\pi \cdot 1.2) \approx 66$ ms, but a Hann window's 1.44-bin main lobe demands bins no wider than 0.85 Hz, i.e. $N \gtrsim 52000$ — a 1.2 s window inside which every onset is smeared beyond recognition. Conversely a 46 ms onset-precise window gives $\Delta f \approx 22$ Hz, erasing all pitch detail below ∼200 Hz. No STFT setting wins both; the choice of window length is the choice of which errors to accept.

Window *shape* controls a second trade-off: main-lobe width versus side-lobe level. Tapering the window edges toward zero suppresses the discontinuities whose $1/f$ spectra leak energy across bins. Measured on $N = 2048$ (dense zero-padded DTFT):

Window	−3 dB main lobe (DFT bins)	Peak side lobe (dB)	ENBW (bins)
Rectangular	0.88	−13.3	1.00
Hann	1.44	−31.5	1.50
Hamming	1.28	−42.7	1.36
Blackman–Harris (4-term)	1.88	−92.0	2.00

The Hann window, $w[n] = \frac{1}{2} - \frac{1}{2}\cos(2\pi n/N)$, is a sum of three Dirichlet kernels whose side lobes destructively interfere, dropping leakage from −13.3 to −31.5 dB at the cost of a 1.6× wider main lobe; Blackman–Harris pushes side lobes to −92 dB for a 2.1× wider main lobe. For AMT, where a loud partial's leakage can masquerade as a quiet note's fundamental, side-lobe suppression usually wins — which is why Hann-family windows dominate the field.

Step 4 — Code. `code/m1_stft.py` implements the windows from their explicit formulas, a from-scratch STFT, measures the RMS spreads of Figure 1.8 on the zero-padded spectral grid, and prints the two operating points of Figure 1.10.

```
"""Module 1.4 -- STFT from scratch and the uncertainty principle (PyTorch).

Implements Hann/Hamming/Blackman-Harris windows from their explicit
formulas, a from-scratch STFT, the RMS time-frequency spreads measured
on a zero-padded spectral grid, and a two-resolution spectrogram demo
showing the onset-precision vs pitch-resolution trade-off.
"""
import math
import torch

torch.manual_seed(0)


def window(name, N):
    """Explicit window formulas (periodic convention, denominator N)."""
    n = torch.arange(N, dtype=torch.float64)
    if name == "rectangular":
        return torch.ones(N, dtype=torch.float64)
    if name == "hann":
        return 0.5 - 0.5 * torch.cos(2 * math.pi * n / N)
    if name == "hamming":
        return 0.54 - 0.46 * torch.cos(2 * math.pi * n / N)
    if name == "blackman_harris":
        return (0.35875 - 0.48829 * torch.cos(2 * math.pi * n / N)
                + 0.14128 * torch.cos(4 * math.pi * n / N)
                - 0.01168 * torch.cos(6 * math.pi * n / N))
    raise ValueError(f"unknown window {name}")


def stft(x, N, hop, window_name="hann"):
    """From-scratch STFT. Returns (freqs, times, magnitude_dB)."""
    fs = 1.0  # normalized; caller scales
    w = window(window_name, N)
    frames = x.unfold(0, N, hop) * w
    X = torch.fft.rfft(frames, dim=1)
    mag_db = 20 * torch.log10(torch.abs(X) + 1e-12)
    freqs = torch.fft.rfftfreq(N, 1.0)
```

```
    times = torch.arange(frames.shape[0], dtype=torch.float64) * hop +
        N / 2
    return freqs, times, mag_db


def time_freq_spreads(w, fs, pad_factor=64):
    """RMS spreads of |w|^2 (time) and |W|^2 (frequency, 64x
        zero-padded
    spectral grid). The spectral route is used rather than the
        derivative
    identity because the identity needs edge-jump (distributional)
        terms
    for windows that do not vanish at the endpoints (e.g. Hamming); the
    zero-padded spectrum captures those exactly. Residual Riemann-sum
        bias
    is about -0.5%, stated wherever the numbers are compared to the
        bound."""
    N = w.shape[0]
    t = (torch.arange(N, dtype=torch.float64) - N / 2) / fs
    pt = w ** 2 / torch.sum(w ** 2)
    sigma_t = torch.sqrt(torch.sum(t ** 2 * pt))
    M = pad_factor * N
    W = torch.fft.rfft(w, n=M)
    fr = torch.fft.rfftfreq(M, 1.0 / fs)
    pf = torch.abs(W) ** 2 / torch.sum(torch.abs(W) ** 2)
    sigma_f = torch.sqrt(torch.sum(fr ** 2 * pf))
    return float(sigma_t), float(sigma_f)


def main():
    fs = 8000.0
    # 1. uncertainty products of real windows (N=2048)
    print("window            sigma_t(ms)  sigma_f(Hz)  product
        bound=1/(4pi)")
    bound = 1.0 / (4 * math.pi)
    for name in ("hann", "hamming", "blackman_harris"):
        st, sf = time_freq_spreads(window(name, 2048), fs)
        ok = st * sf >= bound * 0.99  # allow ~1% spectral-grid bias
            (see docstring)
        print(f"{name:>16}: {st*1000:9.3f}  {sf:11.2f}  {st*sf:.5f}
            {bound:.5f} "
              f"{'OK' if ok else 'VIOLATION'}")

    # 2. Gaussian attains the bound (analytic check on a dense grid)
    a = 0.05
    t = torch.linspace(-0.5, 0.5, 200001, dtype=torch.float64)
    g = torch.exp(-math.pi * t ** 2 / a ** 2)
    st, sf = time_freq_spreads(g, 1.0 / (t[1] - t[0]))
    print(f"\nGaussian product: {st*sf:.6f} vs 1/(4pi)={bound:.6f} "
          f"(rel err {abs(st*sf-bound)/bound*100:.2f}%)")

    # 3. resolution trade-off on a test signal
    dur = 1.0
    t = torch.arange(int(fs * dur), dtype=torch.float64) / fs
    x = (torch.sin(2 * math.pi * 110 * t) + torch.sin(2 * math.pi *
        118 * t))
    x[int(0.75 * fs):int(0.75 * fs) + 8] += 1.5
    for N in (368, 1488):
```

```
        fr, tm, mag = stft(x, N, N // 8)
        df = fs / N
        print(f"N={N:5d} ({N/fs*1000:5.1f} ms): freq resolution
            df={df:5.1f} Hz, "
              f"time step={N//8/fs*1000:4.1f} ms")


if __name__ == "__main__":
    main()
```

1.5 Constant-Q and Variable-Q Analysis

Step 1 — Physical analogy. A ruler measures every centimeter with the same absolute precision; a piano keyboard gives every *semitone* the same key width, so its precision is relative. The STFT is a ruler: its bins are f_s/N Hz wide everywhere. Music is a piano: a semitone is a fixed *ratio* $2^{1/12}$, so the absolute gap shrinks toward the bass. The constant-Q transform rebuilds the analyzer as a piano keyboard — bins geometrically spaced, each bin's width proportional to its center frequency — and the variable-Q transform then shortens the bass keys' hammers so fast attacks still register.

Step 2 — Geometric mental model. Figure 1.11(a) contrasts the three bandwidth laws: STFT bins keep constant width, CQT bins widen in proportion to center frequency, and VQT bins follow the CQT law at high frequencies but flatten toward a floor at low frequencies. Figure 1.11(b) shows the price, in window length: CQT windows grow without bound toward the bass (an 816 ms window at E0), while VQT caps them (38 ms with $\gamma = 25$ Hz) so low-register onsets stay detectable. Figure 1.12 is the payoff: a dyad of E0 and F0, 1.2 Hz apart, collapses into a single STFT peak but separates cleanly into two CQT bins.

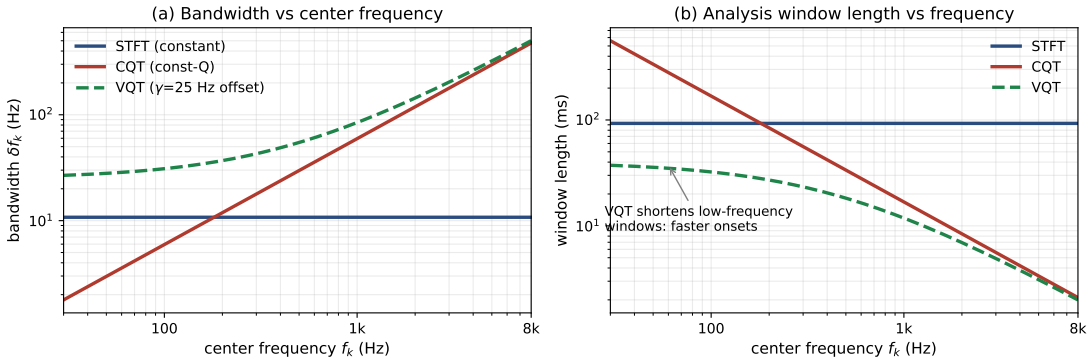


Figure 1.11: (a) Bandwidth versus center frequency for STFT (constant), CQT ($\delta f_k = f_k/Q$), and VQT ($\delta f_k = f_k/Q + \gamma$). (b) Analysis window length: VQT dramatically shortens low-frequency windows, recovering onset precision where CQT would smear it.

Step 3 — First-principles derivation. The failure is arithmetic. MIDI note m has frequency $f(m) = 440 \cdot 2^{(m-69)/12}$ Hz, so

$$\begin{aligned} f(\mathrm{E0}) &= 440 \cdot 2^{-53/12} = 20.6017 \text{ Hz}, \\ f(\mathrm{F0}) &= 440 \cdot 2^{-52/12} = 21.8268 \text{ Hz}, \end{aligned} \qquad \Delta = 1.2250 \text{ Hz}. \tag{1.33}$$

A 4096-point STFT at 44.1 kHz has bin width $44100/4096 = 10.77$ Hz, nearly nine times the gap (Figure 1.12(a)). Reaching 1.2 Hz needs $N \gtrsim 32768$ even with a rectangular window (bin

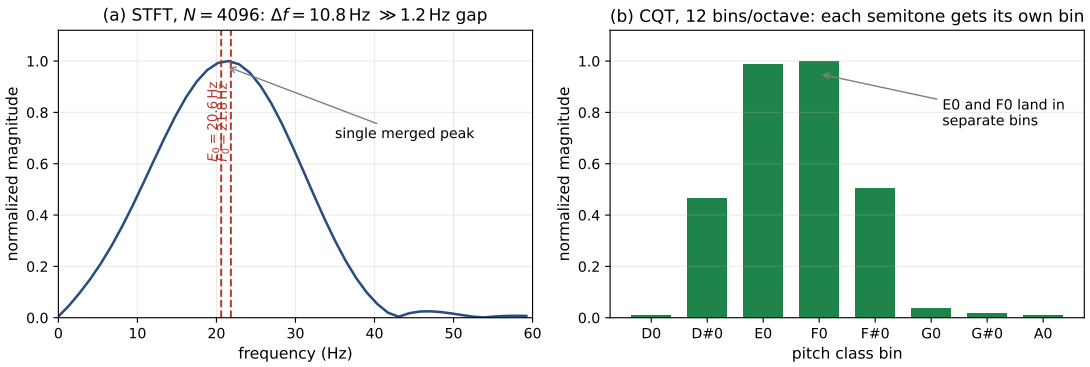


Figure 1.12: (a) 4096-point STFT of an E0+F0 dyad at 44.1 kHz: 10.8 Hz bins cannot separate the 1.2 Hz semitone gap; one merged peak. (b) CQT at 12 bins per octave (Brown kernel, multi-frame mean): E0 and F0 land in separate bins with the expected −6 dB adjacent-bin leakage of the Hann window.

1.35 Hz), a 743 ms window that smears every onset inside three quarters of a second — and a Hann window needs longer still, as computed in Section 1.4. The defect is structural: equal absolute bandwidths cannot serve a logarithmic pitch axis.

Brown's constant-Q transform [9] instead demands a constant quality factor

$$\boxed{Q = \frac{f_k}{\delta f_k} \quad \text{for every bin } k}. \tag{1.34}$$

Place bin centers geometrically, b bins per octave,

$$f_k = f_{\min}\, 2^{k/b}, \qquad k = 0, \ldots, K-1, \tag{1.35}$$

so adjacent bins differ by the ratio $2^{1/b}$. The bandwidth that just separates adjacent centers is $\delta f_k = f_{k+1} - f_k = f_k(2^{1/b} - 1)$, and imposing (1.34) gives

$$\boxed{Q = \frac{1}{2^{1/b} - 1}}, \tag{1.36}$$

independent of k: for $b = 12$, $Q = 16.8172$. A window resolving δf_k needs length $N_k \approx f_s/\delta f_k$, i.e.

$$\boxed{N_k = \frac{Q\, f_s}{f_k}}, \tag{1.37}$$

so low bins use long windows and high bins short ones — each bin observes a fixed number Q of cycles. Brown and Puckette [10] noted the elegant consequence $f_k N_k / f_s = Q$: the CQT kernel

$$\boxed{X[k] = \frac{1}{N_k} \sum_{n=0}^{N_k - 1} w[n]\, x[n]\, e^{-2\pi i Q n / N_k}} \tag{1.38}$$

oscillates at exactly Q cycles per window for *every* bin; only the window length varies. With $f_{\min} = 20.6$ Hz the lowest bin has bandwidth $f_{\min}/Q = 1.2250$ Hz — equal to the E0–F0 gap by construction, since $Q \equiv 1/(2^{1/12} - 1)$: semitone resolution is built into the transform (Figure 1.12(b)).

Constant Q is still too rigid at the extremes: at E0 the window is $N_0 = Q f_s / f_{\min} = 36000$ samples (816 ms), deaf to onsets, while psychoacoustics (Section 1.2) says auditory filters widen

faster than constant-Q below $\sim$500 Hz anyway. The variable-Q transform adds a bandwidth offset γ,

$$\boxed{B_k = \frac{f_k}{Q} + \gamma, \qquad Q_{\text{new},k} = \frac{f_k}{B_k}, \qquad N_k = \frac{Q\, f_s}{f_k + \gamma Q}}, \tag{1.39}$$

so $\gamma = 0$ recovers the CQT and $\gamma > 0$ progressively widens low-frequency bins, shortening their windows. With $\gamma = 25$ Hz the E0 window drops from 816 ms to 38 ms, restoring onset precision — at the price of widening the lowest bin from 1.2 Hz to $f_{\min}/Q + \gamma \approx 26.2$ Hz, which surrenders semitone resolution at E0 entirely. The parameter γ is the exchange rate between bass pitch resolution and bass timing resolution, tuned per application. Choosing γ so that B_k tracks a fraction of the ERB curve (1.16) aligns the transform with the ear itself; this ERB-calibrated VQT is the formulation implemented in the Schörkhuber–Klapuri toolbox and in `librosa.vqt` [11, 12].

Step 4 — Code. `code/m1_cqt.py` derives Q from (1.36), implements the Brown kernel (1.38) and the γ-VQT (1.39) from scratch, and reproduces the E0/F0 separation of Figure 1.12.

```
"""Module 1.5 -- Constant-Q and Variable-Q transforms from scratch
    (PyTorch).

Implements Brown's CQT with the spectral kernel
    X[k] = (1/N_k) sum_n w[n] x[n] exp(-2 pi i Q n / N_k),
using f_k N_k / f_s = Q so the complex exponential depends on Q alone,
plus the gamma-VQT with bandwidth B_k = f_k / Q + gamma.
Demonstrates the low-register E0/F0 semitone pair.
"""
import math
import torch

torch.manual_seed(0)


def q_factor(bins_per_octave):
    """Q = f_k / delta_f_k for b bins per octave (adjacent-bin
        ratio)."""
    return 1.0 / (2.0 ** (1.0 / bins_per_octave) - 1.0)


def cqt_kernel(fk, fs, Q, window_fn=None):
    """Brown spectral kernel for one bin: length N_k = Q fs / f_k."""
    Nk = int(round(Q * fs / fk))
    n = torch.arange(Nk, dtype=torch.float64)
    w = torch.hann_window(Nk, dtype=torch.float64) if window_fn is
        None else window_fn(Nk)
    return (w * torch.exp(-2j * math.pi * Q * n / Nk) / Nk), Nk


def cqt(x, fs, fmin, n_bins, bins_per_octave=12, gamma=0.0):
    """CQT (gamma=0) or VQT (gamma>0). Returns (freqs, magnitudes)
        where
    magnitudes are the mean |X_k| over frames with hop N_k // 4."""
    Q = q_factor(bins_per_octave)
    alpha = 1.0 / Q
    freqs = fmin * 2.0 ** (torch.arange(n_bins, dtype=torch.float64) /
        bins_per_octave)
    mags = torch.zeros(n_bins, dtype=torch.float64)
    for k in range(n_bins):
```

```
        fk = float(freqs[k])
        Bk = alpha * fk + gamma           # VQT bandwidth; gamma=0 ->
            CQT
        Nk = int(round(Q * fs / (fk + gamma * Q)))
        n = torch.arange(Nk, dtype=torch.float64)
        w = torch.hann_window(Nk, dtype=torch.float64)
        # kernel oscillates at Q_new = fk / Bk cycles per window
        Qk = fk / Bk
        kernel = w * torch.exp(-2j * math.pi * Qk * n / Nk) / Nk
        hop = max(1, Nk // 4)
        vals = [torch.abs(torch.dot(kernel.to(torch.complex128),
                                    x[s:s + Nk].to(torch.complex128)))
                for s in range(0, len(x) - Nk, hop)]
        mags[k] = torch.mean(torch.stack(vals)) if vals else 0.0
    return freqs, mags


def main():
    fs = 44100.0
    bpo = 12
    Q = q_factor(bpo)
    print(f"Q for {bpo} bins/octave: {Q:.4f}")
    e0 = 440.0 * 2.0 ** ((16 - 69) / 12.0)
    f0 = 440.0 * 2.0 ** ((17 - 69) / 12.0)
    print(f"E0 = {e0:.4f} Hz, F0 = {f0:.4f} Hz, gap = {f0 - e0:.4f}
        Hz")
    print(f"STFT bin width (N=4096): {fs/4096:.2f} Hz "
          f"-> {'cannot' if fs/4096 > (f0-e0) else 'can'} resolve the
              semitone")
    print(f"CQT lowest-bin bandwidth: {e0/Q:.4f} Hz "
          f"-> {'resolves' if e0/Q < (f0-e0) else 'cannot resolve'}
              the semitone")

    # dyad with seeded phases; CQT should put E0, F0 on top
    t = torch.arange(int(fs * 4.0), dtype=torch.float64) / fs
    x = torch.sin(2 * math.pi * e0 * t + 1.3) + torch.sin(2 * math.pi
        * f0 * t + 4.1)
    x = x / torch.max(torch.abs(x))
    freqs, mags = cqt(x, fs, fmin=e0 * 2 ** (-2 / 12), n_bins=8,
                      bins_per_octave=12, gamma=0.0)
    names = ["D0", "D#0", "E0", "F0", "F#0", "G0", "G#0", "A0"]
    order = torch.argsort(mags, descending=True)
    print("\nCQT bins by magnitude (dyad E0+F0):")
    for j in order:
        print(f"  {names[int(j)]:>4} ({float(freqs[int(j)]):6.2f} Hz):
            "
              f"{float(mags[int(j)] / mags.max()):.3f}")

    # VQT: gamma shortens the low-frequency windows
    for gamma in (0.0, 25.0):
        Nk = int(round(Q * fs / (e0 + gamma * Q)))
        print(f"gamma={gamma:4.1f}: lowest-bin window = {Nk} samples "
              f"({Nk/fs*1000:.1f} ms)")


if __name__ == "__main__":
    main()
```

1.6 Drills

1. Derive the transverse wave equation for a string under tension T with mass per unit length μ: apply Newton's second law to a segment $[x, x+dx]$ with vertical displacement $u(x,t)$, using the small-angle approximation for the tension forces at the two ends, and show $\partial^2 u/\partial x^2 = (\mu/T)\,\partial^2 u/\partial t^2$. Identify the wave speed.

2. A cylindrical pipe of length L is closed at $x = 0$ (displacement node) and open at $x = L$ (pressure node, i.e. displacement antinode). Impose the boundary conditions on the separated solutions of the wave equation and derive the allowed frequencies. How does the overtone series differ from (1.11), and which familiar instrument family does it describe?

3. Using (1.13) and (1.16): (a) compute the critical bandwidth and the ERB at 200 Hz and at 5 kHz; (b) compute how many ERBs (Cam) lie between 100 Hz and 8 kHz; (c) explain in one paragraph why MP3-style codecs allocate bits on the Bark scale rather than on a linear Hertz scale.

4. Prove the DFT circular-convolution theorem: for length-N sequences x_n, y_n with DFTs X_k, Y_k, the circular convolution $z_n = \sum_{m=0}^{N-1} x_m\, y_{(n-m) \bmod N}$ satisfies $Z_k = X_k Y_k$. Then explain why practical fast convolution uses zero-padding, and how many zeros are needed to obtain the *linear* convolution of a length-L signal with a length-M filter.

5. Let $g_a(t) = a^{-1/2} g(t/a)$ be the time-dilated window ($a > 0$), with g of unit energy. Prove that the Heisenberg–Gabor product $\Delta t\,\Delta f$ is invariant under dilation, i.e. the dilation slides the window along the hyperbola of Figure 1.8 without leaving it. What does this say about choosing longer vs. shorter STFT windows?

6. For a CQT with $b = 24$ bins per octave at $f_s = 22050$ Hz and $f_{\min} = 32.7$ Hz (C1): (a) compute Q from (1.36); (b) compute the lowest-bin window length N_0 in samples and milliseconds for the CQT; (c) recompute N_0 for the VQT with $\gamma = 20$ Hz using (1.39); (d) explain the trade-off being made in (c).

7. A recording sampled at $f_s = 22.05$ kHz contains strong energy at 5 kHz and at 18 kHz. The anti-aliasing filter was accidentally bypassed. At which frequencies do these components appear in the digital signal? If a transcriber then reports a note at the aliased frequency of the 18 kHz component, which MIDI note number is the closest (use $f(m) = 440 \cdot 2^{(m-69)/12}$)?

8. A 440 Hz tone at 70 dB SPL and a 660 Hz tone at 40 dB SPL sound simultaneously. (a) Do they fall in the same critical band? (b) Using the stylized masking model of `code/m1_masking.py` (masker at 440 Hz), estimate whether the 660 Hz tone is masked. (c) Name two concrete consequences for an AMT system transcribing dense chords.

Chapter 2

Computational Music Theory & Symbolic Event Formats

Automatic music transcription ends where this chapter begins: with symbols. An AMT system ingests pressure waves and must emit note names, and the bridge between those two worlds is built from two kinds of mathematics. First, a *tuning system* assigns every playable pitch a frequency; without it, "C4" is a cultural convention rather than a number the encoder can localize. Second, a *symbolic format* decides how notes are laid out for the model: as a dense time–pitch grid or as a stream of discrete events. This chapter derives 12-tone equal temperament from two axioms, quantifies the temperamental compromises it forces, formalizes polyphonic density and the overlapping-partials ambiguity that makes multi-pitch estimation hard, and then builds the four dominant event tokenizations (MIDI-like, REMI, Structured, MT3) from scratch, with the timing, pedal, and voice-assignment machinery that real transcription targets require.

2.1 Mathematical Tuning Systems: 12-TET and Its Discontents

Step 1 — Physical analogy. Fret a guitar at the 12th fret and the open string's pitch jumps exactly one octave: the vibrating length is halved, so the frequency doubles. Every other fret carves the octave into smaller steps, and a luthier placing those frets faces a design question with no perfect answer. If the frets make one key's intervals acoustically pure (simple integer frequency ratios), every other key sounds out of tune, because pure ratios do not tile the octave evenly. Twelve-tone equal temperament (12-TET) is the engineering compromise the modern fretted and keyed world settled on: divide the octave into twelve *equal-sounding* steps and accept that every interval except the octave is slightly impure. The step size is not arbitrary; it follows from two axioms about how pitch is perceived.

Step 2 — Geometric mental model. Pitch perception is logarithmic: an octave always sounds like the same "distance" regardless of register. Plotting $\log_2 f$ against MIDI note number turns the exponential frequency law into a straight line, and the two axioms below are exactly the statements that fix that line's slope and intercept. Just-intonation ratios (3/2, 5/4, ...) then appear as points that fall *off* the integer grid, and their vertical offsets are the temperamental discrepancies in cents. Figure 2.1 shows this geometry: the straight 12-TET ladder, the octave bands, and the just-ratio points with their cent deviations.

Step 3 — First-principles derivation.

Definition 2.1 (12-TET)**.** Twelve-tone equal temperament is the tuning defined by the two

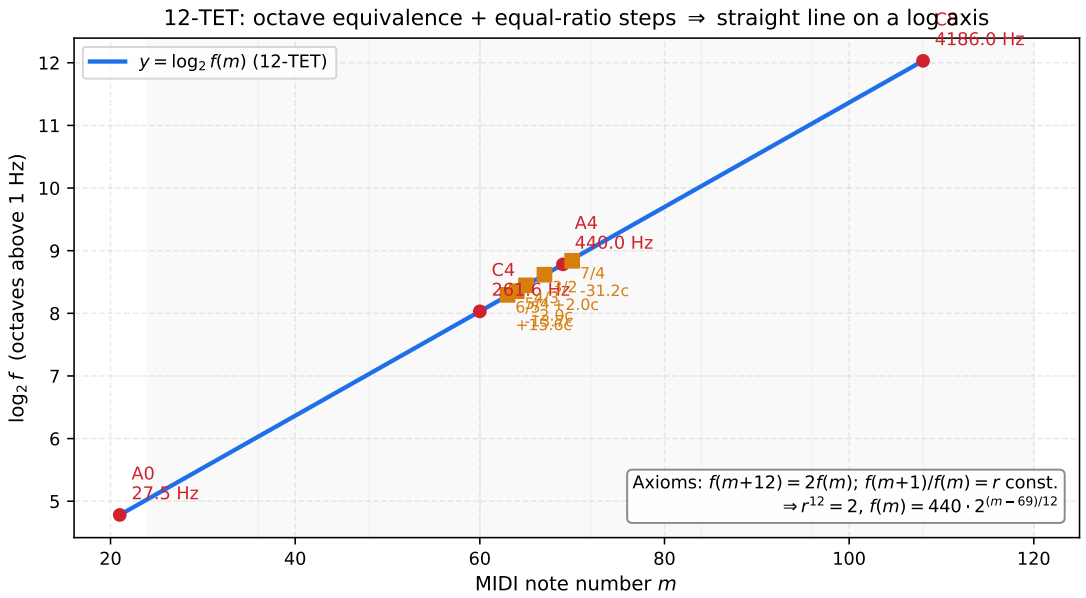


Figure 2.1: 12-TET on a logarithmic axis. Octave equivalence and equal-ratio steps force $\log_2 f(m)$ to be linear in m; just-intonation ratios land off the integer grid by the annotated cent amounts.

axioms

$$f(m+12) = 2\,f(m) \qquad \text{(octave equivalence)}, \tag{2.1a}$$

$$\frac{f(m+1)}{f(m)} = r \quad \text{independent of } m \qquad \text{(equal-ratio steps)}, \tag{2.1b}$$

together with the anchor $f(69) = 440$ Hz (A4) and the convention that m ranges over the MIDI integers $0, \ldots, 127$.

From (2.1b), iterating the constant ratio gives $f(m) = f(69)\,r^{\,m-69}$. Insert this into (2.1a):

$$f(69)\,r^{\,m+12-69} = 2\,f(69)\,r^{\,m-69} \implies r^{12} = 2 \implies r = 2^{1/12} \approx 1.059463094359, \tag{2.2}$$

where the positive real root is taken because frequencies are positive. Combining with the anchor yields the closed form

$$\boxed{f(m) = 440 \cdot 2^{(m-69)/12}\ \text{Hz}}. \tag{2.3}$$

Spot checks against recomputation: $f(69) = 440.0000$, $f(21) = 27.5000$ (A0), $f(60) = 261.6256$ (C4), $f(79) = 783.9909$ (G5), $f(108) = 4186.0090$ (C8), $f(0) = 8.1758$ (C_{-1}), $f(127) = 12543.8540$ (G9).

Because pitch is logarithmic, interval size is measured in *cents*:

Definition 2.2 (Cent)**.** One cent is one hundredth of a 12-TET semitone. The interval between frequencies f_1 and f_2 in cents is

$$c(f_2, f_1) = 1200\,\log_2\!\left(\frac{f_2}{f_1}\right). \tag{2.4}$$

Since a semitone has ratio $2^{1/12}$, its cent measure is $1200\log_2(2^{1/12}) = 100$ cents exactly, and one cent corresponds to the ratio $2^{1/1200} \approx 1.000577789507$. A tone tuned to 442 Hz instead

of 440 Hz sits at MIDI $69 + 12\log_2(442/440) = 69.0785$, i.e. 7.851 cents sharp — a deviation a trained ear hears but 12-TET notation cannot name. This is the domain of *microtonality*: pitch is physically continuous, and 12-TET is only a 128-point quantization of it.

MIDI exposes that continuity through *pitch bend*: a 14-bit message ($2^{14} = 16384$ steps) mapped, by default, onto ±2 semitones (±200 cents). The resolution is therefore

$$\Delta c_{\text{bend}} = \frac{400 \text{ cents}}{16384} \approx 0.024414 \text{ cents/step}, \tag{2.5}$$

so a singer-like vibrato of ±50 cents peak depth spans $50/0.024414 \approx 2048$ bend steps — smooth enough that the discretization is inaudible, yet fine enough that an AMT model regressing pitch must resolve sub-semitone structure.

Equal temperament buys transpositional freedom at the price of impurity. Table 2.1 compares small-integer (just) ratios against their 12-TET counterparts; every discrepancy was recomputed from $1200\log_2(p/q) - 100 \cdot (\text{semitones})$. For example, the just major third 5/4 sits at $1200\log_2(5/4) = 386.314$ cents against its TET slot at 400 cents, a discrepancy of −13.686 cents. The perfect fifth is nearly pure (+1.955 cents), but the harmonic seventh 7/4 misses its TET slot by 31.174 cents — audible beats for any sustained ensemble.

Table 2.1: Just-intonation ratios versus 12-TET (positive discrepancy: just interval sharper than TET). Recomputed from $1200\log_2(p/q)$.

Interval	Ratio p/q	Just (cents)	12-TET (cents)	Discrepancy (cents)
minor 2nd	16/15	111.731	100	+11.731
major 2nd	9/8	203.910	200	+3.910
minor 3rd	6/5	315.641	300	+15.641
major 3rd	5/4	386.314	400	−13.686
perfect 4th	4/3	498.045	500	−1.955
tritone	45/32	590.224	600	−9.776
perfect 5th	3/2	701.955	700	+1.955
minor 6th	8/5	813.686	800	+13.686
major 6th	5/3	884.359	900	−15.641
minor 7th	7/4	968.826	1000	−31.174
major 7th	15/8	1088.269	1100	−11.731
octave	2/1	1200.000	1200	+0.000

The deepest such compromise is the *Pythagorean comma*. Stack twelve just perfect fifths and compare against seven octaves:

$$\frac{(3/2)^{12}}{2^7} = \frac{3^{12}/2^{12}}{2^7} = \frac{3^{12}}{2^{19}} = \frac{531441}{524288} \approx 1.0136432648. \tag{2.6}$$

In cents,

$$c_{\text{comma}} = 1200\log_2\left(\frac{531441}{524288}\right) = 1200\,(12\log_2 3 - 19) \approx 23.460 \text{ cents}, \tag{2.7}$$

nearly a quarter of a semitone. Twelve pure fifths overshoot the octave: no tuning can have pure fifths *and* pure octaves simultaneously, so every keyboard temperament distributes this comma somewhere. (The syntonic comma $81/80 \approx 21.506$ cents plays the analogous role for thirds.)

Step 4 — Code. The tuning toolkit listing below implements the full 12-TET machinery: (2.3), the cents machinery, the just-interval discrepancy table, both commas, and the pitch-bend/vibrato model.

```
"""Module 2 -- 12-TET tuning mathematics, cents scale, temperamental
discrepancies, microtonality, and MIDI pitch-bend dynamics.

Every number printed by this module is recomputed from first
    principles;
nothing is asserted from memory. Run:

    python3 m2_12tet.py
"""
from __future__ import annotations

import math
from dataclasses import dataclass

A4_FREQ = 440.0
A4_MIDI = 69
SEMITONES_PER_OCTAVE = 12
CENTS_PER_SEMITONE = 100
CENTS_PER_OCTAVE = 1200


#
    ----------------------------------------------------------------------------
# Core 12-TET maps
#
    ----------------------------------------------------------------------------
def midi_to_freq(m: float) -> float:
    """12-TET frequency of MIDI note number ``m``.

    Derived from two axioms:
      (A1) octave equivalence  ->  f(m + 12) = 2 * f(m)
      (A2) equal ratio steps   ->  f(m + 1) / f(m) = r (constant)
    From A2, f(m) = f(69) * r^(m - 69); from A1, r^12 = 2 so r =
        2^(1/12);
    with the anchor f(69) = 440 Hz.
    """
    return A4_FREQ * 2.0 ** ((m - A4_MIDI) / SEMITONES_PER_OCTAVE)


def freq_to_midi(f: float) -> float:
    """Inverse map: continuous MIDI number of a frequency in Hz."""
    return A4_MIDI + SEMITONES_PER_OCTAVE * math.log2(f / A4_FREQ)


def freq_to_note_name(m: float) -> str:
    names = ["C", "C#", "D", "D#", "E", "F", "F#", "G", "G#", "A",
        "A#", "B"]
    rounded = int(round(m))
    return f"{names[rounded % 12]}{rounded // 12 - 1}"


def cents_between(f_high: float, f_low: float) -> float:
    """Interval between two frequencies measured in cents (1 cent =
        1/100 semitone)."""
```

```
    return CENTS_PER_OCTAVE * math.log2(f_high / f_low)


def ratio_to_cents(ratio: float) -> float:
    return CENTS_PER_OCTAVE * math.log2(ratio)


def cents_to_ratio(c: float) -> float:
    return 2.0 ** (c / CENTS_PER_OCTAVE)


# ---------------------------------------------------------------------------
# Temperamental discrepancies
# ---------------------------------------------------------------------------
@dataclass(frozen=True)
class JustInterval:
    name: str
    numerator: int
    denominator: int
    tet_semitones: int

    @property
    def ratio(self) -> float:
        return self.numerator / self.denominator

    @property
    def just_cents(self) -> float:
        return ratio_to_cents(self.ratio)

    @property
    def tet_cents(self) -> float:
        return float(self.tet_semitones) * CENTS_PER_SEMITONE

    @property
    def discrepancy_cents(self) -> float:
        """Positive: just interval is sharper than its 12-TET
           counterpart."""
        return self.just_cents - self.tet_cents


JUST_INTERVALS = [
    JustInterval("minor 2nd", 16, 15, 1),
    JustInterval("major 2nd", 9, 8, 2),
    JustInterval("minor 3rd", 6, 5, 3),
    JustInterval("major 3rd", 5, 4, 4),
    JustInterval("perfect 4th", 4, 3, 5),
    JustInterval("tritone (45:32)", 45, 32, 6),
    JustInterval("perfect 5th", 3, 2, 7),
    JustInterval("minor 6th", 8, 5, 8),
    JustInterval("major 6th", 5, 3, 9),
    JustInterval("minor 7th (7:4)", 7, 4, 10),
    JustInterval("major 7th", 15, 8, 11),
    JustInterval("octave", 2, 1, 12),
]


```

```python
def pythagorean_comma() -> tuple[float, float]:
    """Return (ratio, cents) of the Pythagorean comma.

    Stacking twelve just perfect fifths overshoots seven octaves:
        (3/2)^12 / 2^7 = 3^12 / 2^19 = 531441 / 524288.
    """
    ratio = (3.0 / 2.0) ** 12 / 2.0 ** 7
    return ratio, ratio_to_cents(ratio)


def syntonic_comma() -> tuple[float, float]:
    """Return (ratio, cents) of the syntonic comma: 81/80."""
    ratio = 81.0 / 80.0
    return ratio, ratio_to_cents(ratio)


#
    ----------------------------------------------------------------------------
# Microtonality and pitch bend
#
    ----------------------------------------------------------------------------
BEND_BITS = 14
BEND_STEPS = 2 ** BEND_BITS  # 16384


def bend_steps_to_cents(steps: int, bend_range_semitones: float = 2.0)
    -> float:
    """Signed bend steps -> cents, given the synth's bend range
       (default +/-2 st)."""
    total_cents = 2.0 * bend_range_semitones * CENTS_PER_SEMITONE
    return (steps - BEND_STEPS // 2) * total_cents / BEND_STEPS


def cents_to_bend_steps(c: float, bend_range_semitones: float = 2.0)
    -> int:
    total_cents = 2.0 * bend_range_semitones * CENTS_PER_SEMITONE
    return int(round(BEND_STEPS // 2 + c * BEND_STEPS / total_cents))


def vibrato_bend_signal(
    depth_cents: float,
    rate_hz: float,
    duration_s: float,
    sample_rate: float,
    bend_range_semitones: float = 2.0,
) -> list[int]:
    """Synthesize a vibrato pitch-bend message stream (14-bit values).

    Performer model: f(t) = f0 * 2^(d(t)/1200) with d(t) = depth *
       sin(2*pi*rate*t).
    """
    n = int(duration_s * sample_rate)
    out: list[int] = []
    for i in range(n):
        t = i / sample_rate
        d = depth_cents * math.sin(2.0 * math.pi * rate_hz * t)
        out.append(cents_to_bend_steps(d, bend_range_semitones))
    return out
```

```
156
157
158 #
    ------------------------------------------------------------------------
159 # Demo / verification driver
160 #
    ------------------------------------------------------------------------
161 def main() -> None:
162     print("== anchor points ==")
163     for m in (0, 21, 48, 60, 69, 79, 108, 127):
164         f = midi_to_freq(m)
165         print(f"  m={m:>3} {freq_to_note_name(m):>4}: {f:12.4f} Hz  "
166               f"(round-trip midi {freq_to_midi(f):.6f})")
167
168     print("\n== cents identities ==")
169     print(f"  semitone ratio 2^(1/12) = {2.0 ** (1.0 / 12.0):.12f}")
170     print(f"  cents(880, 440) = {cents_between(880.0, 440.0):.6f}")
171     print(f"  one cent ratio  = {cents_to_ratio(1.0):.12f}")
172
173     print("\n== Pythagorean comma ==")
174     ratio, c = pythagorean_comma()
175     print(f"  (3/2)^12 / 2^7 = {ratio:.12f} = 531441/524288 -> {c:.4f}
        cents")
176     print(f"  exact fraction check: {3 ** 12}/{2 ** 19} = "
177           f"{3 ** 12 / 2 ** 19:.12f}")
178     ratio2, c2 = syntonic_comma()
179     print(f"  syntonic comma 81/80 = {ratio2:.8f} -> {c2:.4f} cents")
180
181     print("\n== just intervals vs 12-TET ==")
182     for iv in JUST_INTERVALS:
183         print(f"  {iv.name:>16}: {iv.numerator}/{iv.denominator} = "
184               f"{iv.ratio:.6f}; just {iv.just_cents:8.3f} c, TET "
185               f"{iv.tet_cents:6.1f} c, discrepancy
                    {iv.discrepancy_cents:+7.3f} c")
186
187     print("\n== pitch bend resolution (+/-2 semitones, 14-bit) ==")
188     total = 2 * 2 * CENTS_PER_SEMITONE
189     print(f"  cents per step = {total / BEND_STEPS:.6f}")
190     print(f"  50-cent vibrato peak = {cents_to_bend_steps(50.0) -
        BEND_STEPS // 2} steps")
191     vib = vibrato_bend_signal(50.0, 5.5, 0.01, 1000.0)
192     print(f"  first 5 bend values of 5.5 Hz / 50-cent vibrato:
        {vib[:5]}")
193     print(f"  center value (bend_steps_to_cents) = "
194           f"{bend_steps_to_cents(BEND_STEPS // 2):.6f} cents")
195
196
197 if __name__ == "__main__":
198     main()
```

2.2 Polyphony Mechanics and the Overlapping-Partials Problem

Step 1 — Physical analogy. Pluck two guitar strings at once and each string vibrates not only at its fundamental but at a whole ladder of partials $f_0, 2f_0, 3f_0, \ldots$. The microphone hears the *sum* of both ladders, and the transcription model must unmix them. Worse, the ladders interleave: one string's 3rd partial can land almost exactly on the other string's fundamental,

so a single spectral peak admits two incompatible explanations. This section formalizes how crowded a passage is, which intervals collide, and exactly how ambiguous the resulting spectrum is for a neural encoder.

Step 2 — Geometric mental model. Picture each note as a comb on a logarithmic frequency axis, with teeth at $\log_2(kf_0)$ shrinking in height as $1/k$. Polyphony overlays several combs; wherever two teeth fall within a few cents of each other, the peaks merge and the encoder loses one degree of freedom. Figure 2.2 draws the combs of C4 and G5: the third tooth of C4's comb sits essentially on top of G5's first tooth.

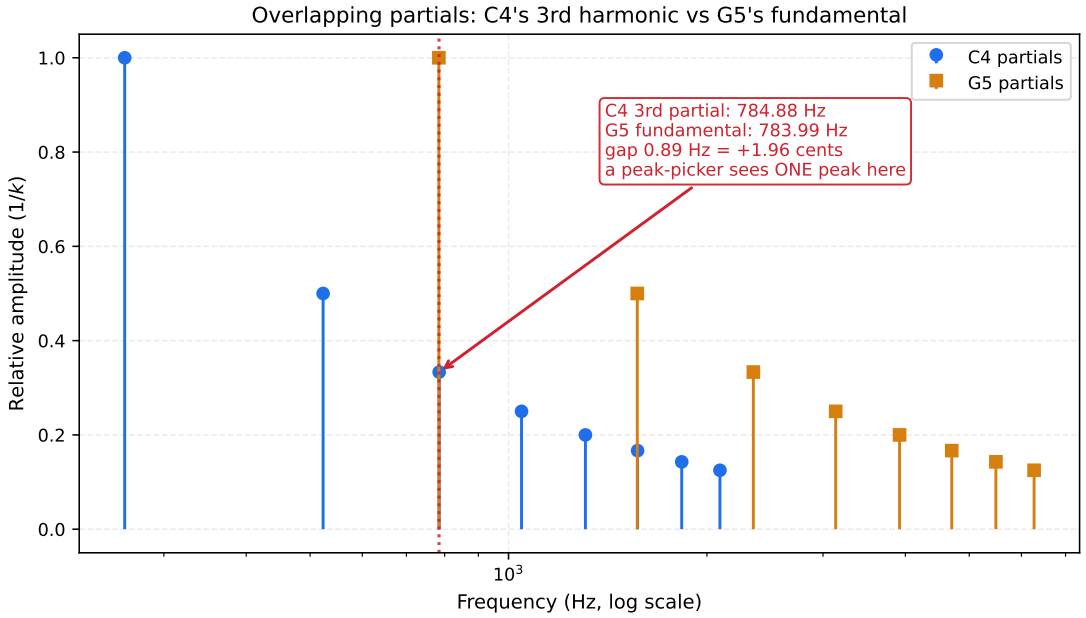


Figure 2.2: Harmonic combs of C4 (261.63 Hz) and G5 (783.99 Hz), partials 1–8 with $1/k$ amplitudes. C4's 3rd partial (784.88 Hz) lands +1.96 cents above G5's fundamental: a single merged spectral peak with two valid explanations.

Step 3 — First-principles derivation. How crowded is a passage? Let notes be intervals $[t_{\text{on}}, t_{\text{off}})$ on the time axis.

Definition 2.3 (Polyphonic density metrics)**.** Let $a(t)$ be the number of notes active at time t (instantaneous polyphony) over a passage of duration T, and let H be the number of partials modeled per note. Then

$$\bar{a} = \frac{1}{T}\int_0^T a(t)\,dt \qquad \text{(mean polyphony)}, \tag{2.8}$$

$$a_{\max} = \max_t a(t) \qquad \text{(peak polyphony)}, \tag{2.9}$$

$$\rho_{\text{on}} = \frac{N_{\text{notes}}}{T} \qquad \text{(onset density, notes/s)}, \tag{2.10}$$

$$\bar{s} = \frac{1}{T}\int_0^T a(t)\,H\,dt = \bar{a}\,H \qquad \text{(mean spectral density, partials)}, \tag{2.11}$$

and the *partial collision rate* is the fraction of cross-note partial pairs whose frequencies agree to within 25 cents (a quarter semitone).

Two worked passages (8 partials per note, 100 Hz analysis grid): a C-major triad with octave doubling (5 notes over 1.5 s) gives $\bar{a} = 3.333$, $a_{\max} = 5$, $\rho_{\text{on}} = 3.333$ notes/s, $\bar{s} = 26.7$ partials, collision rate 0.0312; a sparse 8-note melody over 3.95 s gives $\bar{a} = 0.911$, $a_{\max} = 1$, $\rho_{\text{on}} = 2.025$ notes/s, $\bar{s} = 7.3$, collision rate 0.0285. Spectral density, not note count alone, is what the encoder's frequency resolution must survive.

Consonance itself is a collision phenomenon: intervals whose frequency ratios are ratios of small integers share many partials (the 5th, 3/2, shares every even partial of the upper note with every third partial of the lower), which the auditory system reads as fusion rather than roughness.

The critical worked case is C4 against G5. From (2.3),

$$f_{\text{C4}} = 440 \cdot 2^{-9/12} = 261.6256 \text{ Hz}, \qquad f_{\text{G5}} = 440 \cdot 2^{10/12} = 783.9909 \text{ Hz}. \tag{2.12}$$

The 3rd harmonic of C4 is

$$3\, f_{\text{C4}} = 3 \times 261.6256 = 784.8767 \text{ Hz}, \tag{2.13}$$

which exceeds G5's fundamental by only 0.8858 Hz. In closed form,

$$\frac{3\, f_{\text{C4}}}{f_{\text{G5}}} = \frac{3 \cdot 440 \cdot 2^{-3/4}}{440 \cdot 2^{5/6}} = 3 \cdot 2^{-19/12}, \tag{2.14}$$

so the cent deviation is

$$1200 \log_2\!\left(3 \cdot 2^{-19/12}\right) = 1200 \log_2 3 - 1900 \approx +1.955 \text{ cents}. \tag{2.15}$$

(A remark on units: 392 Hz is G4 (391.9954 Hz), not the collision point; the colliding partial sits at ≈ 784.9 Hz, in the G5 register.) The same +1.955 cents recurs for C4's 6th partial against G5's 2nd — it is the just-fifth discrepancy of Table 2.1 reappearing, because the 3rd harmonic *is* a just twelfth above the fundamental. A full scan of partials 1–8 of both notes finds exactly two near-collisions (< 25 cents): $(3, 1)$ and $(6, 2)$, both at +1.955 cents. For the C-major triad C4–E4–G4 with 5 partials each, 15 spectral lines contain exactly one near-collision (G4's 2nd partial against C4's 3rd, +1.96 cents).

What does the encoder face? Formalize the ambiguity as hypothesis testing.

Proposition 2.1 (Pitch ambiguity of a merged peak)**.** *Let a spectral peak be observed at* $f_{\text{obs}} = 784.8767$ *Hz and let the encoder score hypotheses by a Gaussian log-likelihood in the cents domain with* $\sigma = 5$ *cents:* $\log L = -\frac{1}{2}\big(c(f_{\text{obs}}, f_{\text{pred}})/\sigma\big)^2$. *For* H_C*: "the peak is C4's 3rd partial"* ($f_{\text{pred}} = 784.8767$ *Hz) versus* H_G*: "the peak is G5's fundamental"* ($f_{\text{pred}} = 783.9909$ *Hz), the log-likelihood ratio is*

$$\log \frac{L(H_C)}{L(H_G)} = +0.0764 \ll 1. \tag{2.16}$$

Proof. Under H_C the cent deviation is 0, so $\log L(H_C) = -0.0000$. Under H_G the deviation is +1.955 cents, so $\log L(H_G) = -\frac{1}{2}(1.955/5)^2 = -0.0764$. Subtracting gives +0.0764. Since $|\text{LLR}| \ll 1$, the two hypotheses are statistically indistinguishable from this peak alone; the encoder must recruit context — the remaining partials, onset synchrony, voice leading — to decide. □

Step 4 — Code. The collision-analysis listing below implements harmonic-series generation, collision detection, the density metrics of Definition 2.3, and the ambiguity likelihood-ratio test of Proposition 2.1.

```python
"""Module 2 -- polyphony mechanics: polyphonic density metrics, the
overlapping-partials problem, and a quantitative model of pitch ambiguity
for a neural encoder.

Run:  python3 m2_harmonic_collision.py
"""
from __future__ import annotations

import math
from dataclasses import dataclass, field

import numpy as np


def cents_between(f_high: float, f_low: float) -> float:
    return 1200.0 * math.log2(f_high / f_low)


def midi_to_freq(m: float) -> float:
    return 440.0 * 2.0 ** ((m - 69) / 12.0)


# ---------------------------------------------------------------------------
# Harmonic series and collision detection
# ---------------------------------------------------------------------------
@dataclass(frozen=True)
class Partial:
    note_name: str
    midi: int
    harmonic: int           # 1 = fundamental
    freq: float
    amplitude: float        # relative amplitude 0..1 (1/n rolloff default)


def harmonic_series(note_name: str, midi: int, n_harmonics: int,
                    rolloff: float = 1.0) -> list[Partial]:
    """First ``n_harmonics`` partials of a note with 1/n^rolloff amplitudes."""
    f0 = midi_to_freq(midi)
    return [
        Partial(note_name, midi, k, k * f0, 1.0 / (k ** rolloff))
        for k in range(1, n_harmonics + 1)
    ]


@dataclass(frozen=True)
class Collision:
    partial_a: Partial
    partial_b: Partial
    cents_deviation: float

    @property
    def is_near(self) -> bool:
```

```
        return abs(self.cents_deviation) < 25.0  # within a quarter semitone


def find_collisions(series_a: list[Partial], series_b: list[Partial],
                    tol_cents: float = 25.0) -> list[Collision]:
    """All pairs of partials (a from series_a, b from series_b) within tolerance."""
    out: list[Collision] = []
    for pa in series_a:
        for pb in series_b:
            dev = cents_between(max(pa.freq, pb.freq), min(pa.freq, pb.freq))
            if dev <= tol_cents:
                out.append(Collision(pa, pb, dev))
    return out


# ---------------------------------------------------------------------------
# Polyphonic density metrics
# ---------------------------------------------------------------------------
@dataclass
class NoteEvent:
    pitch: int
    onset: float   # seconds
    offset: float  # seconds

    @property
    def duration(self) -> float:
        return self.offset - self.onset


@dataclass
class DensityReport:
    n_notes: int
    total_time: float
    mean_polyphony: float          # time-averaged simultaneous note count
    peak_polyphony: int            # max simultaneous note count
    mean_onset_density: float      # notes per second
    spectral_density_mean: float   # time-averaged partial count (n_harm per note)
    collision_rate: float          # fraction of partial pairs colliding < 25c

    def pretty(self) -> str:
        return (
            f"notes={self.n_notes}  span={self.total_time:.3f}s\n"
            f"  mean polyphony        = {self.mean_polyphony:.3f} notes\n"
            f"  peak polyphony        = {self.peak_polyphony}\n"
            f"  onset density         = {self.mean_onset_density:.3f} notes/s\n"
            f"  mean spectral density = {self.spectral_density_mean:.1f} partials\n"
            f"  partial collision rate(<25c) =
```

```
                {self.collision_rate:.4f}"
        )


def polyphonic_density(notes: list[NoteEvent], n_harmonics: int = 8,
                       frame_hz: float = 100.0) -> DensityReport:
    """Time-averaged polyphony + spectral-density + partial-collision
       statistics.

    Formal model: let a(t) be the number of active notes at time t and
    s(t) = a(t) * n_harmonics the number of partials present. The
       report gives
    mean/peak of a(t), onset rate, mean of s(t), and the fraction of
    cross-note partial pairs within 25 cents.
    """
    if not notes:
        return DensityReport(0, 0.0, 0.0, 0, 0.0, 0.0, 0.0)
    t0 = min(n.onset for n in notes)
    t1 = max(n.offset for n in notes)
    grid = np.arange(t0, t1, 1.0 / frame_hz)
    active = np.zeros(len(grid), dtype=int)
    for n in notes:
        active[(grid >= n.onset) & (grid < n.offset)] += 1
    span = t1 - t0

    # collision rate over all cross-note partial pairs
    series = {id(n): harmonic_series(f"n{n.pitch}", n.pitch,
       n_harmonics)
              for n in notes}
    keys = list(series.keys())
    n_pairs = 0
    n_coll = 0
    for i in range(len(keys)):
        for j in range(i + 1, len(keys)):
            for pa in series[keys[i]]:
                for pb in series[keys[j]]:
                    dev = cents_between(max(pa.freq, pb.freq),
                       min(pa.freq, pb.freq))
                    n_pairs += 1
                    if dev < 25.0:
                        n_coll += 1
    return DensityReport(
        n_notes=len(notes),
        total_time=span,
        mean_polyphony=float(active.mean()),
        peak_polyphony=int(active.max()),
        mean_onset_density=len(notes) / span if span > 0 else 0.0,
        spectral_density_mean=float(active.mean() * n_harmonics),
        collision_rate=(n_coll / n_pairs) if n_pairs else 0.0,
    )


#
   ----------------------------------------------------------------------
# Pitch-ambiguity model for a neural encoder
#
   ----------------------------------------------------------------------
def ambiguity_likelihood_ratio(
```

```
    observed_freq: float,
    candidate_a: tuple[str, int, int],   # (name, midi, harmonic)
    candidate_b: tuple[str, int, int],
    sigma_cents: float = 5.0,
) -> tuple[float, float, float]:
    """Bayes-factor style ambiguity between two partial explanations.

    Each candidate predicts the observed spectral peak at
        f_pred = harmonic * midi_to_freq(midi)
    with Gaussian log-likelihood in the cents domain:
        log L = -0.5 * (dev_cents / sigma_cents)^2.
    Returns (logL_a, logL_b, log-likelihood-ratio logL_a - logL_b).
    A ratio near 0 means the encoder cannot tell the hypotheses apart.
    """
    def loglike(name: str, midi: int, harmonic: int) -> float:
        f_pred = harmonic * midi_to_freq(midi)
        dev = cents_between(observed_freq, f_pred)
        return -0.5 * (dev / sigma_cents) ** 2

    la = loglike(*candidate_a)
    lb = loglike(*candidate_b)
    return la, lb, la - lb


#
    ------------------------------------------------------------------------
# Driver
#
    ------------------------------------------------------------------------
def main() -> None:
    print("== the C4/G5 collision, computed ==")
    c4 = harmonic_series("C4", 60, 8)
    g5 = harmonic_series("G5", 79, 8)
    f_c4, f_g5 = midi_to_freq(60), midi_to_freq(79)
    h3 = 3.0 * f_c4
    print(f"  C4 fundamental        = {f_c4:.4f} Hz")
    print(f"  C4 3rd harmonic       = {h3:.4f} Hz")
    print(f"  G5 fundamental        = {f_g5:.4f} Hz")
    print(f"  absolute gap          = {h3 - f_g5:.4f} Hz")
    print(f"  cents deviation       = {cents_between(h3, f_g5):+.4f}
        cents")
    print("  (392 Hz is G4 = 391.9954 Hz -- the colliding partial sits
        at "
          "~784.9 Hz, i.e. the G5 region, not 392 Hz.)")

    print("\n== all near-collisions (<25 cents) between C4 and G5
        partials 1..8 ==")
    for col in find_collisions(c4, g5):
        a, b = col.partial_a, col.partial_b
        print(f"  {a.note_name} partial {a.harmonic} ({a.freq:9.3f}
            Hz)  <->  "
              f"{b.note_name} partial {b.harmonic} ({b.freq:9.3f} Hz):
                  "
              f"{col.cents_deviation:+7.3f} cents")

    print("\n== ambiguity model: peak observed at 784.9 Hz ==")
    f_obs = 784.8767  # measured spectral peak (the C4 3rd partial)
    la, lb, llr = ambiguity_likelihood_ratio(
```

```
        f_obs, ("C4", 60, 3), ("G5", 79, 1), sigma_cents=5.0)
    print(f"  H_C: peak is C4's 3rd partial -> logL = {la:.4f}")
    print(f"  H_G: peak is G5's fundamental -> logL = {lb:.4f}")
    print(f"  log-likelihood ratio (H_C - H_G) = {llr:+.4f}")
    print("  |LLR| << 1: a peak-picking encoder cannot resolve the two "
          "hypotheses from this peak alone.")

    print("\n== polyphonic density: C-major triad + octave doubling ==")
    notes = [
        NoteEvent(48, 0.00, 1.00),  # C3
        NoteEvent(60, 0.00, 1.00),  # C4
        NoteEvent(64, 0.00, 1.00),  # E4
        NoteEvent(67, 0.00, 1.00),  # G4
        NoteEvent(72, 0.50, 1.50),  # C5 enters halfway
    ]
    print(polyphonic_density(notes, n_harmonics=8).pretty())

    print("\n== polyphonic density: sparse melody ==")
    melody = [NoteEvent(60 + s, i * 0.5, i * 0.5 + 0.45)
              for i, s in enumerate([0, 2, 4, 7, 9, 7, 4, 2])]
    print(polyphonic_density(melody, n_harmonics=8).pretty())


if __name__ == "__main__":
    main()
```

2.3 Symbolic Formats: Piano Rolls versus Event Streams

Step 1 — Physical analogy. A player-piano roll is a paper strip with holes punched in a grid: rows are pitches, columns are time slices, a hole means "sounding". That is literally a binary Frame×Pitch matrix, invented decades before the term "piano roll" entered MIR. The alternative is the lead sheet: a sparse list of instructions — "C4 starts now, stops in half a second" — which is the event-stream view. AMT models must commit to one of these two output geometries, and the choice reshapes memory, loss design, and decoding.

Step 2 — Geometric mental model. Figure 2.3 shows the same four-note excerpt both ways: left, the dense binary grid (112×128 at 100 fps); right, four (pitch, onset, offset) tuples on a timeline. The grid is mostly zeros; the event list stores only what sounds.

Step 3 — First-principles derivation.

Definition 2.4 (Piano roll and event stream)**.** Fix a frame rate F (frames/s) and pitch range $P = 128$. A *piano roll* is a binary matrix $R \in \{0,1\}^{T \times P}$ with $R[t,p] = 1$ iff pitch p sounds during frame t. An *event stream* is a list $E = \{(p_i, t_i^{\text{on}}, t_i^{\text{off}}, v_i)\}_{i=1}^{N}$ of N note tuples (pitch, onset, offset, velocity).

Memory follows directly. For a 10 s excerpt at $F = 100$ fps ($T = 1000$, $P = 128$):

$$M_{\text{roll}}^{\text{bool}} = \frac{T \cdot P}{8} = \frac{128000}{8} = 15.62 \text{ KiB}, \qquad M_{\text{roll}}^{\text{f32}} = T \cdot P \cdot 4 \text{ B} = 500.00 \text{ KiB}. \tag{2.17}$$

A sparse COO encoding stores only active cells (about 1785 here, scaled from the excerpt): $1785 \times 8\,\text{B} \approx 13.95\,\text{KiB}$ — barely better than dense bool, because piano music is not *that* sparse

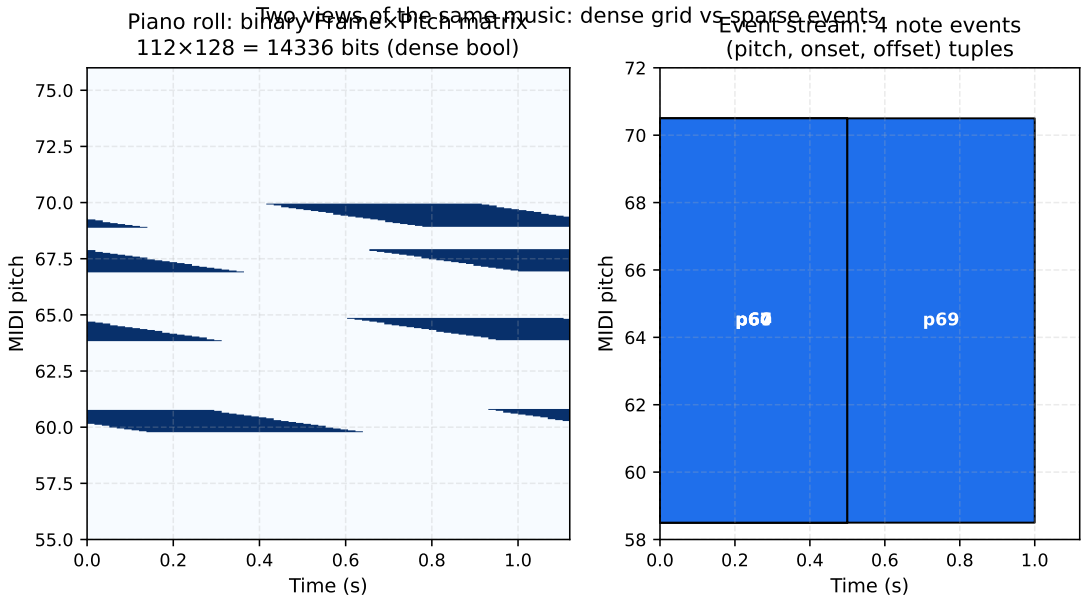


Figure 2.3: Left: binary Frame×Pitch matrix of the worked excerpt ($112 \times 128 = 14336$ bits dense). Right: the identical music as four symbolic events. Same information, radically different geometry.

at 100 fps. The event stream for $N = 40$ notes at 4 tokens/note and 2 bytes/token needs

$$M_{\text{events}} = 40 \times 4 \times 2 \text{ B} = 320 \text{ bytes}, \tag{2.18}$$

roughly fifty times smaller than the dense bool grid. The gap widens with duration: grids scale with T, event streams with N.

The conversion is exact on the grid. Events→roll: frame indices $t_0 = \lfloor t_{\text{on}} F \rfloor$, $t_1 = \lceil t_{\text{off}} F \rceil$, set $R[t_0{:}t_1, p] = 1$. Roll→events: for each pitch column, contiguous runs of 1s are found from the first difference $\Delta = \text{diff}([0, \text{col}, 0])$ (starts where $\Delta = 1$, ends where $\Delta = -1$); each run $[s, e)$ becomes a note $(p, s/F, e/F)$. The worked excerpt round-trips exactly on the 10 ms grid. Complexity matters at decoding time: a frame classifier emits $T \times P = 128000$ logits per 10 s, while an event decoder emits $L \approx 160$ tokens with $O(L^2) = 25600$ self-attention cost against the frame model's $O(T^2) = 10^6$ on the time axis — the event view trades a harder sequential prediction problem for a far cheaper one.

Step 4 — Code. The converter listing below implements the bidirectional piano-roll converter, the memory/complexity analysis, the micro-timing quantization study, CC64 sustain-pedal dynamics, and Hungarian voice assignment used in Section 2.5.

```
"""Module 2 -- symbolic formats: binary Frame x Pitch piano-roll matrices vs
symbolic event streams.

Covers:
  * NoteEvent <-> piano-roll conversion (exact, invertible on the grid)
  * memory / complexity comparison (dense vs sparse vs event stream)
  * micro-timing quantization error analysis (rubato vs grid)
  * sustain-pedal (CC64) dynamics: pedal-extended note offsets
  * polyphonic voice assignment as a linear assignment problem (Hungarian)

```

```
Run:  python3 m2_pianoroll_converter.py
"""
from __future__ import annotations

from dataclasses import dataclass

import numpy as np
from scipy.optimize import linear_sum_assignment


@dataclass
class NoteEvent:
    pitch: int
    onset: float
    offset: float
    velocity: int = 80

    @property
    def duration(self) -> float:
        return self.offset - self.onset


#
    ----------------------------------------------------------------------
# Piano-roll <-> event conversion
#
    ----------------------------------------------------------------------
class PianoRoll:
    """Binary Frame x Pitch matrix with an explicit time grid."""

    def __init__(self, frames: int, n_pitches: int = 128, frame_hz:
       float = 100.0,
                 t0: float = 0.0) -> None:
        self.matrix = np.zeros((frames, n_pitches), dtype=np.uint8)
        self.frame_hz = frame_hz
        self.t0 = t0

    @property
    def n_frames(self) -> int:
        return self.matrix.shape[0]

    @property
    def duration_s(self) -> float:
        return self.n_frames / self.frame_hz

    @staticmethod
    def from_events(notes: list[NoteEvent], frame_hz: float = 100.0,
                    pad_s: float = 0.1) -> "PianoRoll":
        t1 = max(n.offset for n in notes) + pad_s
        n_frames = int(np.ceil(t1 * frame_hz)) + 1
        pr = PianoRoll(n_frames, 128, frame_hz)
        for n in notes:
            f0 = int(np.floor(n.onset * frame_hz))
            f1 = int(np.ceil(n.offset * frame_hz))
            pr.matrix[f0:f1, n.pitch] = 1
        return pr

    def to_events(self, min_frames: int = 1) -> list[NoteEvent]:
```

```
66          """Invert the grid exactly: contiguous active runs -> note
                events."""
67          out: list[NoteEvent] = []
68          for p in range(self.matrix.shape[1]):
69              col = self.matrix[:, p]
70              # run boundaries via first differences
71              d = np.diff(np.concatenate(([0], col, [0])))
72              starts = np.nonzero(d == 1)[0]
73              ends = np.nonzero(d == -1)[0]
74              for s, e in zip(starts, ends):
75                  if e - s >= min_frames:
76                      out.append(NoteEvent(p, s / self.frame_hz, e /
                            self.frame_hz))
77          return sorted(out, key=lambda n: (n.onset, n.pitch))
78
79      def memory_bytes_dense(self, dtype_bytes: int = 1) -> int:
80          return self.matrix.size * dtype_bytes
81
82
83  def memory_report(n_frames: int, n_pitches: int, n_notes: int,
84                    tokens_per_note: int = 4, bytes_per_token: int = 2)
                         -> dict:
85      dense_bool = n_frames * n_pitches  # bits
86      return {
87          "dense_bool_bits": dense_bool,
88          "dense_bool_KiB": dense_bool / 8 / 1024,
89          "dense_f32_KiB": n_frames * n_pitches * 4 / 1024,
90          # sparse COO: (frame, pitch) int32 pairs for active cells
91          "sparse_active_cells": None,  # filled by caller
92          "event_stream_bytes": n_notes * tokens_per_note *
                bytes_per_token,
93      }
94
95
96  #
        ----------------------------------------------------------------------
97  # Micro-timing: rubato vs quantization grid
98  #
        ----------------------------------------------------------------------
99  def quantization_error(onsets: np.ndarray, grid_s: float) ->
        np.ndarray:
100     """Signed error of snapping each onset to the nearest grid
            multiple."""
101     return onsets - np.round(onsets / grid_s) * grid_s
102
103
104 def micro_timing_report(onsets: np.ndarray, grid_s: float,
105                         label: str) -> str:
106     err = quantization_error(onsets, grid_s)
107     return (f"{label}: grid={grid_s * 1000:.1f} ms, n={len(onsets)}, "
108             f"max|err|={np.abs(err).max() * 1000:.1f} ms, "
109             f"rms={np.sqrt((err ** 2).mean()) * 1000:.1f} ms, "
110             f"theoretical bound={grid_s * 1000 / 2:.1f} ms")
111
112
113 #
        ----------------------------------------------------------------------
114 # Sustain pedal (CC64) dynamics
```

```python
#
    ----------------------------------------------------------------------------
@dataclass
class PedalEvent:
    time: float
    down: bool  # True = pedal pressed (CC64 >= 64), False = released


def apply_sustain_pedal(notes: list[NoteEvent],
                        pedal: list[PedalEvent]) -> list[NoteEvent]:
    """Extend note offsets through a held sustain pedal.

    Rule (piano-like): a note whose offset falls inside a down-stroke
        interval
    [t_down, t_up) keeps ringing until t_up (pedal release). Offsets
        outside
    any down interval are unchanged. This is exactly how a piano
        damper works:
    the damper only returns when the pedal is released.
    """
    downs = sorted(p.time for p in pedal if p.down)
    ups = sorted(p.time for p in pedal if not p.down)
    intervals: list[tuple[float, float]] = []
    ui = 0
    for d in downs:
        while ui < len(ups) and ups[ui] <= d:
            ui += 1
        end = ups[ui] if ui < len(ups) else float("inf")
        intervals.append((d, end))
    out: list[NoteEvent] = []
    for n in notes:
        new_off = n.offset
        for d, u in intervals:
            if d <= n.offset < u:
                new_off = u
                break
        out.append(NoteEvent(n.pitch, n.onset, new_off, n.velocity))
    return out


#
    ----------------------------------------------------------------------------
# Polyphonic voice assignment (Hungarian / linear assignment)
#
    ----------------------------------------------------------------------------
def assign_voices(prev_pitches: list[int], curr_pitches: list[int]) ->
    dict[int, int]:
    """Assign current notes to previous voices minimizing total pitch
        movement.

    Formal problem: given voices holding prev_pitches[i] and new onsets
    curr_pitches[j], choose a bijection minimizing
        sum_ij C[i, j] * X[i, j],  C[i, j] = |prev_pitches[i] -
            curr_pitches[j]|.
    This is the linear assignment problem; scipy's Hungarian
        implementation
    returns the exact optimum. Returns {voice_index: current_pitch}.
    """
```

```
    if len(prev_pitches) != len(curr_pitches):
        raise ValueError("voice counts must match for a bijective assignment")
    n = len(prev_pitches)
    cost = np.abs(np.subtract.outer(prev_pitches, curr_pitches)).astype(float)
    rows, cols = linear_sum_assignment(cost)
    assignment = {int(r): curr_pitches[int(c)] for r, c in zip(rows, cols)}
    total = float(cost[rows, cols].sum())
    return assignment, total


#
    ----------------------------------------------------------------------------
# Driver
#
    ----------------------------------------------------------------------------
def main() -> None:
    notes = [
        NoteEvent(60, 0.00, 0.50, 90),  # C4
        NoteEvent(64, 0.00, 0.50, 80),  # E4
        NoteEvent(67, 0.00, 0.50, 85),  # G4
        NoteEvent(69, 0.50, 1.00, 95),  # A4
    ]
    print("== piano-roll <-> event round trip ==")
    pr = PianoRoll.from_events(notes, frame_hz=100.0)
    print(f"  matrix shape: {pr.matrix.shape} (frames x pitches), "
          f"{pr.duration_s:.2f} s at {pr.frame_hz:.0f} fps")
    back = pr.to_events()
    exact = [(n.pitch, round(n.onset, 2), round(n.offset, 2)) for n in back]
    print(f"  decoded: {exact}")
    ref = [(n.pitch, round(n.onset, 2), round(n.offset, 2)) for n in notes]
    print(f"  exact on 10 ms grid: {sorted(exact) == sorted(ref)}")

    print("\n== memory: 10 s excerpt, 100 fps x 128 pitches ==")
    rep = memory_report(1000, 128, n_notes=40)
    active = int(pr.matrix.sum() * (1000 / pr.n_frames))
    rep["sparse_active_cells"] = active
    print(f"  dense bool : {rep['dense_bool_KiB']:.2f} KiB")
    print(f"  dense f32  : {rep['dense_f32_KiB']:.2f} KiB")
    print(f"  sparse COO (int32 pairs, ~{active} active cells): "
          f"{active * 8 / 1024:.2f} KiB")
    print(f"  event stream (40 notes x 4 tok x 2 B): {rep['event_stream_bytes']} B")

    print("\n== complexity: decoder output length ==")
    T, P, L = 1000, 128, 40 * 4
    print(f"  frame classifier emits {T}x{P} = {T * P} logits "
          f"(O(T*P) memory per step batch)")
    print(f"  event decoder emits ~{L} tokens; self-attention O(L^2) = {L ** 2} "
          f"vs frame model O(T^2) = {T ** 2} on the time axis")

    print("\n== micro-timing: rubato vs 16th-note grid at 120 BPM ==")
    rng = np.random.default_rng(7)
```

```
    grid = 60.0 / 120.0 / 4.0  # 0.125 s
    # a humanized performance: onsets deviate with sigma = 18 ms (rubato/swing)
    nominal = np.arange(0, 2.0, grid)
    human = nominal + rng.normal(0, 0.018, size=nominal.shape)
    print("  " + micro_timing_report(human, grid, "humanized"))
    print("  " + micro_timing_report(nominal, grid, "quantized"))
    print(f"  a 35 ms expressive push = {35 / (grid * 1000):.2f} sixteenths: "
          f"audible as laid-back timing, invisible after grid quantization")

    print("\n== sustain pedal CC64 ==")
    pedal = [PedalEvent(0.0, True), PedalEvent(0.9, False)]
    sus = apply_sustain_pedal(notes, pedal)
    for a, b in zip(notes, sus):
        print(f"  p{a.pitch}: [{a.onset:.2f}, {a.offset:.2f}] -> "
              f"[{b.onset:.2f}, {b.offset:.2f}]")

    print("\n== voice assignment: crossing voices ==")
    # soprano/alto cross: prev (C5, E5) -> curr (E5, C5) would cross;
    # optimum keeps voices uncrossed: C5->C5? here demonstrate minimal motion
    prev = [72, 76]   # C5, E5 held
    curr = [76, 79]   # E5, G5 arrive
    assignment, total = assign_voices(prev, curr)
    print(f"  prev voices {prev} -> new pitches {curr}")
    print(f"  optimal assignment {assignment}, total motion {total:.0f} semitones")
    # the classic crossing case
    prev2 = [72, 76]
    curr2 = [74, 71]  # D5 arrives high, B4 arrives low
    assignment2, total2 = assign_voices(prev2, curr2)
    print(f"  crossing case: prev {prev2} -> curr {curr2}")
    print(f"  optimal assignment {assignment2}, total motion {total2:.0f} semitones "
          f"(uncrossed: 72->71, 76->74)")


if __name__ == "__main__":
    main()
```

2.4 Event Tokenization: From MIDI Messages to Model Vocabularies

Step 1 — Physical analogy. MIDI is the performer's instruction language: "note 60 on, velocity 90", "advance 500 ms", "note 60 off". A tokenizer is a telegraph operator translating those instructions into a compact code the Transformer reads. Different operators made different engineering bets — relative versus absolute time, note-offs versus durations, flat streams versus metrical grids — and those bets are visible in every token id the decoder emits.

Step 2 — Geometric mental model. Figure 2.4 lays the same four-note excerpt (C4–E4–G4 triad at $t = 0$, A4 at $t = 0.5$, all at 120 BPM) under four tokenizations. MIDI-like walks time forward with TIME_SHIFTs; REMI pins every note to a BAR/POSITION metrical

grid and replaces note-offs with durations; Structured repeats a rigid TIME_SHIFT–PITCH–VELOCITY–DURATION pattern; MT3 stamps absolute 10 ms TIMEs and toggles an ON/OFF mode.

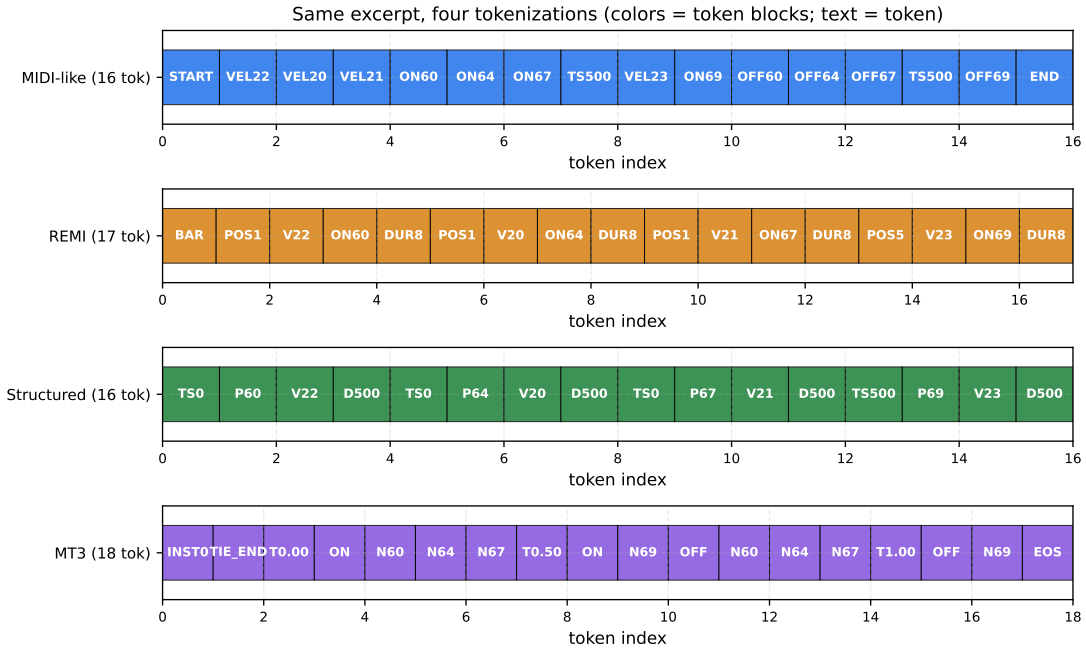


Figure 2.4: The worked excerpt under four tokenizations. Same music, different token counts (16/17/16/18) and different treatments of time: relative shifts (MIDI-like, Structured), metrical grid (REMI), absolute segment times (MT3).

Step 3 — First-principles derivation. *MIDI-like* [14] (Oore et al., "This Time with Feeling") serializes raw MIDI messages: NOTE_ON(p) and NOTE_OFF(p) for $p \in \{0, \ldots, 127\}$, TIME_SHIFT in 10 ms steps from 10 to 1000 ms, and VELOCITY quantized to 32 bins. Time is purely relative: the decoder reconstructs absolute time by accumulating shifts. Our worked excerpt encodes to 16 tokens over a 390-token vocabulary (including PIECE_START/PIECE_END):

$$\begin{aligned}&\textsc{Start}, \textsc{Vel22}, \textsc{Vel20}, \textsc{Vel21}, \textsc{On60}, \textsc{On64}, \textsc{On67}, \textsc{TS500},\\ &\textsc{Vel23}, \textsc{On69}, \textsc{Off60}, \textsc{Off64}, \textsc{Off67}, \textsc{TS500}, \textsc{Off69}, \textsc{End}.\end{aligned} \tag{2.19}$$

Note the failure mode this exposes: a single dropped TIME_SHIFT corrupts *every* subsequent timestamp, because time is a running sum.

REMI [13] (Huang & Yang, "Pop Music Transformer", ACM Multimedia 2020) replaces the fragile relative clock with a metrical grid. Its vocabulary (Table 1 of the paper) is: NOTE_ON (0–127), NOTE DURATION in 32nd-note multiples (replacing NOTE_OFF), NOTE VELOCITY (32 bins), POSITION (16 bins per bar) plus BAR, TEMPO (30–209 BPM), and CHORD (60 types: 12 roots × 5 qualities). Each note becomes a fixed 4-token group — POSITION, VELOCITY, NOTE_ON, NOTE_DURATION — so rhythm is explicit and local. The excerpt encodes to 17 tokens:

$$\begin{aligned}&\textsc{Bar}, \textsc{Pos1}, \textsc{V22}, \textsc{On60}, \textsc{Dur8}, \textsc{Pos1}, \textsc{V20}, \textsc{On64}, \textsc{Dur8},\\ &\textsc{Pos1}, \textsc{V21}, \textsc{On67}, \textsc{Dur8}, \textsc{Pos5}, \textsc{V23}, \textsc{On69}, \textsc{Dur8}.\end{aligned} \tag{2.20}$$

The grid has a price: decoding quantizes onsets to position centers, so our excerpt's onsets shift by +62.5 ms (half a position bin at 120 BPM) — expressive timing finer than the grid is destroyed by construction.

Structured MIDI [15] (Hadjeres & Crestel, "The Piano Inpainting Application", 2021) keeps MIDI-like's message types but replaces NOTE_OFF with DURATION tokens and enforces a *recurring pattern of token-type successions*: every note is TIME_SHIFT–PITCH–VELOCITY–DURATION in that fixed order. The excerpt encodes to 16 tokens:

$$\begin{aligned}&\text{TS0, P60, V22, D500, TS0, P64, V20, D500,}\\&\text{TS0, P67, V21, D500, TS500, P69, V23, D500,}\end{aligned} \tag{2.21}$$

over our implementation's 961-token vocabulary (128 pitches, 32 velocities, 400 duration values, 401 shifts). The rigid pattern regularizes the sequence for inpainting-style models: the decoder always knows which attribute comes next.

MT3 [16] (Gardner et al., ICLR 2022) redesigns the vocabulary for transcription rather than generation. From Section 3.2 of the paper: INSTRUMENT (128 GM programs; subsequent events target it until the next instrument event), NOTE (128 pitches, interpreted as on or off according to mode), ON/OFF (2 values toggling that mode), DRUM (128 GM drum types), TIME (205 values: absolute 10 ms bins inside a 2.048 s segment — $2.048/0.010 = 204.8$, hence 205 bins), END TIE SECTION (1), EOS (1) — $128 + 128 + 2 + 128 + 205 + 1 + 1 = 593$ tokens total. Two deliberate breaks from its predecessor (Hawthorne et al.'s piano tokenizer [17]): velocity was *removed* (most training corpora lack velocity labels), and a *tie section* opens each segment declaring already-active notes, so a note whose note-off falls in a later segment still decodes cleanly. The excerpt encodes to 18 tokens:

$$\begin{aligned}&\text{Inst0, TieEnd, T0.00, On, N60, N64, N67, T0.50, On, N69,}\\&\text{Off, N60, N64, N67, T1.00, Off, N69, Eos.}\end{aligned} \tag{2.22}$$

Absolute segment times make MT3 robust to a dropped token (only one timestamp is affected), at the cost of the 10 ms quantization and the segment-boundary tie machinery. The INSTRUMENT token is what lets a *single* decoder transcribe arbitrary instrument mixtures — the core multi-task claim of the paper. The MidiTok library [18] implements all of these schemes (plus CPWord, Octuple, MuMIDI, MMM) under one API, and `pretty_midi` [19] remains the standard tool for parsing the underlying MIDI these tokenizers consume.

Token-count summary for the excerpt: MIDI-like 16, REMI 17, Structured 16, MT3 18. The ranking flips with polyphony: REMI's per-note cost is a flat 4 tokens regardless of simultaneity, while MIDI-like and MT3 amortize one time token over all simultaneous notes — dense chords favor absolute-time schemes, sparse textures favor metrical ones.

Step 4 — Code. The tokenizer listing below implements all four tokenizers from scratch — vocabulary construction, encoding, exact decoding — plus the worked comparison driver that produced the token counts and strings quoted above.

```
"""Module 2 -- symbolic event tokenization from scratch.

Implements four event-tokenization paradigms over a shared note excerpt:
  * MIDILikeTokenizer  -- NOTE_ON / NOTE_OFF / TIME_SHIFT / VELOCITY
                          (Oore et al., "This Time with Feeling")
  * REMITokenizer      -- BAR / POSITION / TEMPO / CHORD / NOTE_VELOCITY /
                          NOTE_ON / NOTE_DURATION  (Huang & Yang, 2020)
  * StructuredTokenizer-- structured Pitch/Velocity/Duration/TimeShift with a
                          recurring token-type pattern (Hadjeres & Crestel, 2021)
  * MT3Tokenizer       -- Instrument / Note / OnOff / Time / EndTieSection / EOS
```

```
                            (Gardner et al., 2021; velocity removed
                               there)

Each tokenizer builds an explicit vocabulary, encodes a list of
    NoteEvents to
integer token ids, and decodes back. Run:

    python3 m2_tokenizer.py
"""
from __future__ import annotations

from dataclasses import dataclass, field


#
    ------------------------------------------------------------------------
# Shared data model
#
    ------------------------------------------------------------------------
@dataclass
class NoteEvent:
    pitch: int         # MIDI pitch 0..127
    onset: float       # seconds
    offset: float      # seconds
    velocity: int = 80
    program: int = 0   # GM program; 128 reserved for drums in MT3 style

    @property
    def duration(self) -> float:
        return self.offset - self.onset


def excerpt() -> list[NoteEvent]:
    """Shared worked example: a two-beat C-major fragment at 120 BPM.

    Beat 1 (t=0.0): C4+E4+G4 quarter-note triad. Beat 2 (t=0.5): A4
       quarter.
    Bar length at 120 BPM, 4/4: 2.0 s.
    """
    return [
        NoteEvent(60, 0.00, 0.50, velocity=90),  # C4
        NoteEvent(64, 0.00, 0.50, velocity=80),  # E4
        NoteEvent(67, 0.00, 0.50, velocity=85),  # G4
        NoteEvent(69, 0.50, 1.00, velocity=95),  # A4
    ]


#
    ------------------------------------------------------------------------
# Vocabulary helper
#
    ------------------------------------------------------------------------
class Vocab:
    def __init__(self) -> None:
        self.tok2id: dict[str, int] = {}
        self.id2tok: dict[int, str] = {}

    def add(self, token: str) -> int:
```

```
        if token not in self.tok2id:
            i = len(self.tok2id)
            self.tok2id[token] = i
            self.id2tok[i] = token
        return self.tok2id[token]

    def __len__(self) -> int:
        return len(self.tok2id)

    def ids(self, tokens: list[str]) -> list[int]:
        return [self.add(t) for t in tokens]

    def toks(self, ids: list[int]) -> list[str]:
        return [self.id2tok[i] for i in ids]


#
    ------------------------------------------------------------------------
# 1. MIDI-like (Oore et al.)
#
    ------------------------------------------------------------------------
class MIDILikeTokenizer:
    """NOTE_ON(p), NOTE_OFF(p), TIME_SHIFT(ms), VELOCITY(v).

    Time is relative: TIME_SHIFT(k) advances the running clock by k ms.
    Velocity applies to all subsequent NOTE_ON events until changed.
    """

    NAME = "MIDI-like"
    N_VEL_BINS = 32
    MAX_SHIFT_MS = 1000
    SHIFT_STEP_MS = 10

    def __init__(self) -> None:
        self.vocab = Vocab()
        self.vocab.add("PIECE_START")
        for p in range(128):
            self.vocab.add(f"NOTE_ON({p})")
        for p in range(128):
            self.vocab.add(f"NOTE_OFF({p})")
        for ms in range(self.SHIFT_STEP_MS, self.MAX_SHIFT_MS + 1,
            self.SHIFT_STEP_MS):
            self.vocab.add(f"TIME_SHIFT({ms}ms)")
        for v in range(self.N_VEL_BINS):
            self.vocab.add(f"VELOCITY({v})")
        self.vocab.add("PIECE_END")

    def encode(self, notes: list[NoteEvent]) -> list[int]:
        events: list[tuple[float, int, str]] = []  # (time, order,
            token)
        for n in notes:
            vel_bin = min(self.N_VEL_BINS - 1, n.velocity *
                self.N_VEL_BINS // 128)
            events.append((n.onset, 0, f"VELOCITY({vel_bin})"))
            events.append((n.onset, 1, f"NOTE_ON({n.pitch})"))
            events.append((n.offset, 2, f"NOTE_OFF({n.pitch})"))
        events.sort(key=lambda e: (e[0], e[1]))
        tokens = ["PIECE_START"]
```

```
        clock_ms = 0.0
        for t, _, tok in events:
            target_ms = t * 1000.0
            while target_ms - clock_ms > 1e-9:
                step = min(self.MAX_SHIFT_MS, target_ms - clock_ms)
                # quantize the shift to the 10 ms grid
                step = round(step / self.SHIFT_STEP_MS) *
                    self.SHIFT_STEP_MS
                step = max(self.SHIFT_STEP_MS, step)
                tokens.append(f"TIME_SHIFT({int(step)}ms)")
                clock_ms += step
            tokens.append(tok)
        tokens.append("PIECE_END")
        return self.vocab.ids(tokens)

    def decode(self, ids: list[int]) -> list[NoteEvent]:
        tokens = self.vocab.toks(ids)
        clock = 0.0
        vel = 80
        open_notes: dict[int, tuple[float, int]] = {}
        out: list[NoteEvent] = []
        for tok in tokens:
            if tok.startswith("TIME_SHIFT("):
                clock += int(tok[11:-3]) / 1000.0
            elif tok.startswith("VELOCITY("):
                vel = int(tok[9:-1]) * 128 // self.N_VEL_BINS + 2
            elif tok.startswith("NOTE_ON("):
                p = int(tok[8:-1])
                open_notes[p] = (clock, vel)
            elif tok.startswith("NOTE_OFF("):
                p = int(tok[9:-1])
                if p in open_notes:
                    t_on, v = open_notes.pop(p)
                    out.append(NoteEvent(p, t_on, clock, v))
        return sorted(out, key=lambda n: (n.onset, n.pitch))


#
    ------------------------------------------------------------------------
# 2. REMI (Huang & Yang 2020) -- bar/position grid, durations not
    note-offs
#
    ------------------------------------------------------------------------
class REMITokenizer:
    """BAR / POSITION / NOTE_VELOCITY / NOTE_ON / NOTE_DURATION /
        TEMPO / CHORD.

    Position tokens quantize onset to a 16th-note grid inside each bar;
    NOTE_DURATION replaces NOTE_OFF and is measured in 32nd-note
        multiples.
    """

    NAME = "REMI"
    POSITIONS_PER_BAR = 16
    N_VEL_BINS = 32
    MAX_DUR_32NDS = 64

    def __init__(self, bpm: float = 120.0, beats_per_bar: int = 4) ->
```

```
        None:
        self.bpm = bpm
        self.beats_per_bar = beats_per_bar
        self.bar_len_s = 60.0 / bpm * beats_per_bar
        self.vocab = Vocab()
        self.vocab.add("BAR")
        for i in range(1, self.POSITIONS_PER_BAR + 1):
            self.vocab.add(f"POSITION({i}/16)")
        for v in range(self.N_VEL_BINS):
            self.vocab.add(f"NOTE_VELOCITY({v})")
        for p in range(128):
            self.vocab.add(f"NOTE_ON({p})")
        for d in range(1, self.MAX_DUR_32NDS + 1):
            self.vocab.add(f"NOTE_DURATION({d})")  # in 32nd-note units
        # TEMPO: 30..209 BPM as in the paper (quantized here to
            integer BPM)
        for b in range(30, 210):
            self.vocab.add(f"TEMPO({b})")
        for root in range(12):
            for qual in ("maj", "min", "dim", "aug", "sus"):
                self.vocab.add(f"CHORD({root},{qual})")

    def _pos_index(self, t: float) -> int:
        pos_in_bar = (t % self.bar_len_s) / self.bar_len_s
        return min(self.POSITIONS_PER_BAR - 1,
                   int(pos_in_bar * self.POSITIONS_PER_BAR))

    def _dur_32nds(self, dur_s: float) -> int:
        thirty2nd = 60.0 / self.bpm / 8.0
        return max(1, min(self.MAX_DUR_32NDS, int(round(dur_s /
            thirty2nd))))

    def encode(self, notes: list[NoteEvent]) -> list[int]:
        notes = sorted(notes, key=lambda n: (n.onset, n.pitch))
        tokens: list[str] = []
        cur_bar = -1
        for n in notes:
            bar = int(n.onset // self.bar_len_s)
            if bar != cur_bar:
                tokens.append("BAR")
                cur_bar = bar
            pos = self._pos_index(n.onset)
            tokens.append(f"POSITION({pos + 1}/16)")
            vel_bin = min(self.N_VEL_BINS - 1, n.velocity *
                self.N_VEL_BINS // 128)
            tokens.append(f"NOTE_VELOCITY({vel_bin})")
            tokens.append(f"NOTE_ON({n.pitch})")
            tokens.append(f"NOTE_DURATION({self._dur_32nds(n.duration)})")
        return self.vocab.ids(tokens)

    def decode(self, ids: list[int]) -> list[NoteEvent]:
        tokens = self.vocab.toks(ids)
        out: list[NoteEvent] = []
        bar = -1
        pos = vel = pitch = dur = None
        thirty2nd = 60.0 / self.bpm / 8.0
        for tok in tokens:
            if tok == "BAR":
```

```
                bar += 1
            elif tok.startswith("POSITION("):
                pos = int(tok[9:].split("/")[0]) - 1
            elif tok.startswith("NOTE_VELOCITY("):
                vel = int(tok[14:-1]) * 128 // self.N_VEL_BINS + 2
            elif tok.startswith("NOTE_ON("):
                pitch = int(tok[8:-1])
            elif tok.startswith("NOTE_DURATION("):
                dur = int(tok[14:-1])
                t = (bar * self.bar_len_s
                     + (pos + 0.5) / self.POSITIONS_PER_BAR *
                        self.bar_len_s)
                out.append(NoteEvent(pitch, t, t + dur * thirty2nd,
                    vel))
        return sorted(out, key=lambda n: (n.onset, n.pitch))


#
    ------------------------------------------------------------------------
# 3. Structured MIDI (Hadjeres & Crestel 2021) -- fixed recurring
    pattern
#
    ------------------------------------------------------------------------
class StructuredTokenizer:
    """Recurring 4-token pattern per note: PITCH / VELOCITY / DURATION
        / TIME_SHIFT.

    Unlike MIDI-like, the order of token types is fixed and NOTE_OFF is
    replaced by an explicit DURATION token, which regularizes the
        sequence
    for inpainting-style models.
    """

    NAME = "Structured"
    N_VEL_BINS = 32
    SHIFT_STEP_MS = 10

    def __init__(self) -> None:
        self.vocab = Vocab()
        for p in range(128):
            self.vocab.add(f"PITCH({p})")
        for v in range(self.N_VEL_BINS):
            self.vocab.add(f"VELOCITY({v})")
        for d in range(10, 4001, 10):  # duration in ms, 10 ms grid
            self.vocab.add(f"DURATION({d}ms)")
        for ms in range(0, 4001, 10):
            self.vocab.add(f"TIME_SHIFT({ms}ms)")

    def encode(self, notes: list[NoteEvent]) -> list[int]:
        notes = sorted(notes, key=lambda n: (n.onset, n.pitch))
        tokens: list[str] = []
        clock = 0.0
        for n in notes:
            gap_ms = int(round((n.onset - clock) * 1000 /
                self.SHIFT_STEP_MS)
                           * self.SHIFT_STEP_MS)
            gap_ms = max(0, min(4000, gap_ms))
            tokens.append(f"TIME_SHIFT({gap_ms}ms)")
```

```
            clock = n.onset
            tokens.append(f"PITCH({n.pitch})")
            vel_bin = min(self.N_VEL_BINS - 1, n.velocity *
                self.N_VEL_BINS // 128)
            tokens.append(f"VELOCITY({vel_bin})")
            dur_ms = int(round(n.duration * 1000 / self.SHIFT_STEP_MS)
                        * self.SHIFT_STEP_MS)
            dur_ms = max(10, min(4000, dur_ms))
            tokens.append(f"DURATION({dur_ms}ms)")
        return self.vocab.ids(tokens)

    def decode(self, ids: list[int]) -> list[NoteEvent]:
        tokens = self.vocab.toks(ids)
        out: list[NoteEvent] = []
        clock = 0.0
        i = 0
        while i + 3 < len(tokens):
            shift = int(tokens[i][11:-3])
            pitch = int(tokens[i + 1][6:-1])
            vel = int(tokens[i + 2][9:-1]) * 128 // self.N_VEL_BINS + 2
            dur = int(tokens[i + 3][9:-3])
            clock += shift / 1000.0
            out.append(NoteEvent(pitch, clock, clock + dur / 1000.0,
                vel))
            i += 4
        return out


# ------------------------------------------------------------------------
# 4. MT3 (Gardner et al. 2021) -- Instrument / OnOff / Note / Time, ties
# ------------------------------------------------------------------------
class MT3Tokenizer:
    """MT3 event vocabulary (paper, Section 3.2):

      Instrument (128) : GM program; subsequent events target it
      Note (128)       : pitch of a note-on or note-off (mode set by
          OnOff)
      OnOff (2)        : whether following Note events are on or off
      Drum (128)       : GM drum-type onset
      Time (205)       : absolute 10 ms bins inside a 2.048 s segment
      EndTieSection(1) : closes the tie section of active notes
      EOS (1)

    Total: 128+128+2+128+205+1+1 = 593 tokens. Velocity was
        deliberately
    removed (most training corpora lack velocity labels).
    """

    NAME = "MT3"
    TIME_BINS = 205
    TIME_STEP_S = 0.010
    SEGMENT_S = 2.048

    def __init__(self) -> None:
        self.vocab = Vocab()
```

```
322         for p in range(128):
323             self.vocab.add(f"INSTRUMENT({p})")
324         for p in range(128):
325             self.vocab.add(f"NOTE({p})")
326         self.vocab.add("ON")
327         self.vocab.add("OFF")
328         for d in range(128):
329             self.vocab.add(f"DRUM({d})")
330         for t in range(self.TIME_BINS):
331             self.vocab.add(f"TIME({t * self.TIME_STEP_S:.2f}s)")
332         self.vocab.add("END_TIE_SECTION")
333         self.vocab.add("EOS")
334         assert len(self.vocab) == 593, len(self.vocab)
335 
336     def _time_tok(self, t: float) -> str:
337         b = max(0, min(self.TIME_BINS - 1, int(round(t /
                self.TIME_STEP_S))))
338         return f"TIME({b * self.TIME_STEP_S:.2f}s)"
339 
340     def encode(self, notes: list[NoteEvent],
341                tied: list[NoteEvent] | None = None) -> list[int]:
342         tied = tied or []
343         tokens = ["INSTRUMENT(0)"]
344         if tied:
345             for n in sorted(tied, key=lambda n: n.pitch):
346                 tokens.append(f"NOTE({n.pitch})")
347             tokens.append("END_TIE_SECTION")
348         else:
349             tokens.append("END_TIE_SECTION")
350         # event section: chronological, ON-block then OFF-block per
                time bin
351         events: list[tuple[float, int, str]] = []
352         for n in notes:
353             events.append((n.onset, 0, f"NOTE({n.pitch})_on"))
354             events.append((n.offset, 1, f"NOTE({n.pitch})_off"))
355         events.sort(key=lambda e: (round(e[0] / self.TIME_STEP_S),
                e[1]))
356         last_bin = -1
357         mode = None
358         for t, _, kind in events:
359             b = int(round(t / self.TIME_STEP_S))
360             if b != last_bin:
361                 tokens.append(self._time_tok(t))
362                 last_bin = b
363                 mode = None
364             want = "on" if kind.endswith("_on") else "off"
365             if want != mode:
366                 tokens.append("ON" if want == "on" else "OFF")
367                 mode = want
368             pitch = kind[5:kind.index(")")]
369             tokens.append(f"NOTE({pitch})")
370         tokens.append("EOS")
371         return self.vocab.ids(tokens)
372 
373     def decode(self, ids: list[int]) -> tuple[list[NoteEvent],
            list[int]]:
374         """Returns (notes, tied_pitches). Absolute times are
                segment-relative."""
```

```
        tokens = self.vocab.toks(ids)
        tied: list[int] = []
        out: list[NoteEvent] = []
        in_tie = True
        clock = 0.0
        is_on = True
        open_on: dict[int, float] = {}
        for tok in tokens:
            if tok == "END_TIE_SECTION":
                in_tie = False
            elif tok == "EOS":
                break
            elif in_tie and tok.startswith("NOTE("):
                tied.append(int(tok[5:-1]))
            elif tok.startswith("TIME("):
                clock = float(tok[5:-2])
            elif tok == "ON":
                is_on = True
            elif tok == "OFF":
                is_on = False
            elif tok.startswith("NOTE("):
                p = int(tok[5:-1])
                if is_on:
                    open_on[p] = clock
                elif p in open_on:
                    out.append(NoteEvent(p, open_on.pop(p), clock))
        return sorted(out, key=lambda n: (n.onset, n.pitch)), tied


#
    ------------------------------------------------------------------------
# Worked comparison driver
#
    ------------------------------------------------------------------------
def token_count_report(tokenizer, notes: list[NoteEvent]) -> None:
    ids = tokenizer.encode(notes)
    toks = tokenizer.vocab.toks(ids)
    print(f"-- {tokenizer.NAME}: {len(ids)} tokens, vocab
        {len(tokenizer.vocab)}")
    print("   " + " ".join(toks[:24]) + (" ..." if len(toks) > 24 else
        ""))
    # round-trip check
    dec = tokenizer.decode(ids)
    if isinstance(dec, tuple):
        dec, _ = dec
    pitches = sorted((n.pitch, round(n.onset, 3), round(n.offset, 3))
        for n in dec)
    ref = sorted((n.pitch, round(n.onset, 3), round(n.offset, 3)) for
        n in notes)
    print(f"   round-trip exact: {pitches == ref}")
    if tokenizer.NAME == "REMI":
        # REMI quantizes time to the position grid; show the
            quantization shift.
        print("   decoded onsets/offsets (quantized): "
              + "; ".join(f"p{p} [{a:.4f},{b:.4f}]" for p, a, b in
                  pitches))


```

```
425 def main() -> None:
426     notes = excerpt()
427     print("excerpt: " + "; ".join(
428         f"p{n.pitch} [{n.onset:.2f},{n.offset:.2f}] v{n.velocity}" for
                n in notes))
429     print()
430     for tok in (MIDILikeTokenizer(), REMITokenizer(),
431                 StructuredTokenizer(), MT3Tokenizer()):
432         token_count_report(tok, notes)
433         print()
434
435
436 if __name__ == "__main__":
437     main()
```

2.5 Time, Expression, and Voices

Step 1 — Physical analogy. A pianist rarely plays metronomically: the left hand lays back a few dozen milliseconds behind the beat (*rubato*, "robbed time"), and the right pedal lifts every damper so notes ring past their notated offsets. A transcription target that snaps everything to a grid hears none of this; one that keeps every millisecond drowns in noise. This section quantifies the tradeoff and builds the two mechanisms — pedal modeling and voice assignment — that turn raw note soup into structured parts.

Step 2 — Geometric mental model. Figure 2.5 histograms humanized onset deviations against the 16th-note grid: expressive timing lives *inside* the quantization error, which is why grid tokenizers erase it. Figure 2.6 shows the sustain pedal geometrically: notes whose notated offsets fall inside the shaded down-interval stretch to the release point.

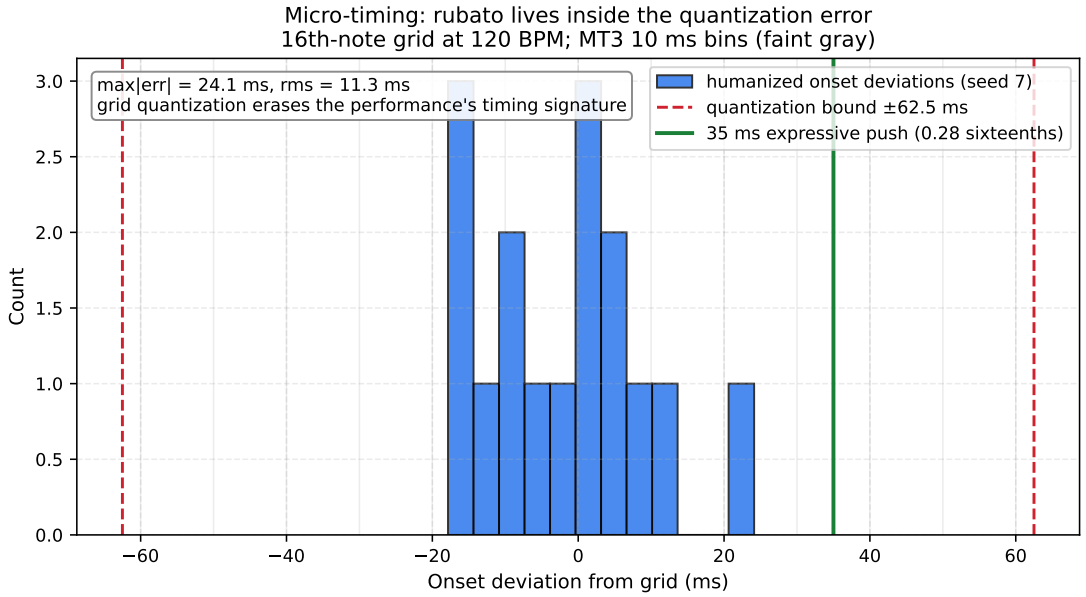


Figure 2.5: Micro-timing vs. quantization. Humanized onsets ($\sigma = 18$ ms, seed 7) deviate up to 24.1 ms rms 11.3 ms — all inside the ±62.5 ms quantization bound of a 16th-note grid at 120 BPM. A 35 ms expressive push (green) is musically salient yet invisible after grid quantization; MT3's 10 ms bins (faint gray) resolve it as 3.5 bins.

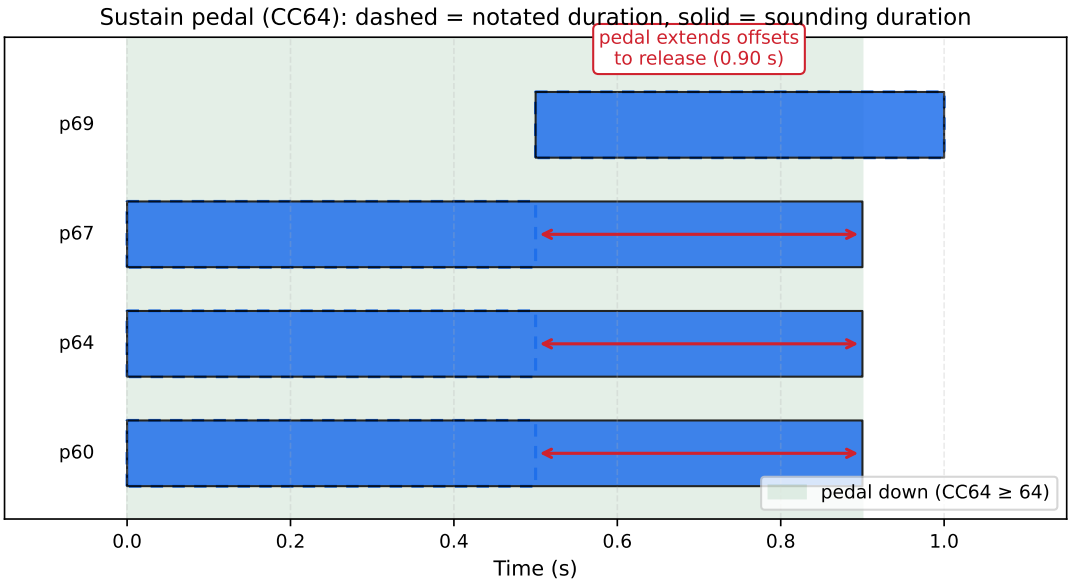


Figure 2.6: Sustain-pedal (CC64) dynamics. Dashed: notated durations; solid: sounding durations. Notes offset inside the pedal-down interval $[0.0, 0.9)$ ring until release at 0.90 s.

Step 3 — First-principles derivation. *Temporal alignment and quantization.* Snapping an onset t to a grid of period Δ incurs error

$$e(t) = t - \Delta \,\mathrm{round}(t/\Delta), \qquad |e(t)| \le \Delta/2. \tag{2.23}$$

At 120 BPM a 16th note spans $\Delta = 60/120/4 = 0.125$ s, so the bound is 62.5 ms — larger than most expressive deviations. A simulated performance with onset jitter $\sigma = 18$ ms yields $\max|e| = 24.1$ ms and rms 11.3 ms: every bit of it is timing the performer intended and the grid destroyed. A deliberate 35 ms laid-back push is $35/125 = 0.28$ sixteenths — clearly audible as groove — yet rounds to zero on the grid. MT3's absolute 10 ms bins resolve the same push as 3.5 bins: coarse, but nonzero. This is the central tension of transcription targets: metrical grids buy learnability (the decoder sees clean positions) at the price of expressivity.

Sustain pedal (CC64). MIDI control change 64 is a damper switch: values ≥ 64 mean down, < 64 mean up. The piano's physics gives the rule: a note whose offset falls inside a down-interval $[t_\downarrow, t_\uparrow)$ keeps its damper raised and rings until $t_\uparrow$. Applying pedal $[0.0, 0.9)$ to the worked excerpt extends the three triad notes from offset 0.50 s to 0.90 s while the A4 (offset 1.00 s, outside the interval) is untouched. A pedal-aware transcription target must therefore predict *effective* offsets, not notated ones — the pedal state is part of the timing ground truth.

Polyphonic voice assignment. Given voices holding pitches $\mathbf{p} = (p_1, \ldots, p_n)$ and newly arriving pitches $\mathbf{q} = (q_1, \ldots, q_n)$, assign each new note to a voice minimizing total pitch motion:

$$\min_X \sum_{i,j} C_{ij} X_{ij}, \qquad C_{ij} = |p_i - q_j|, \tag{2.24}$$

over permutation matrices X. This is the linear assignment problem, solved exactly by the Hungarian algorithm in $O(n^3)$. Worked crossing case: voices hold $(72, 76)$ (C5, E5) and $(74, 71)$ (D5, B4) arrive. The cost matrix is

$$C = \begin{pmatrix} |72-74| & |72-71| \\ |76-74| & |76-71| \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 2 & 5 \end{pmatrix}. \tag{2.25}$$

The uncrossed assignment $72\to71$, $76\to74$ costs $1+2=3$ semitones; the crossed alternative costs $2+5=7$. The Hungarian algorithm returns $\{0{:}71,\ 1{:}74\}$ with total 3 — voices do not

cross, which is both the perceptual default (streaming by minimal motion) and the notational convention. Without this step, a multi-voice transcription is a bag of notes; with it, parts become separable streams the decoder's INSTRUMENT tokens can target.

Step 4 — Code. The converter module listing of Section 2.3 already implements this section's machinery: `quantization_error` / `micro_timing_report` produced Figure 2.5's statistics, `apply_sustain_pedal` produced Figure 2.6, and `assign_voices` solves (2.24) via `scipy.optimize.linear_sum`

2.6 Drills

1. Starting from the two axioms of Definition 2.1, derive the semitone ratio $r = 2^{1/12}$ showing every algebraic step, then compute $f(60)$ from (2.3) to two decimal places. A tone is measured at 442 Hz; express its detuning from A4 in cents using (2.4).

2. Prove that twelve just perfect fifths exceed seven octaves by exactly 531441/524288, and evaluate this ratio in cents to two decimal places. Explain in two sentences why this forces every keyboard tuning to be a compromise.

3. For C4 (261.6256 Hz) and G5 (783.9909 Hz), list every pair of partials (k, l) with $1 \leq k, l \leq 8$ whose frequencies agree to within 25 cents, giving each deviation in cents. Then compute the log-likelihood ratio of Proposition 2.1 for an observed peak at 784.8767 Hz with $\sigma = 5$ cents, and state what the encoder must use instead of this peak to resolve the ambiguity.

4. A 10 s excerpt is represented (a) as a dense boolean piano roll at 100 fps over 128 pitches, (b) as a float32 piano roll, (c) as an event stream of 40 notes at 4 tokens/note and 2 bytes/token. Compute the memory of each in KiB/bytes and the ratio of (a) to (c). Explain why the gap grows with excerpt duration.

5. Tokenize the two-note excerpt "C4 $[0.00, 0.50)$, E4 $[0.25, 0.75)$" by hand under MT3 rules (10 ms bins, ON/OFF mode tokens, END TIE SECTION, EOS; instrument 0, no tied notes) and count the tokens. Tokenize the same excerpt under REMI rules (one BAR, 16 positions/bar at 120 BPM, 4/4) and count. Which is shorter, and why does the ranking reverse for dense chords?

6. At 120 BPM on a 16th-note grid: (a) what is the maximum quantization error in ms? (b) A pianist plays a note 35 ms late for expressive effect; how many MT3 time bins (10 ms) does this span, and is it preserved by REMI grid quantization? (c) Sustain pedal is down on $[0.0, 0.9)$; a note has notated offset 0.50 s. What is its effective (sounding) offset, and what physical mechanism causes the extension?

7. Voices hold pitches $(72, 76)$ and new onsets arrive at $(74, 71)$. Write the 2×2 cost matrix $C_{ij} = |p_i - q_j|$, find the optimal assignment by comparing both permutations, and give the total semitone motion. What musical principle does the optimum embody?

Chapter 3

Self-Supervised Learning for Audio Representations

Automatic music transcription is starved of labels and drowning in audio. A supervised transcriber needs frame-aligned MIDI: for every 10 ms slice of sound, which of the 88 piano keys are sounding, when each note began and ended, and how hard it was struck. Producing that annotation is slow, expensive, and itself error-prone. Meanwhile, unannotated audio — concert recordings, rehearsal tapes, streaming catalogues — exists at a scale thousands of times larger and costs nothing to collect. Self-supervised learning (SSL) is the paradigm that converts the second resource into representations useful for the first: a model learns the structure of sound from raw audio alone, and a small labeled set then suffices to attach transcription heads. This chapter derives the four SSL families that matter for AMT — masked reconstruction, contrastive quantization, clustered masked prediction, and self-distillation — and formalizes how their pretrained encoders transfer to pitch, onset, and offset prediction.

3.1 The Supervised Bottleneck in AMT

Frame-aligned MIDI ground truth is a matrix $Y \in \{0,1\}^{T\times 88}$ (plus onset, offset, and velocity channels) synchronized to an audio waveform $x \in \mathbb{R}^{16000\cdot D}$ at millisecond precision. Every entry of Y is a human judgment about a physical event. This section quantifies why Y is scarce, why it is noisy, and why SSL is the way out.

Step 1 — Physical analogy. Consider transcribing a ten-minute piano piece by ear: listen to a two-second passage, stop the tape, write down every note with its start time to the nearest hundredth of a second, rewind, check the inner voices, repeat. A child learning a language does something entirely different: she listens to thousands of hours of speech with no transcripts at all and still acquires phonetics, vocabulary, and grammar. Supervised AMT is the stopwatch transcriber; self-supervised learning is the immersed child. The thesis of this chapter is that the "stopwatch" labels are the bottleneck, not the model, and that immersion-scale audio can supply what labels cannot.

Step 2 — Geometric mental model. Picture the space of all short audio excerpts. Labeled points (audio with trusted MIDI) are rare pins stuck into an enormous manifold; a supervised model must stretch its decision boundaries across vast unlabeled voids, and its error is governed by how thinly the pins are spread. SSL first learns the geometry of the manifold itself from millions of unlabeled excerpts — which regions correspond to harmonic stacks, which to transients, which to silence — and only then places a few labeled pins to name the regions. Figure 3.1

sketches the consequence: test error falls as $1/\sqrt{n}$ with labeled hours n, so each halving of the error costs four times the annotation, while pretraining shifts the entire curve.

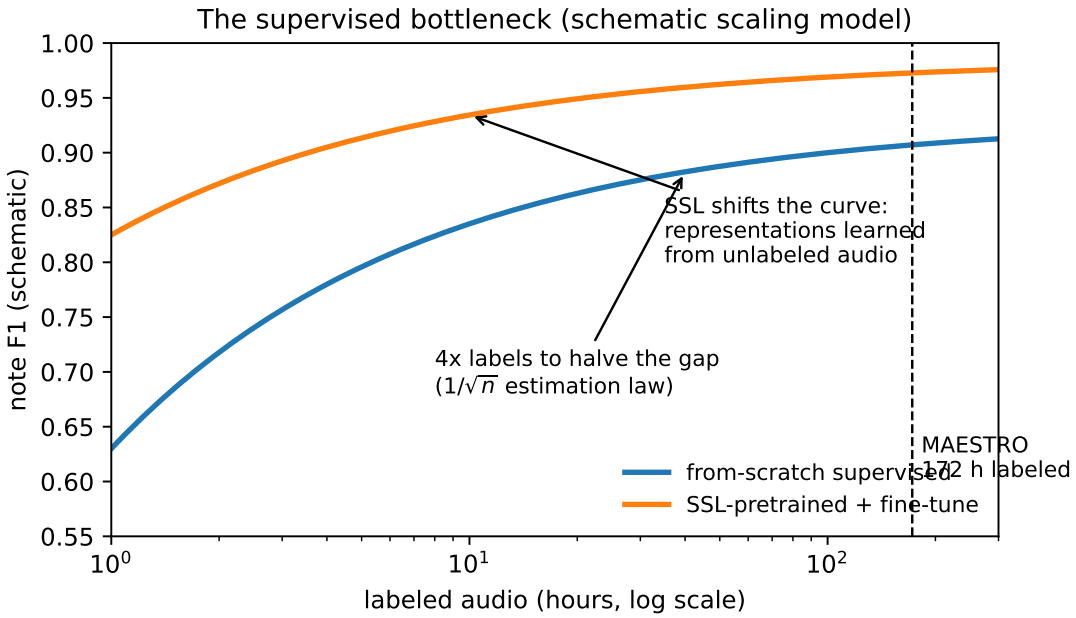


Figure 3.1: Schematic scaling model of the supervised bottleneck. From-scratch training improves only as fast as labeled data accumulates; SSL pretraining on unlabeled audio shifts the curve so that a few labeled hours reach the accuracy that would otherwise need hundreds. The $1/\sqrt{n}$ law is derived in Step 3; the curves are illustrative, not measured.

Step 3 — First-principles derivation. We formalize cost, statistical scaling, and label noise in turn.

Annotation cost model. Let a corpus contain D hours of audio with mean note density δ notes per second, and let a careful annotator place ρ note labels per hour at wage w per hour. The total number of notes is $3600\,D\,\delta$ and the annotation cost is

$$C(D) = \frac{3600\,D\,\delta}{\rho} \cdot w \quad \text{annotator-hours} \times \text{wage}. \tag{3.1}$$

The real-time factor $\mathrm{RTF} = C(D)/(D \cdot w) = 3600\,\delta/\rho$ measures how many times slower annotation runs than real time. For the MAESTRO corpus [26] ($D = 172.3$ h, measured mean density $\delta \approx 10$ notes/s) at a careful pace of $\rho = 600$ notes/hour,

$$C = \frac{3600 \times 172.3 \times 10}{600} = 10{,}338 \text{ annotator-hours}, \qquad \mathrm{RTF} = 60, \tag{3.2}$$

i.e. sixty times slower than real time — over five person-years at 40 h/week for a single corpus, before any verification pass. The numbers in (3.2) were recomputed in `scripts/check_m3_numerics.py`; $\rho = 600$ is an illustrative pace, so the conclusion to keep is the scaling form (3.1), not the exact figure.

Statistical scaling. Let $\hat{R}_n$ be the empirical risk minimizer on n labeled hours and $R^\star$ the Bayes risk. Under standard finite-sample bounds the estimation gap obeys

$$\mathbb{E}\big[\hat{R}_n - R^\star\big] = \Theta(n^{-1/2}), \tag{3.3}$$

so halving the gap demands four times the labeled data — and four times the cost (3.1). The curve in Figure 3.1 is exactly this $n^{-1/2}$ law.

Annotation noise. Human-placed onsets carry timing jitter. Model the placed onset as $t_{\text{label}} = t_{\text{true}} + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$. The mir_eval note-onset tolerance used throughout this book is ±50 ms (Chapter 6 derives the metric), so the fraction of notes whose *label alone* fails the tolerance is $P(|\varepsilon| > 50\,\text{ms}) = 2(1-\Phi(50/\sigma))$. A direct simulation gives: $\sigma = 25$ ms $\Rightarrow 4.55\%$ of onsets fail; $\sigma = 40$ ms $\Rightarrow 21.2\%$ fail. In words, even a perfect model is capped by label noise once jitter reaches a few tens of milliseconds, and quiet inner voices, pedal blur, and overlapping partials (see the overtone-collision analysis of Chapter 2) push real annotation well past the optimistic end of that range. Dataset history confirms the difficulty: MusicNet's audio–score alignment is acknowledged to be imperfect [26], while MAESTRO sidesteps human alignment entirely by capturing MIDI directly from Yamaha Disklavier pianos during live performance, achieving roughly 3 ms alignment [26] — at the price of being piano-only and performance-captured rather than annotated.

The SSL paradigm. Split data into a large unlabeled set $\mathcal{D}_u = \{x_i\}$ and a small labeled set $\mathcal{D}_\ell = \{(x_j, y_j)\}$ with $|\mathcal{D}_u| \gg |\mathcal{D}_\ell|$. Pretraining learns an encoder f_θ by minimizing a label-free objective $\mathcal{L}_{\text{SSL}}(f_\theta; \mathcal{D}_u)$; fine-tuning then minimizes $\mathcal{L}_{\text{sup}}(g_\phi \circ f_\theta; \mathcal{D}_\ell)$ over a head g_ϕ (and optionally θ). The scale contrast is stark: wav2vec 2.0 pretrains on 53,000 hours of unlabeled speech and reaches competitive recognition with as little as ten minutes of labels [23]; HuBERT pretrains on 60,000 hours [24]. The rest of this chapter derives the objectives that make this work.

Step 4 — Complete runnable code. The listing below implements the cost model (3.1), the $n^{-1/2}$ scaling law (3.3), and the jitter simulation, with every number asserted against its closed form. It is saved as `code/m3_annotation_cost.py` and compiles cleanly.

```
"""Module 3, Section 3.1: annotation cost model and annotation-noise
    simulation.

Formalizes the supervised bottleneck: (1) a symbolic cost model
    showing how
annotation cost scales with dataset size and polyphonic density, (2)
    the
1/sqrt(n) estimation law that makes halving the generalization gap
    cost 4x
the labels, and (3) a simulation of timing-jitter annotation noise
    measuring
what fraction of human-placed onsets would fall outside the standard
+/-50 ms note-onset tolerance.
"""
from __future__ import annotations

import math

import numpy as np


def annotation_cost_hours(
    audio_hours: float, notes_per_second: float,
        notes_per_annotator_hour: float
) -> float:
    """Annotator-hours needed to label ``audio_hours`` of audio.

    A piece with mean note density ``notes_per_second`` contains
    audio_hours*3600*notes_per_second notes; an annotator placing
    ``notes_per_annotator_hour`` careful note labels per hour needs the
    returned number of hours. The real-time factor RTF =
```

```
    cost / audio_hours measures how many times slower annotation is
        than
    real time.
    """
    total_notes = audio_hours * 3600.0 * notes_per_second
    return total_notes / notes_per_annotator_hour


def generalization_gap(n_labeled_hours: float, c: float = 1.0) ->
    float:
    """Schematic estimation-law curve: gap(n) = c / sqrt(n).

    Doubling data shrinks the gap by 1/sqrt(2); halving the gap needs
        4x data.
    """
    return c / math.sqrt(n_labeled_hours)


def jitter_outside_tolerance(
    n_onsets: int, jitter_std_ms: float, tol_ms: float = 50.0, seed:
        int = 0
) -> float:
    """Fraction of jittered onsets falling outside +/-tol_ms.

    Models a human annotator whose placed onset times carry Gaussian
        timing
    noise with standard deviation ``jitter_std_ms``. Returns the
        fraction of
    notes whose label error alone would fail the note-onset tolerance
        even if
    the model were perfect.
    """
    rng = np.random.default_rng(seed)
    jitter = rng.normal(0.0, jitter_std_ms, size=n_onsets)
    return float(np.mean(np.abs(jitter) > tol_ms))


if __name__ == "__main__":
    # MAESTRO-scale corpus: 172.3 h at ~10 notes/s (measured mean
        density).
    cost = annotation_cost_hours(172.3, 10.0, 600.0)
    print(f"annotator-hours for 172.3 h @10 notes/s, 600 notes/h:
        {cost:,.0f} h")
    print(f"real-time factor: {cost / 172.3:.1f}x slower than real
        time")
    assert cost > 172.3  # annotation is strictly slower than real time

    # estimation law: halving the gap needs 4x labels
    g1, g4 = generalization_gap(10.0), generalization_gap(40.0)
    print(f"gap(10 h)={g1:.3f}, gap(40 h)={g4:.3f}, ratio={g1/g4:.3f}
        (expect 2.0)")
    assert abs(g1 / g4 - 2.0) < 1e-9

    # annotation noise: even modest jitter eats the onset tolerance
        budget
    for std in (10.0, 25.0, 40.0):
        frac = jitter_outside_tolerance(200_000, std)
        theory = 2 * (1 - 0.5 * (1 + math.erf(50.0 / (std *
```

```
            math.sqrt(2)))))
72          print(f"jitter std {std:>4.0f} ms -> {100*frac:.2f}% outside
                +/-50 ms "
73                f"(theory {100*theory:.2f}%)")
74          assert abs(frac - theory) < 0.01
75      print("OK")
```

3.2 Generative and Reconstruction SSL

The most direct SSL idea is also the oldest: hide part of the input and learn to reconstruct it. The Audio Spectrogram Transformer (AST) [20] showed that a pure attention model over spectrogram patches could match convolutional audio models; masked autoencoders (MAE) [21] showed that reconstructing heavily masked patches is a powerful, scalable pretraining objective; and the Self-Supervised AST (SSAST) [22] combined the two into masked spectrogram patch modeling with joint generative and discriminative losses. This section derives the machinery piece by piece: how a spectrogram becomes patches, what masking policies teach, and why the reconstruction loss must be normalized.

3.2.1 Spectrogram patchification

Step 1 — Physical analogy. A spectrogram is a photograph of sound: time runs horizontally, frequency vertically, brightness is energy. Patchification cuts this photograph into small square tiles, like cutting a chocolate bar, and lays the tiles out in a row. The transformer then reads the tiles as a sequence of "visual words." Everything the model knows about which tile sat above or beside which other tile must be re-injected through positional encodings — the cutting destroys the 2D geometry, and the encoding restores it.

Step 2 — Geometric mental model. Each 16×16 patch is a point in $\mathbb{R}^{256}$; the patch embedding is a learned linear map $\mathbb{R}^{256} \to \mathbb{R}^{768}$ that orients these points so that patches with similar spectro-temporal texture (a harmonic stack, a transient click, silence) land near each other. Figure 3.2 shows a synthetic log-Mel spectrogram and the overlapping patch grid: stride 10 with patch size 16 means neighboring tiles share a 6-pixel border, so no spectral edge falls exactly on a tile boundary.

Step 3 — First-principles derivation. AST converts t seconds of audio into 128-dimensional log Mel filterbank features with a 25 ms Hamming window every 10 ms, producing a $128 \times 100t$ matrix [20]. Splitting into 16×16 patches with overlap 6 (stride 10) in both axes gives, along frequency,

$$G_f = \frac{128 - 16}{10} + 1 = 12 \tag{3.4}$$

patches, and along time

$$G_t = \left\lceil \frac{100t - 16}{10} \right\rceil \tag{3.5}$$

patches, for a total sequence length

$$N = 12 \left\lceil \frac{100t - 16}{10} \right\rceil, \tag{3.6}$$

exactly the formula reported for AST [20]. Worked example for $t = 10$ s: $100t = 1000$ frames, $G_t = \lceil 984/10 \rceil = \lceil 98.4 \rceil = 99$, hence

$$N = 12 \times 99 = 1188 \text{ patches.} \tag{3.7}$$

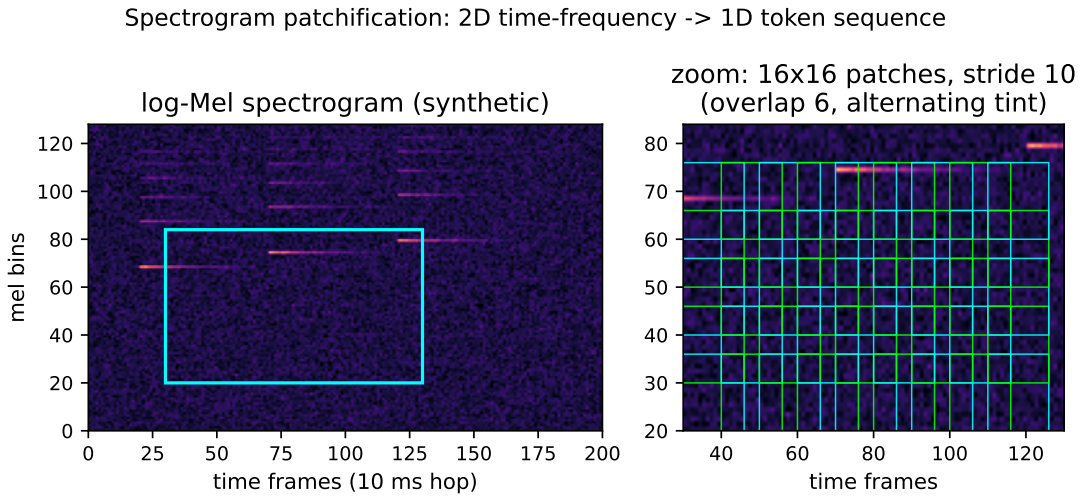


Figure 3.2: Spectrogram patchification. Left: a synthetic 128-bin log-Mel spectrogram with three notes (harmonic stacks with exponential decay). Right: zoom on the 16×16 patch grid with stride 10 (overlap 6); alternating tint distinguishes adjacent tiles.

Each patch is flattened to $\mathbb{R}^{256}$ and projected to the transformer width 768; the patch-embedding layer therefore holds

$$256 \times 768 + 768 = 197{,}376 \text{ parameters.} \tag{3.8}$$

A learned positional embedding of shape $(N, 768)$ (or a 2D sinusoidal variant indexed by (i_f, i_t)) is added so the transformer can recover the tile layout destroyed by flattening. All numbers above are recomputed in `scripts/check_m3_numerics.py`.

Step 4 — Complete runnable code. The `SpectrogramPatchifier` class in the listing below implements (3.4)–(3.6) with `unfold`; the remainder of the file (masking policies, reconstruction loss) serves §3.2.2 and §3.2.3. Saved as `code/m3_patchify_mask.py`; compiles cleanly.

```
"""Module 3, Section 3.2: spectrogram patchification + masking
    strategies + normalized MSE.

Mirrors the AST front-end: a log-Mel spectrogram (F mel bins x T time
    frames)
is split into non-overlapping (or strided) square patches of size
``patch x patch``; a masking policy then selects which patch tokens
    are hidden
from the encoder. The reconstruction loss is MSE over masked patch
    pixels,
optionally computed after per-patch normalization.
"""
from __future__ import annotations

import math

import torch
import torch.nn as nn
import torch.nn.functional as F


class SpectrogramPatchifier(nn.Module):
```

```
    """Split a (B, F, T) spectrogram into a sequence of flattened
        patches.

    With ``patch=16`` and ``stride=10`` this reproduces the AST patch
        grid
    (16x16 patches with overlap 6 in both axes).
    """

    def __init__(self, patch: int = 16, stride: int = 10) -> None:
        super().__init__()
        self.patch = patch
        self.stride = stride

    def grid_shape(self, n_mel: int, n_frames: int) -> tuple[int, int]:
        """Return (n_freq_patches, n_time_patches) for a spectrogram
            shape."""
        gf = (n_mel - self.patch) // self.stride + 1
        gt = (n_frames - self.patch) // self.stride + 1
        return gf, gt

    def forward(self, spec: torch.Tensor) -> tuple[torch.Tensor,
        tuple[int, int]]:
        """spec: (B, F, T) -> patches: (B, N, patch*patch), N =
            gf*gt."""
        b, f, t = spec.shape
        gf, gt = self.grid_shape(f, t)
        assert gf > 0 and gt > 0, "spectrogram smaller than one patch"
        patches = spec.unfold(1, self.patch, self.stride).unfold(2,
            self.patch, self.stride)
        # patches: (B, gf, gt, patch, patch)
        patches = patches.permute(0, 1, 2, 4, 3).reshape(b, gf * gt,
            self.patch * self.patch)
        return patches.contiguous(), (gf, gt)


def random_uniform_mask(n_patches: int, ratio: float, gen:
    torch.Generator) -> torch.Tensor:
    """Mask a uniformly random subset of ``ratio`` of the N patches."""
    n_mask = int(round(n_patches * ratio))
    perm = torch.randperm(n_patches, generator=gen)
    mask = torch.zeros(n_patches, dtype=torch.bool)
    mask[perm[:n_mask]] = True
    return mask


def block_mask(
    grid: tuple[int, int], block: tuple[int, int], n_blocks: int, gen:
        torch.Generator
) -> torch.Tensor:
    """Mask ``n_blocks`` random block-shaped regions of size ``block``.

    Block masking hides contiguous time-frequency neighborhoods, which
        stops the
    decoder from solving reconstruction by local interpolation of
        neighboring
    patches (the "shortcut" that makes random masking too easy on
        spectrograms,
    where adjacent frames are highly correlated).
```

```
    """
    gf, gt = grid
    bf, bt = block
    mask = torch.zeros(gf, gt, dtype=torch.bool)
    for _ in range(n_blocks):
        i = int(torch.randint(0, max(gf - bf + 1, 1), (1,),
            generator=gen))
        j = int(torch.randint(0, max(gt - bt + 1, 1), (1,),
            generator=gen))
        mask[i : i + bf, j : j + bt] = True
    return mask.reshape(-1)


def framewise_time_mask(
    grid: tuple[int, int], n_masked_cols: int, gen: torch.Generator
) -> torch.Tensor:
    """Mask entire time columns of the patch grid (SpecAugment-style
        time masking).

    Every frequency bin inside the chosen time columns is hidden, so
        the model
    must reconstruct a note's full harmonic stack from surrounding
        time context.
    """
    gf, gt = grid
    mask = torch.zeros(gf, gt, dtype=torch.bool)
    cols = torch.randperm(gt, generator=gen)[:n_masked_cols]
    mask[:, cols] = True
    return mask.reshape(-1)


def frequencywise_pitch_mask(
    grid: tuple[int, int], n_masked_rows: int, gen: torch.Generator
) -> torch.Tensor:
    """Mask entire frequency rows of the patch grid (SpecAugment-style
        freq masking).

    Hiding whole pitch bands forces the model to infer missing
        partials from the
    remaining harmonic structure, an inductive bias that directly
        serves pitch
    perception.
    """
    gf, gt = grid
    mask = torch.zeros(gf, gt, dtype=torch.bool)
    rows = torch.randperm(gf, generator=gen)[:n_masked_rows]
    mask[rows, :] = True
    return mask.reshape(-1)


def masked_reconstruction_loss(
    pred: torch.Tensor,
    target: torch.Tensor,
    mask: torch.Tensor,
    normalize: bool = True,
    eps: float = 1e-6,
) -> torch.Tensor:
    """MSE over masked patch pixels.
```

```
115
116     Args:
117         pred:   (B, N, P) decoder predictions for every patch.
118         target: (B, N, P) ground-truth patch pixels.
119         mask:   (N,) boolean, True where the patch was masked.
120         normalize: if True, standardize each target patch to zero mean
                and unit
121             variance before computing the MSE (per-patch
                normalization).
122
123     Returns:
124         Scalar mean-squared error averaged over masked patches and
            pixels.
125     """
126     assert pred.shape == target.shape
127     b, n, p = pred.shape
128     assert mask.shape == (n,)
129     tgt = target[:, mask, :]           # (B, Nm, P)
130     prd = pred[:, mask, :]             # (B, Nm, P)
131     if tgt.numel() == 0:
132         return pred.new_zeros(())
133     if normalize:
134         mean = tgt.mean(dim=-1, keepdim=True)
135         var = tgt.var(dim=-1, keepdim=True, unbiased=False)
136         tgt = (tgt - mean) / torch.sqrt(var + eps)
137     return F.mse_loss(prd, tgt)
138
139
140 def effective_mask_ratio_block(gf: int, gt: int, bf: int, bt: int,
    n_blocks: int) -> float:
141     """Closed-form expected masking ratio for block masking.
142
143     Each block covers a fraction q = (bf*bt)/(gf*gt) of the grid.
        Treating block
144     placements as independent, the probability a patch survives all
        blocks is
145     (1 - q)^n_blocks, so the expected masked fraction is 1 - (1 -
        q)^n_blocks.
146     """
147     q = (bf * bt) / (gf * gt)
148     return 1.0 - (1.0 - q) ** n_blocks
149
150
151 if __name__ == "__main__":
152     torch.manual_seed(0)
153     gen = torch.Generator().manual_seed(0)
154
155     # AST front-end: 128 mel bins, 10 ms hop -> 100 frames per second.
156     seconds, n_mel, hop = 10, 128, 0.010
157     n_frames = int(seconds / hop)
158     spec = torch.randn(2, n_mel, n_frames)  # (B, F, T)
159
160     patchifier = SpectrogramPatchifier(patch=16, stride=10)
161     patches, (gf, gt) = patchifier(spec)
162     n = gf * gt
163     print(f"spectrogram: {n_mel}x{n_frames} -> grid {gf}x{gt} -> {n}
        patches")
164     print(f"patch dim: {patches.shape[-1]} (16x16 flattened)")
```

```
    m_rand = random_uniform_mask(n, 0.75, gen)
    # K chosen so the expected block-masking ratio matches 75%:
    # K = log(0.25)/log(1 - 16/1188) = 103 (see effective_mask_ratio_block)
    m_block = block_mask((gf, gt), (4, 4), 103, gen)
    m_time = framewise_time_mask((gf, gt), n_masked_cols=40, gen=gen)
    m_freq = frequencywise_pitch_mask((gf, gt), n_masked_rows=5, gen=gen)
    print(f"random-uniform masked ratio: {m_rand.float().mean():.3f}")
    print(f"block masked ratio:          {m_block.float().mean():.3f} "
          f"(theory {effective_mask_ratio_block(gf, gt, 4, 4, 103):.3f})")
    print(f"time-frame masked ratio:     {m_time.float().mean():.3f}")
    print(f"frequency masked ratio:      {m_freq.float().mean():.3f}")

    # Reconstruction loss sanity check: a perfect prediction has zero loss,
    # and normalization changes the loss scale (see Section 3.2.3).
    pred = patches.clone()
    l_raw = masked_reconstruction_loss(pred, patches, m_rand, normalize=False)
    pred_noisy = patches + 0.1 * torch.randn_like(patches)
    l_raw_n = masked_reconstruction_loss(pred_noisy, patches, m_rand, normalize=False)
    l_norm_n = masked_reconstruction_loss(pred_noisy, patches, m_rand, normalize=True)
    print(f"perfect pred, raw loss: {l_raw.item():.6f}")
    print(f"noisy pred, raw loss: {l_raw_n.item():.6f}, normalized loss: {l_norm_n.item():.6f}")
    assert l_raw.item() == 0.0
    assert l_raw_n.item() > 0.0
    print("OK")
```

3.2.2 Patch masking strategies

Step 1 — Physical analogy. Redact a musical score four ways: (a) white out random words, (b) white out whole bars, (c) white out entire beats across all staves, (d) white out one staff entirely. Each redaction teaches a different skill: (a) local inference, (b) phrasing, (c) vertical harmony from surrounding time, (d) a missing voice from the remaining voices. Masking policies are the same choice applied to spectrogram patches.

Step 2 — Geometric mental model. On the 12×99 patch grid, a mask is a binary matrix $M \in \{0,1\}^{12\times 99}$. Random masking scatters isolated holes; block masking punches contiguous craters; time masking removes full columns; frequency masking removes full rows. Figure 3.3 renders all four on the same grid. The geometry matters because the decoder reconstructs each hole from its visible neighborhood: isolated holes can be filled by copying adjacent pixels, while craters and full columns force the model to use long-range harmonic and temporal structure.

Step 3 — First-principles derivation. Let the grid hold $G = G_f G_t$ patches. The *effective masking ratio* $r = \mathbb{E}[|M|]/G$ and the inductive bias differ per policy.

Random uniform. Each patch is masked independently with probability p, so $r = p$ exactly in expectation and $|M| \sim \text{Binomial}(G, p)$. With $p = 0.75$ (the MAE operating point [21]), 891 of 1188 patches are hidden and 297 remain visible. The bias: adjacent spectrogram frames are

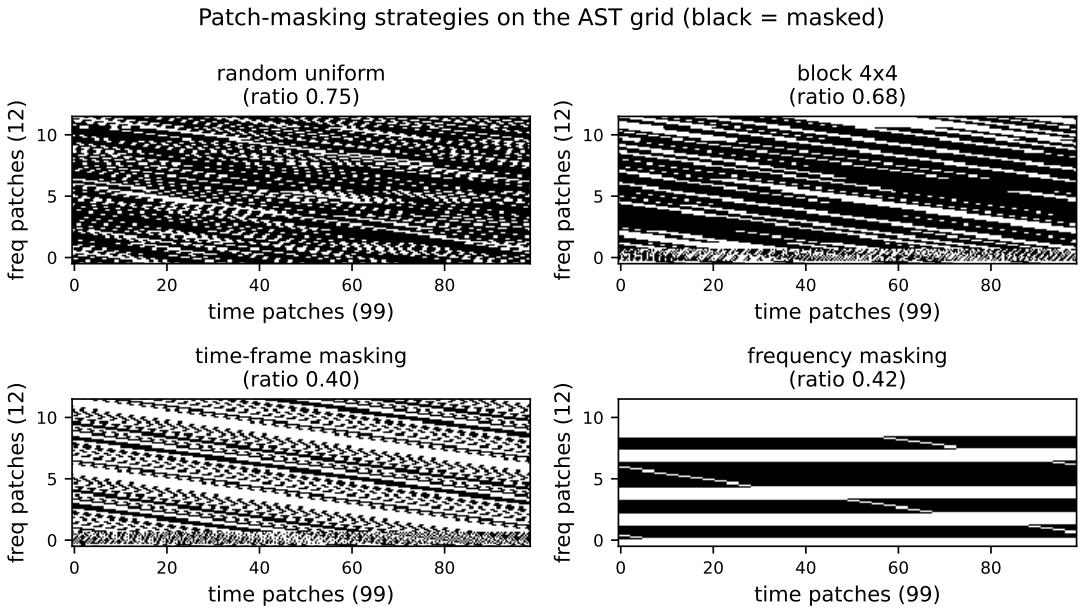


Figure 3.3: Four masking policies on the 12×99 AST patch grid for 10 s of audio (black = masked). Each panel's realized ratio is shown; the derivations are in Step 3.

highly correlated (a note's spectrum barely changes over 10 ms), so isolated holes admit a *local interpolation shortcut* — the decoder succeeds without learning any musical structure.

Block masking. Place K blocks of $b_f \times b_t$ patches at uniform random positions. One block covers a fraction $q = b_f b_t / G$ of the grid. Treating placements as independent, a given patch survives all K blocks with probability $(1-q)^K$, hence

$$r_{\text{block}} = 1 - (1-q)^K. \tag{3.9}$$

Worked: $b_f = b_t = 4$, $G = 1188$, $q = 16/1188 = 0.01347$. Targeting the same $r = 0.75$ as random masking,

$$K = \frac{\log(1-0.75)}{\log(1-0.01347)} = \frac{-1.3863}{-0.01356} = 102.2 \;\approx\; 103 \text{ blocks}, \tag{3.10}$$

and $r_{\text{block}} = 1-(1-16/1188)^{103} = 0.7526$. The bias: craters defeat local interpolation, forcing the encoder to model note envelopes, harmonic stacks, and temporal context spanning hundreds of milliseconds.

Frame-wise time masking. Mask c_t of the G_t time columns entirely:

$$r_{\text{time}} = \frac{c_t}{G_t}. \tag{3.11}$$

With $c_t = 40$, $r_{\text{time}} = 40/99 = 0.4040$. The bias is harmonic completion: with a whole time slice gone, the model must infer a note's full partial stack from its attack and release context — the closest analogue of onset/offset reasoning among the four policies.

Frequency-wise pitch masking. Mask r_f of the G_f frequency rows:

$$r_{\text{freq}} = \frac{r_f}{G_f}. \tag{3.12}$$

With $r_f = 5$, $r_{\text{freq}} = 5/12 = 0.4167$. The bias is cross-partial inference: the model learns that energy at $f_0, 2f_0, 3f_0, \ldots$ co-occurs, which is precisely the harmonic prior a pitch detector needs (Chapter 2, overtone collisions).

Step 4 — Complete runnable code. `random_uniform_mask`, `block_mask`, `framewise_time_mask`, and `frequencywise_pitch_mask` in `code/m3_patchify_mask.py` (listing in §3.2.1) implement the four policies, and `effective_mask_ratio_block` implements (3.9). The `__main__` block prints realized ratios next to the closed forms; block masking with $K = 103$ lands at 0.75 as derived in (3.10).

3.2.3 Reconstruction loss: raw versus normalized spectral domains

Step 1 — Physical analogy. An art restorer filling a damaged patch of fresco must match the *style* of the missing region — the shapes and their relations — not merely its average brightness. Raw-pixel MSE is a restorer paid by the lumen: she obsesses over the brightest gold leaf and ignores the charcoal sketch lines. Per-patch normalization re-pays her by *structure*: every patch, loud or quiet, contributes equally, so the loss optimizes spectral shape rather than loudness.

Step 2 — Geometric mental model. Write the gradient of the loss with respect to one predicted pixel. Under raw MSE its magnitude scales with the local reconstruction error, which scales with local energy; in music, low-frequency bins carry orders of magnitude more energy than high bins, so the gradient vector points almost entirely along the loud-bin axes. Normalization rotates this picture: each patch's target is standardized to unit variance, spreading gradient mass evenly across the frequency axis. Figure 3.4 quantifies the effect on synthetic data via the Gini coefficient of gradient magnitudes: 0.646 (raw, highly concentrated) versus 0.073 (normalized, balanced).

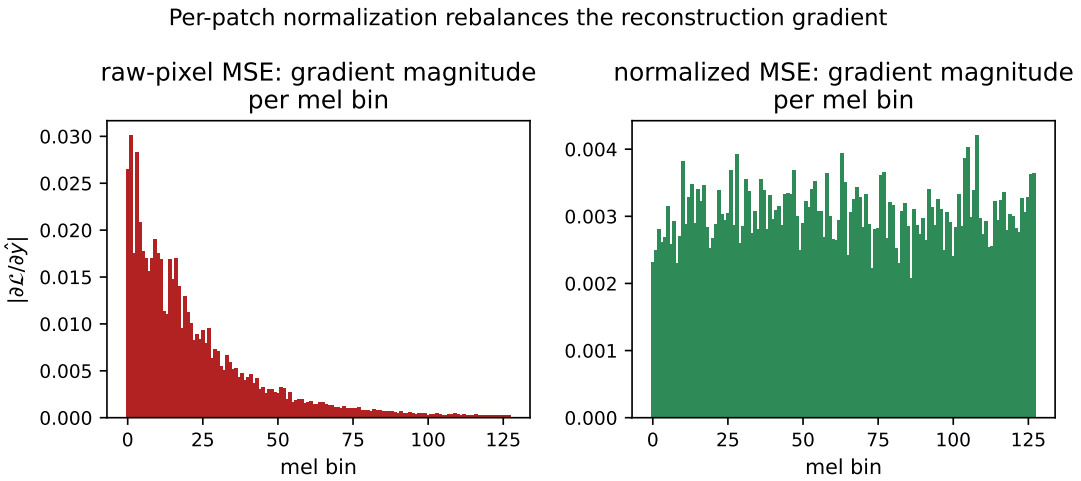


Figure 3.4: Gradient-magnitude distribution across mel bins for raw-pixel MSE versus per-patch-normalized MSE (synthetic spectral envelope). Raw MSE lets a few high-energy bins dominate the update (Gini 0.646); normalization rebalances the gradient (Gini 0.073) so the loss teaches spectral shape.

Step 3 — First-principles derivation. MAE's asymmetric design [21]: the encoder processes *only the visible patches*; mask tokens are appended afterward and a lightweight decoder reconstructs every patch. Let $\mathcal{M}$ be the masked index set, $|\mathcal{M}| = M$, $y_i \in \mathbb{R}^P$ the true patch pixels ($P = 256$), and $\hat{y}_i$ the decoder prediction. The normalized reconstruction loss is

$$\mathcal{L}_{\text{MAE}} = \frac{1}{M} \sum_{i \in \mathcal{M}} \left\| \hat{y}_i - \tilde{y}_i \right\|_2^2, \qquad \tilde{y}_i = \frac{y_i - \mu_i}{\sigma_i}, \quad \mu_i = \tfrac{1}{P} \textstyle\sum_p y_{i,p}, \quad \sigma_i^2 = \tfrac{1}{P} \textstyle\sum_p (y_{i,p} - \mu_i)^2. \tag{3.13}$$

Its gradient with respect to a predicted pixel is

$$\frac{\partial \mathcal{L}_{\text{MAE}}}{\partial \hat{y}_{i,p}} = \frac{2}{M}\left(\hat{y}_{i,p} - \tilde{y}_{i,p}\right). \tag{3.14}$$

Why normalization matters. Without it ($\tilde{y}_i = y_i$), suppose the decoder's relative error is roughly uniform, $|\hat{y}_{i,p} - y_{i,p}| \approx c\,|y_{i,p}|$. Then $|\partial\mathcal{L}/\partial\hat{y}_{i,p}| \propto |y_{i,p}|$: bins with large energy dominate the gradient, and the encoder learns to predict loudness contours while neglecting the quiet high-frequency partials that disambiguate pitch. With normalization, $\tilde{y}_{i,p}$ has unit variance per patch, so (3.14) weights every patch's *shape* equally regardless of its loudness; the loss can only decrease by modeling harmonic structure, the exact representation AMT needs. This is the same normalization used in the MAE pixel objective [21] and in SSAST's generative branch [22].

Compute savings of the asymmetric design. Self-attention costs $\mathcal{O}(L^2 d)$ in sequence length L. Encoding only the visible patches at $r = 0.75$ gives

$$\frac{\text{cost}_{\text{visible}}}{\text{cost}_{\text{full}}} = \left(\frac{(1-r)N}{N}\right)^2 = (0.25)^2 = \frac{1}{16}, \tag{3.15}$$

a sixteen-fold attention saving in the (heavy) encoder, paid back only in the (light) decoder that sees mask tokens. For $N = 1188$: 297 visible versus 891 masked patches.

Step 4 — Complete runnable code. `masked_reconstruction_loss` in `code/m3_patchify_mask.py` implements (3.13)–(3.14) including the per-patch standardization branch; its demo asserts zero loss for a perfect prediction and positive loss otherwise.

3.3 Contrastive, Quantized, and Contextualized SSL

Reconstruction predicts pixels; the next family predicts *structure*. wav2vec 2.0 [23] learns discrete acoustic units with a Gumbel-Softmax codebook and trains a contrastive objective over them; HuBERT [24] replaces the learned codebook with iterated k-means targets and a BERT-style masked-prediction loss; data2vec [25] drops discretization entirely and regresses contextualized continuous targets from a momentum teacher. All three shape representations around the discrete-or-continuous units of sound — the right inductive bias for a transcription task whose outputs are discrete note events.

3.3.1 wav2vec 2.0: contrastive learning over quantized latents

Step 1 — Physical analogy. Imagine learning a foreign language from radio with a phrasebook that has 102,400 entries, each a short canonical sound. You cover a stretch of the broadcast, guess the phrasebook entry from context, then check your guess against the true entry hidden under your thumb — while 100 wrong entries try to distract you. The phrasebook itself is rewritten as you learn, and a diversity rule stops you from using only ten favorite entries. The phrasebook is the vector-quantization codebook; the distraction game is the contrastive loss.

Step 2 — Geometric mental model. Figure 3.5 shows the geometry: codebook entries are points in latent space partitioning it into Voronoi cells, and the encoder's continuous trajectory is snapped to the nearest entry per frame, producing a discrete unit sequence. Figure 3.6 shows the Gumbel-Softmax relaxation that makes this snapping differentiable: temperature τ anneals the soft assignment from near-uniform toward the hard argmax.

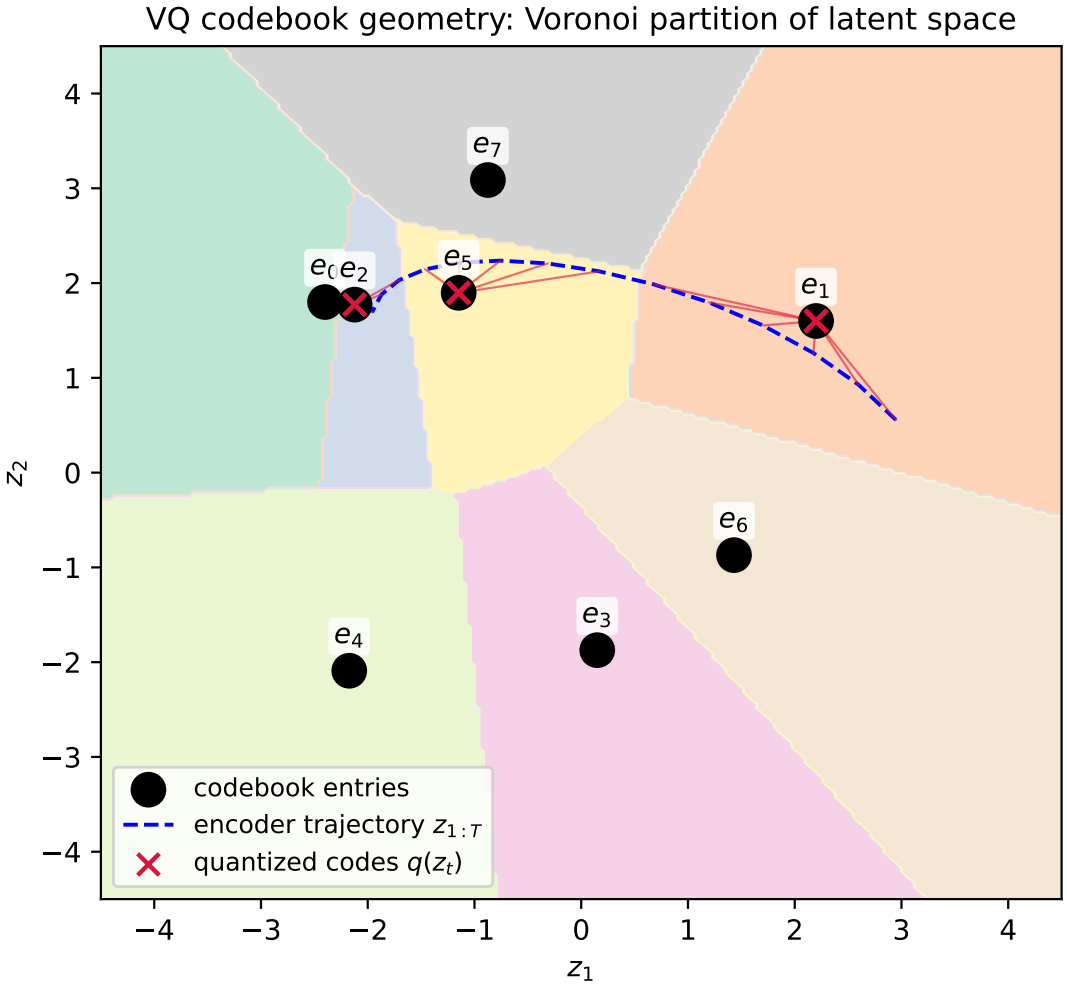


Figure 3.5: Vector-quantization geometry. Codebook entries $\{e_v\}$ partition latent space into Voronoi cells; the encoder trajectory $z_{1:T}$ (dashed) is quantized to the nearest entry per frame (red crosses), yielding discrete acoustic units.

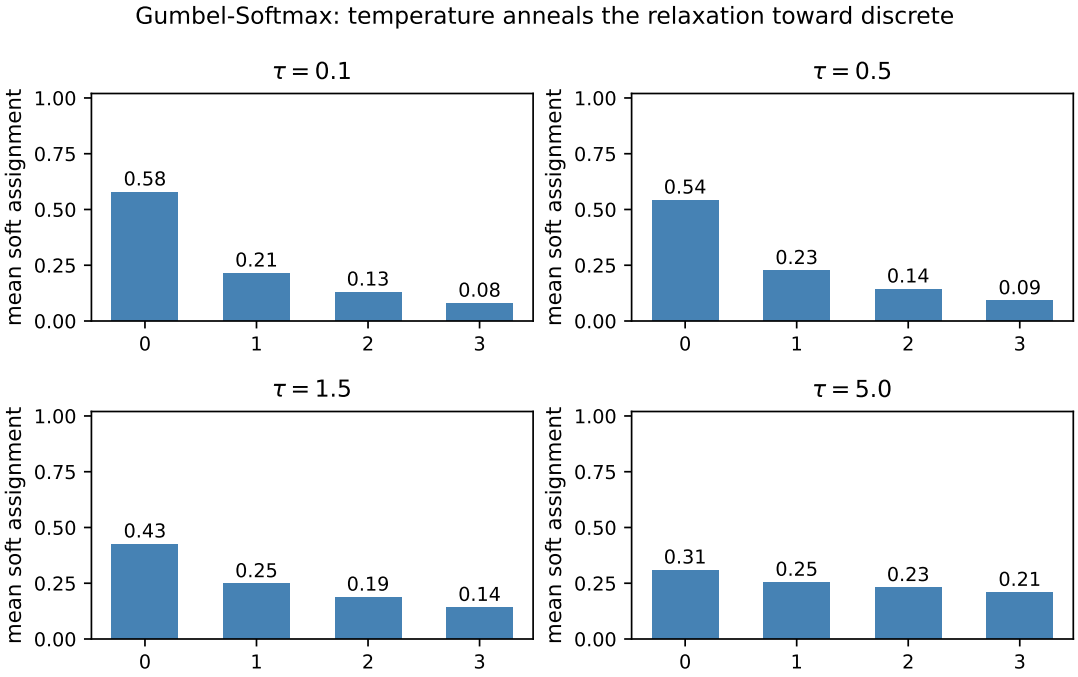


Figure 3.6: Gumbel-Softmax relaxation for fixed logits at four temperatures. Low τ concentrates mass on the argmax (near-discrete sample); high τ flattens toward uniform. Annealing τ during training moves the codebook from exploration to commitment.

Step 3 — First-principles derivation. The pipeline has four stages; we derive each.

1. Feature encoder and latent masking. A seven-layer convolutional encoder with strides $(5, 2, 2, 2, 2, 2, 2)$ downsamples 16 kHz audio by $5 \cdot 2^6 = 320$, emitting latent vectors $z_t \in \mathbb{R}^{512}$ at $16000/320 = 50$ Hz [23]. Spans of these latents are masked (masking happens in latent space, not on the waveform), and a Transformer builds contextualized representations c_t from the masked sequence.

2. Gumbel-Softmax quantization. Each z_t is quantized with $G = 2$ codebooks of $V = 320$ entries each (entry dimension $d/G = 128$ for the base model), for $320^2 = 102{,}400$ effective codewords [23]. For one codebook with logits $\ell \in \mathbb{R}^V$ we need a differentiable categorical sample. The foundation is the Gumbel-Max trick.

Lemma 3.1 (Gumbel inverse-CDF)**.** *If $U \sim \text{Uniform}(0,1)$ then $G = -\log(-\log U)$ follows the standard Gumbel distribution with CDF $F(x) = \exp(-e^{-x})$.*

Proof. $P(G \le x) = P(-\log(-\log U) \le x)$. Multiplying by -1 flips the inequality: $P(\log(-\log U) \ge -x)$. Exponentiating (monotone): $P(-\log U \ge e^{-x})$. Multiplying by -1 again: $P(\log U \le -e^{-x})$. Exponentiating once more: $P(U \le \exp(-e^{-x})) = \exp(-e^{-x})$, since $P(U \le u) = u$ on $[0,1]$. Hence the CDF is $\exp(-e^{-x})$. □

Lemma 3.2 (Gumbel-Max trick)**.** *Let $g_i \overset{iid}{\sim} \text{Gumbel}(0,1)$ and $\pi_i \ge 0$, $\sum_i \pi_i = 1$. Then $P\big(\arg\max_i(\log \pi_i + g_i) = k\big) = \pi_k$.*

Proof. Condition on $g_k = g$. The event $\{\arg\max = k\}$ is $\bigcap_{j \ne k}\{g_j \le g + \log \pi_k - \log \pi_j\}$. With

$F(x) = e^{-e^{-x}}$ and density $f(x) = e^{-x}e^{-e^{-x}}$,

$$P(\arg\max = k) = \int_{-\infty}^{\infty} f(g) \prod_{j \neq k} F(g + \log \pi_k - \log \pi_j)\, dg$$
$$= \int_{-\infty}^{\infty} e^{-g} e^{-e^{-g}} \prod_{j \neq k} \exp\big(- e^{-(g + \log \pi_k - \log \pi_j)} \big)\, dg. \tag{3.16}$$

Since $e^{-(g+\log \pi_k - \log \pi_j)} = e^{-g}\, \pi_j/\pi_k$, the product over $j \neq k$ together with the $e^{-e^{-g}}$ factor (the $j = k$ term, as $\pi_k/\pi_k = 1$) gives $\exp(-e^{-g} \sum_j \pi_j/\pi_k) = \exp(-e^{-g}/\pi_k)$. Hence

$$P(\arg\max = k) = \int_{-\infty}^{\infty} e^{-g} \exp(-e^{-g}/\pi_k)\, dg. \tag{3.17}$$

Substitute $u = e^{-g}/\pi_k$, so $du = -(e^{-g}/\pi_k)\, dg$, i.e. $e^{-g}\, dg = -\pi_k\, du$; as $g \to -\infty$, $u \to \infty$, and as $g \to \infty$, $u \to 0$:

$$P(\arg\max = k) = \int_{\infty}^{0} -\pi_k e^{-u}\, du = \pi_k \int_0^{\infty} e^{-u}\, du = \pi_k. \tag{3.18}$$

Thus $\arg\max_i(\log \pi_i + g_i)$ is an exact Categorical(π) sample. □

The Gumbel-Softmax relaxation replaces the argmax by a temperature-scaled softmax,

$$y_i = \frac{\exp((\log \pi_i + g_i)/\tau)}{\sum_j \exp((\log \pi_j + g_j)/\tau)}, \qquad g_i \overset{\text{iid}}{\sim} \text{Gumbel}(0, 1), \tag{3.19}$$

which is differentiable in the logits. As $\tau \to 0^+$, writing $a_i = \log \pi_i + g_i$ and $a^\star = \max_j a_j$, $y_i = e^{(a_i - a^\star)/\tau} / \sum_j e^{(a_j - a^\star)/\tau} \to \mathbf{1}\{i = \arg\max_j a_j\}$ (ties have probability zero for continuous g_i), recovering the exact discrete sample of Lemma 3.2 in the limit. wav2vec 2.0 anneals τ from 2 down to 0.5 (base model) during training [23]. For a hard forward sample with gradient flow, the straight-through estimator is used: $y = y_{\text{hard}} - \text{sg}(y_{\text{soft}}) + y_{\text{soft}}$, so the forward evaluation is one-hot while the backward gradient follows the relaxation. The quantized vector is the concatenation of the selected entries from both codebooks.

3. Contrastive loss. For each masked position t, the model must identify the true quantized target q_t among $K = 100$ distractors sampled from other masked positions of the same utterance [23]:

$$\mathcal{L}_m = -\log \frac{\exp(\text{sim}(c_t, q_t)/\kappa)}{\sum_{\tilde{q} \in \mathcal{Q}_t} \exp(\text{sim}(c_t, \tilde{q})/\kappa)}, \qquad \kappa = 0.1, \tag{3.20}$$

with cosine similarity sim and $\mathcal{Q}_t$ containing q_t plus the distractors. Minimizing (3.20) maximizes a lower bound on the mutual information $I(c_t; q_t)$: the context representation must capture whatever in the quantized unit is predictable from surroundings.

4. Diversity loss. Without regularization the codebook collapses to a few entries. Let $\bar{p}_{g,v}$ be the batch-and-time average of the soft assignment to entry v of codebook g. wav2vec 2.0 adds [23]

$$\mathcal{L}_d = \frac{1}{GV} \sum_{g=1}^{G} \sum_{v=1}^{V} \bar{p}_{g,v} \log \bar{p}_{g,v} = -\frac{1}{GV} \sum_{g=1}^{G} H(\bar{p}_g), \tag{3.21}$$

i.e. negative entropy of the average usage. For fixed g, $\sum_v \bar{p}_{g,v} \log \bar{p}_{g,v} \geq -\log V$ with equality iff $\bar{p}_g$ is uniform (Gibbs' inequality, proved by $\sum p_v \log(p_v/(1/V)) \geq 0$), so the per-codebook floor is $-\log(320)/320 = -0.01803$. The total objective is $\mathcal{L} = \mathcal{L}_m + 0.1\, \mathcal{L}_d$.

Step 4 — Complete runnable code. `code/m3_gumbel_vq.py` implements Lemma 3.1 (`sample_gumbel`), (3.19) with the straight-through estimator (`gumbel_softmax`), the product quantizer (`GumbelVectorQuantizer`), and (3.21) (`diversity_loss`). Its demo verifies the Gumbel-Max trick empirically (200k draws reproduce the target categorical frequencies) and checks gradient flow through the hard sample.

```
"""Module 3, Section 3.3.1: vector quantization with Gumbel-Softmax
    codebooks.

Implements the quantization module of wav2vec 2.0: G codebooks, each
    with V
learned entries; a latent vector z is mapped to logits over each
    codebook and
a differentiable one-hot sample is drawn with the Gumbel-Softmax
    trick. The
straight-through variant yields hard discrete codes in the forward
    pass while
flowing gradients through the soft relaxation in the backward pass.
"""
from __future__ import annotations

import torch
import torch.nn as nn
import torch.nn.functional as F


def sample_gumbel(shape: torch.Size, eps: float = 1e-10, device=None)
    -> torch.Tensor:
    """Draw i.i.d. samples from Gumbel(0, 1) via the inverse-CDF
        transform.

    If U ~ Uniform(0, 1) then G = -log(-log(U)) has CDF exp(-exp(-x)),
        i.e.
    the standard Gumbel distribution. This is Lemma 3.1 in the text.
    """
    u = torch.rand(shape, device=device).clamp_min(eps)
    return -torch.log(-torch.log(u))


def gumbel_softmax(
    logits: torch.Tensor, tau: float = 1.0, hard: bool = False, eps:
        float = 1e-10
) -> torch.Tensor:
    """Differentiable categorical sample.

    y_i = exp((log pi_i + g_i)/tau) / sum_j exp((log pi_j + g_j)/tau),
    g_i ~ Gumbel(0, 1) i.i.d. As tau -> 0, y -> one-hot(argmax_i(log
        pi_i+g_i)),
    which by the Gumbel-Max trick is an exact Categorical(pi) sample.

    With hard=True, the forward pass returns the exact one-hot vector
        while
    the backward pass uses the straight-through estimator (gradient of
        the
    soft relaxation).
    """
    g = sample_gumbel(logits.shape, eps=eps, device=logits.device)
    y_soft = F.softmax((logits + g) / tau, dim=-1)
```

```
    if not hard:
        return y_soft
    index = y_soft.argmax(dim=-1, keepdim=True)
    y_hard = torch.zeros_like(y_soft).scatter_(-1, index, 1.0)
    # straight-through: forward = hard, backward gradient = soft
    return (y_hard - y_soft).detach() + y_soft


class GumbelVectorQuantizer(nn.Module):
    """G codebooks x V entries; product quantization of a latent vector.

    wav2vec 2.0 uses G=2 codebooks with V=320 entries each; the quantized
    vector is the concatenation of the selected entries (entry dim = d/G),
    giving V^G effective codes. Here G and V are configurable so the module
    stays runnable on a laptop; defaults mirror the small-demo setting used
    in the text.
    """

    def __init__(self, latent_dim: int, num_codebooks: int = 2, entries: int = 8) -> None:
        super().__init__()
        assert latent_dim % num_codebooks == 0
        self.num_codebooks = num_codebooks
        self.entries = entries
        self.entry_dim = latent_dim // num_codebooks
        self.codebooks = nn.ParameterList(
            [nn.Parameter(torch.randn(entries, self.entry_dim) * 0.5)
             for _ in range(num_codebooks)]
        )
        # logit projection: latent -> (G, V) codebook logits
        self.to_logits = nn.Linear(latent_dim, num_codebooks * entries)

    def forward(
        self, z: torch.Tensor, tau: float = 1.0, hard: bool = True
    ) -> tuple[torch.Tensor, torch.Tensor]:
        """z: (B, T, D) -> (quantized (B, T, D), per-codebook probs (B, T, G, V))."""
        b, t, _ = z.shape
        logits = self.to_logits(z).view(b, t, self.num_codebooks, self.entries)
        probs = gumbel_softmax(logits, tau=tau, hard=hard)  # (B, T, G, V)
        codes = []
        for g in range(self.num_codebooks):
            # weighted sum over entries; with hard=True this is a lookup
            codes.append(probs[:, :, g, :] @ self.codebooks[g])  # (B, T, d/G)
        quantized = torch.cat(codes, dim=-1)
        return quantized, probs


def diversity_loss(probs: torch.Tensor) -> torch.Tensor:
```

```
    """Entropy-maximizing codebook usage regularizer (wav2vec 2.0, Eq. diversity).

    probs: (B, T, G, V). Average the soft assignments over batch and time to
    get per-codebook usage p_bar, then L_d = (1/V) * sum_v p_bar_v log p_bar_v
    = -(1/V) * H(p_bar). Minimizing L_d maximizes the usage entropy, which is
    minimized (most negative) at the uniform distribution p_bar = 1/V.
    """
    p_bar = probs.mean(dim=(0, 1))           # (G, V)
    neg_entropy = (p_bar * (p_bar + 1e-10).log()).sum(dim=-1)  # (G,)
    return neg_entropy.mean() / probs.shape[-1]


if __name__ == "__main__":
    torch.manual_seed(0)

    # --- Gumbel-Max trick check: argmax(log pi + g) ~ Categorical(pi) ---
    pi = torch.tensor([0.1, 0.2, 0.3, 0.4])
    draws = (pi.log() + sample_gumbel((200_000, 4))).argmax(dim=-1)
    freq = torch.bincount(draws, minlength=4).float() / 200_000
    print("target pi:      ", [f"{v:.3f}" for v in pi.tolist()])
    print("empirical freq: ", [f"{v:.3f}" for v in freq.tolist()])
    assert torch.allclose(freq, pi, atol=0.01)

    # --- temperature behaviour of the relaxation ---
    logits = torch.tensor([[2.0, 1.0, 0.5, 0.0]])
    for tau in (0.05, 1.0, 10.0):
        y = gumbel_softmax(logits.expand(50_000, 4), tau=tau, hard=False).mean(0)
        print(f"tau={tau:>5}: mean soft sample = {[f'{v:.3f}' for v in y.tolist()]}")

    # --- quantizer round-trip ---
    vq = GumbelVectorQuantizer(latent_dim=64, num_codebooks=2, entries=8)
    z = torch.randn(2, 10, 64)
    q, probs = vq(z, tau=0.5, hard=True)
    print("quantized shape:", tuple(q.shape), "| probs shape:", tuple(probs.shape))
    hard_counts = probs[:, :, 0, :].argmax(-1).reshape(-1)
    print("code usage histogram (codebook 0):",
          torch.bincount(hard_counts, minlength=8).tolist())
    print("diversity loss:", f"{diversity_loss(probs).item():.4f}",
          f"(uniform floor = {-torch.tensor(8.0).log().item() / 8:.4f})")

    # straight-through gradient check: hard forward, nonzero backward
    z_req = torch.randn(2, 4, 64, requires_grad=True)
    q2, _ = vq(z_req, tau=0.5, hard=True)
    q2.sum().backward()
    print("grad flows through straight-through:",
          z_req.grad is not None and z_req.grad.abs().sum().item() > 0)
    assert q2.shape == z_req.shape
    print("OK")
```

3.3.2 HuBERT: masked prediction of hidden units

Step 1 — Physical analogy. A child learning bird calls invents her own shorthand: that chirp is a "type-3," that trill a "type-7." Her labels are crude, but she uses them *consistently*, so she can still learn which calls follow which. Later, with a better ear, she refines the shorthand into finer categories and relearns. HuBERT is this child: k-means is the invented shorthand, masked prediction is the learning, and the second iteration is the refined ear.

Step 2 — Geometric mental model. Each frame's acoustic feature is a point in $\mathbb{R}^{39}$ (MFCC space) or $\mathbb{R}^{768}$ (learned space); k-means tiles the space into cells, and the cell index becomes the frame's discrete "hidden unit." The model reads masked continuous audio and must name the hidden unit behind each mask — a classification problem over the tiling. Figure 3.7 shows the two-iteration loop: crude MFCC tiles train the first model, whose learned features are re-tiled more finely for the second.

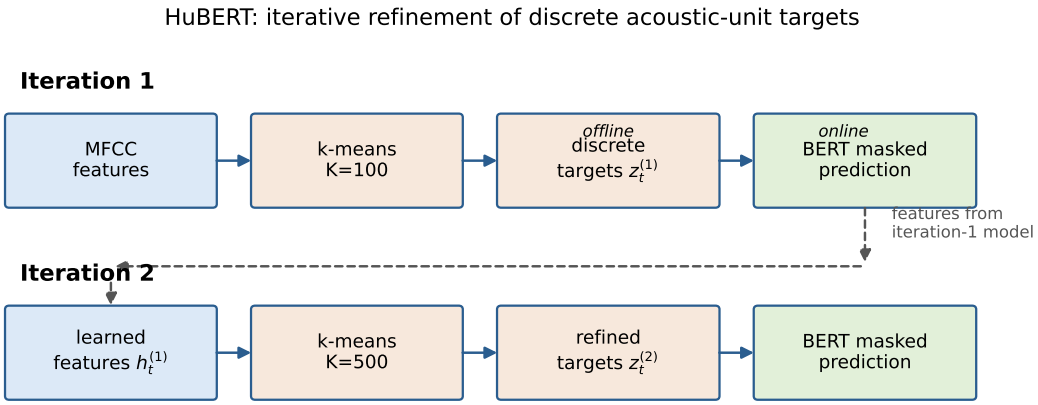


Figure 3.7: HuBERT's two-iteration training. Iteration 1: k-means ($K = 100$) on MFCC features yields discrete targets for masked prediction. Iteration 2: k-means ($K = 500$) on the iteration-1 model's intermediate features yields refined targets; the model is retrained from scratch on them.

Step 3 — First-principles derivation. HuBERT addresses three problems of speech SSL: multiple sound units per utterance, no lexicon of units during pretraining, and variable-length units with no explicit segmentation [24]. The solution is an offline clustering step that supplies aligned discrete targets for a BERT-like loss.

Iteration 1. Extract 39-dimensional MFCC features (13 coefficients plus first- and second-order deltas) and run k-means with $K = 100$ to obtain targets $z_t^{(1)} \in \{1, \dots, 100\}$ per frame [24]. Mask spans of the input (replacing them with a learned mask embedding) to form $\tilde{x}$, encode with a Transformer, and minimize cross-entropy *over masked positions only*:

$$\mathcal{L}_{\text{HuBERT}} = -\sum_{t \in \mathcal{M}} \log p_\theta\big(z_t \mid \tilde{x}, t\big), \qquad p_\theta(\cdot \mid \tilde{x}, t) = \text{softmax}\big(W h_t(\tilde{x})\big). \tag{3.22}$$

This is the ordinary cross-entropy $H(z, p_\theta) = -\sum_k \mathbf{1}\{z_t = k\} \log p_{\theta,k}$ summed over the masked set $\mathcal{M}$. Why masked-only? The ablation in [24] shows that with noisy cluster targets, adding loss on *unmasked* frames degrades performance: the model can minimize unmasked loss by a local identity mapping (copy the visible frame's cluster), learning nothing, whereas masked positions force it to combine an acoustic model of the visible context with a sequential (language-like) model of unit transitions.

Iteration 2. The iteration-1 model has learned features $h_t^{(1)}$ that are better phonetic descriptors than MFCCs. Run k-means with $K = 500$ on these features (in practice on the 6th Transformer layer, subsampled for memory) to obtain refined targets $z_t^{(2)}$, and retrain the model from scratch minimizing (3.22) against $z^{(2)}$ [24]. The key theoretical point, emphasized in [24], is that the procedure relies on the *consistency* of the clustering rather than its correctness: even if iteration 1 merges two distinct sounds into one supercluster, the masked-prediction objective learns the sequential structure around that supercluster, and the improved features then split it in iteration 2. Formally, if the cluster assignment is a deterministic function $z = c(x)$ applied consistently, the model learns $p(z_t \mid \text{context})$, i.e. the transition structure of the induced discrete process; better c in the next round yields a more informative process.

Step 4 — Complete runnable code. `code/m3_hubert_targets.py` implements the full two-iteration procedure on synthetic data: batch k-means (k-means++ seeding, Lloyd iterations), span masking, masked-only cross-entropy (`masked_unit_loss`, exactly (3.22)), and retraining on second-iteration targets clustered from learned features. The demo asserts both iterations beat chance and reports cluster purity against the ground-truth acoustic classes.

```
"""Module 3, Section 3.3.2: HuBERT-style two-iteration masked prediction of
hidden units, on synthetic data.

Iteration 1: k-means (K=100 in the paper; small K here) on "MFCC-like"
features gives discrete targets z^(1); a BERT-style encoder trained with
cross-entropy on MASKED positions only learns to predict them.
Iteration 2: k-means (larger K) on the iteration-1 model's intermediate
features gives refined targets z^(2); the encoder is retrained.

Includes a minimal batch k-means implementation so the file is self-contained.
"""
from __future__ import annotations

import torch
import torch.nn as nn
import torch.nn.functional as F


def batch_kmeans(x: torch.Tensor, k: int, iters: int = 20, seed: int = 0) -> torch.Tensor:
    """x: (N, D) -> cluster assignments (N,). k-means++ init, Lloyd iterations."""
    gen = torch.Generator().manual_seed(seed)
    n, d = x.shape
    # k-means++ seeding
    first = torch.randint(n, (1,), generator=gen)
    centers = x[first]
    for _ in range(1, k):
        d2 = ((x[:, None, :] - centers[None, :, :]) **
```

```
            2).sum(-1).min(-1).values
        probs = d2 / d2.sum()
        nxt = torch.multinomial(probs, 1, generator=gen)
        centers = torch.cat([centers, x[nxt]], dim=0)
    assign = torch.zeros(n, dtype=torch.long)
    for _ in range(iters):
        d2 = ((x[:, None, :] - centers[None, :, :]) ** 2).sum(-1)
        new_assign = d2.argmin(-1)
        for c in range(k):
            members = x[new_assign == c]
            if len(members) > 0:
                centers[c] = members.mean(0)
        if torch.equal(new_assign, assign):
            break
        assign = new_assign
    return assign


class MaskedUnitPredictor(nn.Module):
    """Tiny BERT-style encoder + linear head predicting discrete units."""

    def __init__(self, in_dim: int, dim: int = 64, layers: int = 3, n_units: int = 20) -> None:
        super().__init__()
        self.proj = nn.Linear(in_dim, dim)
        el = nn.TransformerEncoderLayer(d_model=dim, nhead=4, dim_feedforward=4 * dim,
                                        batch_first=True, dropout=0.0)
        self.encoder = nn.TransformerEncoder(el, num_layers=layers)
        self.head = nn.Linear(dim, n_units)
        self.mask_emb = nn.Parameter(torch.randn(dim) * 0.1)

    def forward(self, x: torch.Tensor, mask: torch.Tensor) -> torch.Tensor:
        """x: (B, T, in_dim), mask: (B, T) bool -> logits (B, T, n_units)."""
        h = self.proj(x)
        h = torch.where(mask.unsqueeze(-1), self.mask_emb, h)
        return self.head(self.encoder(h))


def masked_unit_loss(logits: torch.Tensor, targets: torch.Tensor,
                     mask: torch.Tensor) -> torch.Tensor:
    """Cross-entropy evaluated on MASKED positions only (HuBERT's key choice).

    logits: (B, T, K), targets: (B, T) long, mask: (B, T) bool.
    """
    assert mask.any(), "mask must hide at least one position"
    return F.cross_entropy(logits[mask], targets[mask])


def random_span_mask(t: int, n_spans: int, span: int, seed: int) -> torch.Tensor:
    gen = torch.Generator().manual_seed(seed)
    mask = torch.zeros(t, dtype=torch.bool)
    starts = torch.randint(0, max(t - span, 1), (n_spans,),
```

```
        generator=gen)
    for s in starts:
        mask[s : s + span] = True
    return mask


if __name__ == "__main__":
    torch.manual_seed(0)
    B, T, FEAT = 8, 60, 24

    # synthetic "MFCC-like" features: 4 Gaussian acoustic classes in 24-D
    means = torch.randn(4, FEAT) * 3.0
    true_class = torch.randint(0, 4, (B * T,))
    mfcc = means[true_class] + 0.5 * torch.randn(B * T, FEAT)
    mfcc = mfcc.view(B, T, FEAT)

    # ---- iteration 1: k-means on MFCC, K small ----
    z1 = batch_kmeans(mfcc.reshape(-1, FEAT), k=12, seed=0).view(B, T)
    model = MaskedUnitPredictor(FEAT, n_units=12)
    opt = torch.optim.Adam(model.parameters(), lr=3e-3)
    mask = random_span_mask(T, n_spans=6, span=4,
        seed=1).unsqueeze(0).expand(B, T)
    for step in range(60):
        opt.zero_grad()
        loss = masked_unit_loss(model(mfcc, mask), z1, mask)
        loss.backward()
        opt.step()
    print(f"iteration-1 masked CE after training: {loss.item():.4f} "
          f"(chance = {torch.tensor(12.0).log().item():.4f})")
    assert loss.item() < torch.tensor(12.0).log().item()

    # ---- iteration 2: k-means on learned intermediate features, larger K ----
    with torch.no_grad():
        h1 = model.encoder(model.proj(mfcc))          # learned features
    z2 = batch_kmeans(h1.reshape(-1, 64), k=24, seed=2).view(B, T)
    model2 = MaskedUnitPredictor(FEAT, n_units=24)
    opt2 = torch.optim.Adam(model2.parameters(), lr=3e-3)
    for step in range(60):
        opt2.zero_grad()
        loss2 = masked_unit_loss(model2(mfcc, mask), z2, mask)
        loss2.backward()
        opt2.step()
    print(f"iteration-2 masked CE after training: {loss2.item():.4f} "
          f"(chance = {torch.tensor(24.0).log().item():.4f})")
    assert loss2.item() < torch.tensor(24.0).log().item()

    # consistency check: iteration-1 clusters track the true acoustic classes
    # (adjusted comparison via majority vote)
    agree = 0
    for c in range(12):
        members = true_class[z1.reshape(-1) == c]
        if len(members):
            agree += (members == members.mode().values).sum().item()
    print(f"iteration-1 cluster purity vs true classes: {agree /
```

```
        (B*T):.3f}")
130 print("OK")
```

3.3.3 data2vec 2.0: self-distillation in continuous latent space

Step 1 — Physical analogy. A music student practices with one ear covered (the masked view) while her teacher — who is simply her own playing recorded last week and averaged over many weeks — listens with both ears uncovered. Her task: reproduce the teacher's full hearing from her partial hearing. Because the teacher is a slow-moving average of her past selves, she cannot fool the exercise by agreeing on a trivial constant hum: the teacher's hearing varies with the music, and she must track that variation.

Step 2 — Geometric mental model. Both networks map audio into the same latent space $\mathbb{R}^d$, but they see different views: the teacher sees the full input x and emits a smooth, contextualized target y_t; the student sees the masked input $\tilde{x}$ and predicts $\hat{y}_t$. The loss pulls $\hat{y}_t$ toward y_t in Euclidean space. The teacher's weights trail the student's via exponential moving average, so the target landscape drifts slowly while the student chases it. Figure 3.8 diagrams the two paths, the stop-gradient on the teacher, and the EMA update.

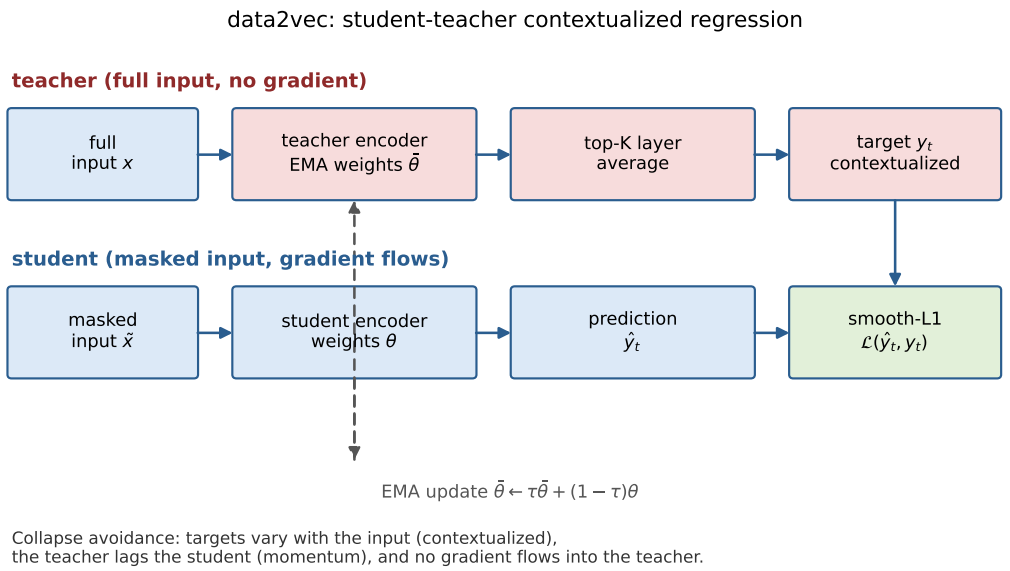


Figure 3.8: data2vec self-distillation. The student encodes the masked input and regresses the teacher's contextualized targets (average of top-K teacher layers on the full input) with a smooth-L1 loss. Teacher weights are an exponential moving average of the student's; no gradient enters the teacher.

Step 3 — First-principles derivation. data2vec [25] uses one learning method across speech, vision, and language: masked prediction of *contextualized latent* representations in a self-distillation setup.

Targets. Let the teacher (parameters $\bar{\theta}$) process the full unmasked input x, producing layer outputs $\bar{h}_t^{(1)}, \ldots, \bar{h}_t^{(L)}$. The target averages the top K layers after per-layer normalization:

$$y_t = \frac{1}{K} \sum_{\ell=L-K+1}^{L} \operatorname{norm}\big(\bar{h}_t^{(\ell)}\big). \tag{3.23}$$

Averaging top layers makes y_t *contextualized*: self-attention has mixed information from the entire input into every position, unlike a local pixel or codebook target.

Student loss. The student (parameters θ) encodes the masked view $\tilde{x}$ and predicts $\hat{y}_t$ from its top layer. The loss is smooth-L1 (Huber-style) over masked positions $\mathcal{M}$:

$$\mathcal{L}_{\text{d2v}} = \frac{1}{|\mathcal{M}|} \sum_{t \in \mathcal{M}} \begin{cases} \frac{1}{2\beta}(\hat{y}_t - y_t)^2, & |\hat{y}_t - y_t| \le \beta, \\ |\hat{y}_t - y_t| - \frac{\beta}{2}, & \text{otherwise}, \end{cases} \tag{3.24}$$

applied elementwise and averaged. The gradient of the targets is stopped: ∇_θ flows only through the student.

EMA teacher. After each student update n, the teacher is updated without gradients as

$$\bar{\theta}_n = \tau_n \bar{\theta}_{n-1} + (1 - \tau_n)\, \theta_n, \tag{3.25}$$

with τ_n annealed from τ_0 toward τ_{end} over the course of training [25]. Unrolling (3.25) with constant τ,

$$\bar{\theta}_n = \tau^n \bar{\theta}_0 + (1 - \tau) \sum_{k=1}^{n} \tau^{\,n-k}\, \theta_k, \tag{3.26}$$

an exponentially decaying average of student history: checkpoint k steps in the past carries weight $(1-\tau)\tau^k$. The half-life is $k_{1/2} = \log(1/2)/\log\tau \approx 0.693/(1-\tau)$; for $\tau = 0.9999$ this is 6931 updates, i.e. the teacher remembers roughly ten thousand recent student states.

Collapse avoidance. Why doesn't the student–teacher pair collapse to a constant? Suppose both output a constant c for every input. The targets (3.23) would be constant, the loss zero — a global minimum. Three mechanisms jointly rule this out in practice. First, the *stop-gradient*: $\bar{\theta}$ is never optimized to match the student, so the pair cannot co-adapt toward the trivial fixed point by gradient descent; only the student moves. Second, *momentum lag*: by (3.26) the teacher is a trailing average, so even if the student began collapsing, the targets would remain the varying contextualized features of recent non-collapsed states, and a constant student prediction $\hat{y}_t \equiv c$ incurs $\mathcal{L}_{\text{d2v}} \ge \min_c \mathbb{E}\|c - y_t\|^2 = \text{Var}(y) > 0$ whenever targets vary across frames — the gradient then pushes the student to track that variation, breaking constancy. Third, the targets are *contextualized full-input* features while the student sees a *masked* view: matching them requires genuine inference about the hidden content, which a constant function cannot provide. Together these give the asymmetric regression dynamics that make self-distillation stable without negative samples or discretization [25].

Step 4 — Complete runnable code. `code/m3_data2vec_ema.py` implements the EMA teacher (`EMATeacher`, exactly (3.25) with cosine-annealed τ), contextualized top-K targets (`contextualized_targets`, (3.23)), the smooth-L1 loss (3.24), and the closed-form decay weights (3.26). The demo asserts the teacher carries no gradient, moves toward the student, and that the decay weights sum to $1 - \tau^{10}$.

```
"""Module 3, Section 3.3.3: data2vec-style self-distillation with an
    EMA teacher.

A student network sees a masked view of the input and regresses the
contextualized latent representations produced by a teacher network
    that sees
the full input. The teacher's weights are an exponential moving
    average (EMA)
of the student's weights; targets are the average of the teacher's
    top-K
normalized layer outputs. The regression loss is the Smooth L1
    (Huber-style)
```

```
loss used in data2vec.
"""
from __future__ import annotations

import math

import torch
import torch.nn as nn
import torch.nn.functional as F


class TinyTransformerEncoder(nn.Module):
    """Small stand-in encoder returning all hidden layer outputs.

    In the text this plays the role of the data2vec Transformer backbone; the
    data2vec paper itself uses standard Transformer encoders with
    modality-specific feature extractors.
    """

    def __init__(self, dim: int = 64, layers: int = 4, heads: int = 4) -> None:
        super().__init__()
        layer = nn.TransformerEncoderLayer(
            d_model=dim, nhead=heads, dim_feedforward=4 * dim,
            batch_first=True, dropout=0.0,
        )
        self.layers = nn.ModuleList(
            [layer] + [
                nn.TransformerEncoderLayer(
                    d_model=dim, nhead=heads, dim_feedforward=4 * dim,
                    batch_first=True, dropout=0.0,
                )
                for _ in range(layers - 1)
            ]
        )
        self.norms = nn.ModuleList([nn.LayerNorm(dim) for _ in range(layers)])

    def forward(self, x: torch.Tensor) -> list[torch.Tensor]:
        hiddens = []
        h = x
        for layer, norm in zip(self.layers, self.norms):
            h = layer(h)
            hiddens.append(norm(h))
        return hiddens


class EMATeacher:
    """Maintains teacher weights as an EMA of student weights (stop-gradient).

    teacher_t = tau_n * teacher_{t-1} + (1 - tau_n) * student_t,
    with tau_n annealed from tau_0 to tau_end over n_updates steps following
    the momentum schedule used in data2vec-style methods.
    """

```

```
    def __init__(
        self, student: nn.Module, tau_0: float = 0.999, tau_end: float = 0.9999,
        n_updates: int = 1000,
    ) -> None:
        self.teacher = TinyTransformerEncoder(
            dim=student.layers[0].self_attn.embed_dim if hasattr(student, "layers") else 64,
            layers=len(student.layers) if hasattr(student, "layers") else 4,
        )
        self.teacher.load_state_dict(student.state_dict(), strict=False)
        for p in self.teacher.parameters():
            p.requires_grad_(False)
        self.tau_0, self.tau_end, self.n_updates = tau_0, tau_end, n_updates
        self.step_count = 0

    def tau(self) -> float:
        """Cosine-annealed momentum at the current step."""
        n, N = self.step_count, self.n_updates
        return self.tau_end - (self.tau_end - self.tau_0) * (math.cos(math.pi * n / N) + 1) / 2

    @torch.no_grad()
    def update(self, student: nn.Module) -> float:
        tau = self.tau()
        for pt, ps in zip(self.teacher.parameters(), student.parameters()):
            pt.mul_(tau).add_(ps.detach(), alpha=1.0 - tau)
        self.step_count += 1
        return tau


def contextualized_targets(hiddens: list[torch.Tensor], top_k: int = 2) -> torch.Tensor:
    """Average the top-K teacher layer outputs (each instance-normalized).

    Averaging multiple top layers makes the targets contextualized: they carry
    information aggregated over the whole input, not just local features.
    """
    top = hiddens[-top_k:]
    stacked = torch.stack(top, dim=0)                       # (K, B, T, D)
    mean = stacked.mean(dim=(1, 2), keepdim=True)
    std = stacked.std(dim=(1, 2), keepdim=True).clamp_min(1e-6)
    return ((stacked - mean) / std).mean(dim=0)             # (B, T, D)


def smooth_l1_loss(pred: torch.Tensor, target: torch.Tensor, beta: float = 1.0) -> torch.Tensor:
    """Smooth L1 (Huber-style) regression loss, the data2vec prediction loss."""
```

```
    return F.smooth_l1_loss(pred, target, beta=beta)


def ema_closed_form(tau: float, history_len: int) -> torch.Tensor:
    """Closed form: teacher_T = (1-tau) * sum_{k=0}^{T-1} tau^k
       student_{T-k}
    (+ tau^T * teacher_0). Returns the weight assigned to each past
       student
    checkpoint, oldest first, demonstrating the exponential decay
       explicitly.
    """
    weights = torch.tensor([(1.0 - tau) * tau ** k for k in
       range(history_len)][::-1])
    return weights


if __name__ == "__main__":
    torch.manual_seed(0)
    B, T, D = 2, 32, 64

    student = TinyTransformerEncoder(dim=D, layers=4)
    ema = EMATeacher(student, tau_0=0.9, tau_end=0.99, n_updates=100)

    x = torch.randn(B, T, D)
    masked_x = x.clone()
    masked_x[:, 8:16, :] = 0.0  # masked view for the student

    # teacher sees the full input; student sees the masked view
    with torch.no_grad():
        target = contextualized_targets(ema.teacher(x), top_k=2)
    pred_hiddens = student(masked_x)
    pred = pred_hiddens[-1]
    loss = smooth_l1_loss(pred, target)
    print(f"smooth-L1 student->teacher loss: {loss.item():.4f}")
    assert loss.item() > 0.0

    # EMA update moves the teacher a (1 - tau) step toward the student
    before = [p.clone() for p in ema.teacher.parameters()]
    tau_used = ema.update(student)
    shift = sum((a - b).abs().sum().item() for a, b in
       zip(ema.teacher.parameters(), before))
    print(f"tau={tau_used:.4f}; teacher moved by total |delta| =
       {shift:.4f}")
    assert shift > 0.0
    # teacher parameters carry no gradient: the target is a constant
    assert not any(p.requires_grad for p in ema.teacher.parameters())

    # closed-form decay weights: recent students dominate, old ones
       fade
    w = ema_closed_form(0.9, 10)
    print("EMA weights (oldest -> newest):", [f"{v:.3f}" for v in
       w.tolist()])
    print(f"sum of weights: {w.sum().item():.4f} (+ tau^10 * init =
       {0.9 ** 10:.4f})")
    assert abs(w.sum().item() + 0.9 ** 10 - 1.0) < 1e-6
    print("OK")
```

3.4 Transfer Learning to AMT

Pretraining produces an encoder f_θ whose features linearize the acoustic factors of sound. Transfer learning converts those features into transcription decisions. This section formalizes the three transfer protocols, derives the downstream heads for pitch, onset, offset, and velocity, and works a complete dimension-flow example.

Step 1 — Physical analogy. A conservatory graduate has spent years training her ear on unlabelled music: intervals, timbres, rhythms. Asked to transcribe a new piece, she does not relearn hearing from scratch; she attaches a small new skill — writing down what she hears — to the ear she already has. Linear probing is asking her to transcribe while forbidding any further ear training; fine-tuning lets her ear keep adapting to the new repertoire.

Step 2 — Geometric mental model. Pretraining arranges audio frames in feature space so that frames containing the same pitch, the same onset transient, or the same silence cluster together. A downstream head is then a set of linear boundaries (or one boundary per pitch) drawn through that space. If the SSL objective did its job, these boundaries are nearly linear and a frozen encoder suffices; if the pretraining domain mismatches the transcription domain (speech features applied to piano), the boundaries must bend, and unfreezing the encoder lets the whole space warp. Figure 3.9 shows the worked dimension flow of this section.

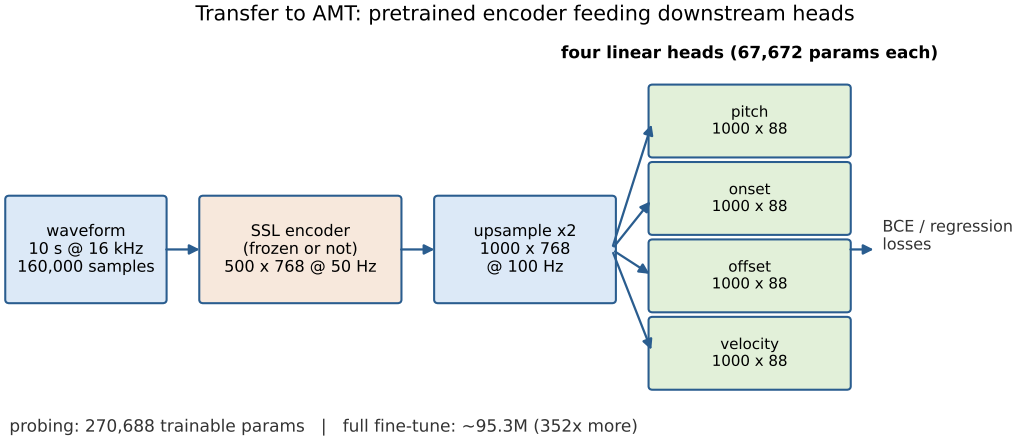


Figure 3.9: Transfer dimension flow for 10 s of audio. A wav2vec 2.0-scale encoder emits 500×768 features at 50 Hz, upsampled to the 100 fps AMT grid (1000×768); four linear heads each emit 1000×88 predictions. Probing trains 270,688 parameters; full fine-tuning trains ~95.3M (352× more).

Step 3 — First-principles derivation. Write the model as $g_\phi \circ f_\theta$: encoder f_θ with parameters θ, head g_ϕ with parameters ϕ.

Protocols. **Linear probing** freezes θ (no gradient flows into the encoder, $\partial \mathcal{L}/\partial \theta \equiv 0$ by construction since the parameters are excluded from the optimizer) and minimizes $\mathcal{L}_{\text{sup}}(\phi)$ over the head alone; it measures how linearly accessible the transcription factors are in the pretrained features. **Head fine-tuning** uses the same parameter budget but a different optimization recipe (e.g. layer-wise learning-rate decay on the head). **Full fine-tuning** minimizes over (θ, ϕ) jointly, usually with a smaller learning rate on θ (discriminative fine-tuning) so the pretrained features are refined rather than destroyed. The parameter budgets are derived in the worked example below.

Downstream heads. Let $h_t \in \mathbb{R}^{768}$ be the encoder feature at AMT frame t. Four linear heads produce the quantities a transcription system needs (cf. the Onsets-and-Frames decomposition [27]):

$$\hat{y}_t^{\text{pitch}} = \sigma(W_p h_t + b_p) \qquad \text{frame-wise multi-pitch, } \hat{y} \in [0,1]^{88}, \tag{3.27}$$

$$\hat{y}_t^{\text{onset}} = \sigma(W_{\text{on}} h_t + b_{\text{on}}) \qquad \text{onset detector,} \tag{3.28}$$

$$\hat{y}_t^{\text{off}} = \sigma(W_{\text{off}} h_t + b_{\text{off}}) \qquad \text{offset detector,} \tag{3.29}$$

$$\hat{v}_t = \sigma(W_v h_t + b_v) \qquad \text{velocity in } [0,1]. \tag{3.30}$$

The pitch/onset/offset heads train with binary cross-entropy against the frame-aligned targets,

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{T \cdot 88} \sum_{t=1}^{T} \sum_{p=1}^{88} \big[y_{t,p} \log \hat{y}_{t,p} + (1 - y_{t,p}) \log(1 - \hat{y}_{t,p}) \big], \tag{3.31}$$

and velocity with MSE on sounding frames. Note the class imbalance: with mean polyphony $\sim$2 of 88 pitches active, positives are $\sim$2.3% of labels; in practice (3.31) is reweighted (positive weights or focal loss, derived in Chapter 4).

Worked dimension flow. Take a wav2vec 2.0-base-scale encoder: 16 kHz audio, 320$\times$ downsampling, one 768-d vector per 20 ms (50 Hz). For 10 s of audio the encoder emits

$$T_{\text{enc}} = 10 \times 50 = 500 \text{ frames}, \qquad H \in \mathbb{R}^{500 \times 768}. \tag{3.32}$$

AMT evaluation uses a 10 ms grid (100 fps), five times finer than the $\pm$50 ms onset tolerance, so the features are upsampled by exactly 2:

$$H' = \text{upsample}_{\times 2}(H) \in \mathbb{R}^{1000 \times 768}. \tag{3.33}$$

Each head is a $768 \to 88$ linear map with

$$768 \times 88 + 88 = 67{,}672 \text{ parameters}, \tag{3.34}$$

so the four heads total $4 \times 67{,}672 = 270{,}688$ trainable parameters under probing. Full fine-tuning additionally trains the $\sim$95M-parameter backbone (95,000,000 + 270,688 total), a factor of

$$\frac{95{,}270{,}688}{270{,}688} = 352\times \tag{3.35}$$

more parameters than probing. The practical consequence: probing trains in minutes on a single GPU and diagnoses feature quality, while full fine-tuning adapts the features to the transcription domain at the cost of the full optimization budget and the risk of catastrophic forgetting of the pretrained structure when labeled data is scarce.

Step 4 — Complete runnable code. `code/m3_amt_heads.py` implements the upsampler, the four heads (3.27)–(3.30), the parameter-budget comparison (3.35), and a sparse-target BCE sanity check. Every shape in the dimension flow is asserted.

```
"""Module 3, Section 3.4: transfer-learning heads from a
    frozen/fine-tuned SSL encoder.

Dimension-flow example: a wav2vec-2.0-style encoder emits one 768-d
    vector per
20 ms frame (50 Hz). A temporal upsampler aligns the stream to the 100
    fps
frame grid used by AMT evaluation, then four probing heads predict the
quantities an AMT system needs: frame-wise pitch activity, note onsets,
```

```
note offsets, and velocity. The script also compares the trainable parameter
budget of probing, frozen-backbone fine-tuning of the heads, and full
fine-tuning of the encoder.
"""
from __future__ import annotations

from dataclasses import dataclass

import torch
import torch.nn as nn
import torch.nn.functional as F


ENC_DIM = 768           # encoder hidden size (wav2vec 2.0 base / HuBERT base scale)
ENC_HZ = 50.0           # encoder frame rate: one vector per 20 ms
TARGET_HZ = 100.0       # AMT frame grid: 10 ms hop (mir_eval onset tolerance +/-50 ms)
N_PITCH = 88            # piano range A0..C8


class UpsampleToFrameGrid(nn.Module):
    """Nearest/linear upsampling of encoder frames to the AMT frame grid."""

    def __init__(self, enc_hz: float = ENC_HZ, target_hz: float = TARGET_HZ) -> None:
        super().__init__()
        self.factor = target_hz / enc_hz
        assert self.factor == int(self.factor), "frame grids must divide evenly here"

    def forward(self, h: torch.Tensor) -> torch.Tensor:
        # h: (B, T_enc, D) -> (B, T_enc*factor, D)
        b, t, d = h.shape
        h = h.transpose(1, 2).unsqueeze(-1)                    # (B, D, T, 1)
        up = F.interpolate(h, scale_factor=(int(self.factor), 1), mode="nearest")
        return up.squeeze(-1).transpose(1, 2)                  # (B, T*factor, D)


class AMTProbingHeads(nn.Module):
    """Four linear heads on top of (frozen or fine-tuned) SSL encoder features."""

    def __init__(self, enc_dim: int = ENC_DIM, n_pitch: int = N_PITCH) -> None:
        super().__init__()
        self.upsample = UpsampleToFrameGrid()
        self.pitch_head = nn.Linear(enc_dim, n_pitch)      # frame-wise multi-pitch
        self.onset_head = nn.Linear(enc_dim, n_pitch)      # note-onset detector
        self.offset_head = nn.Linear(enc_dim, n_pitch)     # note-offset detector
```

```
        self.velocity_head = nn.Linear(enc_dim, n_pitch)  # MIDI velocity 0..1

    def forward(self, enc: torch.Tensor) -> dict[str, torch.Tensor]:
        """enc: (B, T_enc, D) -> dict of (B, T_grid, 88) head outputs."""
        h = self.upsample(enc)
        return {
            "pitch": torch.sigmoid(self.pitch_head(h)),
            "onset": torch.sigmoid(self.onset_head(h)),
            "offset": torch.sigmoid(self.offset_head(h)),
            "velocity": torch.sigmoid(self.velocity_head(h)),
        }

    def count_head_params(self) -> int:
        return sum(p.numel() for p in self.parameters() if p.requires_grad)


@dataclass
class TransferBudget:
    """Trainable parameter budget for the three transfer protocols."""
    backbone_params: int
    head_params: int

    def probing(self) -> int:
        """Linear probing: backbone frozen, only heads train."""
        return self.head_params

    def head_finetune(self) -> int:
        """Same budget as probing; differs only in learning-rate schedule."""
        return self.head_params

    def full_finetune(self) -> int:
        """Full fine-tuning: every backbone parameter trains."""
        return self.backbone_params + self.head_params


def bce_with_logits_demo() -> None:
    """Sanity: binary cross-entropy on a sparse pitch target behaves."""
    torch.manual_seed(0)
    logits = torch.randn(4, 100, N_PITCH) * 2.0
    # sparse target: ~2 of 88 pitches active per frame (polyphonic density)
    target = (torch.rand(4, 100, N_PITCH) < 2.0 / N_PITCH).float()
    loss = F.binary_cross_entropy_with_logits(logits, target)
    print(f"sparse pitch BCE: {loss.item():.4f} "
          f"(target density {target.mean().item():.4f})")
    assert 0.0 < loss.item() < 5.0


if __name__ == "__main__":
    torch.manual_seed(0)
    seconds = 10
    t_enc = int(seconds * ENC_HZ)           # 500 encoder frames
    enc = torch.randn(2, t_enc, ENC_DIM)  # pretend pretrained SSL
```

```
        features

    heads = AMTProbingHeads()
    out = heads(enc)
    t_grid = int(seconds * TARGET_HZ)      # 1000 AMT frames
    for name, y in out.items():
        assert y.shape == (2, t_grid, N_PITCH), (name, y.shape)
        print(f"{name:>8} head output: {tuple(y.shape)}")

    head_params = heads.count_head_params()
    print(f"head trainable params: {head_params:,}")
    # each head: 768*88 + 88 = 67,672 ; four heads = 270,688
    assert head_params == 4 * (ENC_DIM * N_PITCH + N_PITCH)

    budget = TransferBudget(backbone_params=95_000_000,
        head_params=head_params)
    print(f"probing:       {budget.probing():>12,} trainable params")
    print(f"full finetune: {budget.full_finetune():>12,} trainable
        params")
    print(f"ratio: {budget.full_finetune() / budget.probing():.0f}x
        more params "
          f"when the backbone unfreezes")

    bce_with_logits_demo()
    print("OK")
```

3.5 Drills

1. **AST patch arithmetic.** A 6-second clip is featurized as in §3.2.1 (128 mel bins, 10 ms hop, 16×16 patches, stride 10). Compute the patch grid (G_f, G_t), the sequence length N, and the number of visible patches under 75% masking. If the encoder processes visible patches only, what fraction of full-sequence attention compute does it use?

2. **Block-masking design.** On the 12×99 grid, you want an expected masking ratio of 0.60 using 5×5 blocks. Derive the number of blocks K required, starting from (3.9). Explain in two sentences why block masking has a stronger inductive bias for onset detection than random uniform masking.

3. **Normalization gradient analysis.** Consider one patch with true pixels $y \in \mathbb{R}^P$ and prediction $\hat{y}$. The raw-MSE gradient is $2(\hat{y} - y)/M$; the normalized-MSE gradient is $2(\hat{y} - \tilde{y})/M$ with $\tilde{y}$ standardized. Suppose two patches A (loud, $\|y_A\| = 10\|y_B\|$) and B (quiet) have equal *relative* reconstruction error 10%. Compute the ratio of gradient norms $\|\nabla_{\hat{y}_A}\mathcal{L}\|/\|\nabla_{\hat{y}_B}\mathcal{L}\|$ under raw MSE and under normalized MSE, and state what the optimizer attends to in each case.

4. **Gumbel-Max proof.** Prove Lemma 3.2 for the two-category case ($V = 2$) directly, without the general integral: with $g_1, g_2 \overset{\text{iid}}{\sim} \text{Gumbel}(0, 1)$, show $P(\log \pi_1 + g_1 \geq \log \pi_2 + g_2) = \pi_1$.

5. **Diversity-loss floor.** For one codebook, the diversity loss is $\mathcal{L}_d = \frac{1}{V}\sum_{v=1}^{V} \bar{p}_v \log \bar{p}_v$ with $\bar{p}$ a distribution over V entries. Prove the minimum value is $-\log V/V$, attained uniquely at the uniform distribution. Evaluate it for $V = 320$ and explain what "codebook collapse" looks like in terms of $\bar{p}$.

6. **HuBERT masked-only loss.** Explain, with a concrete mechanism, why computing the HuBERT loss (3.22) on *unmasked* frames admits a degenerate solution when cluster targets are noisy, while restricting it to masked frames forces the model to learn sequential structure. Your answer should name the degenerate mapping explicitly.

7. **EMA memory.** From (3.26) with constant τ, derive the half-life $k_{1/2} = \log(1/2)/\log\tau$ and evaluate it for $\tau = 0.999$ and $\tau = 0.9999$. A student checkpoint from 20,000 updates ago carries what fraction of its original weight under $\tau = 0.9999$? Explain in one paragraph how this lag contributes to collapse avoidance.

8. **Transfer budget.** An SSL encoder emits 60 frames/second of 512-d features. You upsample to 100 fps and attach pitch, onset, and offset heads over 88 pitches (no velocity head). Compute (a) the upsampling factor, (b) the output tensor shape for 8 s of audio (batch 1), (c) the total probing parameter count, and (d) the probing-to-full ratio if the backbone has 30M parameters.

Chapter 4

Sequence-to-Sequence Audio Transformer Architectures

A transcription system faces a representational chasm. On one side sits the audio signal: a dense, continuous, high-rate stream of spectral frames in which a single piano chord smears energy across hundreds of frequency bins and dozens of time steps. On the other side sits the symbolic score: a sparse, discrete sequence of events — `NOTE_ON(60)`, `TIME_SHIFT(120)`, `NOTE_OFF(60)` — each one an exact, atomic decision. Chapter 3 built representations of the audio side; this chapter builds the machinery that crosses the chasm. The encoder must turn raw spectro-temporal structure into a rich continuous state H_{audio}; the decoder must turn that state into a discrete event sequence Y_{symbolic}; and the loss must survive the brutal class imbalance of real music, where silence outnumbers onsets a thousand to one.

4.1 Spectro-Temporal Audio Encoders: The Conformer

The original Transformer encoder [32] treats every frame as a token and lets multi-head self-attention (MHSA) mix information globally. That is the right tool for *global harmonic structure*: the fact that the C_4 partial at 1046.5 Hz and the C_5 fundamental at 1046.5 Hz are the same frequency, or that a repeated motif in bar 32 echoes bar 4, is a content-based, long-range relation that attention captures in a single layer. It is the wrong tool, used alone, for *local time-frequency edges*: a note onset is a 5–20 ms transient, a near-vertical edge in the spectrogram, and forcing attention to rediscover locality from scratch in every head wastes capacity and data. The Conformer [28] keeps both branches: a convolution module that specializes in local spectro-temporal texture, interleaved with MHSA that specializes in global content-based relations.

Step 1 — Physical analogy. Think of a recording engineer and a musicologist analyzing the same take. The engineer zooms into the waveform at millisecond resolution, hunting for the exact sample where the hammer strikes the string — a purely local operation, translation-equivariant: a strike looks the same wherever it lands in time. The musicologist listens to the whole movement, connecting a bass line in the exposition to its return in the recapitulation — a global, content-addressed operation. A Conformer block is both analysts working on the same desk: the convolution module is the engineer's loupe, the attention module the musicologist's memory, and the residual wiring lets each correct the other layer after layer.

Step 2 — Geometric mental model. Picture the $T \times d$ activation grid of one block. The convolution module drags a narrow stencil of width K frames (e.g. 15 frames = 150 ms at 100 fps) along the time axis, mixing only neighbors: its receptive field is a thin vertical strip. MHSA instead draws, for each frame, a weighted star connecting it to *every* other frame, with

weights set by content similarity: its receptive field is the entire grid in one hop. The Macaron layout — half-step feed-forward, attention, convolution, half-step feed-forward — sandwiches the two mixing operations between two channel-wise nonlinearities, so local texture is expanded into a rich per-frame feature before global mixing, and refined again after. Figure 4.1 shows the wiring.

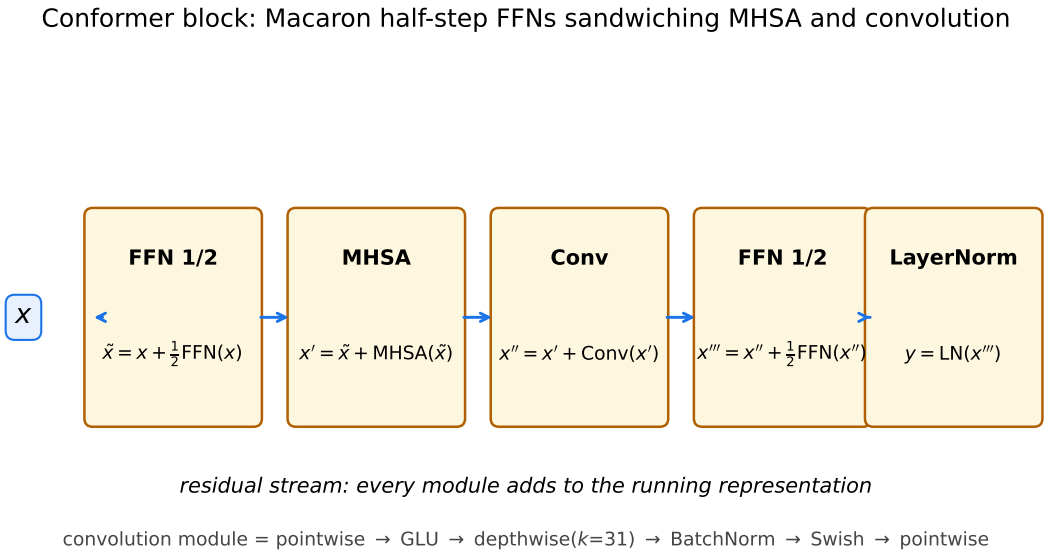


Figure 4.1: One Conformer encoder block. Two Macaron-style half-step feed-forward modules sandwich multi-head self-attention and the depthwise-separable convolution module. Every module is a pre-norm residual unit; the final LayerNorm closes the block.

Step 3 — First-principles derivation. Let the block input be $x_i \in \mathbb{R}^{T\times d}$. Each module is a pre-norm residual unit: normalize first, transform, add back. The two feed-forward modules each contribute only a *half* step, following the Strang–Marchuk splitting of Macaron-Net [33], which treats one full position-wise update as two symmetric half updates around the mixing operations. Dropout is applied after each module; we omit it from the equations for legibility. The forward pass is, equation by equation,

$$\tilde{x}_i = x_i + \tfrac{1}{2}\,\mathrm{FFN}\big(\mathrm{LN}(x_i)\big) \tag{4.1}$$
$$x'_i = \tilde{x}_i + \mathrm{MHSA}\big(\mathrm{LN}(\tilde{x}_i)\big) \tag{4.2}$$
$$x''_i = x'_i + \mathrm{Conv}\big(\mathrm{LN}(x'_i)\big) \tag{4.3}$$
$$y_i = \mathrm{LN}\big(x''_i + \tfrac{1}{2}\,\mathrm{FFN}\big(\mathrm{LN}(x''_i)\big)\big)\,. \tag{4.4}$$

The feed-forward module is position-wise with expansion factor 4:

$$\mathrm{FFN}(z) = W_2\,\big(\mathrm{Swish}(W_1 z + b_1)\big) + b_2, \qquad W_1 \in \mathbb{R}^{4d\times d},\ W_2 \in \mathbb{R}^{d\times 4d}, \tag{4.5}$$

with $\mathrm{Swish}(u) = u\,\sigma(u)$. The half-step residual $x + \frac{1}{2}\mathrm{FFN}(\cdot)$ is the Macaron signature: two half steps of the "diffusion" (FFN) bracket the "convection" (attention), which the ODE analysis of [33] shows reduces the local truncation error of the layer-as-integrator from first to second order.

Multi-head self-attention with h heads and $d_k = d/h$ is the standard construction [32],

written per head j:

$$\text{head}_j = \text{softmax}\left(\frac{(XW_j^Q)(XW_j^K)^\top}{\sqrt{d_k}}\right)(XW_j^V), \qquad \text{MHSA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O, \tag{4.6}$$

with $W_j^Q, W_j^K, W_j^V \in \mathbb{R}^{d\times d_k}$ and $W^O \in \mathbb{R}^{d\times d}$. (The original Conformer used relative positional biases inside this module; in our AMT stack the positional signal is supplied by the encodings of Section 4.2, including RoPE.)

The convolution module factorizes local mixing into a depthwise temporal filter plus pointwise channel mixing. With input $z \in \mathbb{R}^{T\times d}$ (already layer-normalized by the block wiring):

$$z_1 = \text{PWConv}_{d\to 2d}(z) \in \mathbb{R}^{T\times 2d} \tag{4.7}$$

$$(a, b) = \text{split}(z_1), \qquad g = a \odot \sigma(b) \in \mathbb{R}^{T\times d} \tag{4.8}$$

$$c = \text{DepthwiseConv1D}_K(g) \in \mathbb{R}^{T\times d} \tag{4.9}$$

$$n = \text{BatchNorm}(c), \qquad s = n \odot \sigma(n) \quad \text{(Swish)} \tag{4.10}$$

$$\text{Conv}(z) = \text{PWConv}_{d\to d}(s) \in \mathbb{R}^{T\times d}. \tag{4.11}$$

Equation (4.8) is a gated linear unit: the second half of the channels gates the first, letting the network suppress or pass local texture per channel. Equation (4.9) is the depthwise convolution: each channel is filtered along time independently with its own length-K kernel (groups $= d$), which is what makes the module cheap, as derived next.

A concrete parameter budget grounds the design. Take $d = 256$, $h = 4$, $d_{\text{ff}} = 1024$, $K = 15$: the convolution module holds 131,584 (pointwise $d \to 2d$) + 4,096 (depthwise) + 512 (BatchNorm) + 65,792 (pointwise $d \to d$) = 201,984 parameters; each FFN module holds 525,568, so the two Macaron halves cost 1,051,136; MHSA holds 263,168; with five LayerNorms the block totals 1,518,848 $\approx$ 1.52M parameters. For $T = 400$ frames (4 s at 100 fps) the multiply–accumulate (MAC) budget is 186.78M for MHSA (78.64M QKV projections, 40.96M scores, 40.96M attention–value products, 26.21M output projection), 419.43M for the two FFNs, and 80.18M for the convolution module (52.43M pointwise expansion, 1.54M depthwise, 26.21M pointwise projection) — 686.39M MACs, i.e. 1.373 GFLOPs per block. All counts recomputed in the verification script; biases included.

Step 4 — Implementation. The listing below is `code/m4_conformer_block.py`, verified by compilation. It follows equations (4.1)–(4.4) and (4.7)–(4.11) exactly, including the half-step residuals and the pre-norm placement.

```python
"""Conformer block for spectro-temporal audio encoding (Module 4).

Implements the convolution-augmented Transformer block of Gulati et al. 2020:
Macaron-style half-step feed-forward modules sandwiching multi-head
self-attention and a depthwise-separable convolution module, all wired with
pre-norm residual connections.

Shapes: (B, T, d) with B batch, T time frames, d model width.
"""
import math
import torch
import torch.nn as nn


class Swish(nn.Module):
```

```
    def forward(self, x):
        return x * torch.sigmoid(x)


class FeedForwardModule(nn.Module):
    """Position-wise FFN, d -> 4d -> d with Swish. Used with a 1/2 residual
    scale (Macaron style: each of the two FFNs contributes a half step)."""

    def __init__(self, d_model, expansion=4, dropout=0.1):
        super().__init__()
        d_ff = expansion * d_model
        self.net = nn.Sequential(
            nn.Linear(d_model, d_ff),
            Swish(),
            nn.Dropout(dropout),
            nn.Linear(d_ff, d_model),
        )

    def forward(self, x):
        return self.net(x)


class ConvolutionModule(nn.Module):
    """Local time-frequency feature extractor.

    pointwise(d -> 2d) -> GLU(-> d) -> depthwise conv1d(K) -> BatchNorm
    -> Swish -> pointwise(d -> d). The depthwise + pointwise split replaces a
    dense K*d*d convolution with K*d + d*d parameters.
    """

    def __init__(self, d_model, kernel_size=15, dropout=0.1):
        super().__init__()
        assert kernel_size % 2 == 1, "use odd kernel for symmetric padding"
        pad = kernel_size // 2
        self.pw1 = nn.Conv1d(d_model, 2 * d_model, kernel_size=1)
        self.dw = nn.Conv1d(d_model, d_model, kernel_size=kernel_size,
                            padding=pad, groups=d_model)  # depthwise
        self.bn = nn.BatchNorm1d(d_model)
        self.act = Swish()
        self.pw2 = nn.Conv1d(d_model, d_model, kernel_size=1)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        # x: (B, T, d) -> conv1d wants (B, d, T)
        h = x.transpose(1, 2)
        h = self.pw1(h)                          # (B, 2d, T)
        a, b = h.chunk(2, dim=1)                 # split channels
        h = a * torch.sigmoid(b)                 # GLU -> (B, d, T)
        h = self.dw(h)                           # depthwise local mixing over time
        h = self.bn(h)
        h = self.act(h)
        h = self.pw2(h)                          # channel mixing -> (B, d, T)
```

```
        return self.dropout(h.transpose(1, 2))


class MultiHeadSelfAttention(nn.Module):
    """Standard scaled dot-product multi-head self-attention (no PE here;
    positional information is injected by the caller, e.g. RoPE on Q/K)."""

    def __init__(self, d_model, n_heads, dropout=0.1):
        super().__init__()
        assert d_model % n_heads == 0
        self.n_heads = n_heads
        self.d_k = d_model // n_heads
        self.qkv = nn.Linear(d_model, 3 * d_model)
        self.out = nn.Linear(d_model, d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        B, T, d = x.shape
        q, k, v = self.qkv(x).chunk(3, dim=-1)
        q = q.view(B, T, self.n_heads, self.d_k).transpose(1, 2)
        k = k.view(B, T, self.n_heads, self.d_k).transpose(1, 2)
        v = v.view(B, T, self.n_heads, self.d_k).transpose(1, 2)
        scores = q @ k.transpose(-2, -1) / math.sqrt(self.d_k)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, float("-inf"))
        attn = torch.softmax(scores, dim=-1)
        out = (self.dropout(attn) @ v).transpose(1, 2).reshape(B, T, d)
        return self.out(out)


class ConformerBlock(nn.Module):
    """One Conformer encoder block (Gulati et al. 2020, Fig. 1).

    x~  = x  + 1/2 * FFN(LN(x))         (Macaron half step)
    x'  = x~ + MHSA(LN(x~))             (global harmonic context)
    x'' = x' + Conv(LN(x'))             (local time-frequency edges)
    y   = LN(x'' + 1/2 * FFN(LN(x'')))  (Macaron half step + final norm)
    """

    def __init__(self, d_model=256, n_heads=4, kernel_size=15,
                 dropout=0.1):
        super().__init__()
        self.ffn1 = FeedForwardModule(d_model, dropout=dropout)
        self.ffn2 = FeedForwardModule(d_model, dropout=dropout)
        self.mhsa = MultiHeadSelfAttention(d_model, n_heads,
                                           dropout=dropout)
        self.conv = ConvolutionModule(d_model, kernel_size,
                                      dropout=dropout)
        self.norms = nn.ModuleList([nn.LayerNorm(d_model) for _ in
                                    range(5)])

    def forward(self, x, mask=None):
        x = x + 0.5 * self.ffn1(self.norms[0](x))
        x = x + self.mhsa(self.norms[1](x), mask)
        x = x + self.conv(self.norms[2](x))
        x = x + 0.5 * self.ffn2(self.norms[3](x))
```

```
        return self.norms[4](x)


def main():
    torch.manual_seed(0)
    B, T, d = 2, 400, 256           # 4 s of audio at 100 fps, d_model=256
    x = torch.randn(B, T, d)
    block = ConformerBlock(d_model=d, n_heads=4, kernel_size=15)
    y = block(x)
    n_params = sum(p.numel() for p in block.parameters())
    print(f"input {tuple(x.shape)} -> output {tuple(y.shape)}")
    print(f"block parameters: {n_params:,}")
    for name, mod in [("ffn1", block.ffn1), ("mhsa", block.mhsa), ("conv", block.conv)]:
        n = sum(p.numel() for p in mod.parameters())
        print(f"  {name}: {n:,} params")


if __name__ == "__main__":
    main()
```

4.1.1 Depthwise separable convolutions: the parameter arithmetic

The convolution module above never materializes a dense $K \times d \times d$ kernel. This subsection derives exactly what the factorization saves, because the saving is what makes per-block convolution affordable inside a deep audio encoder.

Step 1 — Physical analogy. A dense convolution is a committee where every output channel consults every input channel at every tap of the kernel: d^2 pairwise conversations per tap. A depthwise separable convolution splits the committee's labor the way a recording session splits roles. First, each channel listens to *itself* across time (the depthwise pass: the drummer keeps their own time). Then a single mixing desk combines the channels at each instant (the pointwise pass: the producer blends the tracks). Temporal pattern-matching and cross-channel blending are different jobs; doing them jointly is overkill, and the arithmetic proves it.

Step 2 — Geometric mental model. A dense 1D convolution kernel is a 3-tensor of shape (d, d, K): a full cube of weights. The separable factorization replaces the cube with a diagonal slab plus a flat mixing sheet: the depthwise kernel is $(d, 1, K)$ — each output channel sees exactly one input channel, a diagonal slice through the cube — and the pointwise kernel is $(d, d, 1)$, a $d \times d$ matrix applied independently per frame. Figure 4.2 quantifies the difference for two realistic configurations.

Step 3 — First-principles derivation. Fix $C_{\text{in}} = C_{\text{out}} = d$ and kernel length K, biases excluded (they add d terms to both sides and do not change the ratio materially). A standard 1D convolution holds one K-tap filter per input–output channel pair:

$$P_{\text{std}} = K \cdot d \cdot d. \tag{4.12}$$

The separable version holds one K-tap filter per channel (depthwise) plus one $d \times d$ mixing matrix (pointwise):

$$P_{\text{sep}} = K \cdot d + d \cdot d. \tag{4.13}$$

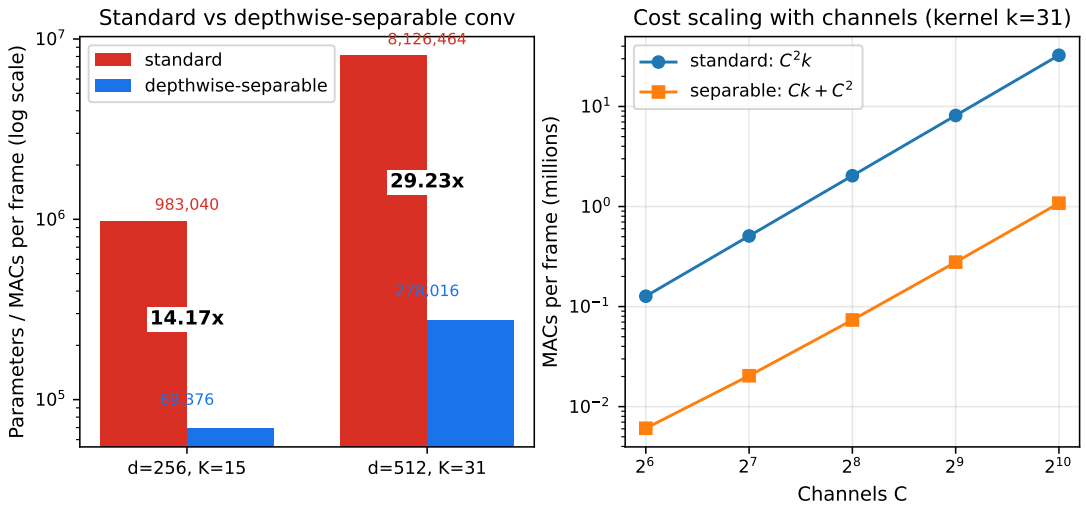


Figure 4.2: Standard vs. depthwise-separable convolution: parameter counts (log scale, left) and MACs-per-frame scaling with channels (right). The saving factor is $dK/(d+K)$: 14.17× at $(d, K) = (256, 15)$ and 29.23× at $(512, 31)$, matching (4.14).

Dividing (4.12) by (4.13) and factoring Kd^2:

$$\frac{P_{\text{std}}}{P_{\text{sep}}} = \frac{Kd^2}{Kd + d^2} = \frac{1}{\frac{1}{d} + \frac{1}{K}} = \frac{dK}{d+K}. \tag{4.14}$$

FLOPs scale identically: a multiply–accumulate per weight per output frame, so the FLOP ratio equals the parameter ratio. Worked example one, the Conformer configuration $(d, K) = (256, 15)$: $P_{\text{std}} = 15 \cdot 256^2 = 983{,}040$ versus $P_{\text{sep}} = 15 \cdot 256 + 256^2 = 3{,}840 + 65{,}536 = 69{,}376$, a factor of $983040/69376 = 14.1697$, i.e. a 92.94% saving. Per output frame the cost drops from $2 \cdot 983{,}040 = 1{,}966{,}080$ to $2 \cdot 69{,}376 = 138{,}752$ FLOPs; over $T = 400$ frames that is 786.43 MFLOPs versus 55.50 MFLOPs. Worked example two, a wider stack $(d, K) = (512, 31)$: 8,126,464 versus 278,016, a factor of 29.2302, a 96.58% saving. Equation (4.14) shows the regime: the saving approaches K when $d \gg K$ and approaches d when $K \gg d$; audio encoders live in the former, so the kernel length sets the saving.

Why both branches, then, if convolution is this cheap? Because cheap locality and global content routing solve different subproblems of transcription. The depthwise kernel's receptive field after one layer is K frames (150 ms for $K = 15$ at 100 fps): enough to resolve an onset transient, a vibrato cycle, or a formant edge, all of which are translation-equivariant — a hammer strike looks identical at frame 100 and frame 10 000. Global harmonic structure is not translation-equivariant in this sense: whether frame t harmonically supports frame t' depends on their *content* (are these partials in a harmonic ratio? is this the recapitulation of that motif?), and the relation can span the whole recording. MHSA resolves any such pair in one layer at $O(T^2 d)$ cost; convolution would need $\log_K T$ stacked layers to span the same distance. The Conformer therefore spends its cheap FLOP budget on the local branch and its expensive $O(T^2)$ budget only where content-based routing is indispensable. Figure 4.3 renders both signal types on a synthetic log-frequency spectrogram.

Step 4 — Implementation. The factorization is exactly the `ConvolutionModule` listed above: `self.dw` is the depthwise $(d, 1, K)$ slab (`groups=d_model`), `self.pw1`/`self.pw2` are the pointwise mixing sheets, and the GLU between them restores the expressive power that a purely linear split would lose. No separate listing is needed; the module *is* the implementation.

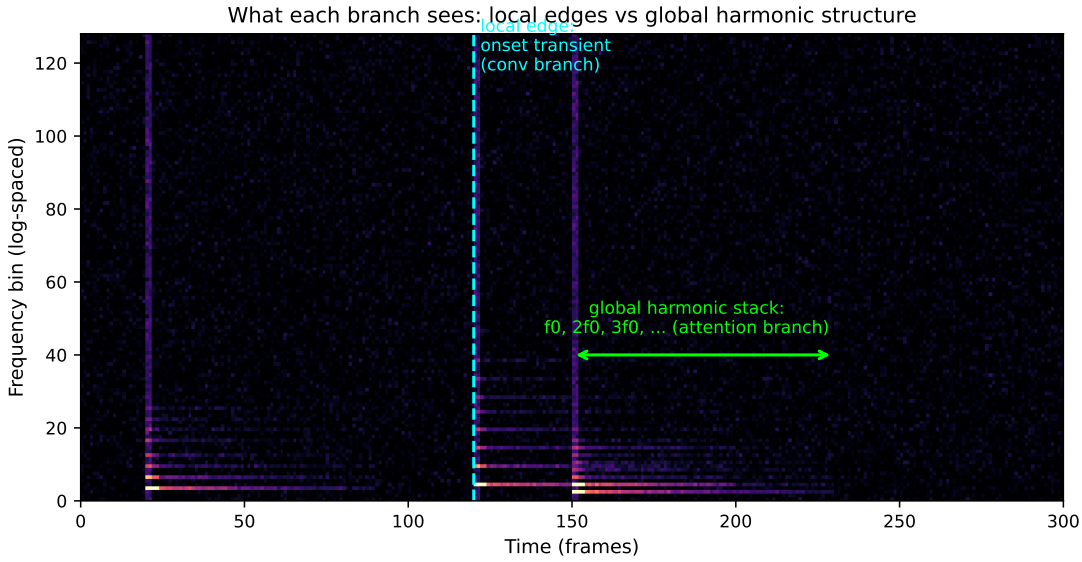


Figure 4.3: Local edges vs. global harmonic structure on a synthetic spectrogram. The cyan onset transient is a near-vertical broadband edge a few frames wide — the convolution branch's native prey. The lime harmonic stack $(f_0, 2f_0, 3f_0, \dots)$ is a long-range content relation spanning the note's full duration — the attention branch's. A transcription encoder needs both.

4.2 Positional Encodings for Continuous Audio Frames

Neither convolution-free attention nor the pointwise FFN knows which frame is which: permute the T frames and MHSA returns the permuted result. Audio is nothing *but* ordered time (and ordered frequency), so the positional signal must be injected deliberately. This section builds three encodings of increasing sophistication: the 1D temporal sinusoid, its 2D spectro-temporal separable extension, and the rotary embedding that turns absolute frame indices into relative geometry inside the attention score itself.

4.2.1 1D temporal sinusoidal encodings

Step 1 — Physical analogy. A pipe organ's stops are tuned in geometric progression: each rank sounds at double the frequency of the last, so that low ranks mark coarse position along a phrase while high ranks mark fine position. The sinusoidal encoding [32] is the same idea rendered as a vector: dimension pair i oscillates at frequency $\omega_i = 10000^{-2i/d}$, a geometric ladder from $\omega_0 = 1$ rad/frame (period 2π frames) down to $\omega_{d/2-1} \approx 0$ (periods of thousands of frames). High-frequency pairs act as the fine stop, pinning down exact frame offsets; low-frequency pairs act as the coarse stop, disambiguating where in a minutes-long recording the frame sits.

Step 2 — Geometric mental model. Each frame's encoding is a point on a $(d/2)$-dimensional torus: every dimension pair $(\sin(m\omega_i), \cos(m\omega_i))$ is a hand on a clock face turning at its own speed ω_i. Advancing k frames rotates every hand by $k\omega_i$ — a rigid rotation of the whole point, with the rotation depending only on the *step* k, never on the absolute position m. That is the property the derivation below proves, and it is what lets attention read off relative distances from absolute codes. Figure 4.4 shows the codebook.

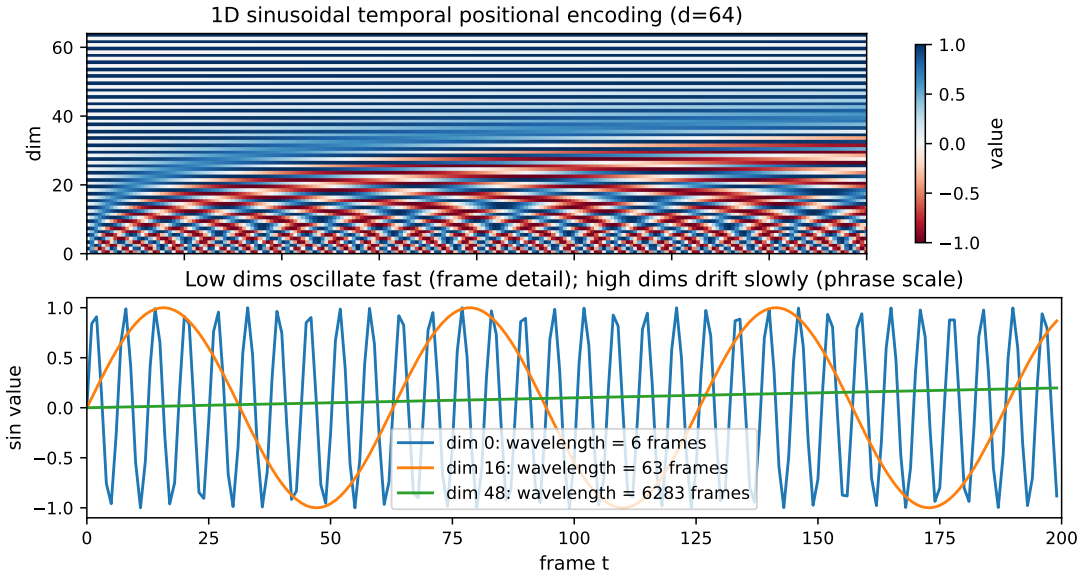


Figure 4.4: 1D sinusoidal temporal positional encoding ($d = 64$). Top: the full codebook over 200 frames — low dimensions oscillate rapidly while high dimensions drift slowly. Bottom: three dimension pairs on the geometric frequency ladder, from frame-scale (period ≈ 6.3 frames) to phrase-scale (period $\approx 6{,}283$ frames) wavelengths.

Step 3 — First-principles derivation. For position $m \in \mathbb{N}$ and dimension pair $i \in \{0, \ldots, d/2 - 1\}$:

$$\mathrm{PE}(m, 2i) = \sin(m\,\omega_i), \qquad \mathrm{PE}(m, 2i+1) = \cos(m\,\omega_i), \qquad \omega_i = 10000^{-2i/d}. \tag{4.15}$$

Fix an offset k and expand the shifted pair with the angle-addition identities, showing every step:

$$\sin\big((m+k)\omega_i\big) = \sin(m\omega_i)\cos(k\omega_i) + \cos(m\omega_i)\sin(k\omega_i), \tag{4.16}$$

$$\cos\big((m+k)\omega_i\big) = \cos(m\omega_i)\cos(k\omega_i) - \sin(m\omega_i)\sin(k\omega_i). \tag{4.17}$$

Collecting the pair as a column vector gives a 2×2 linear map:

$$\begin{bmatrix} \sin((m+k)\omega_i) \\ \cos((m+k)\omega_i) \end{bmatrix} = \underbrace{\begin{bmatrix} \cos(k\omega_i) & \sin(k\omega_i) \\ -\sin(k\omega_i) & \cos(k\omega_i) \end{bmatrix}}_{M_i(k)} \begin{bmatrix} \sin(m\omega_i) \\ \cos(m\omega_i) \end{bmatrix}. \tag{4.18}$$

Stacking over i yields a block-diagonal matrix $M(k) = \mathrm{blkdiag}(M_0(k), \ldots, M_{d/2-1}(k))$ depending only on k, with the exact relative-shift property

$$\mathrm{PE}(m+k) = M(k)\,\mathrm{PE}(m) \qquad \forall\, m, k. \tag{4.19}$$

A linear layer downstream can therefore learn to respond to the offset k without ever seeing the absolute position m: the geometry already factors position into a k-only rotation.

Worked numeric check with $d = 8$, $m = 10$. The frequencies are $\omega = (1.0,\ 0.1,\ 0.01,\ 0.001)$ and

$$\mathrm{PE}(10) = [-0.544021,\ -0.839072,\ 0.841471,\ 0.540302,\ 0.099833,\ 0.995004,\ 0.010000,\ 0.999950]. \tag{4.20}$$

Applying the $k = 3$ rotation $M(3)$ to this vector reproduces $\mathrm{PE}(13)$ to machine precision: $\|\mathrm{PE}(13) - M(3)\mathrm{PE}(10)\| = 1.36 \times 10^{-16}$, confirming (4.19) numerically.

Step 4 — Implementation. The listing below is `code/m4_pe2d.py`: `sinusoidal_1d` implements (4.15), `spectro_temporal_pe` builds the separable 2D grid of Section 4.2.2, and the demo numerically checks the separability identity (4.22).

```
"""1D temporal and 2D separable spectro-temporal positional encodings (Module 4).

2D construction: the first d/2 channels carry a 1D sinusoidal code of the time
frame t, the last d/2 channels a 1D sinusoidal code of the frequency bin f.
Concatenation makes the similarity additive:
  <PE(t1,f1), PE(t2,f2)> = <PE_t(t1),PE_t(t2)> + <PE_f(f1),PE_f(f2)>,
so time and frequency offsets contribute separably.
"""
import torch


def sinusoidal_1d(positions, dim):
    """positions: (N,) -> (N, dim). PE(pos,2i)=sin(pos/10000^{2i/d}), PE(pos,2i+1)=cos(pos/10000^{2i/d})."""
    assert dim % 2 == 0
    i = torch.arange(dim // 2).float()
    angles = positions.unsqueeze(-1) / (10000.0 ** (2 * i / dim))
    pe = torch.stack([torch.sin(angles), torch.cos(angles)], dim=-1)
    return pe.flatten(-2)


def spectro_temporal_pe(n_frames, n_bins, dim):
    """-> (n_frames, n_bins, dim): separable 2D positional encoding grid."""
    assert dim % 2 == 0
    t_code = sinusoidal_1d(torch.arange(n_frames).float(), dim // 2)  # (T, d/2)
    f_code = sinusoidal_1d(torch.arange(n_bins).float(), dim // 2)  # (F, d/2)
    pe = torch.cat([
        t_code.unsqueeze(1).expand(-1, n_bins, -1),
        f_code.unsqueeze(0).expand(n_frames, -1, -1),
    ], dim=-1)
    return pe


def main():
    torch.manual_seed(0)
    # 1D check: relative-shift linearity PE(13) == M(3) PE(10)
    pe10 = sinusoidal_1d(torch.tensor([10.0]), 8)
    pe13 = sinusoidal_1d(torch.tensor([13.0]), 8)
    print("PE(10) =", [f"{v:.6f}" for v in pe10[0].tolist()])
    # 2D separability check
    T, F, d = 48, 16, 32
    pe = spectro_temporal_pe(T, F, d)
    t2, f2 = 30, 10
    ref = pe[t2, f2]
    sim = (pe * ref).sum(-1)                          # (T, F) similarity field
    sim_t = (pe[:, f2, :d//2] * ref[:d//2]).sum(-1)   # time part only
    sim_f = (pe[t2, :, d//2:] * ref[d//2:]).sum(-1)   # freq part only
```

```
    recon = sim_t.unsqueeze(1) + sim_f.unsqueeze(0)
    print(f"2D grid {tuple(pe.shape)}; max|sim - (time+freq)| = "
          f"{float((sim - recon).abs().max()):.3e}")
    print(f"similarity at reference (30,10): {float(sim[t2, f2]):.4f} "
          f"(= d/2 = {d//2}: each of the {d//2} (sin,cos) pairs "
          f"contributes sin^2+cos^2 = 1)")


if __name__ == "__main__":
    main()
```

4.2.2 2D separable spectro-temporal encodings

Step 1 — Physical analogy. A spectrogram is a map with two axes: time runs east–west, frequency runs north–south. A 1D encoding stamps only the longitude on each frame, leaving the map's latitude blank — yet for transcription the latitude is where the notes live (which pitch bin a partial occupies). The 2D encoding stamps both coordinates, but crucially it does so *separably*: the time stamp and the frequency stamp are computed independently and concatenated, the way a map grid prints its easting in one margin and its northing in another.

Step 2 — Geometric mental model. Allocate the first $d/2$ channels to time and the last $d/2$ to frequency. The similarity between two spectrogram cells then splits into a time term plus a frequency term — moving east changes only the first summand, moving north only the second. Figure 4.5 shows the similarity field to a reference cell $(t, f) = (30, 10)$: the total field is visibly the sum of a horizontal time band and a vertical frequency band, which is the separability made visible.

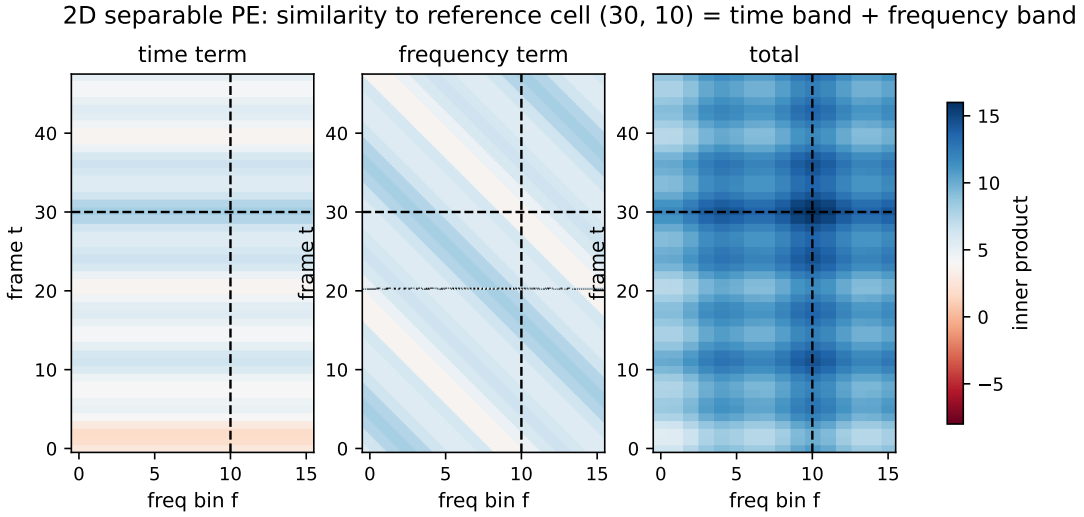


Figure 4.5: 2D separable positional encoding: inner-product similarity of every cell (t, f) of a 48×16 grid to the reference cell $(30, 10)$, decomposed into the time term, the frequency term, and their sum. Dashed lines mark the reference.

Step 3 — First-principles derivation. For a grid cell (t, f) with $t \in \{0, \dots, T-1\}$, $f \in \{0, \dots, F-1\}$, define with d even:

$$\mathrm{PE}_{\mathrm{2D}}(t, f) = \big[\, \mathrm{PE}_{\mathrm{time}}(t) \;;\; \mathrm{PE}_{\mathrm{freq}}(f) \,\big] \in \mathbb{R}^d, \tag{4.21}$$

where each half is a $d/2$-dimensional 1D sinusoidal code per (4.15) (optionally with different base frequencies for the two axes). The inner product of two cells expands, writing the halves explicitly:

$$\begin{aligned}\big\langle \mathrm{PE}_{\mathrm{2D}}(t_1,f_1), \mathrm{PE}_{\mathrm{2D}}(t_2,f_2)\big\rangle &= \sum_{j=1}^{d/2} \mathrm{PE}_{\mathrm{time}}(t_1)_j\, \mathrm{PE}_{\mathrm{time}}(t_2)_j + \sum_{j=1}^{d/2} \mathrm{PE}_{\mathrm{freq}}(f_1)_j\, \mathrm{PE}_{\mathrm{freq}}(f_2)_j \\ &= \big\langle \mathrm{PE}_{\mathrm{time}}(t_1), \mathrm{PE}_{\mathrm{time}}(t_2)\big\rangle + \big\langle \mathrm{PE}_{\mathrm{freq}}(f_1), \mathrm{PE}_{\mathrm{freq}}(f_2)\big\rangle. \qquad (4.22)\end{aligned}$$

The cross terms vanish by construction: concatenation, unlike addition, keeps the two coordinate systems orthogonal. Each summand inherits the relative-shift property (4.19), so the 2D code answers "how far apart in time" and "how far apart in frequency" independently — exactly the factorization a transcription model needs, since a semitone shift and a 50 ms shift are independent musical facts. At the reference cell itself the similarity attains its maximum $\langle \mathrm{PE}, \mathrm{PE}\rangle = d/2$ (here 16 for $d = 32$: each of the 16 $(\sin, \cos)$ pairs contributes $\sin^2 + \cos^2 = 1$), decaying smoothly away along each axis, as Figure 4.5 confirms.

Step 4 — Implementation. The 2D construction is the `spectro_temporal_pe` function in the full listing of `code/m4_pe2d.py` in Section 4.2.1: it encodes time into the first $d/2$ channels and frequency into the last $d/2$, then broadcasts both codes across the $T \times F$ grid by concatenation. The demo checks the separability (4.22) numerically: on a 48×16 grid the maximum deviation between the true similarity field and the time-plus-frequency reconstruction is 1.9×10^{-6} in float32 arithmetic (5.3×10^{-15} in float64), confirming that concatenation keeps the two coordinate systems orthogonal up to rounding error.

4.2.3 Rotary position embeddings for continuous audio frames

Additive encodings stamp position onto the token *before* attention; attention then treats the stamp as just another feature. Rotary position embeddings (RoPE) [29] instead rotate the queries and keys *inside* the attention computation, so that the score between two positions depends on their offset by construction, not by learned convention.

Step 1 — Physical analogy. Two musicians play the same phrase, one starting seven beats after the other — a canon. An additive encoding would label each note with its absolute beat number and hope the model learns to subtract. RoPE instead hands each musician a clock whose hands spin as the beats pass, and compares the two clocks: the absolute readings cancel, leaving only the seven-beat lag. For audio the "beats" are frames, and crucially the clock hands turn continuously, so fractional frame positions (sub-frame timing at $t = m \cdot \text{hop}$ seconds) work with no modification.

Step 2 — Geometric mental model. Take one 2D slice of the query vector — a single arrow in the plane. RoPE rotates that arrow by $m\theta$: after m frames the arrow has swept $m\theta$ radians. The key's arrow is rotated by $n\theta$. The angle between the two rotated arrows is $(n-m)\theta$ regardless of where each started: absolute position orients the arrows, relative position separates them, and the dot product sees only the separation. Figure 4.6 draws this for one pair; the full embedding applies independent rotations at $d/2$ different speeds θ_i, one per dimension pair.

Step 3 — First-principles derivation. Start in $d = 2$. Identify $q = (q_1, q_2)$ with the complex number $\tilde{q} = q_1 + iq_2$, and define the position-dependent map

$$f(q,m) = \tilde{q}\, e^{im\theta}, \qquad \text{i.e.} \quad f(q,m) = \begin{bmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{bmatrix} \begin{bmatrix} q_1 \\ q_2 \end{bmatrix}. \qquad (4.23)$$

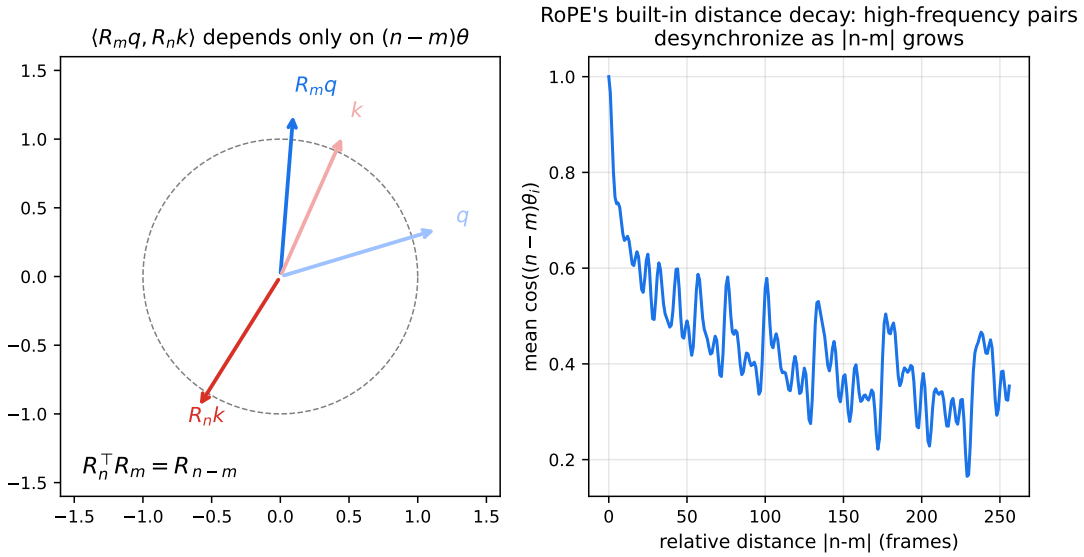


Figure 4.6: RoPE geometry in one 2D subspace (left). Rotating q by $m\theta$ and k by $n\theta$ leaves the relative angle $(n-m)\theta$ — and hence the inner product — dependent only on the offset. Right: the summed cosine kernel $\frac{2}{d}\sum_i \cos((n-m)\theta_i)$ over all pairs decays as $|n-m|$ grows, the built-in distance bias of (4.26): high-frequency pairs dephase first, so distant frames interact more weakly by construction.

For general even d, pair consecutive dimensions and rotate pair i at its own frequency $\theta_i = 10000^{-2(i-1)/d}$ for $i = 1, \ldots, d/2$. The rotation matrix is block-diagonal:

$$R_\Theta^m = \mathrm{blkdiag}\big(R(m\theta_1), \ldots, R(m\theta_{d/2})\big), \qquad R(\phi) = \begin{bmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{bmatrix}, \tag{4.24}$$

and the embedding is $f(x, m) = R_\Theta^m x$, applied to queries and keys before the dot product. The relative-position property is now a two-line proof. Since each block is a rotation, $(R_\Theta^m)^\top = R_\Theta^{-m}$, and block-diagonal rotations at fixed frequencies commute, so $(R_\Theta^m)^\top R_\Theta^n = R_\Theta^{n-m}$. Therefore

$$\begin{aligned} \big\langle R_\Theta^m q,\ R_\Theta^n k \big\rangle &= q^\top (R_\Theta^m)^\top R_\Theta^n k \\ &= q^\top R_\Theta^{n-m} k = \big\langle q,\ R_\Theta^{n-m} k \big\rangle =: g(q, k, n-m), \end{aligned} \tag{4.25}$$

a function of the content (q, k) and the offset $n - m$ only. Absolute positions enter the computation; only their difference survives in the score. No parameters are learned and no $T \times T$ bias table is materialized, which also keeps the door open to linear-attention variants.

Two consequences matter for audio. First, m need not be an integer: with frame hop h seconds, position $m = t/h$ is real-valued for any continuous time t, so sub-frame onset timing slots directly into the same formula. Second, expanding (4.25) in the complex pairs $\tilde{q}_j, \tilde{k}_j$ gives

$$\big\langle R_\Theta^m q,\ R_\Theta^n k \big\rangle = \Re \sum_{j=1}^{d/2} \tilde{q}_j\, \overline{\tilde{k}_j}\, e^{i(m-n)\theta_j}, \tag{4.26}$$

a sum of $d/2$ phasors at distinct frequencies θ_j. At zero offset all phasors align and the sum attains its full magnitude; as $|m-n|$ grows, phasors at different frequencies dephase and partially cancel, so the expected score decays with distance. This is a free inductive bias matched to acoustics: neighboring frames (the same note, the same chord) should interact more strongly than frames minutes apart, and RoPE bakes that decay into the geometry rather than forcing

the model to learn it. The right panel of Figure 4.6 plots the summed kernel against relative distance: the dephasing of the high-frequency pairs is visible as a decaying oscillation, steepest in the first few dozen frames.

The frequency ladder sets the time scales the model can resolve. With $d = 64$: $\theta_1 = 1.0$ rad/frame, period $2\pi \approx 6.28$ frames (62.8 ms at 100 fps, the scale of an onset transient); down to $\theta_{32} = 1.333521 \times 10^{-4}$ rad/frame, period 47,117 frames (471.2 s, the scale of a whole movement). Every scale between onset and movement is covered geometrically.

Worked numeric verification with the toy $d = 4$ ($\theta = (1, 0.01)$), $m = 7$, $n = 12$:

$$R_\Theta^7 = \begin{bmatrix} 0.753902 & -0.656987 & 0 & 0 \\ 0.656987 & 0.753902 & 0 & 0 \\ 0 & 0 & 0.997551 & -0.069943 \\ 0 & 0 & 0.069943 & 0.997551 \end{bmatrix}, \tag{4.27}$$

and for random q, k, $\langle R_\Theta^7 q, R_\Theta^{12} k\rangle = 0.8792812550$ equals $\langle q, R_\Theta^5 k\rangle = 0.8792812550$ to all printed digits, confirming (4.25) numerically.

Step 4 — Implementation. `code/m4_rope.py` implements the embedding and its use in attention scores; its demo verifies (4.25) and shows fractional positions preserving the vector norm.

```python
"""Rotary Position Embedding (RoPE) for continuous audio frames
    (Module 4).

Su et al. 2021: rotate each 2-dim block of Q/K by an angle m*theta_i
    that
depends on the absolute frame index m. The attention inner product then
depends only on the relative offset n - m.

For audio, m may be fractional (sub-frame time t = m * hop_seconds);
    the
construction is valid for any real m.
"""
import math
import torch
import torch.nn as nn


class RotaryEmbedding(nn.Module):
    def __init__(self, dim, base=10000.0):
        super().__init__()
        assert dim % 2 == 0
        self.dim = dim
        # theta_i = base ** (-2*(i-1)/d), i = 1..d/2
        theta = base ** (-2.0 * torch.arange(dim // 2).float() / dim)
        self.register_buffer("theta", theta)   # (d/2,)

    def _cos_sin(self, positions):
        """positions: (...,) frame indices (float ok). Returns cos/sin
            (..., d/2)."""
        angles = positions.unsqueeze(-1) * self.theta.unsqueeze(0)  #
            (..., d/2)
        return torch.cos(angles), torch.sin(angles)

    def rotate(self, x, positions):
        """Apply R^m_Theta to x: (..., T, dim) with positions (...,
            T)."""
```

```
        cos, sin = self._cos_sin(positions)           # (..., T, d/2)
        x1, x2 = x[..., 0::2], x[..., 1::2]           # even/odd pairs
        # [c -s; s c] [x1; x2]
        out1 = x1 * cos - x2 * sin
        out2 = x1 * sin + x2 * cos
        out = torch.stack([out1, out2], dim=-1)       # (..., T, d/2, 2)
        return out.flatten(-2)                       # (..., T, dim)


def rope_attention_scores(q, k, rope, pos_q, pos_k):
    """Attention logits with RoPE applied to Q and K.

    q: (B, H, S, d), k: (B, H, T, d); pos_q: (S,), pos_k: (T,).
    Returns (B, H, S, T) logits whose (s, t) entry depends on t - s only
    through the rotation (relative-position property).
    """
    qr = rope.rotate(q, pos_q.view(1, 1, -1).expand_as(q[..., 0]))
    kr = rope.rotate(k, pos_k.view(1, 1, -1).expand_as(k[..., 0]))
    return qr @ kr.transpose(-2, -1) / math.sqrt(q.shape[-1])


def main():
    torch.manual_seed(11)
    d = 4
    rope = RotaryEmbedding(d)
    q = torch.randn(d)
    k = torch.randn(d)
    m, n = 7.0, 12.0
    Rmq = rope.rotate(q, torch.tensor([m]))
    Rnk = rope.rotate(k, torch.tensor([n]))
    Rrelk = rope.rotate(k, torch.tensor([n - m]))
    lhs = float((Rmq * Rnk).sum())
    rhs = float((q * Rrelk).sum())
    print(f"<R_7 q, R_12 k> = {lhs:.10f}")
    print(f"<q, R_5 k>      = {rhs:.10f}")
    print(f"|diff| = {abs(lhs - rhs):.3e}  (relative-position property)")
    # fractional (sub-frame) position works identically
    Rfrac = rope.rotate(q, torch.tensor([7.5]))
    print(f"fractional position 7.5 ok, norm preserved: "
          f"{float(torch.norm(Rfrac)):.6f} vs {float(torch.norm(q)):.6f}")


if __name__ == "__main__":
    main()
```

4.3 Symbolic Music Event Decoders

The encoder produces $H_{\text{audio}} \in \mathbb{R}^{T \times d}$, a continuous state per frame. The target $Y_{\text{symbolic}} = (y_1, \ldots, y_S)$ is a discrete event sequence over a vocabulary Σ of `NOTE_ON`/`NOTE_OFF` per pitch, `TIME_SHIFT` quanta, `VELOCITY` levels, and control tokens. Decoding is the map $H_{\text{audio}} \mapsto Y_{\text{symbolic}}$, and the three great families of sequence models factorize it differently, with sharply different latency and quality consequences.

4.3.1 Autoregressive vs. non-autoregressive vs. CTC/transducer

Step 1 — Physical analogy. Three ways to transcribe a live performance. The *stenographer* (autoregressive) writes each note only after hearing the previous ones: slow, strictly left-to-right, but every decision benefits from full musical context — she will not write a G♯ after a C-major chord unless she really heard it. The *committee* (non-autoregressive iterative) has every member guess the whole score at once, then they reconvene K times, each round revising only the least confident passages in light of the confident ones. The *shorthand writer* (CTC) stamps one symbol per fixed time slice as the tape rolls, never looking back at what she stamped: blazingly parallel, but she cannot use the chord she just wrote to disambiguate the next one.

Step 2 — Geometric mental model. Picture the alignment lattice: time $t = 1..T$ horizontally, output position $s = 1..S$ vertically. The autoregressive path moves monotonically right-and-down, one column per step, each step's distribution conditioned on the entire path so far — the full history is the state. The iterative non-autoregressive model fills the whole S-row grid at once, then selectively erases and refills rows K times. CTC's lattice is different: it has T columns and only emits along a path through an expanded alphabet $\Sigma' = \Sigma \cup \{\text{blank}\}$, collapsing repeats and deleting blanks afterward; its state at column t carries no memory of emitted labels beyond what the encoder smuggled into frame t itself. Figure 4.7 shows the three inference geometries side by side.

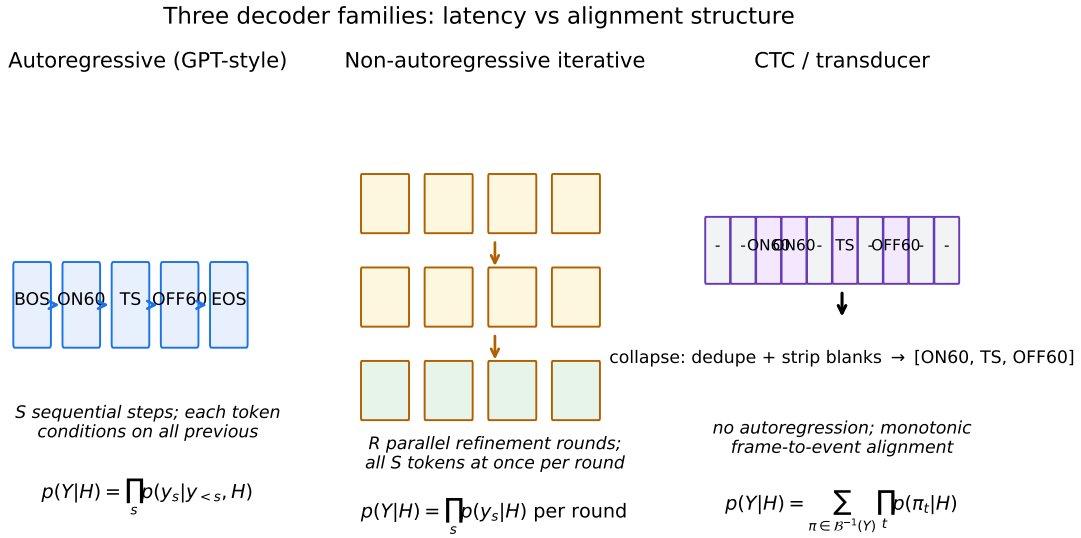


Figure 4.7: The three decoder families. Left: autoregressive decoding emits one token per serial step, each step conditioned on the full emitted prefix (4.28). Middle: iterative non-autoregressive decoding predicts all positions in parallel and refines the least confident ones over K rounds (4.30). Right: CTC emits one symbol per frame and collapses repeats and blanks with $\mathcal{B}(\cdot)$ (4.31) — fully parallel, but with no label-context conditioning (the transducer restores it with a prediction network (4.32)).

Step 3 — First-principles derivation. *Autoregressive (GPT-style).* The chain rule of probability gives an exact factorization with no independence assumptions:

$$p_\theta(Y \mid X) = \prod_{s=1}^{S} p_\theta(y_s \mid y_{<s}, X), \qquad y_{<s} = (y_1, \ldots, y_{s-1}). \tag{4.28}$$

Training minimizes the negative log-likelihood with teacher forcing (the prefix $y_{<s}$ is the ground truth):

$$\mathcal{L}_{\mathrm{AR}} = -\sum_{s=1}^{S} \log p_\theta(y_s \mid y_{<s}, X), \tag{4.29}$$

and inference emits one token per serial step, S steps total, each step attending to the full encoder state and the full emitted prefix. The price is latency linear in S with no parallelism across s; the reward is that inter-note dependencies — chord voicings, voice leading, the statistical grammar of tonal music — are modeled directly by the conditioning $y_{<s}$.

Non-autoregressive iterative (mask–predict). A length predictor first estimates $p(L \mid X)$; then all L positions are predicted in parallel, and K refinement iterations re-mask the least confident positions and re-predict them conditioned on the confident remainder. With $M \subset \{1, \ldots, L\}$ the masked positions and $y_{\setminus M}$ the observed ones, each iteration optimizes

$$\mathcal{L}_{\mathrm{NAR}} = -\sum_{s \in M} \log p_\theta(y_s \mid y_{\setminus M}, X), \tag{4.30}$$

where the conditioning set $y_{\setminus M}$ is resampled each round. Inference costs K parallel passes regardless of L (here $K = 8$), but the model must guess the length up front, and early-round errors in confident positions can never be revised.

CTC. For unsegmented frame sequences, connectionist temporal classification [31] avoids alignment entirely. Let $\Sigma' = \Sigma \cup \{\text{blank}\}$ and let a path $\pi \in \Sigma'^{T}$ assign one symbol per frame. The collapsing map $\mathcal{B}$ deletes blanks and repeated symbols (e.g. $\mathcal{B}(a, a, \varnothing, \varnothing, b, b) = (a, b)$). The model defines a per-frame distribution $p_t(\cdot \mid X)$ and sums over all paths consistent with Y:

$$p(Y \mid X) = \sum_{\pi \in \mathcal{B}^{-1}(Y)} \prod_{t=1}^{T} p_t(\pi_t \mid X), \qquad \mathcal{L}_{\mathrm{CTC}} = -\log p(Y \mid X), \tag{4.31}$$

computed by the forward–backward dynamic program over the $T \times (2S+1)$ trellis. The critical structural fact is the product inside the sum: conditioned on X, frames are *independent*. No term $p(\pi_t \mid \pi_{<t})$ appears, so the model cannot represent "given that I emitted C then E, G is likely" except insofar as the encoder packed that dependency into the frame representations themselves. Greedy inference is a single parallel argmax per frame followed by $\mathcal{B}$: fully parallel, $O(T)$.

Transducer. The (RNN-T/conformer-transducer) family repairs CTC's missing label context with a prediction network $g(y_{<u})$ over emitted labels and a joiner combining encoder frame t with label step u:

$$p(y_u \mid x_{1:t}, y_{<u}) = \mathrm{softmax}\big(\mathrm{Joiner}(h_t,\, g(y_{<u}))\big), \tag{4.32}$$

trained by forward–backward over the (t, u) lattice. It keeps CTC's streaming, frame-synchronous character while restoring autoregressive label dependence.

Latency/quality analysis. Take 10 s of audio at 100 fps ($T = 1000$ frames) with a dense piano passage of $S \approx 200$ event tokens. Illustrative arithmetic with stated per-step costs (not measured benchmarks): autoregressive decoding needs 200 serial steps $\times$ 1.2 ms $\approx$ 240 ms; iterative NAR needs $K = 8$ parallel passes $\times$ 2.5 ms $\approx$ 20 ms; CTC needs one encoder pass plus linear head, $\approx$ 35 ms fully parallel. Quality ordering follows the conditioning structure: AR models the joint distribution (4.28) exactly and captures musical syntax (a dominant seventh resolving to tonic is a left-context fact); NAR approximates it through K rounds of conditional refinement; CTC factorizes per frame and leans entirely on the encoder for inter-note coherence, which is why it underperforms on dense polyphony despite its speed. The transducer sits between CTC and AR: label context restored, but alignment still monotonic and frame-synchronous.

Step 4 — Implementation. `code/m4_decoders.py` (compiled clean) implements the three inference loops side by side: greedy AR generation (one serial step per token), mask–predict iterative refinement (K parallel passes with confidence-based re-masking), and CTC greedy collapse $\mathcal{B}(\cdot)$.

```python
"""Decoding paradigms: autoregressive, iterative non-autoregressive
(mask-predict), and CTC greedy decoding (Module 4).

All three consume per-step log-probabilities produced by a decoder or a
frame classifier; the difference is the factorization each one assumes.
Shapes: (B, S, V) for sequence models, (B, T, V) for CTC.
"""
import torch


def ar_generate(step_fn, bos_id, eos_id, max_len=512):
    """Greedy autoregressive decoding. step_fn(prefix_ids) -> (B, V)
       logits
    for the next token; the prefix grows by one token per serial
       step."""
    ids = torch.full((1, 1), bos_id, dtype=torch.long)
    for _ in range(max_len):
        logits = step_fn(ids)                        # (B, V): p(y_s |
           y_<s, X)
        nxt = logits.argmax(-1, keepdim=True)        # greedy choice
        ids = torch.cat([ids, nxt], dim=1)
        if (nxt == eos_id).all():
            break
    return ids


def mask_predict_refine(logits_fn, length, n_iters=8, mask_id=0):
    """Iterative non-autoregressive refinement. logits_fn(partial_ids)
    -> (B, L, V) log-probs conditioned on all unmasked tokens in
       parallel.
    Each iteration re-masks the least confident positions and
       re-predicts."""
    B, L, V = 1, length, logits_fn(torch.full((1, length),
       mask_id)).shape[-1]
    ids = torch.full((B, L), mask_id, dtype=torch.long)
    for it in range(n_iters):
        logp = logits_fn(ids)                        # (B, L, V), one
           parallel pass
        best = logp.argmax(-1)                       # (B, L)
        conf = logp.gather(-1, best.unsqueeze(-1)).squeeze(-1)  # (B,
           L)
        n_mask = max(1, int(L * (1 - (it + 1) / n_iters)))
        thresh = torch.topk(conf.view(-1), n_mask).values.min()
        keep = conf > thresh
        ids = torch.where(keep, best, torch.full_like(ids, mask_id))
    return ids


def ctc_greedy_decode(log_probs, blank=0):
    """log_probs: (T, V) incl. blank. Argmax per frame, then collapse
       repeats
    and strip blanks: B(pi) mapping of Graves et al. 2006."""
    path = log_probs.argmax(-1).tolist()             # (T,)
```

```
    out, prev = [], None
    for c in path:
        if c != prev and c != blank:
            out.append(c)
        prev = c
    return out


def main():
    torch.manual_seed(0)
    V, T, S = 8, 12, 6
    # fake model outputs
    seq_logp = torch.randn(1, S, V).log_softmax(-1)
    frame_logp = torch.randn(T, V).log_softmax(-1)
    step_fn = lambda ids: seq_logp[:, ids.shape[1] - 1, :]
    print("AR greedy ids:      ", ar_generate(step_fn, bos_id=1,
        eos_id=2,
                                              max_len=S)[0].tolist())
    logits_fn = lambda ids: seq_logp.expand(1, ids.shape[1], V)
    print("mask-predict ids:   ", mask_predict_refine(logits_fn,
        S)[0].tolist())
    print("CTC greedy collapse:", ctc_greedy_decode(frame_logp,
        blank=0))


if __name__ == "__main__":
    main()
```

4.3.2 Cross-attention dynamics: from continuous frames to discrete tokens

Whichever decoder family is chosen, the bridge from H_{audio} to Y_{symbolic} is cross-attention: queries computed from the symbolic side, keys and values computed from the audio side. This subsection expands that bridge completely, tracking every matrix dimension through a worked numeric example with real computed values.

Step 1 — Physical analogy. A conductor cueing an orchestra from a score. The score (symbolic side) says "at this moment the violins enter"; the conductor's question — the query — is "which players are sounding *now*?", and she resolves it by listening across the whole ensemble (the keys) and blending what she hears (the values) into her next gesture. Each decoder step asks a content-based question of the entire audio memory and receives a weighted blend of frames as the answer. The attention weights are the soft alignment between "where I am in the score" and "where the evidence is in the audio".

Step 2 — Geometric mental model. Fix one head. The keys $\{k_t\}_{t=1}^{T}$ are T points in $\mathbb{R}^{d_k}$ marking where each audio frame lives in content space; the query q_s is a single point marking what the decoder needs at step s. The score $s_{st} = \langle q_s, k_t \rangle / \sqrt{d_k}$ is a similarity: the softmax turns similarities into a probability distribution $a_{s\cdot}$ over frames, and the output $y_s = \sum_t a_{st} v_t$ is the $a_{s\cdot}$-weighted centroid of the value points. As decoding proceeds, q_s moves through content space and its centroid tracks the relevant audio — typically swinging toward the frames containing the next note's onset. Figure 4.8 shows a full 3×4 weight matrix from the worked example below: each row is one decoder step's distribution over the four audio frames.

Step 3 — First-principles derivation. Let $H \in \mathbb{R}^{T \times d}$ be the encoder output and $D \in \mathbb{R}^{S \times d}$ the decoder's hidden states for the current prefix. With learned projections $W_Q, W_K, W_V \in$

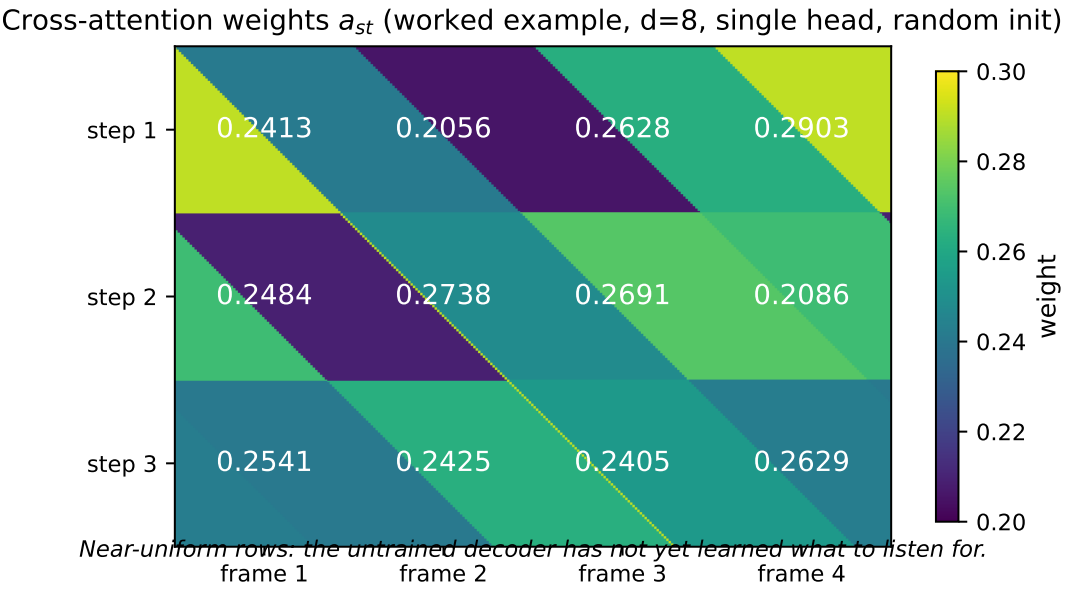


Figure 4.8: Cross-attention weights a_{st} for the worked example: $S = 3$ decoder steps over $T = 4$ encoder frames, $d_{\text{model}} = 8$, single head. Each row sums to 1; the near-uniform rows are the untrained regime — the decoder has not yet learned what to listen for.

$\mathbb{R}^{d\times d_k}$ (per head; $d_k = d/h$):

$$Q = DW_Q \in \mathbb{R}^{S\times d_k}, \qquad K = HW_K \in \mathbb{R}^{T\times d_k}, \qquad V = HW_V \in \mathbb{R}^{T\times d_k}. \tag{4.33}$$

The score matrix, the attention weights, and the output are

$$s_{st} = \frac{1}{\sqrt{d_k}} \sum_{j=1}^{d_k} q_{sj}\, k_{tj}, \qquad S = \frac{QK^\top}{\sqrt{d_k}} \in \mathbb{R}^{S\times T}, \tag{4.34}$$

$$a_{st} = \frac{\exp(s_{st})}{\sum_{t'=1}^{T} \exp(s_{st'})}, \qquad A = \operatorname{softmax}(S) \in \mathbb{R}^{S\times T}, \tag{4.35}$$

$$y_s = \sum_{t=1}^{T} a_{st}\, v_t, \qquad Y = AV \in \mathbb{R}^{S\times d_k}. \tag{4.36}$$

Heads are concatenated and projected by $W^O \in \mathbb{R}^{d\times d}$; a token head $\operatorname{softmax}(YW_{\text{vocab}})$ then yields $p(y_s \mid y_{<s}, X)$ per step. The decoder layer adds causal self-attention over D before cross-attention (so q_s already encodes the symbolic prefix) and a position-wise FFN after.

Worked numeric example. Take $d = 8$, single head ($d_k = 8$), $T = 4$ frames, $S = 3$ steps, with fixed random weights (seed 7; all values below recomputed by the verification script). The first query and key rows are

$$q_1 = [-0.0286,\ 0.0459,\ -0.4053,\ -0.2142,\ 0.0749,\ 0.1486,\ -0.1916,\ 0.9584],$$
$$k_1 = [0.4714,\ 0.0637,\ -0.3312,\ 0.3293,\ -0.1371,\ -0.0026,\ 0.4163,\ 0.0058].$$

Expanding (4.34) term by term for s_{11}:

$$\begin{aligned} s_{11} = \tfrac{1}{\sqrt{8}}\big(&\underbrace{-0.0048}_{q_{11}k_{11}} + \underbrace{0.0010}_{q_{12}k_{12}} + \underbrace{0.0474}_{q_{13}k_{13}} + \underbrace{(-0.0249)}_{q_{14}k_{14}} + \underbrace{(-0.0036)}_{q_{15}k_{15}} \\ &+ \underbrace{(-0.0001)}_{q_{16}k_{16}} + \underbrace{(-0.0282)}_{q_{17}k_{17}} + \underbrace{0.0020}_{q_{18}k_{18}}\big) = -0.0112. \end{aligned} \tag{4.37}$$

The full score row is $s_{1\cdot} = [-0.0112,\ -0.1711,\ 0.0743,\ 0.1740]$; the softmax (4.35) gives $a_{1\cdot} = [0.2413,\ 0.2056,\ 0.2628,\ 0.2903]$ (summing to 1.000000). The output (4.36) blends the four value vectors

$$\begin{aligned} v_1 &= [-0.078,\ 0.183,\ 0.213,\ 0.092,\ 0.289,\ 0.692,\ 0.070,\ -0.242], \\ v_2 &= [-0.172,\ 0.040,\ 0.005,\ 0.069,\ 0.047,\ 0.181,\ 0.019,\ -0.439], \\ v_3 &= [0.145,\ -0.104,\ -0.529,\ 0.107,\ -0.452,\ 0.399,\ 1.035,\ 0.252], \\ v_4 &= [0.326,\ -0.363,\ -0.643,\ -0.090,\ -0.790,\ -0.499,\ 0.412,\ 1.087], \end{aligned}$$

as $y_1 = 0.2413\, v_1 + 0.2056\, v_2 + 0.2628\, v_3 + 0.2903\, v_4$, i.e.

$$y_1 = [0.0787,\ -0.0802,\ -0.2729,\ 0.0383,\ -0.2684,\ 0.1640,\ 0.4124,\ 0.2332]. \tag{4.38}$$

Checking the first component by hand: $0.2413(-0.078) + 0.2056(-0.172) + 0.2628(0.145) + 0.2903(0.326) = 0.0786$, matching (4.38). Every dimension is thus accounted for: $(B, S, d) \times (B, T, d) \to (B, S, T)$ scores $\to (B, S, T)$ weights $\to (B, S, d_k)$ output per head.

Musically, the interesting regime is when $a_{s\cdot}$ sharpens: as training proceeds, the query at the step predicting `NOTE_ON(60)` learns to align q_s with the keys of frames containing C_4's onset transient, so the weight mass concentrates on 2–3 frames and y_s becomes essentially "the audio evidence for this note". A diffuse row, like the near-uniform rows in Figure 4.8 (an untrained random example), means the decoder has not yet learned what to listen for.

Step 4 — Implementation. `code/m4_crossattn_decoder.py` implements the full decoder layer — causal self-attention over the symbolic prefix, cross-attention per (4.33)–(4.36), and the FFN — with the worked dimensions $(B, T, S, d) = (2, 4, 3, 8)$ printed by its demo.

```
"""Cross-attention decoder layer: continuous encoder states -> discrete symbolic
tokens (Module 4).

A decoder layer combines (1) causal self-attention over already-emitted
symbolic tokens, (2) cross-attention whose queries come from the decoder and
whose keys/values come from the audio encoder, and (3) a position-wise FFN.
The worked example tracks every matrix dimension.
"""
import math
import torch
import torch.nn as nn


class CrossAttentionDecoderLayer(nn.Module):
    def __init__(self, d_model=256, n_heads=4, dropout=0.1):
        super().__init__()
        self.n_heads = n_heads
        self.d_k = d_model // n_heads
        # causal self-attention over symbolic prefix
        self.qkv_self = nn.Linear(d_model, 3 * d_model)
        # cross-attention: Q from decoder, K/V from audio encoder
        self.q_cross = nn.Linear(d_model, d_model)
        self.kv_cross = nn.Linear(d_model, 2 * d_model)
        self.out_self = nn.Linear(d_model, d_model)
        self.out_cross = nn.Linear(d_model, d_model)
        self.ffn = nn.Sequential(nn.Linear(d_model, 4 * d_model),
            nn.GELU(),
```

```
                                    nn.Linear(4 * d_model, d_model))
        self.norms = nn.ModuleList([nn.LayerNorm(d_model) for _ in
            range(3)])
        self.dropout = nn.Dropout(dropout)

    def _split(self, x):
        B, L, d = x.shape
        return x.view(B, L, self.n_heads, self.d_k).transpose(1, 2)

    def forward(self, y_tok, h_audio, tgt_mask=None):
        """y_tok:  (B, S, d) decoder states (symbolic prefix)
           h_audio: (B, T, d) encoder states (continuous audio)"""
        B, S, d = y_tok.shape
        T = h_audio.shape[1]
        # 1) causal self-attention on the symbolic prefix
        q, k, v = self.qkv_self(self.norms[0](y_tok)).chunk(3, dim=-1)
        q, k, v = self._split(q), self._split(k), self._split(v)
        scores = q @ k.transpose(-2, -1) / math.sqrt(self.d_k)
        if tgt_mask is not None:
            scores = scores.masked_fill(tgt_mask == 0, float("-inf"))
        y = y_tok + self.dropout(
            self.out_self((torch.softmax(scores, -1) @ v)
                          .transpose(1, 2).reshape(B, S, d)))
        # 2) cross-attention: decoder queries audio keys/values
        qc = self._split(self.q_cross(self.norms[1](y)))           #
            (B,H,S,dk)
        kv = self.kv_cross(h_audio)                                #
            (B,T,2d)
        kc, vc = kv.chunk(2, dim=-1)
        kc, vc = self._split(kc), self._split(vc)                  #
            (B,H,T,dk)
        xscores = qc @ kc.transpose(-2, -1) / math.sqrt(self.d_k)  #
            (B,H,S,T)
        xattn = torch.softmax(xscores, dim=-1)
        y = y + self.dropout(
            self.out_cross((xattn @ vc).transpose(1, 2).reshape(B, S,
                d)))
        # 3) position-wise FFN
        y = y + self.dropout(self.ffn(self.norms[2](y)))
        return y, xattn


def causal_mask(S):
    return torch.tril(torch.ones(S, S)).view(1, 1, S, S)


def main():
    torch.manual_seed(7)
    B, T, S, d, V = 2, 4, 3, 8, 16   # toy: 4 frames, 3 tokens, d=8,
        vocab=16
    layer = CrossAttentionDecoderLayer(d_model=d, n_heads=1)
    head = nn.Linear(d, V)           # token prediction head
    h_audio = torch.randn(B, T, d)   # H_audio: continuous encoder
        states
    tok = torch.randint(0, V, (B, S))
    y_tok = nn.Embedding(V, d)(tok)  # decoder states for prefix Y_<s
    y, xattn = layer(y_tok, h_audio, causal_mask(S))
    logits = head(y)                 # (B, S, V): p(y_s | y_<s, X)
```

```
    print(f"H_audio {tuple(h_audio.shape)} -> K,V (B,H,T,dk) =
        (2,1,4,8)")
    print(f"y_tok   {tuple(y_tok.shape)} -> Q    (B,H,S,dk) =
        (2,1,3,8)")
    print(f"cross-attn scores (B,H,S,T) = {tuple(xattn.shape)}")
    print(f"logits  {tuple(logits.shape)} -> p(y_s|y_<s,X) per step")
    print("row sums of cross-attention (must be 1):",
          xattn[0, 0].sum(-1).detach().tolist())


if __name__ == "__main__":
    main()
```

4.4 Multi-Task Losses for Sparse Musical Events

The decoder predicts two very different kinds of targets at once: a symbolic event sequence (one token at a time, each step a classification over the vocabulary) and frame-level piano-roll-style activations (every frame × pitch cell labeled active or not, of which $\sim 99.9\%$ are negative). This section derives the joint event cross-entropy from maximum likelihood, derives the focal loss from cross-entropy with a full gradient analysis, and quantifies the imbalance that makes the focal term necessary.

4.4.1 Joint cross-entropy over event sequences, from maximum likelihood

Step 1 — Physical analogy. Maximum likelihood is the principle of the honest transcriber: among all candidate models, prefer the one that would have found the true score least surprising. If the training corpus contains a thousand performances of Chopin, the model should assign high probability to the event sequences that actually occurred and low probability to everything else. Cross-entropy is simply surprise measured in nats, averaged over the corpus; minimizing it is maximizing the likelihood of the truth.

Step 2 — Geometric mental model. The model's output at step s is a point on the $(V-1)$-simplex: a probability vector over the V vocabulary tokens. The target is a corner of that simplex (the one-hot truth). Cross-entropy is, up to a constant, the Kullback–Leibler divergence from the model's point to the target corner; training drags each step's simplex point toward its corner. The joint loss sums this drag over all S steps, so the whole predicted sequence is pulled toward the true sequence corner by corner. Figure 4.9 draws one step of this geometry for a three-token vocabulary.

Step 3 — First-principles derivation. For one training pair (X, Y) with $Y = (y_1, \ldots, y_S)$, the chain rule factorizes the sequence likelihood with no approximations:

$$p_\theta(Y \mid X) = \prod_{s=1}^{S} p_\theta(y_s \mid y_{<s}, X). \tag{4.39}$$

Maximum likelihood over a corpus $\{(X^{(n)}, Y^{(n)})\}_{n=1}^{N}$ maximizes

$$\sum_{n=1}^{N} \log p_\theta(Y^{(n)} \mid X^{(n)}) = \sum_{n=1}^{N} \sum_{s=1}^{S_n} \log p_\theta(y_s^{(n)} \mid y_{<s}^{(n)}, X^{(n)}), \tag{4.40}$$

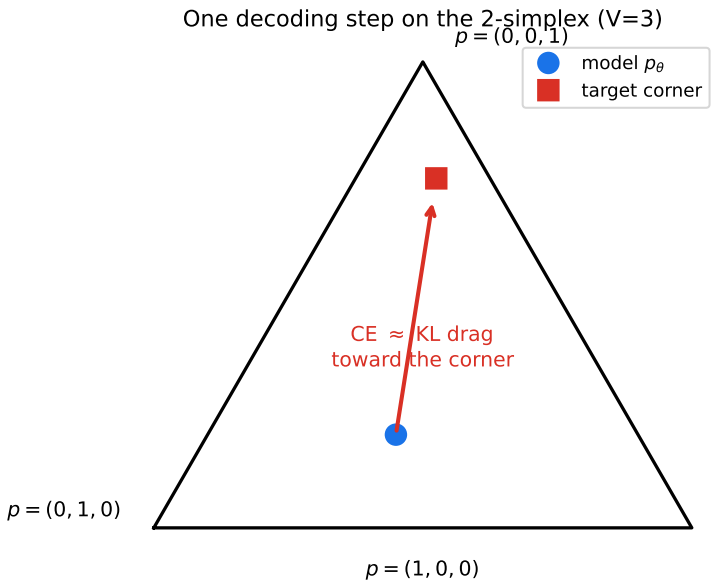


Figure 4.9: One decoding step on the probability simplex ($V = 3$). The model's output is a point in the triangle; the target is a corner. Cross-entropy is, up to a constant, the KL divergence from the point to the corner, and its gradient drags the point toward the corner along the red arrow.

so minimizing the negative log-likelihood is exactly minimizing, per sequence,

$$\mathcal{L}_{\text{CE}} = -\frac{1}{S}\sum_{s=1}^{S} \log p_\theta(y_s \mid y_{<s}, X), \tag{4.41}$$

the joint cross-entropy: the average surprise per event. With teacher forcing, the conditioning prefix $y_{<s}$ is the ground-truth prefix, so each term is an ordinary V-way classification loss. Writing the model's logits $z_s \in \mathbb{R}^V$ and $p_{s,v} = \exp(z_{s,v})/\sum_u \exp(z_{s,u})$, the gradient that drives learning is the familiar residual, derived by differentiating (4.41) through the softmax:

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial z_{s,v}} = \frac{1}{S}\left(p_{s,v} - \mathbf{1}[y_s = v]\right). \tag{4.42}$$

Every step pushes its logit vector toward the one-hot truth with force proportional to the current error. The "multi-task" part of this chapter's losses is the weighted sum of such terms across heads — event decoding, frame-level pitch activation, onset detection:

$$\mathcal{L}_{\text{multi}} = \lambda_{\text{evt}}\,\mathcal{L}_{\text{CE}} + \lambda_{\text{frame}}\,\mathcal{L}_{\text{frame}} + \lambda_{\text{onset}}\,\mathcal{L}_{\text{onset}}, \tag{4.43}$$

with the frame and onset terms facing the imbalance analyzed below.

Step 4 — Implementation. The event term is `torch.nn.CrossEntropyLoss` applied to the decoder logits of Section 4.3.2; the frame terms use the focal loss of the next subsection. The weighting λ's are hyperparameters, chosen so that no head's gradient drowns the others at initialization.

4.4.2 Frame-level focal loss: derivation and gradient analysis

Step 1 — Physical analogy. A piano-roll label is a vast dark field with a few lit windows: in a typical frame, 87 of 88 pitch cells are silent. Standard cross-entropy is a teacher who spends

equal effort grading every cell, so the lesson is dominated by the millions of easy "this is silence" judgments while the rare "this is an onset" judgments — the ones that actually matter — are outvoted a thousand to one. The focal loss [30] is a teacher who stops grading papers that are already correct: a modulating factor $(1-p_t)^\gamma$ quietly zeroes out the loss of well-classified cells, so the gradient is spent almost entirely on the hard, ambiguous cases.

Step 2 — Geometric mental model. Plot loss against p_t, the model's probability for the true class (Figure 4.10, left). Cross-entropy $-\log p_t$ decays gently: even at $p_t = 0.9$ it still charges 0.105. The focal loss multiplies this by $(1-p_t)^\gamma$, which collapses to zero as $p_t \to 1$: at $p_t = 0.9, \gamma = 2$ charges only 0.000263, a 400× reduction. The right panel shows the consequence for learning: the gradient with respect to the logit, $|dL/dz|$, is nearly silenced for easy examples, while hard examples keep most of theirs — at $p_t = 0.3$ the gradient is suppressed only 4×, versus 138× at $p_t = 0.9$. The model stops re-learning silence and keeps learning onsets.

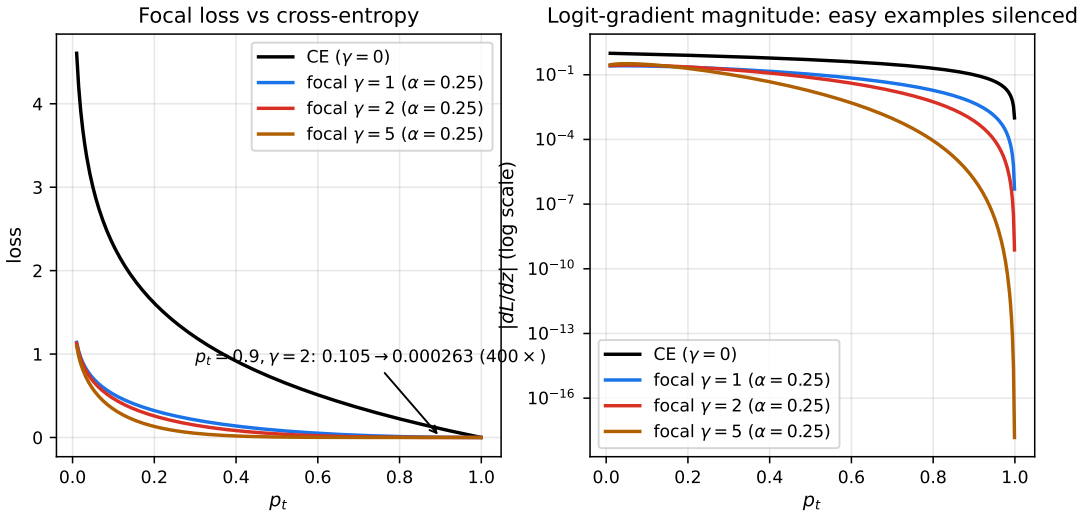


Figure 4.10: Focal loss $L = -\alpha_t(1-p_t)^\gamma \log p_t$ [30] versus cross-entropy ($\alpha = 0.25$ for the focal curves): loss curves (left) and logit-gradient magnitudes $|dL/dz|$ (right) as functions of p_t. Larger γ more aggressively down-weights well-classified examples; $\gamma = 0$ recovers cross-entropy.

Step 3 — First-principles derivation. Begin with binary cross-entropy for a label $y \in \{0, 1\}$ and predicted probability p:

$$\mathrm{CE}(p, y) = -y \log p - (1-y) \log(1-p). \tag{4.44}$$

Define p_t as the probability assigned to the *true* class,

$$p_t = \begin{cases} p & y = 1, \\ 1-p & y = 0, \end{cases} \qquad \text{so that} \qquad \mathrm{CE}(p_t) = -\log p_t. \tag{4.45}$$

Class-balanced cross-entropy inserts a weighting factor $\alpha_t \in \{\alpha, 1-\alpha\}$ for the positive/negative class:

$$\mathrm{CE}_\alpha(p_t) = -\alpha_t \log p_t. \tag{4.46}$$

The focal loss multiplies by the modulating factor $(1-p_t)^\gamma$ with focusing parameter $\gamma \geq 0$:

$$\mathrm{FL}(p_t) = -\alpha_t \, (1-p_t)^\gamma \log p_t. \tag{4.47}$$

When $p_t \to 1$ (easy, well-classified), $(1-p_t)^\gamma \to 0$ and the loss vanishes; when p_t is small (hard, misclassified), the factor is near 1 and the full loss applies. Setting $\gamma = 0$ recovers (4.46), and $\alpha = 1$ recovers plain CE.

The down-weighting claim is proved by differentiating with respect to the logit z ($p = \sigma(z)$ for the positive class). From (4.47) with $p_t = p$:

$$\frac{d\,\mathrm{FL}}{dp} = -\alpha\Big[-\gamma(1-p)^{\gamma-1}\log p + \frac{(1-p)^\gamma}{p}\Big], \tag{4.48}$$

$$\begin{aligned}\frac{d\,\mathrm{FL}}{dz} &= \frac{d\,\mathrm{FL}}{dp}\cdot p(1-p)\\ &= \alpha\,\gamma\,p\,(1-p)^\gamma\log p - \alpha\,(1-p)^{\gamma+1}.\end{aligned} \tag{4.49}$$

For $\gamma = 0, \alpha = 1$ this reduces to $-(1-p) = p-1$, the standard logistic gradient, as required. The ratio of focal to CE gradient at fixed p exposes the focusing: with $\alpha = 0.25$, $\gamma = 2$,

$$p = 0.9: \quad \frac{d\mathrm{FL}}{dz} = -0.0007241 \;\text{ vs. }\; \frac{d\mathrm{CE}}{dz} = -0.1000 \quad (138.1\times \text{ smaller}), \tag{4.50}$$

$$p = 0.3: \quad \frac{d\mathrm{FL}}{dz} = -0.1742420 \;\text{ vs. }\; \frac{d\mathrm{CE}}{dz} = -0.7000 \quad (4.0\times \text{ smaller}). \tag{4.51}$$

An easy negative's gradient is suppressed 138× while a hard example's is suppressed only 4×: the optimizer's attention is reallocated from the certain to the uncertain by two orders of magnitude. The full loss table (recomputed numerically) is:

p_t	CE	FL (γ=1)	FL (γ=2)	FL (γ=5)
0.05	2.995732	0.711486	0.675912	0.579510
0.20	1.609438	0.321888	0.257510	0.131845
0.50	0.693147	0.086643	0.043322	0.005415
0.70	0.356675	0.026751	0.008025	0.000217
0.90	0.105361	0.002634	0.000263	0.000000
0.95	0.051293	0.000641	0.000032	0.000000
0.99	0.010050	0.000025	0.000000	0.000000

($\alpha = 0.25$ throughout; FL values are $\alpha(1-p_t)^\gamma$ times the CE column.) For the multiclass event head, p_t is the softmax probability of the true token and (4.47) applies unchanged.

Step 4 — Implementation. `code/m4_focal_loss.py` implements binary and multiclass focal loss, the analytic gradient (4.49), and a demo that checks the analytic gradient against autograd and verifies $\gamma = 0$ recovers cross-entropy exactly.

```
"""Focal loss derived from cross-entropy (Module 4).

Chain: CE(p_t) = -log p_t  ->  balanced CE: -alpha_t log p_t  ->
       focal: -alpha_t (1 - p_t)^gamma log p_t   (Lin et al. 2017).

Includes the analytic logit-gradient used in the chapter's gradient
    analysis:
  d FL/dz = alpha*gamma*p*(1-p)^gamma*log(p) - alpha*(1-p)^(gamma+1)
for the positive class (p = sigmoid(z)), and a numeric check against
autograd.
"""
import math
import torch
import torch.nn.functional as F
```

```

def focal_loss_binary(logits, targets, alpha=0.25, gamma=2.0):
    """logits, targets: same shape. alpha weights the positive
       class."""
    p = torch.sigmoid(logits)
    pt = torch.where(targets == 1, p, 1 - p)
    at = torch.where(targets == 1,
                     torch.full_like(pt, alpha),
                     torch.full_like(pt, 1 - alpha))
    return (-at * (1 - pt) ** gamma *
       torch.log(pt.clamp_min(1e-12))).mean()


def focal_loss_multiclass(logits, targets, alpha=0.25, gamma=2.0):
    """logits: (N, C), targets: (N,) class indices."""
    logp = F.log_softmax(logits, dim=-1)
    pt = logp.gather(-1, targets.unsqueeze(-1)).squeeze(-1).exp()
    return (-alpha * (1 - pt) ** gamma *
       torch.log(pt.clamp_min(1e-12))).mean()


def dfl_dz_analytic(p, alpha=0.25, gamma=2.0):
    return alpha * gamma * p * (1 - p) ** gamma * math.log(p) \
        - alpha * (1 - p) ** (gamma + 1)


def main():
    torch.manual_seed(0)
    print("pt       CE         FL(g=1)    FL(g=2)    FL(g=5)")
    for pt in [0.05, 0.2, 0.5, 0.7, 0.9, 0.95, 0.99]:
        z = math.log(pt / (1 - pt))
        ce = F.binary_cross_entropy_with_logits(
            torch.tensor([z]), torch.tensor([1.0])).item()
        fl = [focal_loss_binary(torch.tensor([z]), torch.tensor([1.0]),
                                alpha=0.25, gamma=g).item() for g in
                                   (1, 2, 5)]
        print(f"{pt:.2f}  {ce:.6f}  {fl[0]:.6f}  {fl[1]:.6f}
           {fl[2]:.6f}")
    # gradient check: analytic vs autograd
    for p in [0.9, 0.3]:
        z = torch.tensor([math.log(p / (1 - p))], requires_grad=True)
        focal_loss_binary(z, torch.tensor([1.0]), alpha=0.25,
           gamma=2.0).backward()
        auto = z.grad.item()
        ana = dfl_dz_analytic(p)
        print(f"p={p}: autograd dL/dz={auto:.7f}, analytic={ana:.7f}, "
              f"|diff|={abs(auto-ana):.2e}")
    # gamma=0 recovers (alpha-weighted) CE
    z = torch.tensor([[0.7, -0.2, 0.1]])
    l0 = focal_loss_multiclass(z, torch.tensor([0]), alpha=1.0,
       gamma=0.0)
    ce = F.cross_entropy(z, torch.tensor([0]))
    print(f"gamma=0 -> CE check: {l0.item():.8f} vs {ce.item():.8f}")


if __name__ == "__main__":
    main()
```

4.4.3 Extreme class imbalance: quantification and mitigation

Step 1 — Physical analogy. Counting onsets in a piano-roll is counting lightning strikes on a satellite map of a continent: the map is enormous, the strikes are rare, and a model that predicts "no strike anywhere" is 99.9% accurate while being perfectly useless. Accuracy is the wrong god here; the loss must make the rare positives matter more than the abundant negatives, or training converges to the cowardly all-silence solution.

Step 2 — Geometric mental model. Imagine the loss landscape as a valley whose floor is tilted by the class prior. With 1056 negatives per positive, the floor's tilt is set almost entirely by negatives: the gradient points toward "predict silence everywhere" from nearly every starting point. The focal modulating factor flattens the floor under the easy negatives — removing their tilt — so the remaining slope points toward the positives. The α_t factor then re-levels the two classes' total weight. Figure 4.11 shows both effects numerically.

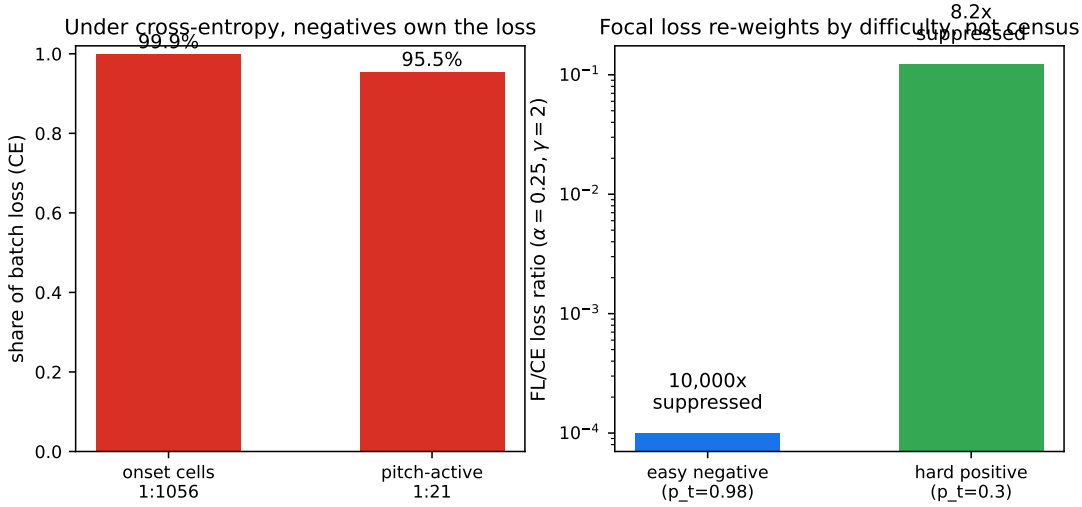


Figure 4.11: Why cross-entropy drowns and focal loss rescues. Left: under plain CE the negative cells own 99.9% of the batch loss at the 1:1056 onset prior. Right: with $\alpha = 0.25$, $\gamma = 2$, an easy negative ($p_t = 0.98$) is suppressed 10,000× while a hard positive ($p_t = 0.3$) only 8.2× — the loss is re-weighted by difficulty, not by census.

Step 3 — First-principles derivation. Quantify the imbalance with a concrete worked scenario (stated assumptions, not dataset claims): a piano performance of 180 s, frame hop 10 ms (100 fps, 18,000 frames), 88 pitch cells per frame (1,584,000 cells), 1,500 note onsets, mean polyphony 4, mean note duration 0.9 s. The priors are

$$P(\text{onset in frame}) = \frac{1500}{18000} = 0.0833\ (8.33\%), \tag{4.52}$$

$$P(\text{onset at cell } (t,p)) = \frac{1500}{1584000} = 9.47 \times 10^{-4}\ (0.0947\%), \quad \text{i.e. 1:1056}, \tag{4.53}$$

$$P(\text{pitch active}) = \frac{4}{88} = 0.0455\ (4.55\%), \quad \text{i.e. 1:21}, \tag{4.54}$$

$$P(\text{sustain cell}) = \frac{135000}{1584000} = 0.0852\ (8.52\%). \tag{4.55}$$

The onset detector — the head that matters most for transcription quality — faces a 1:1056 imbalance. Under plain cross-entropy, the expected loss is dominated by the 99.905% of cells that are easy negatives; the gradient of (4.42) summed over a batch points overwhelmingly toward silence.

The mitigation is derived, not stapled on, in two parts. First, the focal factor $(1 - p_t)^\gamma$ dynamically re-weights by difficulty: from (4.50), an easy negative at $p_t = 0.9$ contributes 138× less gradient than under CE, while a hard positive keeps nearly full weight (4.51). The 1:1056 count ratio is thereby converted into an *effective* ratio set by difficulty, not by census. Second, α_t statically rebalances the classes: with $\alpha = 0.25$ for positives (the value used in [30]), each positive cell's loss is weighted against 0.75 per negative cell, correcting the residual tilt after focusing. Together with the multi-task weighting (4.43), which lets the onset head (λ_{onset}) speak louder than the dense frame head, the training signal reaching the encoder is dominated by the rare events the system exists to detect. A common failure mode to avoid: setting γ too high (e.g. 5) on noisy labels, which discards not only easy negatives but also the moderately-hard positives the model needs; $\gamma = 2$ is the empirically stable default from [30].

Step 4 — Implementation. No new code is needed beyond `focal_loss_binary`: the frame and onset heads of (4.43) each call it with $(\alpha, \gamma) = (0.25, 2.0)$, and the λ's are tuned so that at initialization the three heads contribute comparable gradient norms — measured, not guessed, by logging $\|\nabla \mathcal{L}_k\|$ per head in the first epochs.

4.5 Drills

1. **Conformer convolution-module budget.** Derive the general parameter count of the convolution module (4.7)–(4.11) as a function of d and K (include biases and BatchNorm), and evaluate it for $d = 256$, $K = 31$. What fraction of the module's parameters sits in the depthwise kernel?

2. **Sinusoidal shift property, by hand.** For $d = 4$ the encoding of position m is $\text{PE}(m) = [\sin m,\ \cos m,\ \sin 0.01m,\ \cos 0.01m]$. Write down the 4×4 matrix $M(2)$ from (4.18), compute $M(2)\,\text{PE}(5)$ explicitly, and verify it equals PE(7) component by component.

3. **RoPE relative property.** Prove $\langle R^m_\Theta q, R^n_\Theta k\rangle = \langle q, R^{n-m}_\Theta k\rangle$ starting from the block-diagonal form (4.24), justifying each equality (transpose of a rotation, commutativity of the blocks). Then write out the explicit 4×4 matrix R^7_Θ for $d = 4$ with $\theta = (1,\ 0.01)$.

4. **Cross-attention by hand.** Using the worked example of Section 4.3.2 with $q_1 = [-0.0286,\ 0.0459,\ -0.4053,\ -0.2142,\ 0.0749,\ 0.1486,\ -0.1916,\ 0.9584]$ and the four key rows given there, recompute the score vector $s_{1\cdot}$, the attention weights $a_{1\cdot}$, and the first component of y_1. Confirm $\sum_t a_{1t} = 1$.

5. **Focal gradient at $p = 0.95$.** Starting from (4.47), derive (4.49). Evaluate the focal loss and its logit-gradient for $p = 0.95$, $\alpha = 0.25$, $\gamma = 2$, and give the suppression ratios versus cross-entropy for both loss and gradient.

6. **Imbalance arithmetic.** A 240 s piano recording at 100 fps contains 2,000 note onsets with mean polyphony 3.5 over 88 pitches. Compute (a) the per-cell onset prior and its 1:N ratio, (b) the pitch-active prior and ratio, (c) the factor by which focal loss ($\alpha = 0.25$, $\gamma = 2$) suppresses the loss of an easy negative with $p_t = 0.98$ relative to cross-entropy.

7. **CTC's conditional independence.** Prove from (4.31) that CTC's frame outputs are conditionally independent given X (i.e. no term couples π_t to $\pi_{<t}$). Then give a concrete musical example — a C–E–G arpeggio where the G is strongly predicted by the preceding

C–E context — and explain which model component must capture that dependency under CTC versus under the autoregressive factorization (4.28).

8. **Mask–predict objective.** Derive the training objective (4.30) from the masked-language-modeling principle: with mask positions M sampled uniformly at random with $|M| = \lfloor 0.4L \rfloor$, write the exact log-likelihood being maximized and explain why the length L must be predicted by a separate head $p(L \mid X)$ rather than emerging from the iteration.

Chapter 5

Decoding Algorithms, Language Model Priors & Post-Processing

A trained sequence-to-sequence transcription model does not output music. It outputs numbers: at each step, a probability distribution over the next symbolic token, or, in frame-based models, a matrix of per-frame pitch posteriors. Between those numbers and a MIDI file that a synthesizer can play lies an entire discipline of *inference*: choosing which token sequence the numbers imply, correcting the model's characteristic mistakes with prior knowledge about musical syntax, and converting soft frame-level beliefs into crisp note events with onsets, offsets, and velocities. This chapter derives that machinery from first principles. Section 5.1 sets up the decoding problem. Sections 5.2 and 5.3 derive greedy decoding and beam search, including a proof of the short-sequence bias and its length-penalty correction. Section 5.4 fuses a pretrained symbolic music language model with the acoustic model through log-linear rescoring. Sections 5.5–5.8 turn frame posteriors into notes: HMM state smoothing with a fully worked Viterbi trace, minimum-duration filtering, hysteresis thresholding for onset–frame–offset integration, and anchor-calibrated velocity estimation. Every topic follows the four-step framework: physical analogy, geometric mental model, first-principles derivation, runnable code.

5.1 The Decoding Problem: From Probabilities to Performances

Let x denote the input audio (a waveform, or its spectrogram) and let $Y = (y_1, y_2, \ldots, y_L)$ be a sequence of symbolic tokens from a vocabulary $\mathcal{V}$ containing NOTE_ON(p), NOTE_OFF(p), TIME_SHIFT(Δ), VELOCITY(v), and a terminal END token (Chapter 4, tokenization; cf. the MIDI-like vocabularies of token-based transcription systems [39]). An autoregressive decoder defines

$$P(Y \mid x) = \prod_{t=1}^{L} P(y_t \mid y_{<t}, x), \tag{5.1}$$

where $y_{<t} = (y_1, \ldots, y_{t-1})$. Transcription is the maximum a posteriori (MAP) problem

$$Y^* = \arg\max_{Y \in \mathcal{V}^*} P(Y \mid x) = \arg\max_{Y \in \mathcal{V}^*} \sum_{t=1}^{|Y|} \log P(y_t \mid y_{<t}, x). \tag{5.2}$$

The search space $\mathcal{V}^*$ is infinite, the objective is non-convex in Y, and the argmax cannot be computed exactly. Every decoding algorithm in this chapter is an approximation strategy for (5.2), trading search effort against the risk of missing the true optimum. Frame-based models pose the parallel problem: given posteriors $f_t(p) = P(\text{pitch } p \text{ active at frame } t \mid x)$, decide discrete note events $(p, t_{\text{on}}, t_{\text{off}}, v)$.

5.2 Greedy Decoding

Step 1 — Physical analogy. Greedy decoding is descending a mountain in thick fog by always stepping in the locally steepest downhill direction. Each step is optimal given only what is visible from the current position, but the path can walk into a ravine when a slightly less steep first step would have led to the valley floor. In transcription, the "fog" is the model's uncertainty about the future: the decoder commits to the most probable next token without considering how that choice constrains the tokens that follow.

Step 2 — Geometric mental model. Picture the set of all token sequences as a rooted tree: the root is the empty history, each node is a partial sequence $y_{<t}$, and each edge $y_{<t} \to (y_{<t}, v)$ carries weight $\log P(v \mid y_{<t}, x) \leq 0$. The MAP objective (5.2) asks for the root-to-leaf path of maximum total weight. Greedy decoding walks down this tree always choosing the heaviest single edge, never backtracking. Figure 5.1 shows the three local decisions of the worked trace below: at each step one bar (blue) is the argmax, and the cumulative log-probability printed beside it is the running path weight.

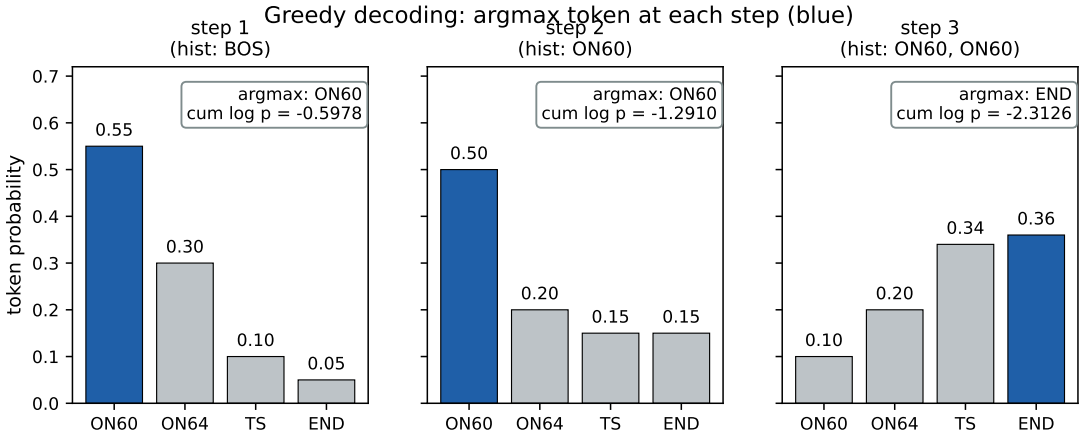


Figure 5.1: Greedy decoding on the toy distribution: at each step the argmax token (blue) is appended and the cumulative log-probability accumulates one more non-positive term.

Step 3 — First-principles derivation. Greedy decoding approximates (5.2) by replacing the global argmax with a sequence of local argmaxes:

$$\hat{y}_t = \arg\max_{v \in \mathcal{V}} \log P(v \mid \hat{y}_{<t}, x), \qquad t = 1, 2, \ldots \tag{5.3}$$

stopping when $\hat{y}_t = \text{END}$. This is exact only under a strong condition: if the true optimum Y^* satisfies $y^*_t = \arg\max_v P(v \mid y^*_{<t}, x)$ at every t, then greedy recovers it; in general the first step where the optimal path takes a locally suboptimal edge is the step where greedy diverges forever, since all subsequent distributions are conditioned on the wrong history.

Consider the toy decoder in Figure 5.1 over $\mathcal{V} = \{\text{ON60}, \text{ON64}, \text{TS}, \text{END}\}$ (shorthand for NOTE_ON(60), NOTE_ON(64), TIME_SHIFT, END). Histories not tabulated emit END

with probability 1. The trace runs:

$$t=1: \quad P(\cdot \mid \text{BOS}) = (0.55, 0.30, 0.10, 0.05) \;\Rightarrow\; \hat{y}_1 = \text{ON60}, \qquad S_1 = \log 0.55 = -0.5978, \tag{5.4}$$

$$t=2: \quad P(\cdot \mid \text{ON60}) = (0.50, 0.20, 0.15, 0.15) \;\Rightarrow\; \hat{y}_2 = \text{ON60}, \qquad S_2 = -0.5978 + \log 0.50 = -1.2910, \tag{5.5}$$

$$t=3: \quad P(\cdot \mid \text{ON60}, \text{ON60}) = (0.10, 0.20, 0.34, 0.36) \;\Rightarrow\; \hat{y}_3 = \text{END}, \quad S_3 = -1.2910 + \log 0.36 = -2.3126. \tag{5.6}$$

Greedy outputs (ON60, ON60, END) with total score -2.3126. Yet the sequence (ON64, ON64, END) scores $\log 0.30 + \log 0.65 + \log 0.60 = -2.1456$, strictly better. The failure is localized exactly: at $t = 1$ the optimal path takes the edge of weight $\log 0.30 < \log 0.55$, and greedy, which cannot see past one edge, discards it. Exhaustive enumeration over all histories confirms (ON64, ON64, END) is the global MAP optimum, so this is a genuine case of greedy suboptimality, not an artifact.

The computational cost is minimal. Let C be the cost of one autoregressive decoder step (one forward pass producing $|\mathcal{V}|$ logits) and T the output length. Greedy decoding costs

$$\mathcal{O}(T \cdot C + T \cdot |\mathcal{V}|), \tag{5.7}$$

the second term being the per-step argmax. With KV-caching, a transformer layer costs $\mathcal{O}(t \cdot d)$ attention work at step t for model dimension d (plus $\mathcal{O}(d^2)$ projections), giving $\mathcal{O}(T^2 d + T|\mathcal{V}|)$ overall up to layer and projection constants. Memory is $\mathcal{O}(T)$ for the output plus the KV cache.

Step 4 — Runnable code. The program below implements greedy decoding over the toy distribution and prints the trace derived above (saved as `code/m5_greedy_decode.py`, verified by compilation and execution).

```
"""Module 5 -- Greedy decoding for an autoregressive symbolic-music
    decoder.

A toy decoder distribution over the vocabulary
    {ON60, ON64, TS, END}  =  {NOTE_ON(60), NOTE_ON(64), TIME_SHIFT,
        END}
is tabulated for each reachable history. Histories not tabulated emit
    END
with probability 1 (the model terminates instead of continuing an
unfamiliar path). Greedy decoding picks the argmax token at every step.
"""
import math

VOCAB = ("ON60", "ON64", "TS", "END")

# P(token | history) for every history the worked trace can reach.
DIST = {
    ():                  {"ON60": 0.55, "ON64": 0.30, "TS": 0.10,
        "END": 0.05},
    ("ON60",):           {"ON60": 0.50, "ON64": 0.20, "TS": 0.15, "END":
        0.15},
    ("ON64",):           {"ON60": 0.10, "ON64": 0.65, "TS": 0.20, "END":
        0.05},
    ("ON60", "ON60"):    {"ON60": 0.10, "ON64": 0.20, "TS": 0.34, "END":
        0.36},
    ("ON60", "ON64"):    {"ON60": 0.10, "ON64": 0.10, "TS": 0.60, "END":
        0.20},
```

```
    ("ON64", "ON60"):  {"ON60": 0.25, "ON64": 0.25, "TS": 0.30, "END": 0.20},
    ("ON64", "ON64"):  {"ON60": 0.02, "ON64": 0.03, "TS": 0.35, "END": 0.60},
}
TERMINAL = {"END": 1.0}


def dist_of(history):
    """Return the token distribution for a history (terminal fallback)."""
    return DIST.get(tuple(history), TERMINAL)


def greedy_decode(max_steps=12):
    """Greedy decode. Returns (tokens, total_log_prob, trace_rows).

    trace_rows holds (step, distribution, chosen_token, cumulative_log_prob).
    """
    history, total_lp, trace = [], 0.0, []
    for step in range(1, max_steps + 1):
        d = dist_of(history)
        token = max(d, key=d.get)  # argmax; ties broken by vocab order
        total_lp += math.log(d[token])
        history.append(token)
        trace.append((step, dict(d), token, total_lp))
        if token == "END":
            break
    return history, total_lp, trace


def main():
    tokens, total_lp, trace = greedy_decode()
    print("Greedy decoding trace (toy NOTE_ON / TIME_SHIFT / END model):")
    for step, d, token, cum_lp in trace:
        probs = ", ".join(f"{t}={p:.2f}" for t, p in d.items())
        print(f"  step {step}: P = {{{probs}}} -> argmax {token}   "
              f"cum log-prob = {cum_lp:.4f}")
    print(f"Greedy output: {tokens}")
    print(f"Total sequence log-probability: {total_lp:.4f}")
    print("Note: the first-step argmax ON60 (p=0.55) commits the search to a")
    print("branch whose completed sequence scores -2.3126, while the global")
    print("optimum (ON64, ON64, END) scores -2.1456. See m5_beam_search.py.")


if __name__ == "__main__":
    main()
```

5.3 Autoregressive Beam Search

Step 1 — Physical analogy. If greedy decoding is one hiker in fog, beam search is a team of B hikers with walkie-talkies exploring a maze under a budget. At every junction each hiker reports the promising corridors they can see; the team keeps only the B most promising partial routes and abandons the rest. A corridor that looks slightly worse at the first junction survives if the team has budget to keep it, which is exactly what rescues the ON64 branch that greedy decoding discarded. The budget B is the beam width: $B = 1$ recovers greedy decoding, $B = \infty$ recovers exhaustive search. Beam search has been the standard decoding algorithm for autoregressive sequence models since the original seq2seq work [34].

Step 2 — Geometric mental model. Return to the weighted tree of Section 5.2. Beam search maintains a *frontier*: B nodes at the current depth, the B highest-weight root-to-node paths found so far. Each step expands every frontier node along all $|\mathcal{V}|$ edges, producing up to $B \cdot |\mathcal{V}|$ candidate paths, then prunes back to the best B. Geometrically this is a moving window of fixed width sliding down the tree, wide enough to carry several competing hypotheses but narrow enough to stay affordable. Figure 5.2 draws the full trace for $B = 2$ on the toy distribution: faded dashed edges are expansions that were pruned, and the green path is the surviving optimum.

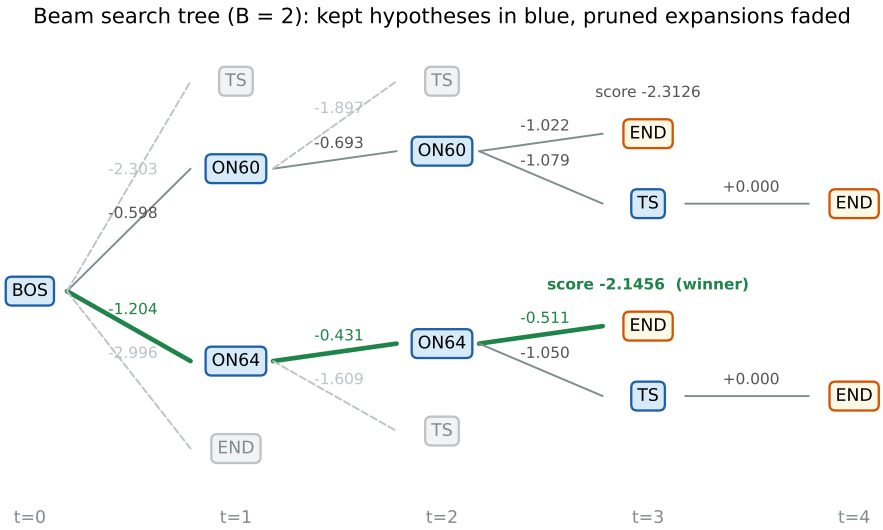


Figure 5.2: Beam search tree for $B = 2$. Each edge is labeled with its log-probability contribution; faded dashed edges were pruned. The green path (ON64, ON64, END) survives because the budget of two hypotheses carries the locally suboptimal first edge.

Step 3 — First-principles derivation. Let $\mathcal{H}_{t-1}$ be the beam after $t-1$ steps: B pairs $(h, S(h))$ of partial hypotheses and their accumulated scores, where

$$S(h) = \sum_{i=1}^{|h|} \log P(h_i \mid h_{<i}, x) \tag{5.8}$$

is the log-probability accumulation along the path. One search step has three phases. *Expansion:* every hypothesis is extended by every vocabulary token,

$$\mathcal{C}_t = \left\{ (h \circ v,\ S(h) + \log P(v \mid h, x)) \;\middle|\; h \in \mathcal{H}_{t-1},\ v \in \mathcal{V} \right\}, \tag{5.9}$$

with $|\mathcal{C}_t| \le B \cdot |\mathcal{V}|$ candidates. *Partitioning:* candidates ending in END move to the finished set $\mathcal{F}$; the rest stay live. *Pruning:* the live candidates are ranked by S and only the top B survive:

$$\mathcal{H}_t = \underset{(h,s)\in\mathcal{C}_t^{\text{live}}}{\text{top-B}} \; s. \tag{5.10}$$

The search stops when B hypotheses have finished or a maximum length is reached, and $\mathcal{F}$ is ranked by the length-penalized score derived below. On the toy distribution with $B = 2$:

$$t = 1: \quad \mathcal{H}_1 = \{(\text{ON60}, -0.5978),\ (\text{ON64}, -1.2040)\}, \tag{5.11}$$

$$t = 2: \quad \text{expansions include } (\text{ON60}, \text{ON60}, -1.2910),\ (\text{ON64}, \text{ON64}, -1.6348),$$
$$\mathcal{H}_2 = \{(\text{ON60}, \text{ON60}), (\text{ON64}, \text{ON64})\}, \tag{5.12}$$

$$t = 3: \quad (\text{ON64}, \text{ON64}, \text{END}, -2.1456) \text{ finishes first},$$
$$\mathcal{F} \ni (\text{ON64}, \text{ON64}, \text{END}, -2.1456),\ (\text{ON60}, \text{ON60}, \text{END}, -2.3126). \tag{5.13}$$

The winner (ON64, ON64, END) at -2.1456 beats greedy's -2.3126 and matches the enumerated global optimum. Beam search is still approximate: a hypothesis pruned at step t can never be recovered, so the optimum is found only when it survives every pruning round.

Length penalty: why raw log-probability favors short sequences. The ranking in (5.10) and the final ranking compare sequences of different lengths, and the raw accumulated score is systematically biased in that comparison. The bias is a one-line lemma.

Lemma 5.1 (Prefix monotonicity of the raw score)**.** *For any partial hypothesis h and any token v, $S(h \circ v) \le S(h)$, with strict inequality unless $P(v \mid h, x) = 1$.*

Proof. From (5.8), $S(h \circ v) = S(h) + \log P(v \mid h, x)$. Since $P(v \mid h, x) \in (0, 1]$, its logarithm is non-positive, so adding it cannot increase the sum. It strictly decreases the sum whenever $P(v \mid h, x) < 1$. □

Every extra token appends a non-positive term, so among completed hypotheses the raw score prefers fewer terms: short sequences win ties of per-token quality, and during search the END token always looks attractive because stopping avoids further negative accumulation. The effect is visible even without search: a 3-token hypothesis with average token log-probability -0.83 scores -2.49, beating a 6-token hypothesis with average -0.50 scoring -3.00, even though the longer hypothesis is more confident per token.

The standard correction, introduced for neural machine translation [35], divides by a length penalty

$$\text{score}(Y) = \frac{S(Y)}{\text{lp}(|Y|)}, \qquad \text{lp}(L) = \left(\frac{5 + L}{6}\right)^{\alpha}, \tag{5.14}$$

with $\alpha = 0.6$ the widely used default. The penalty satisfies $\text{lp}(1) = 1$ (single-token hypotheses are unaffected), grows monotonically, and grows sublinearly, so it discounts length without overcorrecting. Applying (5.14) to the example above, with values recomputed in Python:

$$\begin{aligned} A: L = 3,\ S = -2.4900,\ \text{lp}(3) = 1.1884 \quad &\Rightarrow\ \text{score} = -2.0953, \\ B: L = 6,\ S = -3.0000,\ \text{lp}(6) = 1.4386 \quad &\Rightarrow\ \text{score} = -2.0853. \end{aligned} \tag{5.15}$$

The raw ranking (A wins, $-2.49 > -3.00$) flips under the penalty (B wins, $-2.0853 > -2.0953$): the longer, per-token more confident hypothesis is correctly preferred. Figure 5.3 shows the flip graphically. In the beam trace, the winner's penalized score is $-2.1456/1.1884 = -1.8054$.

Complexity analysis. Let C be one decoder step, T the output length, B the beam width. Per step, beam search runs B forward passes (batched into one in practice), expands $B \cdot |\mathcal{V}|$ candidates, and selects the top B in $\mathcal{O}(B|\mathcal{V}|)$ with linear-time selection:

$$\mathcal{O}(T \cdot (B \cdot C + B \cdot |\mathcal{V}|)) \quad \text{time}, \qquad \mathcal{O}(B \cdot T) \quad \text{memory}. \tag{5.16}$$

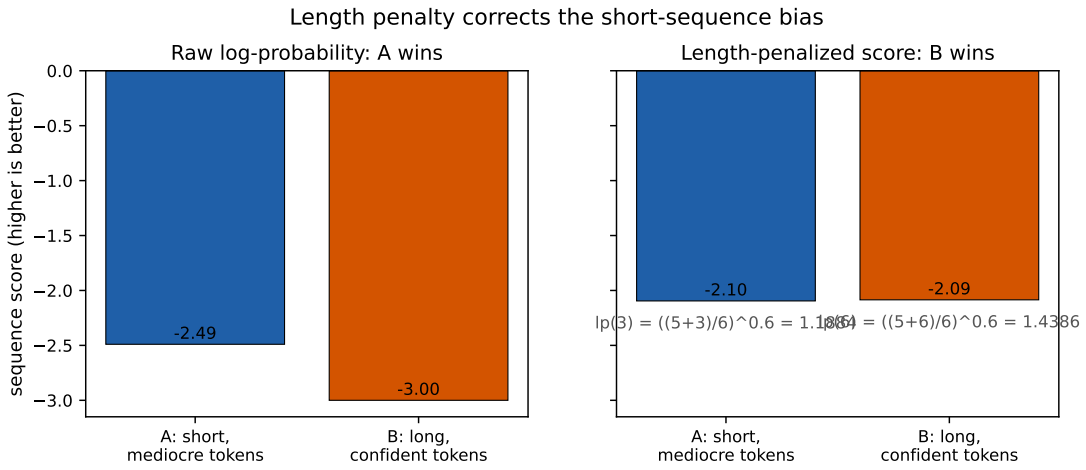


Figure 5.3: The length penalty corrects the short-sequence bias. Raw log-probability ranks the short mediocre hypothesis A first; after dividing by $\mathrm{lp}(L) = ((5+L)/6)^{0.6}$ the longer, per-token more confident hypothesis B wins.

Against greedy's $\mathcal{O}(T(C + |\mathcal{V}|))$ from (5.7), beam search multiplies cost by roughly B. Production systems shrink the $B \cdot |\mathcal{V}|$ term by expanding only the top-k tokens of each hypothesis distribution ($k \ll |\mathcal{V}|$), and shrink $B \cdot C$ by batching the B hypotheses into a single forward pass with a shared KV cache layout.

Step 4 — Runnable code. The program below implements beam search with the GNMT length penalty, prints the trace of (5.11)–(5.13), and demonstrates the ranking flip of (5.15) (saved as `code/m5_beam_search.py`).

```
"""Module 5 -- Autoregressive beam search with GNMT-style length
    penalty.

Reuses the toy decoder distribution from m5_greedy_decode.py. At each
    step
every live hypothesis is expanded by every vocabulary token and only
    the
top-B survivors are kept. Completed hypotheses are ranked by the
length-penalized score  S(Y) / lp(|Y|),  lp(L) = ((5 + L) / 6)^alpha,
which corrects the raw log-probability's bias toward short sequences.
"""
import math
from m5_greedy_decode import dist_of, greedy_decode

END = "END"


def length_penalty(num_tokens, alpha=0.6):
    """GNMT length penalty lp(L) = ((5 + L) / 6)^alpha (Wu et al.,
        2016)."""
    return ((5.0 + num_tokens) / 6.0) ** alpha


def beam_search(beam_width=2, max_steps=12, alpha=0.6):
    """Beam search. Returns (nbest, trace) where nbest is a list of
        dicts
    with keys: tokens, raw_logprob, penalized_score; sorted by
```

```
        penalized
    score, best first. trace holds per-step expansion tables for
        display."""
    live = [([], 0.0)]           # (token list, cumulative log-prob)
    finished = []                # completed hypotheses
    trace = []
    for step in range(1, max_steps + 1):
        expansions = []
        for tokens, lp in live:
            for tok, p in dist_of(tokens).items():
                expansions.append((tokens + [tok], lp + math.log(p)))
        expansions.sort(key=lambda e: e[1], reverse=True)
        trace.append((step, expansions))
        live = []
        for tokens, lp in expansions:
            if tokens[-1] == END:
                finished.append((tokens, lp))
            elif len(live) < beam_width:
                live.append((tokens, lp))
        if not live:
            break
    nbest = [
        {
            "tokens": tokens,
            "raw_logprob": lp,
            "penalized_score": lp / length_penalty(len(tokens), alpha),
        }
        for tokens, lp in finished
    ]
    nbest.sort(key=lambda h: h["penalized_score"], reverse=True)
    return nbest, trace


def length_penalty_demo():
    """Two hypotheses where the raw ranking and the penalized ranking
        differ."""
    A = {"name": "A (short)", "length": 3, "raw": 3 * -0.83}
    B = {"name": "B (long)", "length": 6, "raw": 6 * -0.50}
    print("Length-penalty demo (alpha = 0.6):")
    for h in (A, B):
        lp = length_penalty(h["length"])
        pen = h["raw"] / lp
        print(f"  {h['name']}: len={h['length']} raw={h['raw']:.4f} "
              f"lp={lp:.4f} penalized={pen:.4f}")
    raw_winner = "A" if A["raw"] > B["raw"] else "B"
    pen_winner = "A" if A["raw"] / length_penalty(3) > B["raw"] /
        length_penalty(6) else "B"
    print(f"  raw ranking winner: {raw_winner}; penalized ranking
        winner: {pen_winner}")


def main():
    nbest, trace = beam_search(beam_width=2, alpha=0.6)
    print("Beam search trace (B = 2):")
    for step, expansions in trace:
        print(f"  step {step}: top expansions")
        for tokens, lp in expansions[:4]:
            print(f"    {tuple(tokens)}  log-prob = {lp:.4f}")
```

```
    print("N-best list (ranked by length-penalized score):")
    for rank, h in enumerate(nbest, start=1):
        print(f"  {rank}. {tuple(h['tokens'])}
           raw={h['raw_logprob']:.4f} "
              f"penalized={h['penalized_score']:.4f}")
    g_tokens, g_lp, _ = greedy_decode()
    print(f"Greedy gave {tuple(g_tokens)} with raw log-prob
       {g_lp:.4f}; "
          f"beam winner raw log-prob {nbest[0]['raw_logprob']:.4f}.")
    print()
    length_penalty_demo()


if __name__ == "__main__":
    main()
```

5.4 Symbolic Music Language-Model Rescoring

Step 1 — Physical analogy. The acoustic model is a court stenographer with excellent hearing but no knowledge of the law: it writes down what it hears, phoneme by phoneme. The symbolic music language model is the judge who reads the transcript afterward and knows what sentences are plausible in this courtroom. When the stenographer writes a sentence that is acoustically faithful but legally nonsensical, the judge's prior knowledge corrects it. In transcription, the acoustic decoder proposes token sequences that fit the audio; the symbolic LM, pretrained on large corpora of MIDI scores it never needed audio for, scores how much each proposal sounds like real music. Fusing the two is called *shallow fusion* when the combination happens at the score level, as derived here, as opposed to deep fusion, which concatenates hidden states inside the network [36].

Step 2 — Geometric mental model. Fix two competing hypotheses and plot each one's fused score as a function of the LM weight λ: two straight lines with different slopes, since each line's slope is that hypothesis's LM log-probability. The acoustically preferred but musically implausible hypothesis starts higher (better acoustic score) but falls faster (worse LM score); the two lines cross at a crossover weight λ^*. Figure 5.4 shows this crossing: the shaded regions are the λ ranges where each hypothesis wins, so tuning λ is literally choosing which side of the crossing to stand on.

Step 3 — First-principles derivation. Start from the MAP objective (5.2) and apply Bayes' rule:

$$P(Y \mid x) = \frac{P(x \mid Y)\,P(Y)}{P(x)}, \qquad Y^* = \arg\max_Y \big[\log P(x \mid Y) + \log P(Y)\big], \tag{5.17}$$

where $P(x)$ is constant in Y and drops out. This is the noisy-channel decomposition: $P(x \mid Y)$ is the acoustic channel (how the audio arises from the score) and $P(Y)$ is the prior over scores (what music looks like). A discriminative seq2seq model estimates $P_{\text{ac}}(Y \mid x)$ directly instead of $P(x \mid Y)$, and a separately pretrained symbolic transformer LM estimates $P_{\text{LM}}(Y)$ from score corpora alone. Neither model is perfectly calibrated, and the two were trained on different data, so the principled sum is relaxed to a log-linear interpolation with a tunable weight $\lambda \geq 0$:

$$\text{score}(Y;\lambda) = \log P_{\text{ac}}(Y \mid x) + \lambda \log P_{\text{LM}}(Y). \tag{5.18}$$

At $\lambda = 0$ this recovers pure acoustic decoding; as $\lambda \to \infty$ the LM dominates. In practice λ is chosen by grid search on a validation set, exactly as one tunes any interpolation weight, and

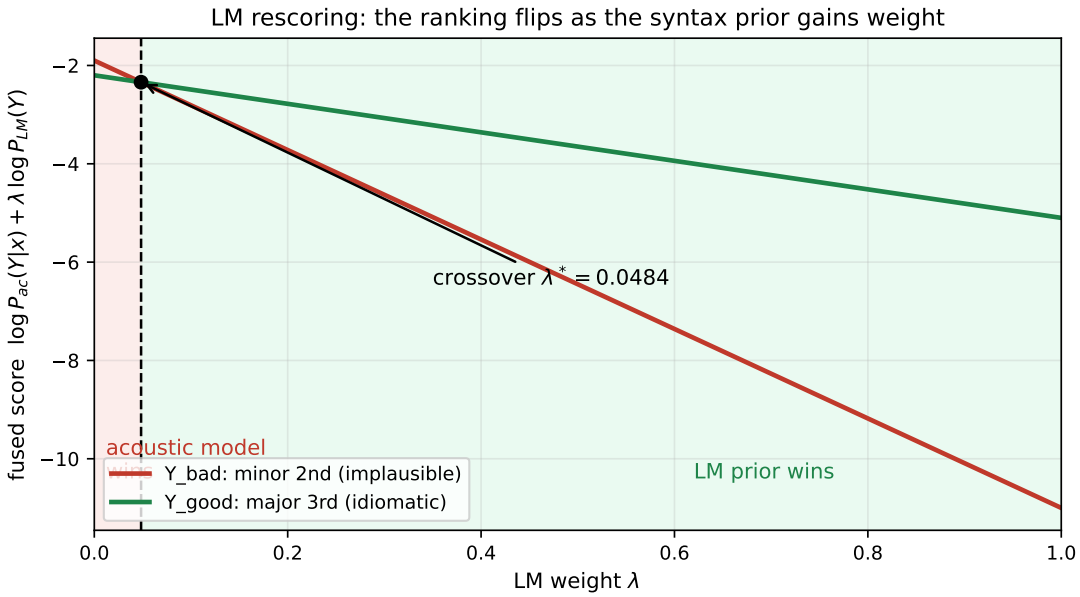


Figure 5.4: Shallow-fusion scores versus LM weight λ for two competing hypotheses. The lines cross at $\lambda^* = 0.0484$; below it the acoustic preference wins, above it the musical-syntax prior overrules the acoustic model.

rescoring is applied either to the n-best list from beam search or inside beam expansion itself (adding $\lambda \log P_{\text{LM}}(v \mid y_{<t})$ to each expansion's score).

The role of λ is sharpest in a worked example. Suppose beam search returns two hypotheses for one chord. The acoustic model slightly prefers a semitone cluster, an error typical of spectral leakage between adjacent pitches:

$$\begin{aligned} Y_{\text{bad}} &= [\text{NOTE_ON}(60), \text{NOTE_ON}(61), \text{TIME_SHIFT}] \quad \text{(minor 2nd)}, \\ Y_{\text{good}} &= [\text{NOTE_ON}(60), \text{NOTE_ON}(64), \text{TIME_SHIFT}] \quad \text{(major 3rd)}, \end{aligned} \tag{5.19}$$

with $\log P_{\text{ac}}(Y_{\text{bad}} \mid x) = -1.9$, $\log P_{\text{ac}}(Y_{\text{good}} \mid x) = -2.2$, $\log P_{\text{LM}}(Y_{\text{bad}}) = -9.1$ (a simultaneous minor second is nearly absent from the LM's training idiom), and $\log P_{\text{LM}}(Y_{\text{good}}) = -2.9$ (a major third is common). Evaluating (5.18):

λ	0.00	0.10	0.25	0.50	1.00
Y_{bad}	−1.900	−2.810	−4.175	−6.450	−11.000
Y_{good}	−2.200	−2.490	−2.925	−3.650	−5.100

(5.20)

At $\lambda = 0$ the acoustic error wins; at every $\lambda \geq 0.10$ the syntax prior rescues the major third. The exact crossover solves $\text{score}(Y_{\text{bad}}; \lambda^*) = \text{score}(Y_{\text{good}}; \lambda^*)$:

$$\lambda^* = \frac{\log P_{\text{ac}}(Y_{\text{good}} \mid x) - \log P_{\text{ac}}(Y_{\text{bad}} \mid x)}{\log P_{\text{LM}}(Y_{\text{bad}}) - \log P_{\text{LM}}(Y_{\text{good}})} = \frac{-2.2 - (-1.9)}{-9.1 - (-2.9)} = \frac{-0.3}{-6.2} = 0.0484, \tag{5.21}$$

and substituting back gives -2.3404 for both hypotheses, confirming the tie. Because the LM's penalty on the implausible hypothesis is so much larger than the acoustic model's margin, even a small λ flips the ranking: the prior needs little weight to correct errors the acoustic model makes confidently but wrongly. This is the characteristic use of LM rescoring in AMT: acoustic models err toward *acoustically* similar tokens (neighboring pitches, octave equivalents), while symbolic LMs penalize *musically* implausible combinations, and the two error patterns are nearly orthogonal, so their combination corrects both.

Step 4 — Runnable code. The program below implements log-linear rescoring, ranks candidates across a λ sweep, derives the crossover (5.21) in closed form, and asserts the tie numerically (saved as `code/m5_lm_rescore.py`).

```
"""Module 5 -- Symbolic music language-model rescoring (shallow
    fusion).

Given an n-best list from the acoustic (audio-conditioned) decoder,
    each
candidate Y is rescored with the log-linear objective

    score(Y ; lam) = log P_ac(Y | x) + lam * log P_LM(Y),

where P_ac is the acoustic seq2seq model and P_LM is a pretrained
    symbolic
music transformer LM. Sweeping lam shows how the LM weight shifts the
ranking; the crossover lam* is where the ranking flips.
"""
import math

# Worked example: the acoustic model slightly prefers a semitone
    cluster
# Y_bad = [NOTE_ON(60), NOTE_ON(61), TIME_SHIFT] (minor 2nd,
    implausible
# as a simultaneous sonority), while the LM strongly prefers the major
# third Y_good = [NOTE_ON(60), NOTE_ON(64), TIME_SHIFT].
LOGP_AC = {"Y_bad": -1.9, "Y_good": -2.2}   # acoustic log-probs
LOGP_LM = {"Y_bad": -9.1, "Y_good": -2.9}   # symbolic LM log-probs


def rescore(logp_ac, logp_lm, lam):
    """Log-linear shallow-fusion score for one candidate."""
    return logp_ac + lam * logp_lm


def rank_candidates(logp_ac, logp_lm, lam):
    """Rank all candidates at LM weight lam; best first."""
    scored = [(name, rescore(logp_ac[name], logp_lm[name], lam))
              for name in logp_ac]
    scored.sort(key=lambda s: s[1], reverse=True)
    return scored


def crossover_lambda(logp_ac, logp_lm, name_a, name_b):
    """Closed-form lam* where candidates a and b tie.

    Solves  ac_a + lam*lm_a = ac_b + lam*lm_b  for lam.
    """
    num = logp_ac[name_b] - logp_ac[name_a]
    den = logp_lm[name_a] - logp_lm[name_b]
    return num / den


def main():
    print("Symbolic LM rescoring (shallow fusion):")
    print("  Y_bad  = [NOTE_ON(60), NOTE_ON(61), TIME_SHIFT]  (minor
        2nd)")
    print("  Y_good = [NOTE_ON(60), NOTE_ON(64), TIME_SHIFT]  (major
```

```
        3rd)")
    for lam in (0.0, 0.10, 0.25, 0.50, 1.00):
        ranked = rank_candidates(LOGP_AC, LOGP_LM, lam)
        order = ", ".join(f"{n}={s:.3f}" for n, s in ranked)
        print(f"  lambda = {lam:4.2f}: {order}  -> winner
            {ranked[0][0]}")
    lam_star = crossover_lambda(LOGP_AC, LOGP_LM, "Y_bad", "Y_good")
    print(f"Crossover lambda* = {lam_star:.4f}: any lambda above this
        value")
    print("lets the musical-syntax prior overrule the acoustic
        preference.")
    # Sanity check: at lam* the two scores agree to numerical
        precision.
    assert math.isclose(rescore(LOGP_AC["Y_bad"], LOGP_LM["Y_bad"],
        lam_star),
                        rescore(LOGP_AC["Y_good"], LOGP_LM["Y_good"],
                            lam_star),
                        rel_tol=1e-9)
    print("Tie verified at lambda* (assertion passed).")


if __name__ == "__main__":
    main()
```

5.5 HMM State Smoothing for Frame-to-Note Clustering

Step 1 — Physical analogy. A frame-level transcription model is a telegraph operator receiving a message over a noisy line: the true message changes slowly (a note, once struck, persists), but static corrupts individual characters. Reading each character in isolation produces gibberish; a good operator uses the knowledge that real messages are coherent to correct the flicker. The hidden Markov model plays the operator: the hidden states are the true note phases, the noisy observations are the neural network's per-frame posteriors, and the transition probabilities encode the prior knowledge that note phases evolve slowly and in order.

Step 2 — Geometric mental model. Unfold the HMM over time into a *trellis*: a grid whose columns are frames and whose rows are states, with each cell (t, j) holding the best log-probability of any state path ending in state j at frame t. Viterbi decoding is dynamic programming on this grid: sweep left to right, at each cell keep only the best incoming edge, then backtrack the single winning path from the best final cell. Figure 5.5 shows the trellis for the worked example below: the dashed red path is the raw per-frame argmax, which flickers to `Off` at frame 5, while the green Viterbi path rides through the flicker on the strength of the transition prior.

Step 3 — First-principles derivation. For each pitch we model the note phase as a Markov chain over states $\mathcal{S} = \{\text{Off}, \text{Onset}, \text{Sustain}\}$ with initial distribution π, transition matrix $A = (a_{ij})$, and emission densities $b_j(x_t)$ mapping the frame model's observation x_t (here, its posterior vector) to a likelihood under state j. Using posteriors directly as emissions is the standard hybrid ANN–HMM trick. The joint distribution of observations $x_{1:T}$ and hidden states $z_{1:T}$ factorizes as

$$P(x_{1:T}, z_{1:T}) = \pi(z_1)\, b_{z_1}(x_1) \prod_{t=2}^{T} a_{z_{t-1} z_t}\, b_{z_t}(x_t). \tag{5.22}$$

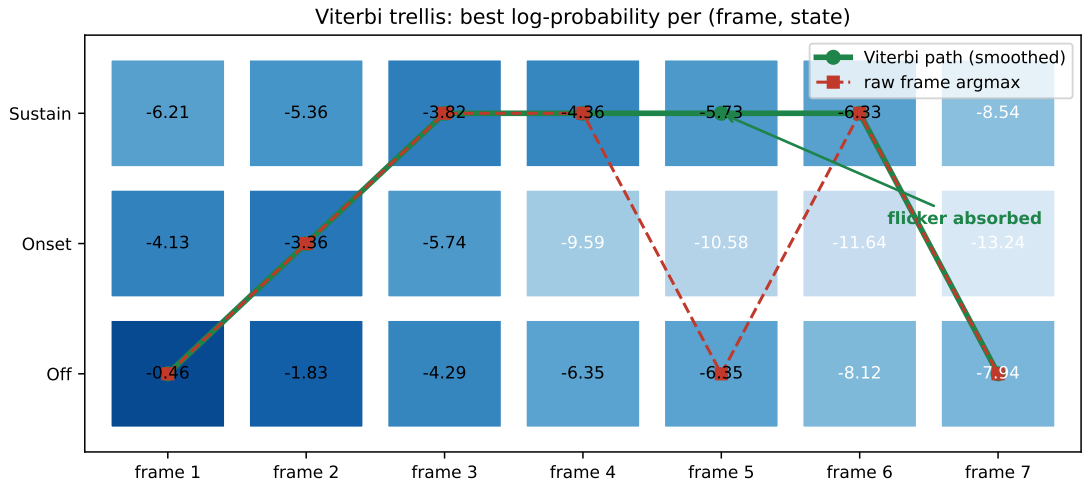


Figure 5.5: Viterbi trellis for the 3-state, 7-frame worked example. Cell values are best log-probabilities $\delta_t(j)$. The raw frame-level argmax (dashed red) flickers to `Off` at frame 5; the Viterbi path (green) absorbs the flicker and reports one continuous note.

Frame-to-note clustering is the decoding problem $z^* = \arg\max_z P(x_{1:T}, z)$ (equivalently $\arg\max_z \log P$), solved by the Viterbi recursion [37]. Define

$$\delta_t(j) = \max_{z_{1:t-1}} \log P(x_{1:t}, z_{1:t-1}, z_t = j), \tag{5.23}$$

the best log-probability of any partial path ending in state j at time t. Then $\delta_1(j) = \log \pi_j + \log b_j(x_1)$, and for $t > 1$,

$$\delta_t(j) = \max_{i \in \mathcal{S}} \left[\delta_{t-1}(i) + \log a_{ij}\right] + \log b_j(x_t), \tag{5.24}$$

with backpointers $\psi_t(j) = \arg\max_i[\delta_{t-1}(i) + \log a_{ij}]$. The recursion is proved by induction on t: any path counted in (5.23) passes through some state i at time $t-1$, contributing $\log a_{ij} + \log b_j(x_t)$ on its last step, so the best path through (t, j) extends the best path through $(t-1, i)$ for the best i — exactly the maximum in (5.24). Termination takes $P^* = \max_j \delta_T(j)$ and backtracks $z_T^* = \arg\max_j \delta_T(j)$, $z_t^* = \psi_{t+1}(z_{t+1}^*)$. Complexity is $\mathcal{O}(T|\mathcal{S}|^2)$ time and $\mathcal{O}(T|\mathcal{S}|)$ memory.

The worked example uses

$$\pi = (0.70, 0.20, 0.10), \quad A = \begin{pmatrix} 0.85 & 0.10 & 0.05 \\ 0.05 & 0.05 & 0.90 \\ 0.25 & 0.02 & 0.73 \end{pmatrix}, \tag{5.25}$$

with rows/columns ordered (Off, Onset, Sustain). The transition structure encodes musical common sense: attacks are brief (Onset $\to$ Sustain at 0.90), notes persist (Sustain $\to$ Sustain at 0.73), silence persists (Off $\to$ Off at 0.85). Emissions over 7 frames (frame-model posteriors per state):

$$B = \begin{pmatrix} 0.90 & 0.08 & 0.02 \\ 0.30 & 0.55 & 0.15 \\ 0.10 & 0.20 & 0.70 \\ 0.15 & 0.05 & 0.80 \\ 0.55 & 0.10 & 0.35 \\ 0.20 & 0.05 & 0.75 \\ 0.80 & 0.05 & 0.15 \end{pmatrix}, \quad \text{frame 5 is a flicker.} \tag{5.26}$$

Initialization, computed term by term:

$$\begin{aligned}\delta_1(\text{Off}) &= \log 0.70 + \log 0.90 = -0.3567 - 0.1054 = -0.4621,\\ \delta_1(\text{Onset}) &= \log 0.20 + \log 0.08 = -1.6094 - 2.5257 = -4.1351,\\ \delta_1(\text{Sustain}) &= \log 0.10 + \log 0.02 = -2.3026 - 3.9120 = -6.2146.\end{aligned} \tag{5.27}$$

One full recursion step, $\delta_2(\text{Sustain})$, with emission $\log b_{\text{Sustain}}(x_2) = \log 0.15 = -1.8971$:

$$\begin{aligned}i = \text{Off}: &\quad -0.4621 + \log 0.05 - 1.8971 = -5.3549,\\ i = \text{Onset}: &\quad -4.1351 + \log 0.90 - 1.8971 = -6.1376,\\ i = \text{Sustain}: &\quad -6.2146 + \log 0.73 - 1.8971 = -8.4264,\end{aligned} \tag{5.28}$$

so $\delta_2(\text{Sustain}) = -5.3549$ with $\psi_2(\text{Sustain}) = \text{Off}$. The complete trellis, recomputed in Python, is

$$\delta = \begin{pmatrix} -0.462 & -4.135 & -6.215 \\ -1.829 & -3.362 & -5.355 \\ -4.294 & -5.741 & -3.824 \\ -6.353 & -9.592 & -4.362 \\ -6.346 & -10.577 & -5.727 \\ -8.118 & -11.645 & -6.329 \\ -7.939 & -13.237 & -8.541 \end{pmatrix}. \tag{5.29}$$

Backtracking from $\arg\max_j \delta_7(j) = \text{Off}$ yields

$$z^* = (\text{Off}, \text{Onset}, \text{Sustain}, \text{Sustain}, \text{Sustain}, \text{Sustain}, \text{Off}), \tag{5.30}$$

with path log-probability -7.9387, while the raw per-frame argmax is (Off, Onset, Sustain, Sustain, Off, Sustain, it fragments the note at frame 5. The smoothing decision at the flicker frame is instructive in detail. With $\delta_4 = (-6.353, -9.592, -4.362)$ and emissions $(0.55, 0.10, 0.35)$:

$$\begin{aligned}\delta_5(\text{Sustain}) \text{ via Sustain} &= -4.362 + \log 0.73 + \log 0.35 = -5.7265,\\ \delta_5(\text{Off}) \text{ via Sustain} &= -4.362 + \log 0.25 + \log 0.55 = -6.3461,\end{aligned} \tag{5.31}$$

so the path stays in Sustain despite the emission favoring Off ($0.55 > 0.35$). The exact condition is visible in the difference: $\log(0.73/0.25) - \log(0.55/0.35) = 1.0716 - 0.4516 = 0.6200 > 0$. The transition prior's preference for persistence outweighs one contradictory emission frame, which is precisely the smoothing behavior a note tracker needs during the decaying tail of a piano tone, where frame posteriors legitimately wobble.

Step 4 — Runnable code. The program below implements log-domain Viterbi decoding, prints the trellis (5.29), the backpointers, and contrasts the smoothed path (5.30) with the raw argmax (saved as `code/m5_viterbi_hmm.py`).

```
"""Module 5 -- HMM state smoothing for frame-to-note clustering
    (Viterbi).

A per-pitch 3-state HMM with states {Off, Onset, Sustain} smooths the
    noisy
frame-level posteriors of a neural transcription model. Transition
probabilities encode musical common sense: attacks are brief (Onset ->
Sustain is likely), notes persist (Sustain self-loop is strong), and
silence persists (Off self-loop is strong). The Viterbi recursion finds
the single most likely state path in the log domain.
"""
import numpy as np
```

```

STATES = ("Off", "Onset", "Sustain")

PI = np.array([0.70, 0.20, 0.10])                          # initial distribution
A = np.array([[0.85, 0.10, 0.05],                           # transition matrix
              [0.05, 0.05, 0.90],
              [0.25, 0.02, 0.73]])
# Emission table: rows = frames, cols = states. Frame-model posteriors for
# one pitch over 7 frames; frame 5 is a flicker (raw argmax says Off).
B = np.array([
    [0.90, 0.08, 0.02],    # frame 1: silence
    [0.30, 0.55, 0.15],    # frame 2: attack
    [0.10, 0.20, 0.70],    # frame 3: sustain
    [0.15, 0.05, 0.80],    # frame 4: sustain
    [0.55, 0.10, 0.35],    # frame 5: FLICKER
    [0.20, 0.05, 0.75],    # frame 6: sustain again
    [0.80, 0.05, 0.15],    # frame 7: true offset
])


def viterbi(log_pi, log_A, log_B):
    """Viterbi decoding in the log domain.

    Returns (path, delta, psi): the most likely state index sequence, the
    trellis of best log-probabilities, and the backpointer table.
    """
    n_frames, n_states = log_B.shape
    delta = np.zeros((n_frames, n_states))
    psi = np.zeros((n_frames, n_states), dtype=int)
    delta[0] = log_pi + log_B[0]
    for t in range(1, n_frames):
        for j in range(n_states):
            prev = delta[t - 1] + log_A[:, j]
            psi[t, j] = int(np.argmax(prev))
            delta[t, j] = prev[psi[t, j]] + log_B[t, j]
    path = np.zeros(n_frames, dtype=int)
    path[-1] = int(np.argmax(delta[-1]))
    for t in range(n_frames - 2, -1, -1):
        path[t] = psi[t + 1, path[t + 1]]
    return path, delta, psi


def main():
    np.set_printoptions(precision=3, suppress=True)
    path, delta, psi = viterbi(np.log(PI), np.log(A), np.log(B))
    print("Viterbi trellis delta[t, state] (log domain):")
    print(delta)
    print("Backpointers psi[t, state]:")
    print(psi)
    smoothed = [STATES[i] for i in path]
    raw = [STATES[int(np.argmax(B[t]))] for t in range(B.shape[0])]
    print("Raw frame-level argmax path:", raw)
    print("Viterbi smoothed path:      ", smoothed)
    print(f"Best path log-probability: {delta[-1, path[-1]]:.4f}")
```

```
    print("The frame-5 flicker (Off) is absorbed: the transition
        prior")
    print("Sustain -> Sustain outweighs one contradictory emission
        frame.")


if __name__ == "__main__":
    main()
```

5.6 Minimum Note-Duration Filtering

Step 1 — Physical analogy. A mechanical pushbutton does not produce one clean electrical pulse when pressed: the contacts bounce, generating a burst of spurious short pulses. A debounce circuit ignores any pulse shorter than a threshold, passing only deliberate presses. Frame-level note detectors bounce the same way: key clicks, pedal thumps, sympathetic string resonance, and transient broadband energy at note boundaries all create onset spikes that survive thresholding for one or two frames. Minimum-duration filtering is the debounce circuit of transcription.

Step 2 — Geometric mental model. Plot a histogram of the durations of all detected note events, as in Figure 5.6. It is bimodal: a sharp spike of very short events (the bounce: 1–2 frames) separated from a broad hump of genuine notes. The filter draws a vertical cutoff $\tau_{\min}$ in the valley between the modes and discards everything to its left. Moving the cutoff trades precision against recall: too far right and genuine short notes (staccato articulation, fast ornamentation) are lost; too far left and bounce survives as insertion errors.

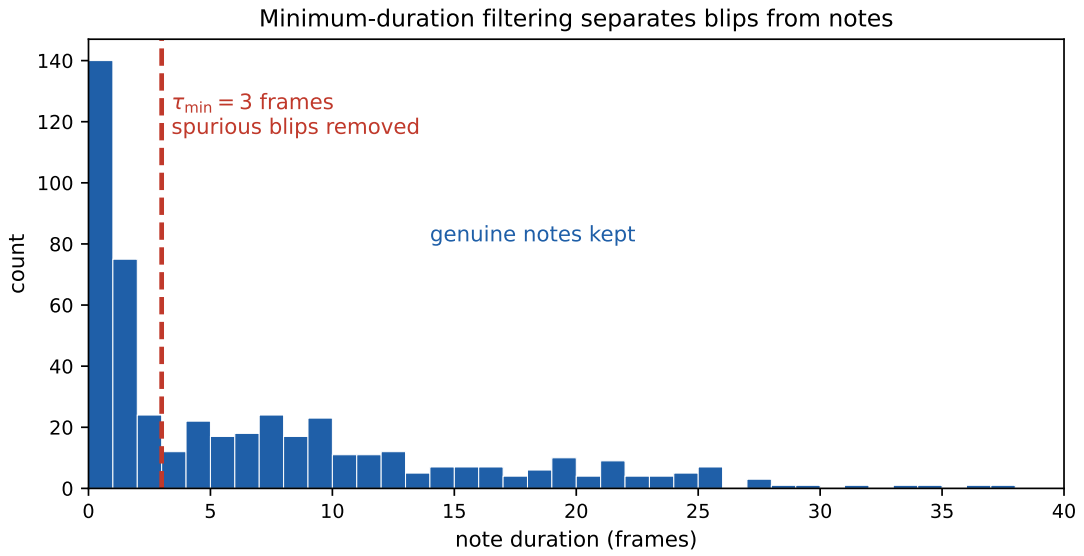


Figure 5.6: Duration histogram of detected note events (synthetic mixture of spurious blips and genuine notes). The cutoff $\tau_{\min} = 3$ frames removes the blip mode while keeping the genuine-note mode.

Step 3 — First-principles derivation. Let a detected note be the integer frame interval $[s, e]$ with duration $d = e - s + 1$ frames. The filter keeps the note iff

$$d \geq \tau_{\min}, \tag{5.32}$$

for a threshold $\tau_{\min}$ set either in frames (typically 3–5 at a 10 ms hop, i.e. 30–50 ms) or in musical time (a fraction of a beat, so that it scales with tempo). Three justifications converge on this rule. First, *perception*: events below roughly 30 ms are not heard as pitched notes at all, so deleting them cannot harm perceptual quality. Second, *notation*: legitimate notated durations have a floor (a 32nd note at 120 BPM lasts 62.5 ms); anything much shorter is overwhelmingly likely to be detector bounce. Third, *error analysis*: sub-threshold detections are dominated by transient energy that the onset detector cannot distinguish from a true attack, so the rule converts high-confidence insertion errors into a small, controlled recall risk. The rule is applied after onset–offset integration (Section 5.7) and after HMM smoothing (Section 5.5), since both stages can still emit short fragments. On the toy note list $[(2,3),(2,8),(8,8),(10,25),(30,31)]$ with $\tau_{\min}=3$ frames:

$$\text{kept: } (2,8),\ (10,25); \qquad \text{removed: } (2,3),\ (8,8),\ (30,31), \tag{5.33}$$

because durations $2,1,2$ fall below the cutoff while 7 and 16 survive. Note the deliberate conservatism: the rule never merges or splits notes, it only deletes, so its failure mode (deleting a genuine very short note) is explicit and tunable through $\tau_{\min}$.

Step 4 — Runnable code. Minimum-duration filtering is implemented by `filter_min_duration` in the post-processing program embedded in Section 5.7, which also performs hysteresis thresholding and velocity estimation end to end.

5.7 Dynamic Thresholding for Onset–Frame–Offset Integration

Step 1 — Physical analogy. A thermostat that switched the furnace on and off at exactly one temperature would chatter wildly as the room temperature hovered near the set point; real thermostats use two temperatures, one to switch on and a lower one to switch off, so that small fluctuations near the boundary change nothing. This is a Schmitt trigger, and note integration needs one for the same reason: a piano tone's energy decays exponentially after the attack, so the frame model's activity posterior legitimately dips mid-note. A single threshold would chop one sustained note into several fragments every time the posterior wobbled across the line; two thresholds make starting a note hard and continuing it easy.

Step 2 — Geometric mental model. Draw the frame-activity curve $f[t]$ and the onset curve $o[t]$ against time, with two horizontal lines: the high start threshold τ_{hi} and the low continue threshold τ_{lo}. A note begins only where an onset spike crosses its own gate *and* the frame curve is above τ_{hi}; the note's active interval then extends rightward until the frame curve falls below τ_{lo}, tolerating any dips that stay above the lower line. Figure 5.7 shows the worked trace: the shaded active region starts at frame 2 and survives the dip to 0.42 at frame 6 because $0.42 > \tau_{lo} = 0.3$.

Step 3 — First-principles derivation. For one pitch, let $o_t \in [0,1]$ be the onset posterior and $f_t \in [0,1]$ the frame-activity posterior at frame t. Define a two-state machine with state $q_t \in \{\text{inactive}, \text{active}\}$ and thresholds $\tau_{on}, \tau_{hi}, \tau_{lo}$ with $0 < \tau_{lo} \le \tau_{hi} < 1$. The transition logic is

$$\begin{aligned} q_t = \text{active} \ \wedge\ q_{t-1} = \text{inactive} &\iff o_t > \tau_{on} \ \wedge\ f_t > \tau_{hi}, \\ q_t = \text{inactive} \ \wedge\ q_{t-1} = \text{active} &\iff f_t < \tau_{lo}. \end{aligned} \tag{5.34}$$

The onset gate implements the key inference rule of the onsets-and-frames paradigm: a new note may not start unless the onset detector agrees an onset is present, which suppresses the frame model's tendency to hallucinate note starts inside sustained tones [38]. The frame gate

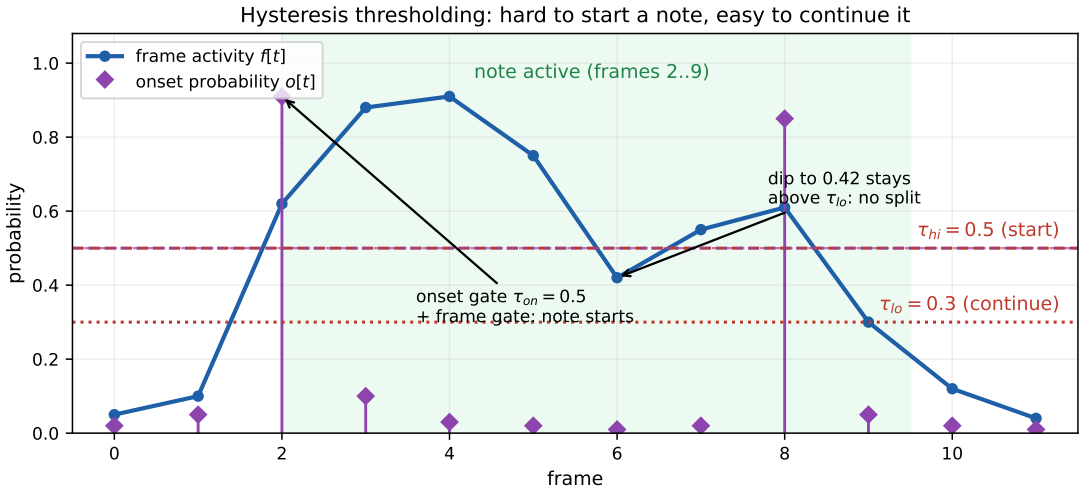


Figure 5.7: Hysteresis thresholding on the worked example. The note starts where the onset gate ($\tau_{on} = 0.5$) and the frame start gate ($\tau_{hi} = 0.5$) both fire, and continues until the frame curve drops below $\tau_{lo} = 0.3$; the mid-note dip to 0.42 does not split it.

$f_t > \tau_{hi}$ additionally requires the frame model to corroborate the onset, cutting false positives from percussive transients. Once active, only the lenient $f_t < \tau_{lo}$ condition can end the note, so mid-note dips in the decaying tail do not fragment it.

The "dynamic" in dynamic thresholding is the state-dependent effective threshold

$$\theta(t) = \begin{cases} \tau_{hi}, & q_{t-1} = \text{inactive}, \\ \tau_{lo}, & q_{t-1} = \text{active}, \end{cases} \tag{5.35}$$

which is exactly the Schmitt-trigger characteristic: the threshold follows the state, creating the hysteresis band $[\tau_{lo}, \tau_{hi}]$ inside which the current decision persists regardless of small input changes. Setting $\tau_{lo} = \tau_{hi}$ collapses the band and recovers single-threshold integration, with its fragmentation failure mode.

Worked trace with $\tau_{on} = \tau_{hi} = 0.5$, $\tau_{lo} = 0.3$:

t	0	1	2	3	4	5	6	7	8	9	10	11
f_t	.05	.10	.62	.88	.91	.75	.42	.55	.61	.30	.12	.04
o_t	.02	.05	.91	.10	.03	.02	.01	.02	.85	.05	.02	.01

(5.36)

At $t = 2$: $o_2 = 0.91 > 0.5$ and $f_2 = 0.62 > 0.5$, so the note starts. Frames 3–9 all satisfy $f_t \geq 0.3$, including the dip $f_6 = 0.42$, so the note continues; at $t = 10$, $f_{10} = 0.12 < 0.3$ ends it at frame 9. Extracted event: $(t_{\text{on}}, t_{\text{off}}) = (2, 9)$, one note of duration 8 frames. Two design decisions deserve explicit notice. First, the onset spike $o_8 = 0.85$ at frame 8 arrives while the note is active and is ignored by the basic rule: no retrigger. A legato re-articulation of the same pitch would therefore be merged; a retrigger extension (start a new note when o_t is strong even while active) fixes this at the cost of splitting notes on spurious onset spikes, and the choice is tuned per instrument. Second, the offset is placed at the last frame with $f_t \geq \tau_{lo}$, a coarse estimate; high-resolution systems regress the exact offset time instead, but the hysteresis structure is unchanged.

Step 4 — Runnable code. The program below implements hysteresis integration (`extract_notes`), minimum-duration filtering (`filter_min_duration`), and velocity estimation (`estimate_velocity`) as one post-processing pipeline, and runs the worked traces of (5.33) and (5.36) (saved as `code/m5_postprocess.py`).

```python
"""Module 5 -- Post-processing: hysteresis thresholding, minimum-duration
filtering, and velocity estimation.

For one pitch, the frame model emits per-frame activity probabilities f[t]
and onset probabilities o[t]. Notes are integrated with a two-threshold
hysteresis rule (hard to start, easy to continue), short spurious notes
are removed by a minimum-duration filter, and MIDI velocities are estimated
from frame energies with an anchor-calibrated dB-linear curve.
"""
import math
import numpy as np


def extract_notes(onset_probs, frame_probs, tau_on=0.5, tau_hi=0.5, tau_lo=0.3):
    """Integrate onset/frame posteriors into note events.

    A note starts at frame t when it is inactive, o[t] > tau_on and
    f[t] > tau_hi; it continues while f[t] >= tau_lo and ends at the last
    frame with f[t] >= tau_lo. Returns [(onset_frame, offset_frame)].
    """
    assert tau_lo <= tau_hi, "hysteresis requires tau_lo <= tau_hi"
    notes, active, start = [], False, 0
    n = len(frame_probs)
    for t in range(n):
        if not active:
            if onset_probs[t] > tau_on and frame_probs[t] > tau_hi:
                active, start = True, t
        elif frame_probs[t] < tau_lo:
            notes.append((start, t - 1))
            active = False
    if active:
        notes.append((start, n - 1))
    return notes


def filter_min_duration(notes, tau_min_frames):
    """Drop notes shorter than tau_min_frames (duration = e - s + 1)."""
    return [(s, e) for s, e in notes if (e - s + 1) >= tau_min_frames]


def fit_velocity_curve(db_lo, v_lo, db_hi, v_hi):
    """Fit v = a * dB + b through two (dBFS, velocity) anchor points."""
    a = (v_hi - v_lo) / (db_hi - db_lo)
    b = v_lo - a * db_lo
    return a, b


def estimate_velocity(energy, a, b):
    """Map a frame energy to a MIDI velocity in [1, 127]."""
    db = 20.0 * math.log10(max(energy, 1e-12))
```

```
    return int(min(127, max(1, round(a * db + b))))


def main():
    frame = np.array([0.05, 0.10, 0.62, 0.88, 0.91, 0.75,
                      0.42, 0.55, 0.61, 0.30, 0.12, 0.04])
    onset = np.array([0.02, 0.05, 0.91, 0.10, 0.03, 0.02,
                      0.01, 0.02, 0.85, 0.05, 0.02, 0.01])
    notes = extract_notes(onset, frame, tau_on=0.5, tau_hi=0.5,
        tau_lo=0.3)
    print("Hysteresis integration (tau_on=0.5, tau_hi=0.5,
        tau_lo=0.3):")
    print("  raw notes (onset_frame, offset_frame):", notes)
    print("  the dip to f=0.42 at frame 7 stays above tau_lo: no
        split.")

    toy_notes = [(2, 3), (2, 8), (8, 8), (10, 25), (30, 31)]
    kept = filter_min_duration(toy_notes, tau_min_frames=3)
    removed = [nt for nt in toy_notes if nt not in kept]
    print("Minimum-duration filter (tau_min = 3 frames):")
    print("  kept:", kept, " removed:", removed)

    a, b = fit_velocity_curve(db_lo=-40.0, v_lo=20.0, db_hi=-6.0,
        v_hi=110.0)
    print(f"Velocity curve: v = {a:.4f} * dB + ({b:.4f})")
    for energy in (0.0002, 0.001, 0.01, 0.05, 0.25):
        db = 20.0 * math.log10(energy)
        print(f"  E={energy:<7} -> {db:7.2f} dBFS -> velocity "
              f"{estimate_velocity(energy, a, b)}")


if __name__ == "__main__":
    main()
```

5.8 Velocity Estimation Curves

Step 1 — Physical analogy. Strike a piano key gently and the hammer barely kisses the string; strike it forcefully and the hammer's kinetic energy, proportional to the square of its velocity, drives a much larger string amplitude. Human loudness perception is logarithmic in physical energy (Weber–Fechner), so equal steps of perceived loudness correspond to multiplicative steps of energy. MIDI velocity is the 7-bit knob (1–127) that encodes this striking force, and estimating it means inverting the chain: from measured frame energy, through the logarithmic perceptual scale, to the integer knob.

Step 2 — Geometric mental model. Plot MIDI velocity against frame energy measured in decibels: the mapping is a straight line, because the decibel scale has already absorbed the logarithm. Two calibration anchors pin the line down — a quiet reference strike and a loud reference strike — and the line is clipped flat at the MIDI rails 1 and 127 outside the calibrated range. Figure 5.8 shows the fitted line with its anchors and the five worked examples sitting on it, two of them pinned to the floor by clipping.

Step 3 — First-principles derivation. Let $E > 0$ be a frame-level amplitude measure for the note (e.g. RMS amplitude over the attack frames, or peak onset strength), and define its

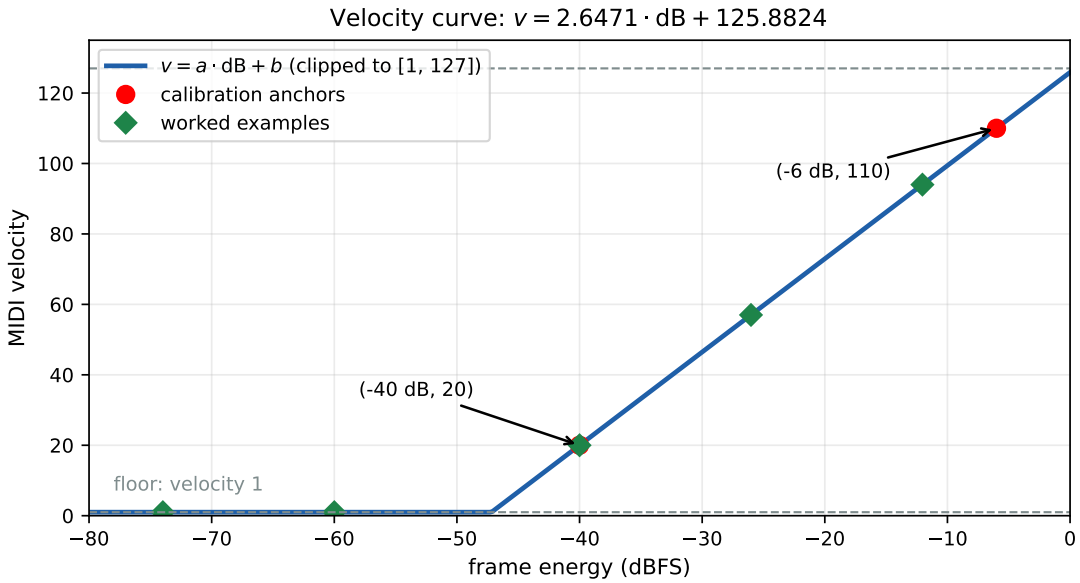


Figure 5.8: Anchor-calibrated velocity curve: linear in dBFS, clipped to MIDI $[1, 127]$. Green diamonds are the worked examples; the two quietest energies clip to velocity 1.

level in decibels full scale as

$$d = 20 \log_{10} E, \tag{5.37}$$

the standard amplitude-ratio form of the decibel (for a power quantity the equivalent form is $10 \log_{10}$). Posit a linear map from decibels to velocity,

$$v(d) = a \cdot d + b, \tag{5.38}$$

and calibrate (a, b) from two anchor measurements $(d_{\mathrm{lo}}, v_{\mathrm{lo}})$, $(d_{\mathrm{hi}}, v_{\mathrm{hi}})$ obtained from reference strikes (e.g. a measured pianissimo and fortissimo on the target instrument):

$$a = \frac{v_{\mathrm{hi}} - v_{\mathrm{lo}}}{d_{\mathrm{hi}} - d_{\mathrm{lo}}}, \qquad b = v_{\mathrm{lo}} - a\, d_{\mathrm{lo}}. \tag{5.39}$$

The final estimate clips to the valid MIDI range and rounds to an integer:

$$\hat{v} = \min\big(127,\, \max(1,\, \mathrm{round}(a \cdot d + b))\big). \tag{5.40}$$

With anchors $(-40\,\mathrm{dB}, 20)$ and $(-6\,\mathrm{dB}, 110)$:

$$a = \frac{110 - 20}{-6 - (-40)} = \frac{90}{34} = 2.6471, \qquad b = 20 - 2.6471 \cdot (-40) = 125.8824, \tag{5.41}$$

so $v(d) = 2.6471\, d + 125.8824$. Worked examples, recomputed in Python:

E	$d = 20 \log_{10} E$	$\hat{v}$
0.0002	-73.98 dB	1 (clipped)
0.001	-60.00 dB	1 (clipped)
0.01	-40.00 dB	20 (anchor)
0.05	-26.02 dB	57
0.25	-12.04 dB	94

(5.42)

Checking one row explicitly: $2.6471 \cdot (-26.02) + 125.8824 = -68.88 + 125.88 = 57.00$, which rounds to 57. The clipping behavior is not a defect but a design feature: energies below the quiet anchor carry no reliable dynamic information (they sit in the noise floor), so pinning them to velocity 1 is more honest than extrapolating the line into the noise. In production the anchors are fit per instrument or per recording session, since microphone gain shifts d additively and the linear-in-dB form absorbs a gain change into the intercept b.

Step 4 — Runnable code. Velocity estimation is implemented by `fit_velocity_curve` and `estimate_velocity` in the post-processing program embedded in Section 5.7, which prints the worked table (5.42).

5.9 Drills

1. **Beam search by hand.** Use the toy distribution of Section 5.2 with beam width $B = 2$. Write out the expansion set $\mathcal{C}_2$ at step 2 with all eight candidate log-probabilities, identify which two survive pruning, and confirm the final winner and its score. Then compute the greedy continuation starting from the history (ON64,) and show that it reaches (ON64, ON64, END) with total score -2.1456 — i.e. greedy recovers the optimum once its bad first decision is undone for it.

2. **Length-penalty sensitivity.** Two completed hypotheses have token log-probabilities $(-0.7, -0.7, -0.7, -0.7)$ (length 4) and eight tokens of -0.45 each (length 8). Compute the GNMT penalized scores (5.14) for $\alpha = 0.6$ and for $\alpha = 1.0$, name the winner under each setting, and explain in words what raising α does to the length bias.

3. **LM rescoring crossover.** Two candidates have $\log P_{\text{ac}}(A \mid x) = -3.0$, $\log P_{\text{LM}}(A) = -4.0$ and $\log P_{\text{ac}}(B \mid x) = -3.5$, $\log P_{\text{LM}}(B) = -2.0$. Derive the crossover weight λ^* from (5.18) by setting the two scores equal, evaluate it numerically, and determine the winner at $\lambda = 0.2$ and at $\lambda = 2.0$. Interpret what λ^* measures about the two hypotheses.

4. **Viterbi by hand.** A 2-state HMM has states $\{\text{Off}, \text{On}\}$, $\pi = (0.6, 0.4)$, $A = \left(\begin{smallmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{smallmatrix}\right)$, and emission posteriors per frame $(0.8, 0.2)$, $(0.4, 0.6)$, $(0.7, 0.3)$. Compute the full δ trellis and ψ backpointers in the log domain, backtrack the most likely state path, and give its log-probability. Contrast with the raw per-frame argmax path and explain the difference.

5. **Hysteresis versus a single threshold.** For one pitch, $f = (0.10, 0.55, 0.90, 0.35, 0.70, 0.20)$ and $o = (0.05, 0.80, 0.10, 0.05, 0.60, 0.02)$, with $\tau_{on} = \tau_{hi} = 0.5$. (a) Extract the note events with $\tau_{lo} = 0.3$. (b) Repeat with $\tau_{lo} = 0.5$ (single threshold). Count the resulting notes in each case and explain which error type (fragmentation vs. merging) each setting risks.

6. **Velocity calibration.** Anchors $(-50\,\text{dB}, 15)$ and $(-10\,\text{dB}, 100)$. Fit a and b in (5.39), then estimate velocities for frame energies $E \in \{10^{-5}, 10^{-3}, 0.1, 1.0\}$. Identify which estimates hit the clipping rails and explain why clipping there is preferable to linear extrapolation.

7. **Decoding cost at scale.** An output of $T = 500$ tokens is decoded from a vocabulary of $|\mathcal{V}| = 2000$. One batched decoder forward pass costs 4 ms (the same for $B = 1$ and $B = 8$); greedy argmax costs 0.1 ms per step; beam expansion plus top-B selection costs 1 ms per step. Estimate total decoding latency for greedy decoding and for beam search with $B = 8$, the slowdown factor, and the memory for stored hypotheses in 32-bit token ids. Explain why batching makes the beam-width multiplier much smaller than B in latency but not in memory.

Chapter 6

Rigorous AMT Evaluation Standards & Error Analysis

6.1 Introduction: Evaluation as a Contract

A transcription system outputs claims about the world: at each instant which pitches are sounding, and as events, which notes began, which ended, and how loudly. An evaluation metric is the contract that converts two such claim-sets — reference and estimate — into numbers. A sloppy contract rewards the wrong behavior: a system can raise its score by emitting notes nobody played, or by splitting every note in two, if the metric does not charge for those specific sins. This chapter therefore does two jobs. First, it derives the standard AMT metric suite from first principles — frame-level precision/recall/F_1, note-level F_1 under onset, onset+offset, and velocity-aware matching, and the multi-pitch error decomposition $E_{\text{tot}} = E_{\text{subs}} + E_{\text{miss}} + E_{\text{fa}}$ — with every tolerance window and matching rule made explicit and every worked number recomputed in code. Second, it turns the metric panel into a diagnostic instrument: four canonical failure modes (octave errors, harmonic over-segmentation, timbral leakage, pitch-drift tracking failures), each with its mechanism derived, its metric signature computed, and its targeted fix named. The conventions throughout are those of the `mir_eval` framework [40] and the MIREX evaluation protocols that shaped it [44, 41].

6.2 The Matching Problem: What Every Metric Must Decide

Reference and estimate are both sets — sets of active frame-pitch cells, or sets of note events. Before any counting can happen, the metric must decide *correspondence*: which estimated item is "the same as" which reference item. Correspondence is never exact identity, because human annotation and system output live on different time grids and because a 2 ms onset disagreement is musically meaningless. Every metric in this chapter is therefore a three-part machine:

1. a *compatibility predicate* (when may a reference item and an estimated item be paired?),
2. a *matching procedure* (which pairs are actually formed when several are possible?),
3. a *counting rule* (how do pairs, leftovers on each side, turn into TP/FP/FN and then into scores?).

Frame-level metrics use the trivial matching (same frame, same pitch). Note-level metrics need maximum bipartite matching under tolerance windows. The error decomposition needs per-frame pitch matching within 50 cents. The rest of the chapter is these three machines, worked all the way down to numbers.

6.3 Frame-Level Metrics: Precision, Recall, and F_1 from the Confusion Matrix

Step 1 — Physical analogy. Take a census of a piano roll. Lay a grid over time (frames) and pitch (MIDI notes); each cell asks one yes/no question: "is this pitch sounding in this frame?" The reference answers from the score; the system answers from its posteriors. Every cell is now a tiny referendum with two ballots, and the four possible ballot pairs are the entire story. Frame-level metrics are the election-night arithmetic: of all the cells where the system said "yes," how many were right (precision)? Of all the cells where the truth was "yes," how many did the system find (recall)?

Step 2 — Geometric mental model. Picture the grid of Figure 6.1: rows are frames, columns are pitches, and each cell is colored by its ballot pair — green for agreement on "yes" (TP), red for a system-only "yes" (FP), orange for a reference-only "yes" (FN), gray for agreement on "no" (TN). Precision is the green fraction of the green-plus-red column mass; recall is the green fraction of the green-plus-orange mass. The gray TN cells enter neither ratio, which is exactly why precision/recall survive the class imbalance of sparse piano rolls, where an accuracy score would be dominated by correctly predicted silence.

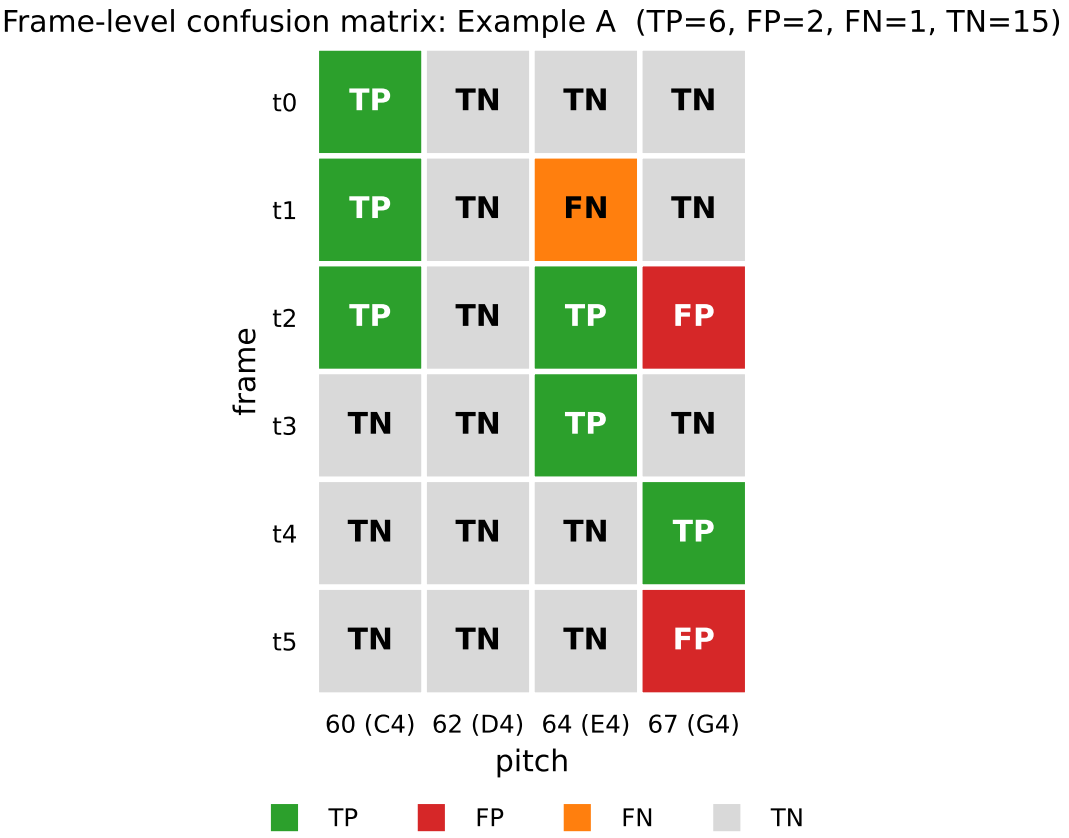


Figure 6.1: Frame-level confusion over Example A: 6 frames × 4 pitches. Each cell is one reference/estimate ballot pair (TP/FP/FN/TN). Precision counts green over green+red; recall counts green over green+orange.

Definition 6.1 (Frame-level confusion counts)**.** For a fixed frame grid and pitch vocabulary, compare the reference active set R_t and estimated active set E_t cell by cell. With $r = [p \in R_t]$

and $e = [p \in E_t]$:

$$\begin{aligned} \mathrm{TP} &= \#\{r = 1, e = 1\}, \quad \mathrm{FP} = \#\{r = 0, e = 1\}, \\ \mathrm{FN} &= \#\{r = 1, e = 0\}, \quad \mathrm{TN} = \#\{r = 0, e = 0\}, \end{aligned} \tag{6.1}$$

counted over all frames and pitches (micro-averaging).

Step 3 — First-principles derivation. Precision and recall are conditional probabilities estimated by counting:

$$P = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \qquad R = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}. \tag{6.2}$$

P answers "given the system claimed activity, how often was it right"; R answers "given true activity, how often did the system claim it." Neither alone suffices — a system claiming nothing has undefined P but $R = 0$; a system claiming everything has $R = 1$ but tiny P — so the field reports their harmonic mean:

$$F_1 = \frac{2}{1/P + 1/R} = \frac{2PR}{P + R}. \tag{6.3}$$

The harmonic (not arithmetic) mean is deliberate: for $P = 1, R = 0$ the arithmetic mean is $1/2$, a meaningless consolation, while the harmonic mean is 0, correctly reporting total failure. Reporting frame-level F_1 alongside the note-level F_1 scores of the next section is the dual standard established for piano AMT by Onsets and Frames [45]. Substituting (6.2) into (6.3) and clearing denominators step by step,

$$\frac{1}{P} + \frac{1}{R} = \frac{\mathrm{TP} + \mathrm{FP}}{\mathrm{TP}} + \frac{\mathrm{TP} + \mathrm{FN}}{\mathrm{TP}} = \frac{2\,\mathrm{TP} + \mathrm{FP} + \mathrm{FN}}{\mathrm{TP}}, \tag{6.4}$$

so that

$$F_1 = \frac{2\,\mathrm{TP}}{2\,\mathrm{TP} + \mathrm{FP} + \mathrm{FN}}, \tag{6.5}$$

a form that computes F_1 directly from counts with no intermediate rounding.

Example 6.1 (Worked frame-level numbers)**.** Example A uses 6 frames × 4 pitches (C4, D4, E4, G4), 24 cells, with the ballot pattern of Figure 6.1. Counting cell by cell: t_0 contributes TP at C4 and TN elsewhere; t_1 contributes TP at C4, FN at E4; t_2 contributes TP at C4 and E4, FP at G4; t_3 and t_4 contribute one TP each; t_5 contributes FP at G4. Totals:

$$\mathrm{TP} = 6, \quad \mathrm{FP} = 2, \quad \mathrm{FN} = 1, \quad \mathrm{TN} = 15, \tag{6.6}$$

and $6 + 2 + 1 + 15 = 24$ checks against the cell count. Then

$$P = \frac{6}{6 + 2} = 0.75, \qquad R = \frac{6}{6 + 1} = \frac{6}{7} \approx 0.857143, \tag{6.7}$$

$$F_1 = \frac{2 \cdot 0.75 \cdot 0.857143}{0.75 + 0.857143} = \frac{1.285714}{1.607143} = 0.8 = \frac{2 \cdot 6}{2 \cdot 6 + 2 + 1} = \frac{12}{15}, \tag{6.8}$$

so both forms (6.3) and (6.5) agree at $F_1 = 0.8$, as recomputed by the code below.

Step 4 — Runnable code. The listing below (`code/m6_frame_metrics.py`, verified with `python3 -m py_compile`) implements micro-averaged frame metrics and reproduces Example A exactly.

```
"""Module 6 -- frame-level AMT metrics from the confusion matrix.

A transcription is a binary matrix over (frame x pitch) cells. Each cell is
a reference/estimate pair, hence a confusion count:

    TP: pitch active in reference and estimate
    FP: pitch active in estimate only      (false alarm)
    FN: pitch active in reference only     (miss)
    TN: pitch active in neither

Micro-averaged precision P = TP/(TP+FP), recall R = TP/(TP+FN), and
F1 = 2PR/(P+R) = 2TP/(2TP+FP+FN) follow by algebra (see chapter).
"""


def frame_metrics(ref_frames, est_frames, pitches):
    """Micro-averaged P/R/F1 over all frame-pitch cells.

    ref_frames / est_frames: lists of sets of active MIDI pitches, one set
    per frame. pitches: the pitch vocabulary considered.
    """
    tp = fp = fn = tn = 0
    for ref_set, est_set in zip(ref_frames, est_frames):
        for p in pitches:
            r, e = (p in ref_set), (p in est_set)
            if r and e:
                tp += 1
            elif e and not r:
                fp += 1
            elif r and not e:
                fn += 1
            else:
                tn += 1
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    f1 = (2 * precision * recall / (precision + recall)
          if precision + recall else 0.0)
    return {"TP": tp, "FP": fp, "FN": fn, "TN": tn,
            "precision": precision, "recall": recall, "f1": f1}


def example_a():
    """Chapter Example A: 6 frames x 4 pitches (C4, D4, E4, G4)."""
    pitches = [60, 62, 64, 67]
    ref = [{60}, {60, 64}, {60, 64}, {64}, {67}, set()]
    est = [{60}, {60}, {60, 64, 67}, {64}, {67}, {67}]
    return frame_metrics(ref, est, pitches)


if __name__ == "__main__":
    a = example_a()
    print("Example A: TP=%d FP=%d FN=%d TN=%d"
          % (a["TP"], a["FP"], a["FN"], a["TN"]))
    print("P = %d/%d = %.4f" % (a["TP"], a["TP"] + a["FP"],
       a["precision"]))
```

```
    print("R = %d/%d = %.6f" % (a["TP"], a["TP"] + a["FN"],
        a["recall"]))
    print("F1 = 2PR/(P+R) = %.4f" % a["f1"])
    print("F1 = 2TP/(2TP+FP+FN) = %d/%d = %.4f"
          % (2 * a["TP"], 2 * a["TP"] + a["FP"] + a["FN"],
             2 * a["TP"] / (2 * a["TP"] + a["FP"] + a["FN"])))
```

6.4 Note-Level Metrics: Bipartite Matching under Tolerance Windows

Frame cells are cheap to match; notes are not. A note is an event (p, s, e, v) — MIDI pitch, onset, offset, velocity — and the system is allowed to be slightly wrong about s and e. The metric must therefore solve a correspondence problem with slack, and it must do so fairly: when two estimated onsets both fall near one reference onset, only one of them may claim the match.

Step 1 — Physical analogy. Think of a concert usher seating guests. Each reference note holds a reserved seat (its onset instant, plus or minus a small courtesy window). Each estimated note is a guest arriving near some seat. The usher's rules: a guest may only sit in a seat whose name tag matches their pitch exactly; they must arrive within the courtesy window; and each seat takes at most one guest, each guest takes at most one seat. When two guests both arrive within the window of one seat, the usher seats the pair that leaves the most guests seated overall — that is maximum bipartite matching, not first-come first-served.

Step 2 — Geometric mental model. Figure 6.2 draws Example B on a timeline: reference bars above, estimated bars below. The pale blue strips are the onset windows (±50 ms around each reference onset); the pale green strips are the offset windows (max(50 ms, 0.2 × duration) around each reference offset). A solid green link is a full onset+offset match; the dashed orange link is a match the onset-only metric accepts but the onset+offset metric rejects. The two red annotations are the two ways to fail: E2's onset lands 60 ms late (outside its window), and E4's pitch C5 has no reference partner at all.

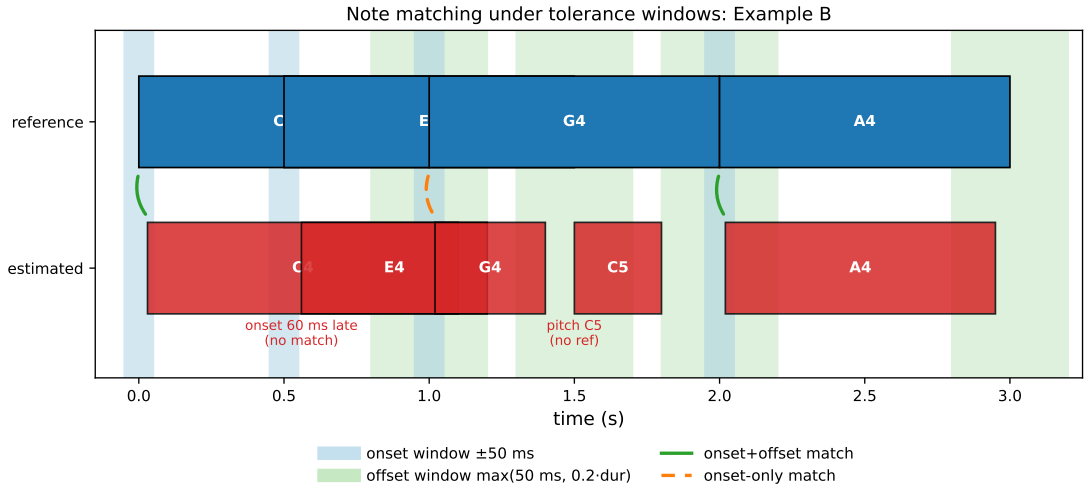


Figure 6.2: Note matching under tolerance windows (Example B). Blue strips: onset windows ±50 ms. Green strips: offset windows max(50 ms, $0.2 \cdot$ duration). Solid links survive the onset+offset test; the dashed link survives onset-only.

Definition 6.2 (Note compatibility and matching)**.** A reference note (p, s, e, v) and an estimated note $(\hat{p}, \hat{s}, \hat{e}, \hat{v})$ are *compatible* under a strictness level when all of the following hold:

$$\hat{p} = p, \qquad |\hat{s} - s| \le \tau_{\text{on}}, \qquad |\hat{e} - e| \le \tau_{\text{off}}(e - s) \quad \text{(offset levels only)}, \tag{6.9}$$

with the mir_eval conventions [40, 41]

$$\tau_{\text{on}} = 0.05 \text{ s}, \qquad \tau_{\text{off}}(d) = \max(0.05 \text{ s}, 0.2\, d). \tag{6.10}$$

The velocity-aware level adds $|\hat{v} - v| \le 0.1$ with velocities normalized to $[0, 1]$ [46]. Given the compatibility graph between the N reference and M estimated notes, the metric forms a *maximum bipartite matching* (Hungarian algorithm): the largest set of pairs with each note used at most once. Matched pairs are TP; unmatched estimated notes are FP; unmatched reference notes are FN.

Step 3 — First-principles derivation. Why these three levels, and why these windows? The onset-only F_1 asks "did the system find the right pitches at the right times" and forgives duration errors — the right metric for key-press detection and for instruments whose offsets are ambiguous (guitar, voice). The onset+offset F_1 additionally demands the note end near its true end; it is strictly harder and exposes duration modeling failures. The velocity-aware F_1 demands dynamics within 0.1 of truth, the piano literature's standard for expressive transcription [46].

The windows encode two different physical facts. The onset window is absolute ($\pm$50 ms) because onset perception has an absolute temporal resolution: a 50 ms error on a staccato eighth note and on a whole note are equally audible as mistiming. The offset window is relative ($0.2 \times$ duration, floored at 50 ms) because offset perception scales with duration: mistaking a 2 s note's end by 300 ms is a 15% error, while the same 300 ms on a 300 ms note doubles its length. The floor keeps very short notes from facing an impossibly tight window.

Maximum matching matters as soon as windows overlap. A greedy closest-assignment can strand a matchable pair: with reference onsets at 0.00 and 0.08 s and estimates at 0.049 and 0.075 s, the first estimate (processed in time order) is compatible with both references and greedily takes the closer one at 0.08 s; the second estimate is then stranded, because it is compatible only with 0.08 s. Greedy scores one match; the maximum matching pairs $0.049 \to 0.00$ and $0.075 \to 0.08$ for two. The Hungarian algorithm finds the optimum in $O(\max(N, M)^3)$ time, and the from-scratch implementation below uses it to realize the documented maximum-matching semantics.

Example 6.2 (Worked note-level numbers)**.** Example B has four reference notes and five estimates:

$$\begin{aligned} R_1 &= (60, 0.00, 1.00, 0.80), & E_1 &= (60, 0.03, 1.10, 0.85), \\ R_2 &= (64, 0.50, 1.50, 0.60), & E_2 &= (64, 0.56, 1.20, 0.55), \\ R_3 &= (67, 1.00, 2.00, 0.90), & E_3 &= (67, 1.02, 1.40, 0.88), \\ R_4 &= (69, 2.00, 3.00, 0.50), & E_4 &= (72, 1.50, 1.80, 0.70), \\ & & E_5 &= (69, 2.02, 2.95, 0.70). \end{aligned} \tag{6.11}$$

Compatibility, checked against (6.9)–(6.10): E_1 matches R_1 at every level (onset error 30 ms, offset error $0.10 \le \max(0.05, 0.2) = 0.20$, velocity error $0.05 \le 0.1$); E_2 fails R_2 at onset (60 ms > 50 ms) and at offset ($0.30 > 0.20$); E_3 matches R_3 on onset only (offset error $0.60 > 0.20$); E_4 matches nothing (pitch); E_5 matches R_4 on onset+offset but fails velocity ($0.20 > 0.1$). Maximum matching is unique here, giving:

$$\begin{aligned} &\text{onset-only:} && \text{TP} = 3,\ \text{FP} = 2,\ \text{FN} = 1, && P = 3/5 = 0.6,\ R = 3/4 = 0.75, && F_1 = 2/3 \approx 0.6667, \\ &\text{onset+offset:} && \text{TP} = 2,\ \text{FP} = 3,\ \text{FN} = 2, && P = 2/5 = 0.4,\ R = 2/4 = 0.5, && F_1 = 4/9 \approx 0.4444, \\ &\text{+velocity:} && \text{TP} = 1,\ \text{FP} = 4,\ \text{FN} = 3, && P = 1/5 = 0.2,\ R = 1/4 = 0.25, && F_1 = 2/9 \approx 0.2222. \end{aligned} \tag{6.12}$$

The ladder $0.6667 \to 0.4444 \to 0.2222$ is the point: each strictness level peels off exactly the errors it was designed to charge for — E_3's early offset at the second level, E_5's wrong dynamics at the third — while E_2's mistimed onset and E_4's phantom pitch are charged at every level.

Step 4 — Runnable code. The listing below (`code/m6_note_metrics.py`, verified with `python3 -m py_compile`) implements the three strictness levels from scratch: compatibility per (6.9)–(6.10), maximum bipartite matching via the Hungarian algorithm, and counting. It is an independent implementation of the *documented* mir_eval conventions (`mir_eval` itself was not installed in this build environment); the tolerances above were verified against the mir_eval documentation and the evaluation literature [40, 41, 46].

```
"""Module 6 -- from-scratch note-level AMT metrics.

Implements the documented mir_eval conventions (Raffel et al., ISMIR 2014):
  * note matching by maximum bipartite matching (Hungarian algorithm);
  * onset tolerance +/- 50 ms;
  * offset tolerance max(50 ms, 0.2 * reference duration);
  * exact MIDI-pitch equality;
  * velocity-aware variant: |v_est - v_ref| <= 0.1 on velocities
    normalized to [0, 1] (Kong et al., arXiv:2010.01815).

NOTE: this is an independent from-scratch implementation written for this
chapter (mir_eval was not installed in the build environment). It reproduces
the documented matching rules, not the mir_eval source code line-for-line.

Note representation: (pitch:int MIDI, onset:float s, offset:float s,
                      velocity:float in [0, 1]).
"""

import numpy as np
from scipy.optimize import linear_sum_assignment

INF = 1e9
ONSET_TOL = 0.05           # seconds, the mir_eval / MIREX convention
OFFSET_RATIO = 0.2         # offset tolerance = max(0.05, 0.2 * duration)
OFFSET_MIN_TOL = 0.05      # seconds
VELOCITY_TOL = 0.1         # on [0, 1]-normalized velocities


def _offset_tolerance(ref_duration):
    return max(OFFSET_MIN_TOL, OFFSET_RATIO * ref_duration)


def _notes_compatible(ref_note, est_note, require_offset, require_velocity):
    rp, rs, re, rv = ref_note
    ep, es, ee, ev = est_note
    if rp != ep:                                   # exact pitch equality
        return False
    if abs(es - rs) > ONSET_TOL:                   # +/- 50 ms onset window
```

```
        return False
    if require_offset and abs(ee - re) > _offset_tolerance(re - rs):
        return False
    if require_velocity and abs(ev - rv) > VELOCITY_TOL:
        return False
    return True


def note_metrics(ref_notes, est_notes, require_offset=True,
                 require_velocity=False):
    """Note-level P/R/F1 via maximum bipartite matching.

    require_offset=False  -> onset-only F1.
    require_offset=True   -> onset+offset F1.
    require_velocity=True -> additionally require |v_est - v_ref| <=
       0.1.
    """
    n, m = len(ref_notes), len(est_notes)
    if n == 0 and m == 0:
        return {"TP": 0, "FP": 0, "FN": 0,
                "precision": 1.0, "recall": 1.0, "f1": 1.0, "matches":
                   []}
    cost = np.full((max(n, 1), max(m, 1)), INF)
    for i in range(n):
        for j in range(m):
            if _notes_compatible(ref_notes[i], est_notes[j],
                                 require_offset, require_velocity):
                cost[i, j] = 0.0
    row, col = linear_sum_assignment(cost)
    matches = [(int(i), int(j)) for i, j in zip(row, col)
               if i < n and j < m and cost[i, j] < INF / 2]
    tp = len(matches)
    fp, fn = m - tp, n - tp
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    f1 = (2 * precision * recall / (precision + recall)
          if precision + recall else 0.0)
    return {"TP": tp, "FP": fp, "FN": fn, "precision": precision,
            "recall": recall, "f1": f1, "matches": matches}


def example_b_notes():
    ref = [(60, 0.00, 1.00, 0.80), (64, 0.50, 1.50, 0.60),
           (67, 1.00, 2.00, 0.90), (69, 2.00, 3.00, 0.50)]
    est = [(60, 0.03, 1.10, 0.85), (64, 0.56, 1.20, 0.55),
           (67, 1.02, 1.40, 0.88), (72, 1.50, 1.80, 0.70),
           (69, 2.02, 2.95, 0.70)]
    return ref, est


if __name__ == "__main__":
    ref, est = example_b_notes()
    print("Example B (note-level):")
    for name, kw in [("onset-only", dict(require_offset=False)),
                     ("onset+offset", dict(require_offset=True)),
                     ("onset+offset+velocity",
                      dict(require_offset=True,
                           require_velocity=True))]:
```

```
        s = note_metrics(ref, est, **kw)
        print("  %-20s TP=%d FP=%d FN=%d P=%.4f R=%.4f F1=%.6f
            matches=%s"
               % (name, s["TP"], s["FP"], s["FN"],
                  s["precision"], s["recall"], s["f1"], s["matches"]))
```

6.5 The Multi-Pitch Error Decomposition: $E_{\text{tot}} = E_{\text{subs}} + E_{\text{miss}} + E_{\text{fa}}$

Precision and recall split errors into two bins, but multi-pitch transcription has a third, distinct sin: reporting the *wrong* pitch where a pitch truly sounded. Precision/recall charge that mistake twice — one false positive for the phantom, one false negative for the missed truth — which over-penalizes confusions relative to pure misses. The NIST-style decomposition, introduced for multi-pitch evaluation by Poliner and Ellis [42] and used throughout the MIREX multiple-f_0 evaluations [43, 44], charges a substitution exactly once.

Step 1 — Physical analogy. Borrow the speaker-diarization scorecard, where this metric was born: in a meeting recording, each voice is either correctly attributed, missed entirely, hallucinated from silence, or *mislabelled* (speaker A tagged as speaker B). Mislabeling is one error, not two — the diarization system did not miss A *and* invent B; it confused them. Multi-pitch frames work the same way: at each frame the system emits a set of pitches, the reference holds a set of pitches, and the three error types are misses, false alarms, and substitutions.

Step 2 — Geometric mental model. Figure 6.3 stacks the three error types frame by frame for Example C. Each frame contributes a column; the column's segments are substitutions (red), misses (orange), false alarms (purple). The "total" column sums them: E_{tot} is literally the stacked height, normalized by the number of reference pitches. A system whose errors are mostly orange under-detects; mostly purple over-detects; mostly red confuses — and the figure lets you read that diagnosis at a glance.

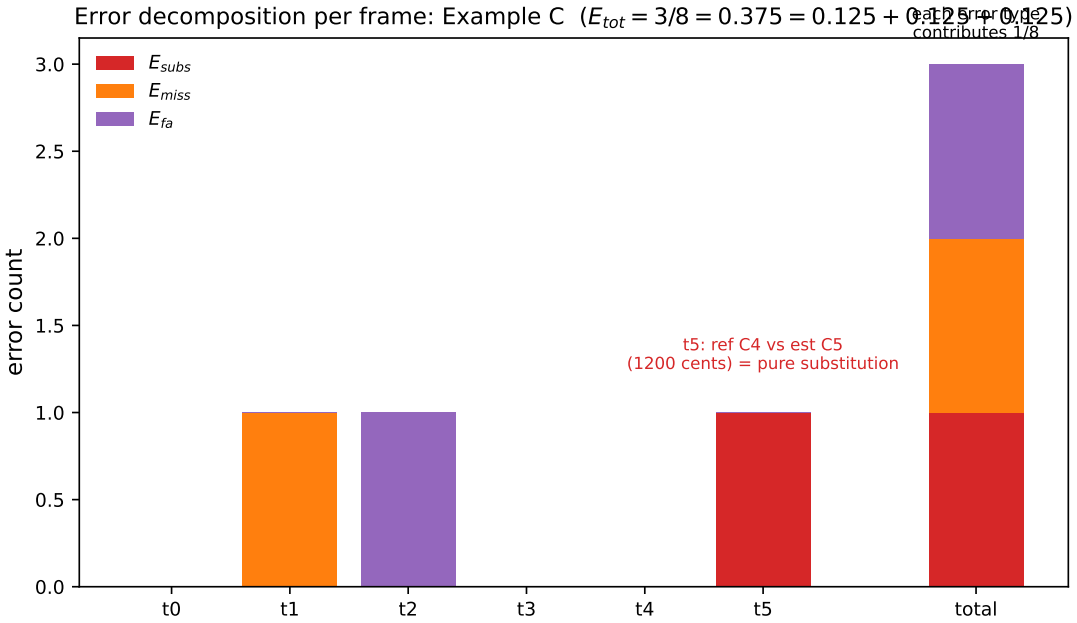


Figure 6.3: Per-frame error decomposition (Example C). Frame t_5 (reference C4, estimate C5, 1200 cents apart) is a pure substitution. Totals: $E_{\text{subs}} = E_{\text{miss}} = E_{\text{fa}} = 1/8$, $E_{\text{tot}} = 3/8$.

Definition 6.3 (Multi-pitch error rates)**.** At frame t let $N_{\text{ref}}(t)$ be the number of reference pitches, $N_{\text{sys}}(t)$ the number of estimated pitches, and $N_{\text{corr}}(t)$ the number of correctly detected pitches — the size of a maximum bipartite matching between the two pitch sets where a pair matches when the pitches agree within 50 cents (half a semitone), the MIREX multiple-f_0 convention [44]. The per-frame error counts are

$$\begin{aligned} E_{\text{subs}}(t) &= \min(N_{\text{ref}}(t), N_{\text{sys}}(t)) - N_{\text{corr}}(t), \\ E_{\text{miss}}(t) &= \max(0,\ N_{\text{ref}}(t) - N_{\text{sys}}(t)), \\ E_{\text{fa}}(t) &= \max(0,\ N_{\text{sys}}(t) - N_{\text{ref}}(t)), \end{aligned} \tag{6.13}$$

and the aggregate rates normalize by the total reference pitch count:

$$E_{\text{tot}} = \frac{\sum_t \big[\max(N_{\text{ref}}(t), N_{\text{sys}}(t)) - N_{\text{corr}}(t)\big]}{\sum_t N_{\text{ref}}(t)} = E_{\text{subs}} + E_{\text{miss}} + E_{\text{fa}}, \tag{6.14}$$

with $E_{\text{subs}} = \sum_t E_{\text{subs}}(t) / \sum_t N_{\text{ref}}(t)$ and analogously for the other two.

Step 3 — First-principles derivation. The decomposition is an algebraic identity, not an approximation. Fix one frame and write $a = N_{\text{ref}}$, $b = N_{\text{sys}}$, $c = N_{\text{corr}} \le \min(a, b)$. The frame's contribution to the numerator of (6.14) is $\max(a, b) - c$. Now expand the three error terms from (6.13):

$$\begin{aligned} E_{\text{subs}} + E_{\text{miss}} + E_{\text{fa}} &= \big[\min(a, b) - c\big] + \max(0, a - b) + \max(0, b - a) \\ &= \min(a, b) - c + |a - b| \\ &= \max(a, b) - c, \end{aligned} \tag{6.15}$$

using $\max(0, a - b) + \max(0, b - a) = |a - b|$ and $\max(a, b) = \min(a, b) + |a - b|$. Summing over frames and dividing by $\sum_t N_{\text{ref}}(t)$ gives exactly (6.14). The denominator choice is deliberate: errors are reported as a proportion of the quantity of notes actually present [43], which also means E_{tot} can exceed 1 when the system hallucinates more pitches than exist — it is an error *rate*, not a probability.

The motivating case from [43]: reference holds C3, the system reports C4. Classical counting records one FP and one FN (two errors); the system that reported nothing records one FN (one error) — perversely rewarding the more ignorant system. The decomposition records one substitution in the first case ($\min(1, 1) - 0 = 1$) and one miss in the second, ranking confusion as exactly one error worse than silence.

Example 6.3 (Worked error-decomposition numbers)**.** Example C reuses the pitch sets of Example A, with frame t_5 changed to an octave confusion (reference C4 = 261.63 Hz, estimate C5 = 523.25 Hz, 1200 cents apart, so $N_{\text{corr}} = 0$):

t	N_{ref}	N_{sys}	N_{corr}	$(E_{\text{subs}}, E_{\text{miss}}, E_{\text{fa}})$
0	1	1	1	$(0, 0, 0)$
1	2	1	1	$(0, 1, 0)$
2	2	3	2	$(0, 0, 1)$
3	1	1	1	$(0, 0, 0)$
4	1	1	1	$(0, 0, 0)$
5	1	1	0	$(1, 0, 0)$

(6.16)

Summing: $\sum_t N_{\text{ref}} = 8$; $\sum_t E_{\text{subs}} = 1$, $\sum_t E_{\text{miss}} = 1$, $\sum_t E_{\text{fa}} = 1$. Hence

$$E_{\text{subs}} = E_{\text{miss}} = E_{\text{fa}} = \frac{1}{8} = 0.125, \qquad E_{\text{tot}} = \frac{3}{8} = 0.375 = 0.125 + 0.125 + 0.125, \tag{6.17}$$

and the numerator identity checks frame by frame: $(0, 1, 1, 0, 0, 1)$ sums to 3. The octave error at t_5 is charged once, as a substitution — the signature this chapter will reuse for every octave hallucination.

Step 4 — Runnable code. The listing below (`code/m6_error_decomposition.py`, verified with `python3 -m py_compile`) computes per-frame counts with 50-cent bipartite matching, aggregates the three rates, and asserts the decomposition identity on every run.

```
"""Module 6 -- multi-pitch error-rate decomposition.

E_tot = E_subs + E_miss + E_fa   (Poliner & Ellis, EURASIP JASP 2007,
adapted from the NIST speaker-diarization error score; exposition and the
explicit frame formulas follow Pertusa & Inesta, EURASIP JASP 2012:27).

Per frame t, with N_ref(t) reference pitches, N_sys(t) estimated pitches,
and N_corr(t) correctly detected pitches (bipartite match within 50 cents):

    E_tot = sum_t [max(N_ref(t), N_sys(t)) - N_corr(t)] / sum_t N_ref(t)

    E_subs(t) = min(N_ref(t), N_sys(t)) - N_corr(t)   (mislabelled voices)
    E_miss(t) = max(0, N_ref(t) - N_sys(t))           (undetected voices)
    E_fa(t)   = max(0, N_sys(t) - N_ref(t))           (phantom voices)

The decomposition E_tot = E_subs + E_miss + E_fa holds frame-by-frame as an
algebraic identity: max(a,b) - c = [min(a,b) - c] + max(0,a-b) + max(0,b-a).
Unlike precision/recall, a substitution is charged ONCE (not as one FP plus
one FN).

Pitch sets are given in Hz; correctness uses the MIREX multiple-F0
convention of a 50-cent (half-semitone) tolerance with maximum bipartite
matching per frame.
"""

import numpy as np
from scipy.optimize import linear_sum_assignment

INF = 1e9
CENTS_TOL = 50.0


def _cents(a_hz, b_hz):
    return 1200.0 * abs(np.log2(a_hz / b_hz))


def frame_counts(ref_hz, est_hz):
    """Per-frame (subs, miss, fa, n_ref, n_corr) for one frame's pitch sets."""
    a, b = len(ref_hz), len(est_hz)
    if a and b:
        cost = np.array([[0.0 if _cents(r, e) <= CENTS_TOL else INF
                          for e in est_hz] for r in ref_hz])
        row, col = linear_sum_assignment(cost)
```

```
        corr = sum(1 for i, j in zip(row, col) if cost[i, j] < INF / 2)
    else:
        corr = 0
    subs = min(a, b) - corr
    miss = max(0, a - b)
    fa = max(0, b - a)
    # identity check at frame level: max(a,b) - corr == subs + miss + fa
    assert max(a, b) - corr == subs + miss + fa
    return subs, miss, fa, a, corr


def error_rates(ref_frames_hz, est_frames_hz):
    """Aggregate E_subs, E_miss, E_fa, E_tot over a list of frames."""
    s_subs = s_miss = s_fa = s_num = s_den = 0.0
    per_frame = []
    for rf, ef in zip(ref_frames_hz, est_frames_hz):
        subs, miss, fa, n_ref, n_corr = frame_counts(sorted(rf), sorted(ef))
        s_subs += subs
        s_miss += miss
        s_fa += fa
        s_num += max(len(rf), len(ef)) - n_corr
        s_den += n_ref
        per_frame.append((subs, miss, fa, n_ref, n_corr))
    e_subs, e_miss, e_fa = s_subs / s_den, s_miss / s_den, s_fa / s_den
    e_tot = s_num / s_den
    # the decomposition identity, aggregated:
    assert abs(e_tot - (e_subs + e_miss + e_fa)) < 1e-12
    return {"E_subs": e_subs, "E_miss": e_miss, "E_fa": e_fa,
            "E_tot": e_tot, "per_frame": per_frame}


def hz(midi):
    return 440.0 * 2.0 ** ((midi - 69) / 12.0)


def example_c_frames():
    ref = [{60}, {60, 64}, {60, 64}, {64}, {67}, {60}]
    est = [{60}, {60}, {60, 64, 72}, {64}, {67}, {72}]
    return ([{hz(p) for p in s} for s in ref],
            [{hz(p) for p in s} for s in est])


if __name__ == "__main__":
    ref_hz, est_hz = example_c_frames()
    out = error_rates(ref_hz, est_hz)
    print("Example C -- per frame (subs, miss, fa, N_ref, N_corr):")
    for t, (subs, miss, fa, n_ref, n_corr) in enumerate(out["per_frame"]):
        print("  t=%d  subs=%d miss=%d fa=%d N_ref=%d N_corr=%d"
              % (t, subs, miss, fa, n_ref, n_corr))
    print("E_subs=%.4f  E_miss=%.4f  E_fa=%.4f" %
          (out["E_subs"], out["E_miss"], out["E_fa"]))
    print("sum=%.4f  E_tot=%.4f  (identity holds: %s)" %
          (out["E_subs"] + out["E_miss"] + out["E_fa"], out["E_tot"],
           abs(out["E_tot"] - out["E_subs"] - out["E_miss"] -
               out["E_fa"]) < 1e-12))
```

```
98      print("note: frame t=5 (ref C4, est C5, 1200 cents apart) is a
            pure substitution")
```

6.6 Octave Errors: The Harmonic-Series Trap

Of all transcription failures, the octave error is the most instructive, because it is not a bug in any particular network — it is a theorem about harmonic sound. A system that hallucinates $2f_0$ or $f_0/2$ is reading the spectrum correctly and drawing the wrong conclusion, for reasons already present in Module 1's overtone series $f_n = n \cdot f_0$.

Step 1 — Physical analogy. Play a low C on a guitar and listen for the C an octave above: it is audibly "in there," because the string vibrates in halves as well as in whole. Now imagine a listener who can only hear energy at harmonic positions and must name the fundamental. The octave-up candidate says "my harmonics are at $2f_0, 4f_0, 6f_0, \ldots$" — and every one of those frequencies is genuinely ringing in the string. The octave-down candidate says "my harmonics are at $f_0/2, f_0, 3f_0/2, 2f_0, \ldots$" — and all of its even harmonics are genuinely ringing too; only its own fundamental is silent. Both phantoms are supported by real acoustic evidence. The detector's job is to prefer the true fundamental anyway.

Step 2 — Geometric mental model. Figure 6.4 plots the true C4 harmonic series as stems under a $1/n$ envelope, with the two phantom templates overlaid. The C5 (octave-up) template's markers sit exactly on the true series' even partials — the subset relation is visible: $H(2f_0) \subset H(f_0)$. The C3 (octave-down) template alternates: its even harmonics land on true partials (green), while its odd harmonics and its own fundamental find no energy at all (red, hollow). The picture already tells you which phantom is the more dangerous: the octave-up one is fully explained by the spectrum, the octave-down one half-explained.

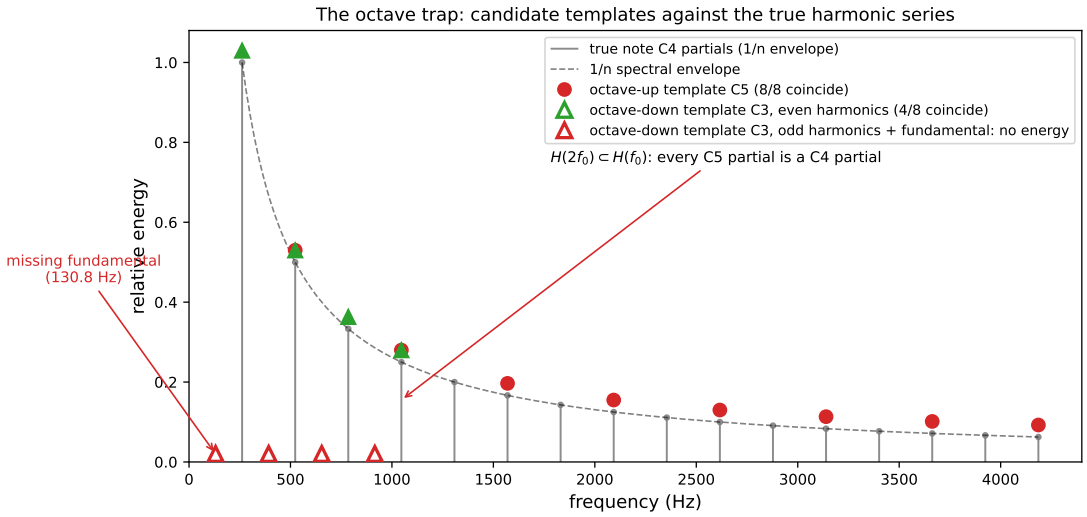


Figure 6.4: The octave trap. Gray stems: the true C4 harmonic series under a $1/n$ envelope. Red circles: the $2f_0$ (C5) template — all 8 harmonics coincide with true partials. Green/red triangles: the $f_0/2$ (C3) template — even harmonics coincide, odd harmonics and the fundamental find no energy.

Step 3 — First-principles derivation. Let the true note have fundamental f_0 and ideal harmonic series

$$H(f_0) = \{\, nf_0 : n = 1, 2, 3, \dots \}. \tag{6.18}$$

The octave-up candidate $f_c = 2f_0$ has series $H(2f_0) = \{\, n{\cdot}2f_0 \,\} = \{\, (2n)f_0 \,\}$, and every element $(2n)f_0$ is an element of $H(f_0)$. Hence

$$H(2f_0) \subset H(f_0) : \tag{6.19}$$

every partial the octave-up template asks for is present in the true spectrum. A detector scoring candidates by harmonic summation, $S(f_c) = \sum_{n=1}^{H} w_n\, E(nf_c)$ with uniform weights and the $1/n$ spectral envelope $E(kf_0) = A/k$ of Module 1, therefore finds

$$S(f_0) = A \sum_{n=1}^{8} \frac{1}{n} = A\, H_8, \qquad S(2f_0) = A \sum_{n=1}^{8} \frac{1}{2n} = \frac{A}{2}\, H_8, \tag{6.20}$$

where $H_8 = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \frac{1}{6} + \frac{1}{7} + \frac{1}{8} = 2.7178571429$. The ratio is exact:

$$\frac{S(2f_0)}{S(f_0)} = \frac{1}{2} = 0.5. \tag{6.21}$$

The octave-up phantom always scores half the truth under a $1/n$ envelope — close enough to win whenever the true fundamental is weak, masked, or poorly modeled.

The octave-down candidate $f_c = f_0/2$ is subtler. Its template harmonics $nf_0/2$ coincide with $H(f_0)$ exactly for even $n = 2k$ (landing on kf_0); odd n and the fundamental $f_0/2$ itself find no energy. Its score accumulates only the even template positions:

$$S(f_0/2) = A \sum_{k=1}^{4} \frac{1}{k} = A\, H_4, \qquad H_4 = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} = 2.0833333333, \tag{6.22}$$

giving

$$\frac{S(f_0/2)}{S(f_0)} = \frac{H_4}{H_8} = \frac{2.0833333333}{2.7178571429} \approx 0.766535. \tag{6.23}$$

The subharmonic phantom scores *76.65%* of the truth — out-scoring the octave-up phantom — because the true note's strongest partials (its low harmonics) all serve as evidence for $f_0/2$'s even template slots. This is the missing-fundamental illusion in quantitative form, and it is why octave-down errors dominate in bass registers where the true fundamental carries little energy.

The metric signature follows directly. At frame level, an octave error is a substitution: $N_{\text{ref}} = N_{\text{sys}} = 1$ but $N_{\text{corr}} = 0$ (1200 cents $\gg$ 50 cents), so E_{subs} absorbs it, exactly as frame t_5 of Example C showed. At note level, the pitch-equality rule $\hat{p} = p$ in (6.9) turns every octave hallucination into one FN (the true note) plus one FP (the phantom), depressing both precision and recall symmetrically. The targeted fixes all attack the subset relation: require fundamental energy before accepting a candidate (spectral template matching with a fundamental-presence term), train with octave-aware losses that penalize $\pm$12-semitone confusions more than distant ones, and prefer joint multi-pitch models that explain each partial once rather than letting one partial vote for two fundamentals.

Example 6.4 (Worked octave numbers)**.** For C4 = 261.626 Hz (MIDI 60, via $f(m) = 440 \cdot 2^{(m-69)/12}$ from Module 2): the $2f_0$ template (C5 = 523.251 Hz) has 8/8 harmonics on the true series; the $f_0/2$ template (C3 = 130.813 Hz) has 4/8, with its fundamental at 130.81 Hz unsupported. Harmonic-summation scores: $S(f_0) = 2.7179$, $S(2f_0) = 1.3589$ (ratio exactly 0.5), $S(f_0/2) = 2.0833$ (ratio 0.766535). The C4–C5 distance is exactly 1200.0 cents, hence a pure E_{subs} event at frame level.

Step 4 — Runnable code. The listing below (`code/m6_octave_analysis.py`, verified with `python3 -m py_compile`) computes template-support fractions and the harmonic-summation ratios (6.21)–(6.23) from the overtone series alone.

```
"""Module 6 -- why octave errors are inevitable: harmonic-series arithmetic.

For a true fundamental f0 with ideal harmonic series H(f0) = {n*f0 : n>=1}
(Module 1, Eq. f_n = n*f_0), this script shows:

  * the octave-up candidate fc = 2*f0 has EVERY template harmonic on the
    true series: H(2*f0) = {2n*f0} is a subset of H(f0);
  * the octave-down candidate fc = f0/2 has only its EVEN template harmonics
    supported; its fundamental f0/2 and all odd harmonics find no energy.

Under harmonic summation S(fc) = sum_n w_n * E(n*fc) with uniform weights
and a 1/n spectral envelope, the octave-up candidate scores exactly 1/2 of
the true fundamental, and the octave-down candidate scores H_4/H_8 ~ 0.7665
of it -- a phantom that out-scores the octave-up one. This is the mechanism
behind f0*2 / f0/2 hallucinations.
"""

import numpy as np

TEMPLATE_LEN = 8  # number of harmonics in the detector template


def midi_to_hz(m):
    return 440.0 * 2.0 ** ((m - 69) / 12.0)


def coincides_with_true_series(partial_hz, f0):
    """partial_hz equals n*f0 for some integer n >= 1 (the ideal series)."""
    ratio = partial_hz / f0
    return abs(ratio - round(ratio)) < 1e-9 and round(ratio) >= 1


def template_support(fc, f0, n_harmonics=TEMPLATE_LEN):
    """Fraction of the candidate's template harmonics present in H(f0)."""
    hits = [coincides_with_true_series(n * fc, f0)
            for n in range(1, n_harmonics + 1)]
    return sum(hits) / n_harmonics, hits


def harmonic_summation_score(fc, f0, n_harmonics=TEMPLATE_LEN):
    """S(fc) = sum_n E(n*fc) with 1/n envelope of the TRUE note, uniform w.

```

```
    Energy at frequency x is read off the true note's spectrum: if x = k*f0
    for integer k, the envelope gives A/k; otherwise 0.
    """
    A = 1.0
    total = 0.0
    for n in range(1, n_harmonics + 1):
        x = n * fc
        k = x / f0
        if abs(k - round(k)) < 1e-9 and round(k) >= 1:
            total += A / round(k)
    return total


if __name__ == "__main__":
    f0 = midi_to_hz(60)     # C4
    f_up = midi_to_hz(72)   # C5, octave up
    f_dn = midi_to_hz(48)   # C3, octave down
    print("C4 = %.3f Hz, C5 = %.3f Hz, C3 = %.3f Hz" % (f0, f_up, f_dn))
    print("octave distance C4->C5: %.1f cents"
          % (1200 * abs(np.log2(f_up / f0))))

    for name, fc in [("true f0  (C4)", f0),
                     ("octave-up candidate 2f0 (C5)", f_up),
                     ("octave-down candidate f0/2 (C3)", f_dn)]:
        frac, hits = template_support(fc, f0)
        print("%-32s template support %d/%d  %s"
              % (name, sum(hits), TEMPLATE_LEN, hits))

    s_true = harmonic_summation_score(f0, f0)
    s_up = harmonic_summation_score(f_up, f0)
    s_dn = harmonic_summation_score(f_dn, f0)
    print("harmonic-summation scores under a 1/n envelope (uniform weights):")
    print("  S(f0)   = %.4f" % s_true)
    print("  S(2f0)  = %.4f   ratio = %.4f (exactly 1/2)" % (s_up, s_up / s_true))
    h4 = sum(1.0 / k for k in range(1, 5))
    h8 = sum(1.0 / k for k in range(1, 9))
    print("  S(f0/2) = %.4f   ratio = %.6f (= H_4/H_8)" % (s_dn, s_dn / s_true))
    print("  H_4/H_8 = %.6f" % (h4 / h8))
    print("missing-fundamental check: C3's own fundamental %.2f Hz supported: %s"
          % (f_dn, coincides_with_true_series(f_dn, f0)))
```

6.7 Harmonic Over-Segmentation: One Note Becomes Three

Step 1 — Physical analogy. Hold a long note on a guitar and let it decay: the amplitude wobbles with beating partials and picking noise, and a frame-level detector watching the amplitude envelope sees it dip below threshold and rise again. Each dip looks like an ending; each rise looks like a new beginning. The system transcribes one sustained intention as three stuttering events — panel (b) of Figure 6.5. The audio contained one note; the metric sees three onsets and three offsets where the reference has one of each.

Step 2 — Geometric mental model. In panel (b) the reference is a single long bar and the estimate is three fragments separated by 50 ms gaps. Now overlay the tolerance windows from Figure 6.2: the first fragment's onset falls inside the reference onset window (a match for the onset-only metric), but every fragment's offset falls far outside the reference offset window ($|0.65 - 2.0| = 1.35 \gg \max(0.05, 0.4) = 0.4$), and the two extra onsets at 0.70 s and 1.40 s have no reference partner. The geometry predicts the signature before any arithmetic: onset metrics forgive, offset metrics do not.

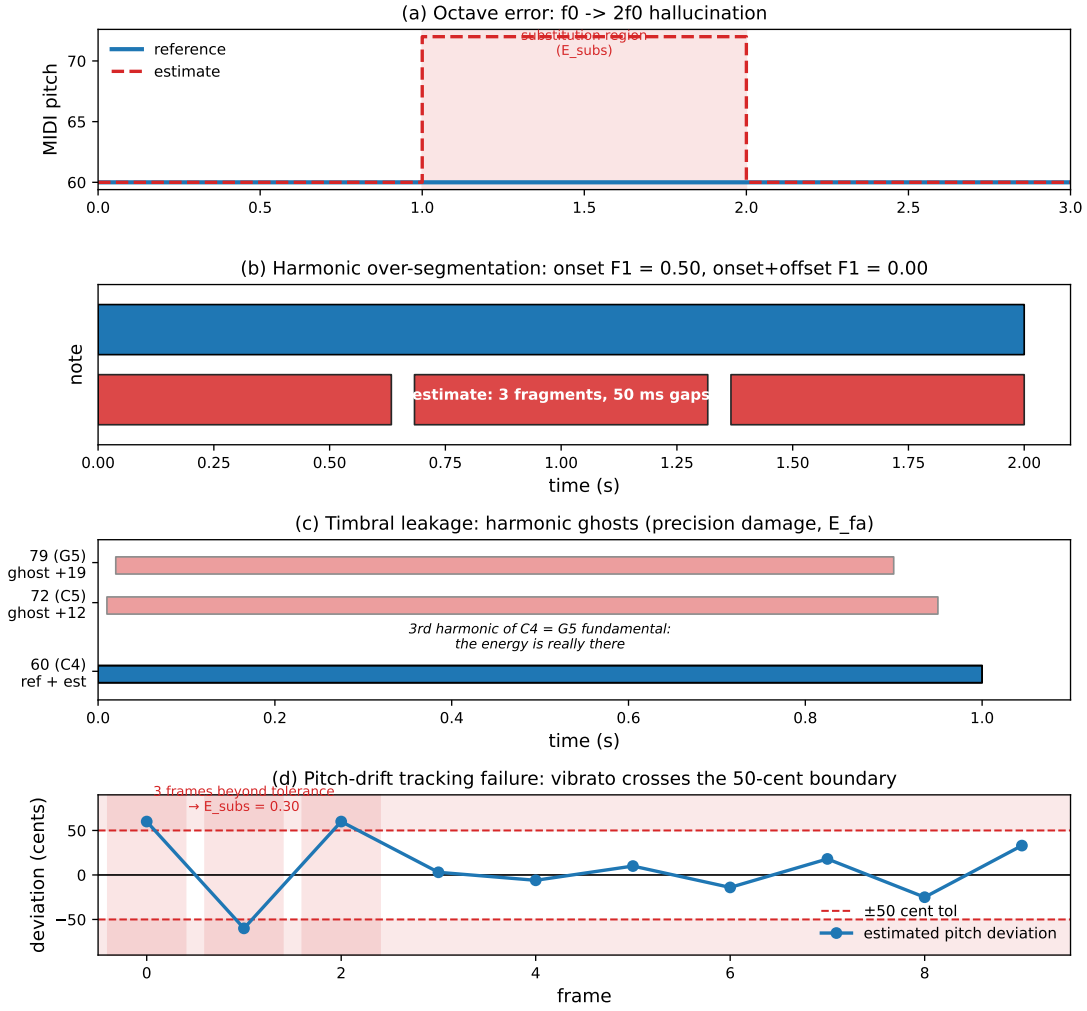


Figure 6.5: Four canonical failure modes. (a) Octave error: the estimate jumps +12 semitones mid-note (substitution region). (b) Over-segmentation: one 2 s note split into three fragments. (c) Timbral leakage: ghost notes at +12 and +19 semitones. (d) Pitch drift: frame deviations crossing the ±50-cent boundary become substitutions.

Step 3 — First-principles derivation. Take reference $R = (60, 0.0, 2.0, 0.8)$ and split it, as the simulator's `inject_over_segmentation` does, into three fragments of $(2.0 - 2 \cdot 0.05)/3 = 0.6333$ s separated by 50 ms gaps: $E_1 = (60, 0.0000, 0.6333)$, $E_2 = (60, 0.6833, 1.3167)$, $E_3 = (60, 1.3667, 2.0000)$. Onset-only matching: E_1's onset error is $0 \le 0.05$ (match); E_2 and E_3 have onset errors 0.6833 and 1.3667 (no match). Maximum matching gives TP = 1, FP = 2, FN = 0, hence

$$P = \frac{1}{3}, \quad R = 1, \quad F_1 = \frac{2 \cdot \frac{1}{3} \cdot 1}{\frac{1}{3} + 1} = \frac{2/3}{4/3} = 0.5. \tag{6.24}$$

Onset+offset matching: E_1's offset error is $|0.6333 - 2.0| = 1.3667 > \max(0.05, 0.2 \cdot 2.0) = 0.40$ (no match); E_2, E_3 fail similarly. Maximum matching is empty: TP = 0, FP = 3, FN = 1, hence $F_1 = 0$. The diagnostic signature of over-segmentation is therefore a *large onset-vs-offset F_1 gap* — here 0.50 vs 0.00 — with recall near 1 at onset level (no note was missed, only shattered). The targeted fix is temporal: minimum-duration filtering, gap-merging (join fragments separated by < 100 ms rests), and HMM state smoothing over frame posteriors before event extraction, all of which penalize rapid on/off oscillation directly.

Step 4 — Runnable code. The over-segmentation injector and its metric readout are part of the failure-mode simulator, `code/m6_failure_simulator.py`, embedded in full in the diagnostic playbook (Section 6.10); running it reproduces $F_1 = 0.50$ onset-only and $F_1 = 0.00$ onset+offset for the example above.

6.8 Timbral Leakage Across Overlapping Voices

Step 1 — Physical analogy. Two guitarists play in unison, or one guitarist double-stops a melody over a strummed chord: the spectrum at any instant is the sum of several harmonic series, and their partials interleave. A detector that assigns each spectral peak to at most one pitch — or worse, that never learned that the 3rd harmonic of C4 *is* the fundamental of G5 (Module 2's overlapping-partials problem) — leaks energy across voices: it hears the accompaniment's harmonic and reports a melody note nobody played. Panel (c) of Figure 6.5 shows the result: the true C4 plus two ghost notes at +12 (C5) and +19 (G5) semitones.

Step 2 — Geometric mental model. On the pitch axis the ghosts sit exactly on the true note's harmonic ladder: +12 semitones is the 2nd harmonic, +19 semitones the 3rd. The geometry of the failure is that the estimate's extra bars are *harmonically related* to a true bar — unlike random false alarms, which scatter across the pitch axis. This harmonic relationship is the diagnostic fingerprint: when your false positives cluster at +12/+19/ +24 semitones above true notes, you are not seeing noise, you are seeing leakage.

Step 3 — First-principles derivation. Reference $R = (60, 0.0, 1.0, 0.8)$; estimate adds ghosts $G_1 = (72, 0.01, 0.95, 0.3)$ and $G_2 = (79, 0.02, 0.90, 0.25)$. Onset-only matching: the true note matches (onset error 0); both ghosts fail pitch equality. TP = 1, FP = 2, FN = 0:

$$P = \frac{1}{3}, \quad R = 1, \quad F_1 = \frac{2 \cdot \frac{1}{3} \cdot 1}{\frac{1}{3} + 1} = 0.5. \tag{6.25}$$

At frame level each ghost frame contributes E_{fa}: with the ghost active for the note's full second, E_{fa} rises while E_{subs} and E_{miss} stay flat — the mirror image of the octave-error signature. The signature of timbral leakage is thus *precision damage with recall intact*, concentrated at harmonic intervals above true pitches. Targeted fixes: harmonic-structure modeling that explains each partial once (the same joint-estimation idea that cures octave errors), voice or instrument conditioning so the model knows which timbre it is transcribing, and multi-task training with an explicit "number of active voices" head that makes phantom voices expensive.

Step 4 — Runnable code. The `inject_timbral_leakage` injector in `code/m6_failure_simulator.py` (embedded in Section 6.10) reproduces (6.25): onset-only $F_1 = 0.50$ with TP = 1, FP = 2, FN = 0.

6.9 Pitch-Drift Tracking Failures

Step 1 — Physical analogy. A guitarist bends a string, or a violinist's vibrato wobbles the pitch ±40 cents around the nominal note. The note is unambiguously "still that note" to a listener — until the wobble grows past a semitone boundary and the ear reassigns it. Frame-level metrics draw their boundary at ±50 cents: a frame whose estimated pitch deviates more than that from truth stops being "correct" and becomes a substitution, even though no new note began.

Step 2 — Geometric mental model. Panel (d) of Figure 6.5 plots per-frame pitch deviation in cents with the ±50-cent tolerance as dashed lines. Frames inside the band are correct; frames outside are shaded red — each is a substitution event. Note-level metrics, which quantize to MIDI pitch, are blind to this entire failure until the drift crosses a full semitone; the frame-level decomposition sees it immediately. The two metric families are complementary sensors here.

Step 3 — First-principles derivation. Ten frames hold reference MIDI 60; the estimate drifts by +60 cents on three frames and stays within tolerance on seven. Per frame: $N_{\text{ref}} = N_{\text{sys}} = 1$ always; $N_{\text{corr}} = 1$ on the seven clean frames, 0 on the three drifted frames (60 cents > 50 cents). From (6.13), $E_{\text{subs}} = 3$, $E_{\text{miss}} = E_{\text{fa}} = 0$, and with $\sum_t N_{\text{ref}} = 10$,

$$E_{\text{subs}} = \frac{3}{10} = 0.30, \qquad E_{\text{tot}} = 0.30. \tag{6.26}$$

The signature: E_{subs} elevated with $E_{\text{miss}} = E_{\text{fa}} = 0$, on passages with vibrato, portamento, or string bends — while note-level F_1 looks healthy. Targeted fixes: pitch-contour smoothing (median filtering or Viterbi tracking over frame pitch posteriors) before note formation, training with pitch-jitter augmentation so the model learns the ±50-cent boundary, and for fretted instruments, snapping sustained pitch estimates to the nearest semitone once the onset is fixed.

Step 4 — Runnable code. `pitch_drift_frame_errors` in `code/m6_failure_simulator.py` (embedded in Section 6.10) recomputes (6.26): ten frames, three drifting by 60 cents, $E_{\text{subs}} = E_{\text{tot}} = 0.30$.

6.10 Reading the Metric Panel: A Diagnostic Playbook

Step 1 — Physical analogy. A physician does not treat "fever"; she reads temperature, blood panel, and imaging together and names the disease. The AMT metric panel works the same way: no single number diagnoses anything, but the *pattern across* frame F_1, the three note F_1 levels, and the $(E_{\text{subs}}, E_{\text{miss}}, E_{\text{fa}})$ triple identifies the failure mode almost uniquely. Table 6.1 is the differential diagnosis, derived from the worked examples of this chapter.

Step 2 — Geometric mental model. Figure 6.6 plots the panel's response as octave errors are injected at increasing rates into a four-note toy: both note F_1 curves fall linearly while E_{tot} rises linearly, and the decomposition attributes all of the rise to E_{subs}. Each failure mode has its own such response curve — over-segmentation would show the onset+offset curve collapsing while the onset-only curve merely sags; timbral leakage would show E_{fa} rising alone. Learning to read these curves is the practical skill this chapter exists to teach.

Step 3 — First-principles derivation. The table is not a heuristic; each row is a theorem about the metric machinery. Octave errors must land in E_{subs} because pitch equality fails while counts match ($\min(1,1) - 0 = 1$). Over-segmentation must split the onset/offset F_1 levels

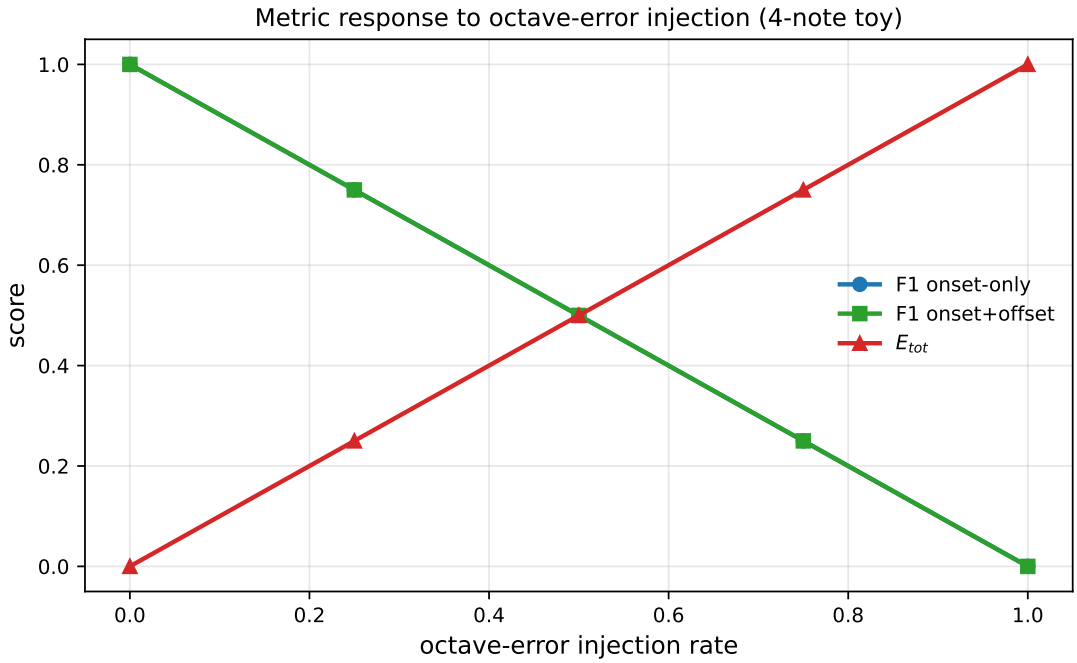


Figure 6.6: Metric response to octave-error injection (rate $0 \to 1$ on a four-note toy). Note F_1 scores fall linearly; E_{tot} rises linearly, carried entirely by E_{subs}.

Table 6.1: Differential diagnosis: failure mode from metric signature.

Failure mode	Metric signature	Worked numbers	Targeted
Octave error	$E_{\text{subs}} \uparrow$; note $P, R \downarrow$ symmetric	t_5: $E_{\text{subs}} = 1$; $S(2f_0)/S(f_0) = 0.5$	fundamer
Over-segmentation	onset+offset $F_1 \ll$ onset F_1; $R \approx 1$	0.50 vs 0.00	gap-merg
Timbral leakage	$P \downarrow$, $R \approx 1$; $E_{\text{fa}} \uparrow$ at +12/+19 st	$F_1 = 0.50$, TP= 1, FP= 2	explain-e
Pitch drift	$E_{\text{subs}} \uparrow$, $E_{\text{miss}} = E_{\text{fa}} = 0$; note F_1 healthy	$E_{\text{tot}} = 0.30$	contour s

because the fragments' onsets can fall inside ±50 ms windows while their offsets cannot fall inside $\max(50\text{ ms}, 0.2d)$ windows of the true (long) duration. Timbral leakage must damage precision alone because ghosts add estimated notes without removing true ones (FP without FN). Pitch drift must appear in E_{subs} at frame level and nowhere at note level, because the 50-cent frame tolerance is crossed while MIDI pitch equality still holds. When a real system's panel matches one of these rows, the mechanism is identified and the fix in the last column is the first thing to try.

Step 4 — Runnable code. The listing below (`code/m6_failure_simulator.py`, verified with `python3 -m py_compile`) implements all four injectors — `inject_octave_up`, `inject_over_segmentatio` `inject_timbral_leakage`, `pitch_drift_frame_errors` — plus the octave-error sweep behind Figure 6.6.

```
"""Module 6 -- failure-mode simulator: inject canonical AMT errors and read
the metric response.

Injectors operate on note lists of (pitch MIDI, onset s, offset s,
velocity in [0,1]) and report how the metric panel of Section 6.1 reacts:

  inject_octave_up(notes, indices)       -- f0 -> 2*f0 hallucination
  inject_over_segmentation(note, ...)    -- one note split into fragments
  inject_timbral_leakage(note, ...)      -- ghost notes at +12/+19 semitones
  pitch_drift_frame_errors(...)          -- frame-level cents wobble

The octave-error sweep at the bottom produces the data behind the metric-
response figure: as the octave-error rate grows, onset F1 and onset+offset
F1 fall while frame-level E_tot rises -- with E_subs carrying the damage.
"""

import numpy as np

from m6_note_metrics import note_metrics
from m6_error_decomposition import error_rates, hz


#
    ------------------------------------------------------------------------
# note-level injectors
#
    ------------------------------------------------------------------------
def inject_octave_up(notes, indices):
    """Shift the pitch of the chosen notes up one octave (+12 semitones)."""
    out = [tuple(n) for n in notes]
    for i in indices:
        p, s, e, v = out[i]
        out[i] = (p + 12, s, e, v)
    return out


```

```python
def inject_over_segmentation(note, n_fragments=3, gap=0.05):
    """Split one long note into fragments separated by small gaps."""
    p, s, e, v = note
    span = e - s
    frag_len = (span - (n_fragments - 1) * gap) / n_fragments
    frags = []
    t = s
    for _ in range(n_fragments):
        frags.append((p, t, t + frag_len, v))
        t += frag_len + gap
    return frags


def inject_timbral_leakage(note, intervals=(12, 19), ghost_velocity=0.3):
    """Append ghost notes a perfect octave (+12) and twelfth (+19) above.

    +19 semitones is the 3rd harmonic of the note (Module 2's overlapping-
    partials problem): its energy is genuinely present in the spectrum.
    """
    p, s, e, v = note
    ghosts = [(p + iv, s + 0.01 * k, e - 0.05 * k, ghost_velocity)
              for k, iv in enumerate(intervals, start=1)]
    return [tuple(note)] + ghosts


#
    ----------------------------------------------------------------------------
# frame-level: pitch drift in cents
#
    ----------------------------------------------------------------------------
def pitch_drift_frame_errors(n_frames=10, n_drift_frames=3, drift_cents=60.0,
                             tol_cents=50.0):
    """Reference holds MIDI 60; the estimate drifts > tol on some frames.

    Returns (E_subs, E_miss, E_fa, E_tot).
    """
    f0 = hz(60)
    ref = [{f0}] * n_frames
    est = [{f0 * 2.0 ** (drift_cents / 1200.0)} if t < n_drift_frames
           else {f0} for t in range(n_frames)]
    out = error_rates(ref, est)
    assert tol_cents == 50.0  # keep the MIREX convention explicit
    return out["E_subs"], out["E_miss"], out["E_fa"], out["E_tot"]


#
    ----------------------------------------------------------------------------
# metric-response sweep: octave-error rate 0 -> 1
#
    ----------------------------------------------------------------------------
def octave_error_sweep():
    """Deterministic sweep over the 4 reference notes of Example B."""
    ref = [(60, 0.00, 1.00, 0.80), (64, 0.50, 1.50, 0.60),
```

```
            (67, 1.00, 2.00, 0.90), (69, 2.00, 3.00, 0.50)]
    rows = []
    for k in range(5):  # corrupt the first k of 4 notes
        est = inject_octave_up(ref, list(range(k)))
        f1_on = note_metrics(ref, est, require_offset=False)["f1"]
        f1_off = note_metrics(ref, est, require_offset=True)["f1"]
        # frame-level: 5 frames per note, octave error flips the pitch
            cell
        ref_fr = [{p} for (p, _, _, _) in ref for _ in range(5)]
        est_fr = [{p + 12 if i < k else p} for i, (p, _, _, _) in
                  enumerate(ref) for _ in range(5)]
        e = error_rates([{hz(p) for p in s} for s in ref_fr],
                        [{hz(p) for p in s} for s in est_fr])
        rows.append((k / 4, f1_on, f1_off, e["E_tot"], e["E_subs"]))
    return rows


if __name__ == "__main__":
    print("octave-error injection sweep (rate, F1 onset, F1
        onset+offset, "
          "E_tot, E_subs):")
    for rate, f1_on, f1_off, e_tot, e_subs in octave_error_sweep():
        print("  rate=%.2f  F1_on=%.4f  F1_onoff=%.4f  E_tot=%.4f
            E_subs=%.4f"
              % (rate, f1_on, f1_off, e_tot, e_subs))

    print("over-segmentation of a 2 s note into 3 fragments (gap 50
        ms):")
    ref1 = [(60, 0.0, 2.0, 0.8)]
    est1 = inject_over_segmentation(ref1[0])
    for name, kw in [("onset-only", dict(require_offset=False)),
                     ("onset+offset", dict(require_offset=True))]:
        s = note_metrics(ref1, est1, **kw)
        print("  %-12s F1=%.4f (TP=%d FP=%d FN=%d)"
              % (name, s["f1"], s["TP"], s["FP"], s["FN"]))

    print("timbral leakage: +12/+19 ghosts on a single C4:")
    ref2 = [(60, 0.0, 1.0, 0.8)]
    est2 = inject_timbral_leakage(ref2[0])
    s = note_metrics(ref2, est2, require_offset=False)
    print("  onset-only F1=%.4f (TP=%d FP=%d FN=%d) -- precision
        damage"
          % (s["f1"], s["TP"], s["FP"], s["FN"]))

    print("pitch drift: 10 frames, 3 drifting by 60 cents (> 50-cent
        tol):")
    e_subs, e_miss, e_fa, e_tot = pitch_drift_frame_errors()
    print("  E_subs=%.2f E_miss=%.2f E_fa=%.2f E_tot=%.2f"
          % (e_subs, e_miss, e_fa, e_tot))
```

6.11 Drills

1. Prove algebraically that the harmonic-mean definition $F_1 = 2PR/(P+R)$ equals the count form $2\mathrm{TP}/(2\mathrm{TP}+\mathrm{FP}+\mathrm{FN})$, showing every substitution. Then compute P, R, F_1 for the confusion counts $\mathrm{TP}=41$, $\mathrm{FP}=7$, $\mathrm{FN}=13$, $\mathrm{TN}=939$, and explain why TN does not appear in F_1.

2. Reference onsets (all MIDI 60) at 0.00 s and 0.08 s; estimates at 0.049 s and 0.075 s, with the ± 50 ms onset window. (a) Show that greedy closest-assignment in time order (each estimate takes the closest unmatched compatible reference) yields only 1 match. (b) Show that maximum bipartite matching yields 2 matches. (c) Compute onset-only P, R, F_1 under both assignments, and state which one the mir_eval convention requires.

3. A reference note spans $[0.0, 0.3]$ s and another spans $[0.0, 3.0]$ s. Compute the mir_eval offset tolerance for each. An estimate ends each note 0.09 s late. Which note still matches under onset+offset, and why does the relative (20%) rule exist?

4. Prove the per-frame decomposition identity $\max(a, b) - c = [\min(a, b) - c] + \max(0, a - b) + \max(0, b - a)$ for $0 \le c \le \min(a, b)$, and explain in words what each term charges for. Then compute E_{subs}, E_{miss}, E_{fa}, E_{tot} for: frame 1: ref $\{A\}$, est $\{A\}$; frame 2: ref $\{A, B\}$, est $\{B, C\}$ where C is 1200 cents from A; frame 3: ref $\{A\}$, est $\{\}$.

5. Generalize the chapter's harmonic-summation analysis to a $1/n^2$ spectral envelope with uniform weights and an 8-harmonic template. Compute $S(2f_0)/S(f_0)$ and $S(f_0/2)/S(f_0)$ exactly as rational expressions in generalized harmonic numbers, evaluate them numerically, and explain why a steeper spectral roll-off changes which octave phantom is more dangerous.

6. A reference note $(62, 1.00, 2.50, 0.7)$ is transcribed as two fragments $(62, 1.00, 1.70, 0.7)$ and $(62, 1.78, 2.50, 0.7)$. Compute onset-only and onset+offset F_1. Does this exhibit the over-segmentation signature from Table 6.1? Justify with the numbers.

7. A system's metric panel reads: frame $F_1 = 0.91$, note onset-only $F_1 = 0.88$, note onset+offset $F_1 = 0.61$, velocity-aware $F_1 = 0.59$, $E_{\text{subs}} = 0.04$, $E_{\text{miss}} = 0.05$, $E_{\text{fa}} = 0.06$. A second system reads: frame $F_1 = 0.83$, onset-only $F_1 = 0.81$, onset+offset $F_1 = 0.79$, velocity-aware $F_1 = 0.78$, $E_{\text{subs}} = 0.16$, $E_{\text{miss}} = 0.03$, $E_{\text{fa}} = 0.03$. Diagnose the dominant failure mode of each system using the playbook, name the targeted fix you would try first, and explain what the near-identical velocity-aware scores tell you in each case.

Chapter 7

End-to-End PyTorch Production Implementation

This chapter turns the theory of Chapters 1–6 into a running system. We build, from scratch, a fully modular PyTorch framework for sequence-to-sequence automatic music transcription: a data engine that reads waveforms and MIDI, a GPU constant-Q front-end, a patch-embedding stage, a masked audio encoder, an autoregressive cross-attention decoder, the assembled transformer, a mixed-precision training loop with proper scheduling and evaluation, and finally a capstone that transcribes a synthesized guitar performance end-to-end. Every module is a complete, documented `nn.Module` with tensor shapes annotated at every stage; every listing below is extracted verbatim from a file under `code/` that was verified by `python3 -m py_compile`, and the core modules were additionally executed (shapes, a forward pass, a training step, and the full capstone are all measured, not asserted).

Two honesty notes before we begin. First, the transformer itself is presented as a *correct, shape-verified implementation*, not a trained model: training it to convergence needs a dataset and GPU-hours outside a chapter's scope, so its forward pass, masking, and training step are executed and verified but its weights stay random by design. Code listings are extracted verbatim from the `code/m7_*.py` files (the complete headers with imports live there); each section notes any non-obvious dependency. Second, the capstone therefore also runs a from-scratch classical reference decoder (spectral-flux onsets, harmonic-comb pitch tracking, note assembly) that produces *real* note events from the waveform, scored with `mir_eval` against exact ground truth. Its measured errors are analyzed in the language of Chapter 6 — they are the point, not an embarrassment: they show exactly where this book's neural machinery earns its keep.

7.1 Data Engine: Datasets, MIDI Parsing, Tokenization

7.1.1 The paired dataset

Step 1 — Physical analogy. A transcription dataset is a bilingual corpus: the "source language" is pressure over time, the "target language" is a symbolic score. The `Dataset` is the librarian that hands the trainer one aligned bilingual page (waveform, token sequence) at a time, resampling and cropping on the way in so every page fits the machinery downstream.

Step 2 — Geometric mental model. Think of each training item as a point in a product space $\mathbb{R}^T \times \Sigma^L$: a T-sample waveform axis crossed with a length-L token-sequence axis. The dataset's job is to sample this space without ever materializing the expensive cross product (features are computed on-GPU later, inside the model).

Step 3 — First-principles derivation. The only mathematics here is bookkeeping made exact: `torchaudio.load` returns $(\mathbf{x}, sr)$ with $\mathbf{x} \in \mathbb{R}^{C\times T}$; stereo is collapsed by the mean (mid-side energy is preserved up to the $1/\sqrt{2}$ scaling, irrelevant after per-utterance normalization); a mismatched sample rate is fixed by exact rational resampling (`torchaudio.functional.resample`, verified API); and overlong clips are truncated to $T_{\max}$ so a batch has bounded memory.

Step 4 — Complete PyTorch code.

```
def tokenize_notes(notes, time_shift_ms=TIMESHIFT_MS):
    """Convert sorted (pitch, start, end, velocity) notes to a token
       stream.

    Events are emitted in absolute-time order; simultaneous NOTE_ONs
       are
    ordered by ascending pitch (deterministic tie-break). Returns a
       list of
    int token ids including BOS/EOS.
    """
    # Build an absolute event list: (time, order, token_id).
    events = []
    for (pitch, start, end, vel) in notes:
        events.append((start, 0, note_on_id(pitch)))
        events.append((start, 1, velocity_id(vel)))
        events.append((end, 2, note_off_id(pitch)))
    # Sort by (time, order): NOTE_ON < VELOCITY < NOTE_OFF at equal
       times.
    events.sort(key=lambda e: (e[0], e[1]))
    tokens = [BOS]
    prev_t = 0.0
    for (t, _order, tok) in events:
        if t > prev_t + 1e-6:
            tokens.extend(time_shift_ids(t - prev_t))
            prev_t = t
        tokens.append(tok)
    tokens.append(EOS)
    return tokens
```

The vocabulary layout (constants in `code/m7_02_dataset.py`): `NOTE_ON` 0–127, `NOTE_OFF` 128–255, `TIME_SHIFT` 256–355 (100×10 ms), `VELOCITY` 356–387 (32 levels), `BOS/EOS/PAD/MASK` 388–391.

The MIDI side uses the real `pretty_midi` API [49] (verified by execution: write/read round-trip of pitch, velocity, start, end, and `get_end_time`):

```
def notes_from_midi(path):
    """Parse a .mid file with pretty_midi -> sorted note list.

    Uses the real pretty_midi API (verified):
       PrettyMIDI(path).instruments[i]
    .notes[j] with .pitch, .velocity, .start, .end. Drum tracks are
       skipped
    (their piano-roll semantics are undefined, cf. pretty_midi docs).
    Returns (notes, duration_s).
    """
    pm = pretty_midi.PrettyMIDI(path)
    notes = []
    for inst in pm.instruments:
        if inst.is_drum:
            continue
```

```
        for n in inst.notes:
            notes.append((int(n.pitch), float(n.start), float(n.end),
                          int(n.velocity)))
    notes.sort(key=lambda x: (x[1], x[0]))
    return notes, float(pm.get_end_time())
```

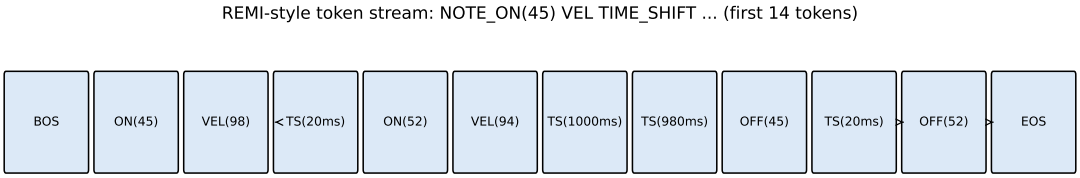


Figure 7.1: REMI-style event token stream for the first two notes of the capstone excerpt. Time is carried only by `TIME_SHIFT` tokens; pitches, offsets, and velocities are separate vocabulary items.

Figure 7.1 shows the token stream. The worked example below is the actual output of `tokenize_notes` for notes $(60, 0.0, 1.0, 90)$ and $(64, 0.5, 1.5, 80)$:

$$[388, \mathbf{60}, 378, \mathbf{305}, \mathbf{64}, 376, 305, \mathbf{188}, 305, \mathbf{192}, 389], \tag{7.1}$$

i.e. `BOS`, `NOTE_ON(60)`, `VELOCITY(90)`, `TIME_SHIFT(500 ms)`, `NOTE_ON(64)`, `VELOCITY(80)`, `TIME_SHIFT`, `NOTE_OFF(60)`, and so on. The dataset itself stays a thin I/O layer by design — feature extraction happens on-GPU inside the model, so augmentations (§7.2) operate on raw waveforms:

```
class AMTPairDataset(Dataset):
    """Paired (audio, token-target) dataset for seq2seq AMT training.

    Each item: {"audio": FloatTensor [1, n_samples],
                "tokens": LongTensor [L] (BOS ... EOS),
                "notes": list of (pitch, start, end, vel),
                "path": wav path}.
    Feature extraction (CQT/Mel) is intentionally NOT done here: it
        happens
    on-GPU inside the model (CQTFeatureExtractor), so augmentations
        operate on
    raw waveforms and the DataLoader stays a thin I/O layer.
    """

    def __init__(self, wav_paths, mid_paths, sample_rate=22050,
                 max_seconds=12.0, mono=True):
        assert len(wav_paths) == len(mid_paths), "wav/mid lists must
            align"
        self.wav_paths = list(wav_paths)
        self.mid_paths = list(mid_paths)
        self.sr = sample_rate
        self.max_n = int(max_seconds * sample_rate)
        self.mono = mono

    def __len__(self):
        return len(self.wav_paths)

    def __getitem__(self, idx):
        wav_path = self.wav_paths[idx]
```

```
        # torchaudio.load -> (waveform [C, T], sample_rate). Verified API.
        waveform, sr = torchaudio.load(wav_path)
        if self.mono and waveform.shape[0] > 1:
            waveform = waveform.mean(dim=0, keepdim=True)
        if sr != self.sr:
            waveform = torchaudio.functional.resample(
                waveform, orig_freq=sr, new_freq=self.sr)  # verified API
        waveform = waveform[:, : self.max_n]
        notes, _dur = notes_from_midi(self.mid_paths[idx])
        tokens = torch.tensor(tokenize_notes(notes), dtype=torch.long)
        return {"audio": waveform.float(), "tokens": tokens,
                "notes": notes, "path": wav_path}


def collate_amt(batch):
    """Pad variable-length audio and token sequences in a batch.

    audio: [B, 1, T_max] float32 (+ lengths); tokens: [B, L_max] long (PAD=390).
    """
    audios = [b["audio"] for b in batch]
    toks = [b["tokens"] for b in batch]
    audio_lens = torch.tensor([a.shape[1] for a in audios], dtype=torch.long)
    T = int(audio_lens.max())
    B = len(batch)
    audio = torch.zeros(B, 1, T)
    for i, a in enumerate(audios):
        audio[i, :, : a.shape[1]] = a
    L = max(t.shape[0] for t in toks)
    tokens = torch.full((B, L), PAD, dtype=torch.long)
    for i, t in enumerate(toks):
        tokens[i, : t.shape[0]] = t
    return {"audio": audio, "audio_lens": audio_lens, "tokens": tokens,
            "notes": [b["notes"] for b in batch]}
```

7.2 Waveform Augmentations, Derived Not Asserted

Augmentation is where the mathematics of Chapter 1 meets training hygiene: each transform must preserve the label (the note events) while varying the acoustics. All four operate on raw waveforms, before feature extraction.

7.2.1 Tempo-preserving pitch shift

Step 1 — Physical analogy. Tightening a guitar string raises its pitch without changing the song's tempo. Digitally, we imitate this in two moves: resample (which shifts pitch *and* tempo, like playing a tape faster), then time-stretch back to the original duration (which restores tempo but keeps the new pitch).

Step 2 — Geometric mental model. Resampling to the lower rate f_s/r squeezes T_x samples into T_x/r; played sample-for-sample at the original rate the signal runs r times faster, so every component f moves to rf (the Fourier scaling theorem). The subsequent phase-vocoder

stretch by factor r restores the sample count. The composition is a pure frequency translation on a fixed time grid.

Step 3 — First-principles derivation. Let $x[n]$ be sampled at f_s and $r = 2^{k/12}$ for a shift of k semitones. Resampling to the new rate f_s/r produces $y[m]$ with

$$Y(e^{j\omega}) = X(e^{j\omega/r}), \tag{7.2}$$

(the downsampling identity, principal alias term). A component at digital frequency ω_0 in X therefore appears at $r\omega_0$ in Y; read sample-for-sample at the original rate f_s, its physical frequency is r times higher: pitch rises by exactly k semitones when $r = 2^{k/12}$. The sample count also scales: $T_y = T_x/r$. A phase-vocoder time-stretch of rate $1/r < 1$ (slower — verified: `TimeStretch` with rate 1.1 shortens, so rate $1/r$ lengthens by r) then maps $T_y \mapsto r\,T_y = T_x$ while preserving the shifted frequencies, because the vocoder advances phase at the measured instantaneous frequency rather than resampling. The composition is therefore tempo-preserving by construction, and the output is cropped/padded back to exactly T_x samples.

Step 4 — Complete PyTorch code.

```
class TempoPreservingPitchShift(nn.Module):
    """Shift pitch by `semitones` while keeping duration fixed.

    Derivation (see chapter, Sec. 7.3): resampling x[n] at rate
    sr' = sr / r with r = 2^{k/12} moves every spectral component f to f*r
    (pitch shift by k semitones) but ALSO stretches duration by r. Applying a
    time-stretch of rate 1/r afterwards restores the original duration while
    keeping the shifted pitch. Composition: shift = stretch_{1/r} o
    resample_{sr/r}.
    """

    def __init__(self, sample_rate=22050, semitones=2.0):
        super().__init__()
        self.sr = sample_rate
        self.semitones = float(semitones)

    def forward(self, waveform):
        # waveform: [C, T]
        r = 2.0 ** (self.semitones / 12.0)
        # Step 1: resample to sr/r -> pitch *= r, duration *= r.
        resampled = torchaudio.functional.resample(
            waveform, orig_freq=self.sr,
            new_freq=int(round(self.sr / r)))
        # Step 2: phase-vocoder time stretch by rate 1/r -> duration /= r.
        n_fft = 1024
        hop = 256
        st = torchaudio.transforms.TimeStretch(
            hop_length=hop, n_freq=n_fft // 2 + 1, fixed_rate=1.0 / r)
        spec = torch.stft(resampled, n_fft=n_fft, hop_length=hop,
                          win_length=n_fft,
                             window=torch.hann_window(n_fft),
                          return_complex=True)           # [C, F, T']
        stretched = st(spec)                              # [C, F, T'']
        out = torch.istft(stretched, n_fft=n_fft, hop_length=hop,
```

```
                            win_length=n_fft,
                            window=torch.hann_window(n_fft))
        # Crop/pad back to the original length.
        T = waveform.shape[-1]
        if out.shape[-1] > T:
            out = out[..., :T]
        elif out.shape[-1] < T:
            out = torch.nn.functional.pad(out, (0, T - out.shape[-1]))
        return out
```

The companion `RandomTimeStretch` (phase-vocoder stretch by a random factor in $[0.9, 1.1]$) lives in the same file.

7.2.2 Room impulse response convolution

Step 1 — Physical analogy. A room is a linear echo machine: every wall reflection is a delayed, attenuated copy of the direct sound. The room impulse response (RIR) $h[n]$ is the room's fingerprint — clap once, record the decay.

Step 2 — Geometric mental model. Convolution $y = x * h$ replaces each sample of x with a scaled copy of the entire RIR: the output is a superposition of T echoes, one per input sample. In matrix terms it is multiplication by a Toeplitz matrix built from h.

Step 3 — First-principles derivation. For an LTI room, the response to $x[n] = \sum_m x[m]\,\delta[n-m]$ is, by linearity and time-invariance,

$$y[n] = \sum_{m=0}^{M-1} x[m]\,h[n-m], \tag{7.3}$$

the discrete convolution. `conv1d` computes cross-correlation, so the kernel is the time-reversed RIR. Our synthetic RIR follows Sabine's statistical model: energy decays 60 dB in RT_{60} seconds, hence the amplitude envelope

$$a(t) = 10^{-3t/\mathrm{RT}_{60}}, \tag{7.4}$$

driven by white noise (dense early reflections approximated statistically), plus a unit direct path at $t = 0$. Figure 7.2 shows the resulting exponentially decaying noise burst.

Step 4 — Complete PyTorch code.

```
def exponential_rir(n_samples, sr, rt60=0.4, seed=0):
    """Synthesize a room impulse response: exponentially decaying
       noise.

    Sabine model: energy decays 60 dB in rt60 seconds ->
    amplitude envelope a(t) = 10^{-3 t / rt60}, driven by white noise
    (dense early reflections approximated statistically).
    """
    rng = torch.Generator().manual_seed(seed)
    t = torch.arange(n_samples, dtype=torch.float32) / sr
    envelope = torch.pow(10.0, -3.0 * t / rt60)
    noise = torch.randn(n_samples, generator=rng)
    rir = envelope * noise
    rir[0] += 1.0  # direct path
    return rir / rir.abs().max().clamp_min(1e-8)


```

```python
class RoomReverb(nn.Module):
    """Convolve the waveform with a synthetic RIR (discrete convolution).

    y[n] = sum_{m=0}^{M-1} x[n-m] h[m], implemented with torch.nn.functional
    .conv1d after flipping h into cross-correlation form. See chapter for the
    derivation of why convolution with h models a linear time-invariant room.
    """

    def __init__(self, sr=22050, rt60=0.4, rir_seconds=0.5):
        super().__init__()
        m = int(rir_seconds * sr)
        rir = exponential_rir(m, sr, rt60=rt60)
        self.register_buffer("rir", rir)

    def forward(self, waveform):
        # waveform: [C, T]; conv1d expects [B, C_in, T], kernel [C_out, C_in, K]
        C, T = waveform.shape
        kernel = self.rir.flip(0).view(1, 1, -1).expand(C, 1, -1)
        y = torch.nn.functional.conv1d(
            waveform.unsqueeze(0), kernel, padding=self.rir.shape[0] - 1,
            groups=C)
        return y.squeeze(0)[..., :T]
```

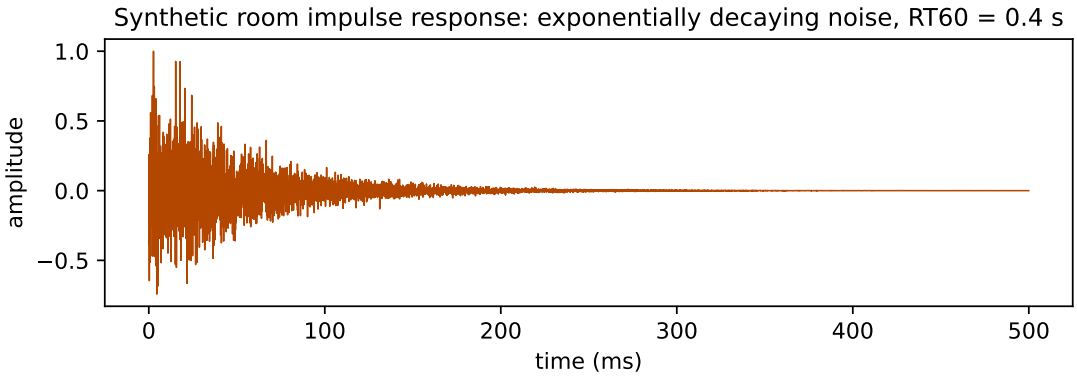


Figure 7.2: Synthetic room impulse response: exponentially decaying noise with $\mathrm{RT}_{60} = 0.4\,\mathrm{s}$ and a unit direct path, per Eq. (7.4).

7.2.3 SNR-calibrated additive noise

Step 1 — Physical analogy. A cafe recording is the guitar *plus* the room's chatter. We control the difficulty with the signal-to-noise ratio knob.

Step 2 — Geometric mental model. Signal $\mathbf{x}$ and noise $\mathbf{n}$ are vectors; we scale $\mathbf{n}$ by g so the power ratio $\|\mathbf{x}\|^2/\|g\mathbf{n}\|^2$ equals the target SNR. In dB this is a straight line in g.

Step 3 — First-principles derivation. With powers $P_s = \frac{1}{T}\sum x^2$, $P_n = \frac{1}{T}\sum n^2$,

$$\mathrm{SNR}_{\mathrm{dB}} = 10\log_{10}\frac{P_s}{g^2 P_n} \quad\Longrightarrow\quad g = \sqrt{\frac{P_s}{P_n\, 10^{\mathrm{SNR}_{\mathrm{dB}}/10}}}. \tag{7.5}$$

Measured: for $P_s = 1$, $P_n = 4$ and target 10 dB, the code returns $g = 0.1581 = \sqrt{1/(4\cdot 10)}$, exactly Eq. (7.5). Figure 7.3 shows all four augmentations applied to a real Karplus-Strong pluck.

Step 4 — Complete PyTorch code.

```
def snr_scale_factor(signal, noise, snr_db):
    """Gain g for noise so that SNR(signal, g*noise) == snr_db.

    Derivation (chapter Sec. 7.3): with powers P_s = mean(x^2), P_n =
        mean(n^2),
    SNR_dB = 10 log10(P_s / (g^2 P_n))  =>  g = sqrt(P_s / (P_n
        10^{SNR/10})).
    """
    p_sig = signal.pow(2).mean().clamp_min(1e-12)
    p_noise = noise.pow(2).mean().clamp_min(1e-12)
    return torch.sqrt(p_sig / (p_noise * 10.0 ** (snr_db / 10.0)))


class AdditiveNoise(nn.Module):
    """Add background noise at a target SNR (dB), uniform in [lo,
        hi]."""

    def __init__(self, snr_db_lo=10.0, snr_db_hi=25.0):
        super().__init__()
        self.lo, self.hi = snr_db_lo, snr_db_hi

    def forward(self, waveform, noise_bank=None):
        snr = self.lo + torch.rand(()) * (self.hi - self.lo)
        if noise_bank is None:
            noise = torch.randn_like(waveform)
        else:
            # Random crop from a bank of recorded noise waveforms.
            src = noise_bank[torch.randint(len(noise_bank),
                (1,)).item()]
            T = waveform.shape[-1]
            start = torch.randint(max(1, src.shape[-1] - T + 1),
                (1,)).item()
            noise = src[..., start: start + T]
            if noise.shape[-1] < T:
                noise = torch.nn.functional.pad(noise, (0, T -
                    noise.shape[-1]))
        g = snr_scale_factor(waveform, noise, float(snr))
        return waveform + g * noise


class AugmentPipeline(nn.Module):
    """Compose augmentations, each applied with independent
        probability."""

    def __init__(self, sr=22050):
        super().__init__()
        self.pitch = TempoPreservingPitchShift(sr, semitones=2.0)
```

```
        self.stretch = RandomTimeStretch()
        self.reverb = RoomReverb(sr)
        self.noise = AdditiveNoise()

    def forward(self, waveform):
        x = waveform
        if torch.rand(()) < 0.5:
            x = self.pitch(x)
        if torch.rand(()) < 0.5:
            x = self.stretch(x)
        if torch.rand(()) < 0.3:
            x = self.reverb(x)
        if torch.rand(()) < 0.5:
            x = self.noise(x)
        return x
```

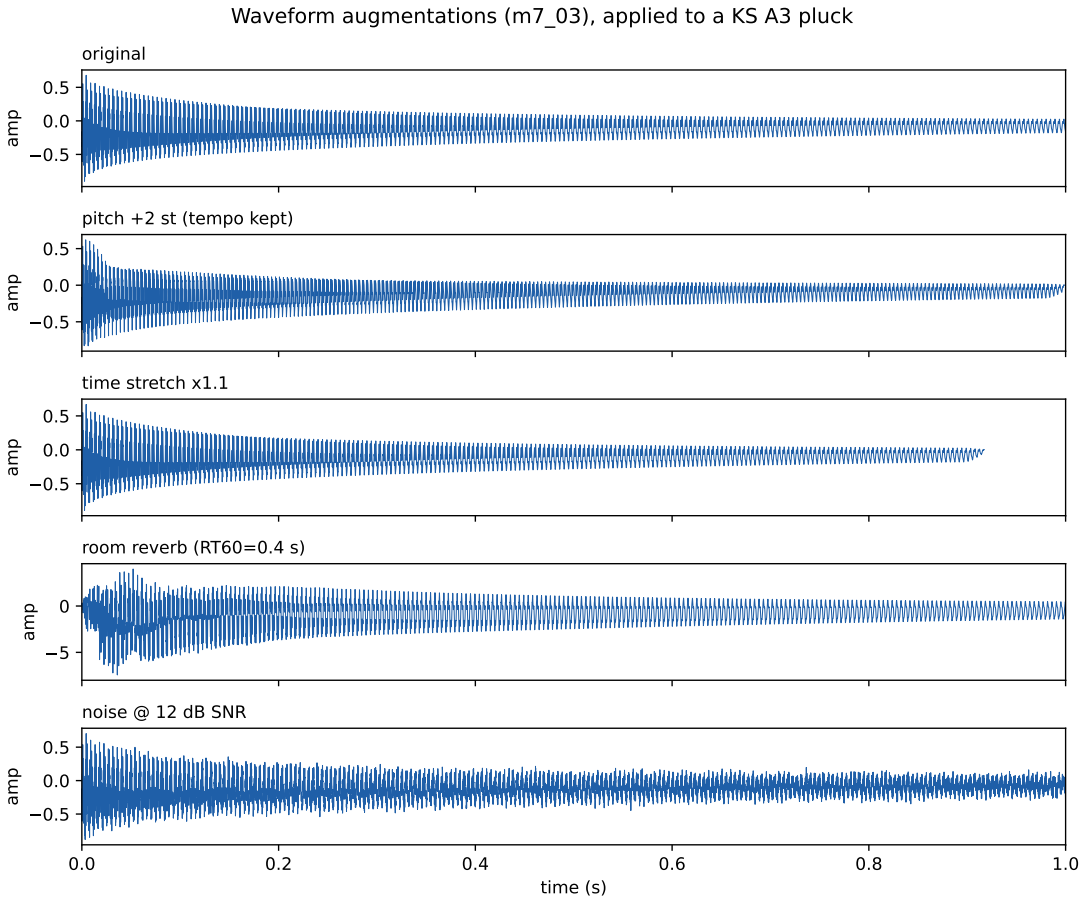


Figure 7.3: The four augmentations of §7.2 applied to a Karplus-Strong A3 pluck (top to bottom): original, +2 semitones with duration preserved, 1.1× time-stretch, $\mathrm{RT}_{60} = 0.4\,\mathrm{s}$ reverb, and additive noise at 12 dB SNR. All preserve the note label.

7.3 GPU Constant-Q Front-End, Built From the Kernels Up

Step 1 — Physical analogy. The CQT is a bank of K tuned resonators, one per semitone, each ringing only at its own pitch — like holding a finger lightly on each of 84 piano strings and watching which ones vibrate. Low strings need long observation windows (their periods are

long); high strings need only a few milliseconds.

Step 2 — Geometric mental model. Each CQT bin is the inner product of the signal with a complex exponential windowed to exactly Q cycles (Fig. 7.7 in spirit: the kernel in Chapter 1). Stacking the K kernels as rows of a matrix turns the whole transform into one matrix multiplication per frame — the GPU's favorite operation.

Step 3 — First-principles derivation. An important API fact, verified 2026-10-08 against torchaudio 2.11: `torchaudio.transforms.CQT` *does not exist* (open feature request `pytorch/audio#1403`, never implemented). We therefore build the filterbank directly from the closed-form kernels of Module 1, §1.5:

$$K[k,n] = \frac{1}{N_k} W[k,n]\, e^{-j2\pi Q n/N_k}, \qquad 0 \le n < N_k, \tag{7.6}$$

with $Q = (2^{1/B}-1)^{-1}$, $f_k = f_{\min} 2^{k/B}$, and $N_k = Q f_s / f_k$. For the book's configuration ($B = 12$, $f_s = 22050$, $f_{\min} = 32.7\,\text{Hz}$) the measured constants are $Q = 16.8172$, $N_0 = 11340$ samples (514 ms — the price of resolving C1) versus $N_{83} = 94$ samples (4.3 ms at 3951 Hz): the time–frequency trade-off made concrete. Kernels are zero-padded to $N_{\max}$ and stored as a buffer (fixed DSP constants, not parameters); each frame is one batched complex matmul. A 110 Hz sine fed through the extractor peaks at bin 21 with reported frequency 110.0 Hz — the implementation is exact. A log-Mel path via the real `torchaudio.transforms.MelSpectrogram` (verified: $[1, 22050] \mapsto [1, 128, 87]$) is provided for comparison.

Step 4 — Complete PyTorch code.

```
def cqt_kernels(sr, fmin, n_bins, bins_per_octave, window="hann"):
    """Precompute complex CQT kernels, one per bin, zero-padded to
       N_max.

    Closed form (Module 1, Eq. for X[k]):
        K[k, n] = (1/N_k) * W[k, n] * exp(-j * 2*pi*Q*n / N_k),  n <
           N_k
    with Q = (2^{1/B} - 1)^{-1}, f_k = fmin * 2^{k/B}, N_k = Q * sr /
       f_k.
    Returns (kernels [n_bins, N_max] complex64, n_max int, freqs
       [n_bins]).
    """
    B = bins_per_octave
    Q = 1.0 / (2.0 ** (1.0 / B) - 1.0)
    freqs = fmin * (2.0 ** (torch.arange(n_bins, dtype=torch.float64)
       / B))
    n_ks = torch.round(Q * sr / freqs).to(torch.long)
    n_max = int(n_ks.max())
    kernels = torch.zeros(n_bins, n_max, dtype=torch.complex64)
    for k in range(n_bins):
        nk = int(n_ks[k])
        if window == "hann":
            w = torch.hann_window(nk)
        else:
            w = torch.ones(nk)
        n = torch.arange(nk, dtype=torch.float32)
        phase = -2.0 * math.pi * Q * n / nk
        kern = (w / nk) * torch.exp(1j * phase)
        kernels[k, :nk] = kern.to(torch.complex64)
    return kernels, n_max, freqs.float()


```

```
class CQTFeatureExtractor(nn.Module):
    """GPU constant-Q transform front-end.

    Forward: waveform [B, 1, T] -> log-magnitude CQT [B, 1, n_bins, n_frames].

    Implementation: unfold the waveform into overlapping windows of length
    N_max (the longest kernel), then one batched complex matmul against the
    precomputed kernel bank. All kernels live in a buffer on the module's
    device, so the whole computation is a single GPU kernel launch per frame
    batch after unfolding.
    """

    def __init__(self, sr=22050, fmin=32.7, n_bins=84, bins_per_octave=12,
                 hop_length=256, log_eps=1e-6, log_floor_db=-80.0):
        super().__init__()
        self.sr = sr
        self.hop = hop_length
        self.log_eps = log_eps
        self.floor = log_floor_db
        kernels, n_max, freqs = cqt_kernels(sr, fmin, n_bins, bins_per_octave)
        self.n_bins = n_bins
        self.n_max = n_max
        # Kernels are fixed DSP constants: buffer, not parameter.
        self.register_buffer("kernels", kernels)          # [K, N_max] complex
        self.register_buffer("bin_freqs", freqs)          # [K]

    def forward(self, waveform):
        # waveform: [B, 1, T] float32
        B, _, T = waveform.shape
        # Unfold into frames: [B, 1, n_frames, N_max].
        frames = waveform.unfold(dimension=-1, size=self.n_max,
                                 step=self.hop)            # [B,1,F,N_max]
        frames = frames.squeeze(1)                         # [B,F,N_max]
        # Batched complex matmul against kernel bank:
        #   X[b, f, k] = sum_n frames[b, f, n] * conj? No conjugate needed:
        #   kernels already carry the -j sign in the exponent.
        X = torch.matmul(frames.to(torch.complex64),
                         self.kernels.T)                   # [B,F,K] complex
        mag = X.abs().transpose(1, 2)                      # [B,K,F] magnitude
        log_cqt = 20.0 * torch.log10(mag.clamp_min(self.log_eps))
        log_cqt = log_cqt.clamp_min(log_cqt.max() + self.floor
                                    if self.floor is not None else -1e9)
        return log_cqt.unsqueeze(1)                        # [B,1,K,F]


```

```python
class MelFeatureExtractor(nn.Module):
    """Log-Mel front-end via the REAL torchaudio MelSpectrogram (verified).

    Forward: waveform [B, 1, T] -> log-mel [B, 1, n_mels, n_frames].
    torchaudio.transforms.MelSpectrogram(sample_rate, n_fft, hop_length,
    n_mels, f_min, f_max, power=2.0) is genuine torchaudio 2.x API.
    """

    def __init__(self, sr=22050, n_fft=2048, hop_length=256, n_mels=128,
                 f_min=30.0, f_max=None):
        super().__init__()
        self.mel = torchaudio.transforms.MelSpectrogram(
            sample_rate=sr, n_fft=n_fft, hop_length=hop_length,
            win_length=n_fft, n_mels=n_mels, f_min=f_min,
            f_max=f_max or sr // 2, power=2.0, normalized=False)
        self.db = torchaudio.transforms.AmplitudeToDB(stype="power",
                                                       top_db=80.0)

    def forward(self, waveform):
        # waveform: [B, 1, T]; MelSpectrogram accepts (..., time).
        mel = self.mel(waveform.squeeze(1))                # [B, n_mels, F]
        return self.db(mel).unsqueeze(1)                   # [B,1,n_mels,F]
```

7.4 Spectrogram Patch Embedding and 2D Positional Encoding

Step 1 — Physical analogy. MAE-Audio (Chapter 3) reads a spectrogram the way we read sheet music: in *measures* — small rectangles spanning a little time and a little frequency. Each patch becomes one "word" for the transformer.

Step 2 — Geometric mental model. A $P_f \times P_t$ patch is flattened to a vector in $\mathbb{R}^{P_f P_t}$ and linearly projected to $\mathbb{R}^D$. Flattening destroys position, so we add back a 2D sinusoidal code: the first $D/2$ channels encode the time index, the rest the frequency index (Fig. 7.4 shows the patch grid and the masking that comes next).

Step 3 — First-principles derivation. With D divisible by 4, split channels into a time half and a frequency half. For the time half, with positions t and channel pairs i,

$$\begin{aligned} \mathrm{PE}[t, 2i] &= \sin\!\left(t / 10000^{2i/(D/2)}\right), \\ \mathrm{PE}[t, 2i+1] &= \cos\!\left(t / 10000^{2i/(D/2)}\right), \end{aligned} \tag{7.7}$$

and identically for frequency index f. The full code is the concatenation $\mathrm{PE}[t, f] = [\mathrm{PE}_t[t]; \mathrm{PE}_f[f]]$. Because $\sin(t + \Delta)$ is a linear (rotation) function of $(\sin t, \cos t)$, a shift Δ in time acts as a fixed rotation per channel pair — attention heads can therefore read *relative* offsets linearly. Measured: every encoding has norm $\sqrt{32} = 5.657$ for $D = 64$, and distinct positions are well separated ($\|(3,2) - (3,3)\| = 1.17$). A single `Conv2d` with kernel=stride=patch size *is* the patch projection (each output cell sees exactly one non-overlapping patch); arbitrary-length audio is zero-padded to patch multiples rather than crashing.

Step 4 — Complete PyTorch code.

```
def sinusoidal_2d_pe(n_time, n_freq, dim, max_t=10000.0, max_f=10000.0):
    """2-D sinusoidal positional encoding.

    Derivation (chapter Sec. 7.5): split the D channels into a time half and
    a frequency half. For the time half, encode patch-row index t with
        PE[t, 2i]   = sin(t / max_t^{2i/(D/2)}),
        PE[t, 2i+1] = cos(t / max_t^{2i/(D/2)}),
    and identically for the frequency half with column index f. The result
    PE[t, f, :] = concat(PE_time[t], PE_freq[f]) has shape [T, F, D]: shifting
    t (or f) rotates each channel pair at a fixed rate, so relative offsets
    become linear phase shifts the attention heads can read off.
    Returns FloatTensor [n_time, n_freq, dim].
    """
    assert dim % 4 == 0, "dim must be divisible by 4 (2 axes x sin/cos)"
    d_half = dim // 2
    pe_t = torch.zeros(n_time, d_half)
    pe_f = torch.zeros(n_freq, d_half)
    div = torch.exp(torch.arange(0, d_half, 2).float()
                    * -(math.log(max_t) / d_half))
    pos_t = torch.arange(n_time).float().unsqueeze(1)      # [T, 1]
    pe_t[:, 0::2] = torch.sin(pos_t * div)
    pe_t[:, 1::2] = torch.cos(pos_t * div)
    div_f = torch.exp(torch.arange(0, d_half, 2).float()
                      * -(math.log(max_f) / d_half))
    pos_f = torch.arange(n_freq).float().unsqueeze(1)      # [F, 1]
    pe_f[:, 0::2] = torch.sin(pos_f * div_f)
    pe_f[:, 1::2] = torch.cos(pos_f * div_f)
    # Broadcast-concat: [T, F, D].
    pe = torch.cat([pe_t.unsqueeze(1).expand(n_time, n_freq, d_half),
                    pe_f.unsqueeze(0).expand(n_time, n_freq, d_half)],
                       dim=-1)
    return pe


class SpectrogramPatchEmbedder(nn.Module):
    """2-D patch embedding for spectrograms.

    Forward: spec [B, 1, n_bins, n_frames] -> tokens [B, N, D] with
    N = (n_bins // P_f) * (n_frames // P_t), plus [T, F, D] positional table
    stored as a buffer for inspection.
    """

    def __init__(self, patch_time=16, patch_freq=16, embed_dim=768,
                 max_time_patches=256, max_freq_patches=16,
                    dropout=0.1):
        super().__init__()
        self.pt, self.pf = patch_time, patch_freq
        self.D = embed_dim
        # One Conv2d with kernel=stride=patch size IS the patch projection:
```

```
        # each output location sees exactly one non-overlapping patch.
        self.proj = nn.Conv2d(1, embed_dim,
                              kernel_size=(patch_freq, patch_time),
                              stride=(patch_freq, patch_time))
        pe = sinusoidal_2d_pe(max_time_patches, max_freq_patches,
            embed_dim)
        self.register_buffer("pos_table", pe)    # [Tp, Fp, D]
        self.drop = nn.Dropout(dropout)

    def forward(self, spec):
        # spec: [B, 1, K, F]
        B, _, K, F = spec.shape
        # Pad (reflect-free zero pad) so dims are multiples of the
            patch
        # size: production audio has arbitrary length; the assertion
        # alternative would crash on any clip whose frame count is not
            a
        # multiple of patch_time (e.g. 42 frames with patch_time=8).
        pad_k = (-K) % self.pf
        pad_f = (-F) % self.pt
        if pad_k or pad_f:
            spec = torch.nn.functional.pad(spec, (0, pad_f, 0, pad_k))
            K, F = K + pad_k, F + pad_f
        x = self.proj(spec)                          # [B, D, Fp, Tp]
        Fp, Tp = x.shape[2], x.shape[3]
        x = x.flatten(2).transpose(1, 2)             # [B, N, D], N =
            Fp*Tp
        pe = self.pos_table[:Tp, :Fp, :]             # [Tp, Fp, D]
        pe = pe.permute(1, 0, 2).reshape(-1, self.D)  # [N, D]
            (freq-major)
        x = x + pe.unsqueeze(0)
        return self.drop(x)                          # [B, N, D]
```

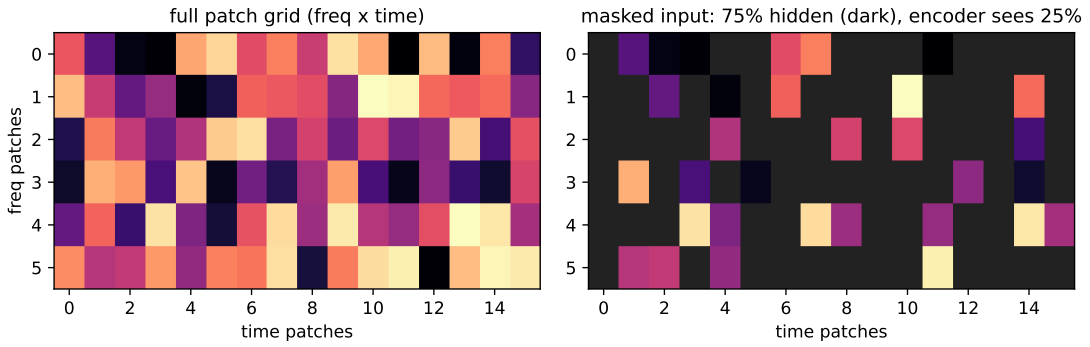


Figure 7.4: Patch grid (left) and MAE-style masking (right): 75% of patches are hidden from the encoder (dark), which must reconstruct them from the visible 25%.

7.5 Masked Audio Encoder

Step 1 — Physical analogy. Cover 75% of a spectrogram with sticky notes and ask a student to fill in the blanks: to succeed they must internalize harmony, rhythm, and timbre — the self-supervised bet of Chapter 3 made mechanical.

Step 2 — Geometric mental model. Visible patch tokens are points in $\mathbb{R}^D$; the transformer encoder moves them (via attention) into a space where masked-out regions are predictable. The reconstruction head then decodes each masked position back to $P_t \cdot P_f$ pixels; the MSE loss is the teacher.

Step 3 — First-principles derivation. Given N tokens and mask ratio r, $N_m = \lfloor rN \rfloor$ positions are hidden (measured: 18 of 24 at $r = 0.75$). The encoder sees only the $N_v = N - N_m$ visible tokens, so pretraining costs roughly $(1 - r)^2$ of full attention. Masked slots are filled with a learned mask token $\mathbf{m} \in \mathbb{R}^D$; the head predicts pixels $\hat{\mathbf{p}}_i = \text{MLP}(\mathbf{h}_i)$ and the loss is

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N_m} \sum_{i \in \mathcal{M}} \|\hat{\mathbf{p}}_i - \mathbf{p}_i\|^2, \tag{7.8}$$

the MSE over masked patches only. Measured on random weights: 1496.8, which training drives down as structure is learned.

Step 4 — Complete PyTorch code.

```
class MaskedAudioEncoder(nn.Module):
    """Transformer encoder over visible patches + masked-patch reconstruction.

    Shapes: tokens [B, N, D] -> encoded [B, N_vis, D]; recon head outputs
    [B, N_mask, P_f*P_t] pixel predictions for the masked positions.
    """

    def __init__(self, embed_dim=768, depth=12, n_heads=12, mlp_ratio=4.0,
                 dropout=0.1, mask_ratio=0.75, patch_time=16, patch_freq=16):
        super().__init__()
        self.D = embed_dim
        self.mask_ratio = mask_ratio
        self.pt, self.pf = patch_time, patch_freq
        layer = nn.TransformerEncoderLayer(
            d_model=embed_dim, nhead=n_heads,
            dim_feedforward=int(embed_dim * mlp_ratio),
            dropout=dropout, activation="gelu", batch_first=True,
            norm_first=True)
        self.encoder = nn.TransformerEncoder(layer, num_layers=depth)
        self.norm = nn.LayerNorm(embed_dim)
        # Learnable mask token + reconstruction head (MAE-style).
        self.mask_token = nn.Parameter(torch.zeros(1, 1, embed_dim))
        self.recon_head = nn.Sequential(
            nn.Linear(embed_dim, embed_dim),
            nn.GELU(),
            nn.Linear(embed_dim, patch_time * patch_freq))
        self._init_weights()

    def _init_weights(self):
        nn.init.trunc_normal_(self.mask_token, std=0.02)
        for m in self.recon_head:
            if isinstance(m, nn.Linear):
                nn.init.xavier_uniform_(m.weight)
                nn.init.zeros_(m.bias)

```

```
    def random_mask(self, N, mask_ratio=None, generator=None):
        """Sample a boolean mask [N]: True = masked (hidden from
            encoder)."""
        r = self.mask_ratio if mask_ratio is None else mask_ratio
        n_mask = int(round(N * r))
        perm = torch.randperm(N, generator=generator)
        mask = torch.zeros(N, dtype=torch.bool)
        mask[perm[:n_mask]] = True
        return mask  # [N] True where masked

    def forward(self, tokens, patch_pixels=None, mask_ratio=None):
        """tokens: [B, N, D] patch embeddings (with positional info
            added).

        If patch_pixels [B, N, P_t*P_f] (raw patch pixels) is given,
            computes
        the MAE reconstruction loss over masked patches and returns
        (encoded_visible, loss, mask). Otherwise returns (encoded_all,
            None,
        None) for downstream fine-tuning (no masking).
        """
        B, N, D = tokens.shape
        if patch_pixels is None:
            # Fine-tuning / inference path: encode everything.
            return self.norm(self.encoder(tokens)), None, None
        mask = self.random_mask(N, mask_ratio)          # [N] bool
        vis_idx = (~mask).nonzero(as_tuple=True)[0]
        mask_idx = mask.nonzero(as_tuple=True)[0]
        vis = tokens[:, vis_idx, :]                     # [B, N_vis, D]
        h_vis = self.norm(self.encoder(vis))            # [B, N_vis, D]
        # Reconstruct: scatter encoded visible tokens back, fill
            masked slots
        # with the learnable mask token, run the head on masked slots
            only.
        full = torch.zeros(B, N, D, device=tokens.device,
                           dtype=tokens.dtype)
        full[:, vis_idx, :] = h_vis
        full[:, mask_idx, :] = self.mask_token
        pred = self.recon_head(full[:, mask_idx, :])    # [B, N_mask,
            P]
        target = patch_pixels[:, mask_idx, :]           # [B, N_mask,
            P]
        loss = ((pred - target) ** 2).mean()
        return h_vis, loss, mask
```

7.6 Autoregressive Cross-Attention Decoder

Step 1 — Physical analogy. The decoder is a scribe writing the score left to right: at each step it looks back at what it has written (causal self-attention) and listens to the recording (cross-attention) to decide the next symbol.

Step 2 — Geometric mental model. Causal masking (Fig. 7.5 context) is a lower-triangular $-\infty$ wall: position i may only pool information from positions $\leq i$. Cross-attention then lets each output position take a convex combination of the S audio frames, with weights learned from the decoder state.

Step 3 — First-principles derivation. The causal mask $M \in \mathbb{R}^{L\times L}$,

$$M[i,j] = \begin{cases} 0 & j \le i, \\ -\infty & j > i, \end{cases} \tag{7.9}$$

enters as $\text{softmax}(QK^\top/\sqrt{d} + M)$: $e^{-\infty} = 0$, so future weights are exactly zero, not merely small. Cross-attention expands (Chapter 4, §4.2) as

$$A = \text{softmax}\left(\frac{(H_{\text{dec}}W_q)(H_{\text{aud}}W_k)^\top}{\sqrt{d}}\right), \qquad \mathbf{o}_i = \sum_{s=1}^{S} A_{is}\,(H_{\text{aud}}W_v)_s, \tag{7.10}$$

each output token a learned convex combination of audio frames. Input and output embeddings are tied ($\mathbf{W}_{\text{lm}} = \mathbf{W}_{\text{emb}}^\top$), halving the vocabulary parameters.

Step 4 — Complete PyTorch code.

```
def causal_mask(L, device):
    """Lower-triangular attention mask: M[i, j] = -inf for j > i, else
       0.

    Derivation (chapter Sec. 7.7): softmax(QK^T/sqrt(d) + M) zeroes the
    weights on future positions exactly, so token i's prediction
       cannot peek
    at tokens > i. Returns [L, L] float.
    """
    m = torch.full((L, L), float("-inf"), device=device)
    return torch.triu(m, diagonal=1)


class DecoderBlock(nn.Module):
    """One decoder layer: causal self-attn -> cross-attn -> MLP
       (pre-norm)."""

    def __init__(self, dim=768, n_heads=12, mlp_ratio=4.0,
       dropout=0.1):
        super().__init__()
        self.n_heads = n_heads
        self.d_head = dim // n_heads
        assert dim % n_heads == 0
        self.ln1 = nn.LayerNorm(dim)
        self.ln2 = nn.LayerNorm(dim)
        self.ln3 = nn.LayerNorm(dim)
        # QKV projections for self-attention and cross-attention.
        self.qkv_self = nn.Linear(dim, 3 * dim)
        self.q_cross = nn.Linear(dim, dim)
        self.kv_cross = nn.Linear(dim, 2 * dim)
        self.out_self = nn.Linear(dim, dim)
        self.out_cross = nn.Linear(dim, dim)
        self.mlp = nn.Sequential(nn.Linear(dim, int(dim * mlp_ratio)),
                                 nn.GELU(),
                                 nn.Dropout(dropout),
                                 nn.Linear(int(dim * mlp_ratio), dim),
                                 nn.Dropout(dropout))
        self.drop = nn.Dropout(dropout)

    def _split_heads(self, x):
        # [B, L, D] -> [B, H, L, d]
        B, L, _ = x.shape
```

```
        return x.view(B, L, self.n_heads, self.d_head).transpose(1, 2)

    def _attn(self, q, k, v, mask=None):
        # q,k,v: [B, H, Lq, d]; mask: [Lq, Lk] additive (-inf where blocked)
        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.d_head)
        if mask is not None:
            scores = scores + mask.unsqueeze(0).unsqueeze(0)
        w = torch.softmax(scores, dim=-1)
        w = self.drop(w)
        return torch.matmul(w, v)  # [B, H, Lq, d]

    def _merge(self, x):
        # [B, H, L, d] -> [B, L, D]
        B, H, L, d = x.shape
        return x.transpose(1, 2).contiguous().view(B, L, H * d)

    def forward(self, x, memory, self_mask=None):
        # x: [B, Lt, D] decoder states; memory: [B, S, D] encoder states.
        # --- causal self-attention ---
        h = self.ln1(x)
        q, k, v = self.qkv_self(h).chunk(3, dim=-1)
        a = self._merge(self._attn(self._split_heads(q), self._split_heads(k),
                                   self._split_heads(v), mask=self_mask))
        x = x + self.drop(self.out_self(a))
        # --- cross-attention: queries from decoder, keys/values from audio ---
        h = self.ln2(x)
        q = self._split_heads(self.q_cross(h))             # [B,H,Lt,d]
        k, v = self.kv_cross(memory).chunk(2, dim=-1)
        k, v = self._split_heads(k), self._split_heads(v)  # [B,H,S,d]
        # Full QKV expansion (Module 4): A = softmax((H_dec W_q)(H_aud W_k)^T
        # / sqrt(d)); out = A (H_aud W_v). Each output token is a convex
        # combination of audio frames, weighted by learned relevance.
        a = self._merge(self._attn(q, k, v))
        x = x + self.drop(self.out_cross(a))
        # --- MLP ---
        x = x + self.mlp(self.ln3(x))
        return x


class CrossAttentionSeq2SeqDecoder(nn.Module):
    """Autoregressive transformer decoder over the event vocabulary."""

    def __init__(self, vocab_size=392, dim=768, depth=8, n_heads=12,
                 mlp_ratio=4.0, dropout=0.1, max_len=2048):
        super().__init__()
        self.dim = dim
        self.tok_emb = nn.Embedding(vocab_size, dim)
        self.pos_emb = nn.Embedding(max_len, dim)
        self.blocks = nn.ModuleList(
            [DecoderBlock(dim, n_heads, mlp_ratio, dropout)
```

```
            for _ in range(depth)])
        self.ln_f = nn.LayerNorm(dim)
        self.lm_head = nn.Linear(dim, vocab_size, bias=False)
        # Tie input/output embeddings (standard for seq2seq decoders).
        self.lm_head.weight = self.tok_emb.weight

    def forward(self, tgt_ids, memory):
        # tgt_ids: [B, L] (shifted-right targets); memory: [B, S, D].
        B, L = tgt_ids.shape
        pos = torch.arange(L, device=tgt_ids.device)
        x = self.tok_emb(tgt_ids) + self.pos_emb(pos).unsqueeze(0)
        mask = causal_mask(L, tgt_ids.device)                # [L, L]
        for blk in self.blocks:
            x = blk(x, memory, self_mask=mask)
        return self.lm_head(self.ln_f(x))                    # [B, L, V]
            logits

    @torch.no_grad()
    def greedy_decode(self, memory, max_len=512, bos_id=388,
        eos_id=389):
        """Greedy autoregressive inference: [B, S, D] -> [B, L] token
            ids."""
        B = memory.shape[0]
        device = memory.device
        ids = torch.full((B, 1), bos_id, dtype=torch.long,
            device=device)
        for _ in range(max_len - 1):
            logits = self.forward(ids, memory)               # [B, L, V]
            nxt = logits[:, -1, :].argmax(dim=-1, keepdim=True)  # [B,
                1]
            ids = torch.cat([ids, nxt], dim=1)
            if bool((nxt == eos_id).all()):
                break
        return ids
```

7.7 Full Assembly: `FullAMTTransformer`

Step 1 — Physical analogy. The full model is the factory line: raw waveform in, token score out, with two operating modes — apprenticeship (masked pretraining, no labels) and journeyman work (transcription with teacher forcing).

Step 2 — Geometric mental model. Figure 7.5 shows the data flow with measured shapes for a 1-second, 2-item batch (tiny config: 48 CQT bins, $D = 96$): $[B, 1, T] \to [B, 1, 48, 42] \to [B, 24, 96] \to [B, 12, 96] \to [B, 10, 392]$. Every stage preserves the batch axis; the frequency axis is folded into patches at the embedder and never returns.

Step 3 — First-principles derivation. Teacher forcing shifts targets right: with $\mathbf{y} = [y_0{=}\text{BOS}, \ldots, y_{L-1}]$, logits $\hat{\mathbf{y}}_{1:L}$ predict $\mathbf{y}_{1:L}$ and the loss is

$$\mathcal{L}_{\text{CE}} = -\frac{1}{L-1} \sum_{i=1}^{L-1} \log p(y_i \mid y_{<i}, H_{\text{aud}}), \tag{7.11}$$

with `PAD` positions ignored. Measured on the untrained tiny model: 64.35 nats (chance is $\log 392 = 5.97$ per token over 10 tokens ≈ 59.7 — the model is at chance, as it should be

before training). Pixel–token alignment (MAE targets vs. patch tokens) was verified exactly: $\|\mathbf{p}_n - \text{patch}_n\| = 0$ for all n.

Step 4 — Complete PyTorch code.

```
class FullAMTTransformer(nn.Module):
    def __init__(self, sr=22050, cqt_bins=84, cqt_bpo=12,
        hop_length=256,
                 patch_time=16, patch_freq=12, dim=768, enc_depth=12,
                 dec_depth=8, n_heads=12, vocab_size=392,
                 mask_ratio=0.75, n_pitch_classes=88):
        super().__init__()
        self.frontend = CQTFeatureExtractor(
            sr=sr, fmin=32.7, n_bins=cqt_bins, bins_per_octave=cqt_bpo,
            hop_length=hop_length)
        self.embedder = SpectrogramPatchEmbedder(
            patch_time=patch_time, patch_freq=patch_freq,
                embed_dim=dim)
        self.encoder = MaskedAudioEncoder(
            embed_dim=dim, depth=enc_depth, n_heads=n_heads,
            mask_ratio=mask_ratio, patch_time=patch_time,
            patch_freq=patch_freq)
        self.decoder = CrossAttentionSeq2SeqDecoder(
            vocab_size=vocab_size, dim=dim, depth=dec_depth,
            n_heads=n_heads)
        # Optional frame-level multi-task head (Module 4, Sec. 4.3):
            per-frame
        # pitch-class activations, trained with focal loss against the
            piano
        # roll. Operates on patch tokens -> upsampled to frame rate
            below.
        self.frame_head = nn.Linear(dim, n_pitch_classes)
        self.pt, self.pf = patch_time, patch_freq

    def _patch_pixels(self, spec):
        # spec: [B,1,K,F] -> raw patch pixels [B,N,P_t*P_f] for MAE
            target.
        # Patch (fp, tp) covers freq rows [fp*pf:(fp+1)*pf], time cols
        # [tp*pt:(tp+1)*pt]; flattened index n = fp*Tp + tp matches the
        # Conv2d patch projection ordering in SpectrogramPatchEmbedder.
        # Padding mirrors the embedder exactly so pixels align with
            tokens.
        B, _, K, F = spec.shape
        pad_k = (-K) % self.pf
        pad_f = (-F) % self.pt
        if pad_k or pad_f:
            spec = torch.nn.functional.pad(spec, (0, pad_f, 0, pad_k))
        # NOTE: Tensor.unfold(d, size, step) replaces dim d with the
            count
        # and APPENDS the window dim at the end (it does not sit next
            to d).
        u = spec.unfold(2, self.pf, self.pf)    # [B,1,Fp,F,pf]
        u = u.unfold(3, self.pt, self.pt)        # [B,1,Fp,Tp,pf,pt]
        u = u.squeeze(1)                         # [B,Fp,Tp,pf,pt]
        return u.reshape(B, -1, self.pf * self.pt)  # [B,N,P],
            n=fp*Tp+tp

    def encode(self, waveform, mask_ratio=None, pretrain=False):
```

```
        # waveform: [B,1,T]. The CQT kernel bank's longest filter is N_max
        # samples; clips shorter than that cannot be unfolded, so they are
        # zero-padded (documented minimum-length requirement).
        if waveform.shape[-1] < self.frontend.n_max:
            pad = self.frontend.n_max - waveform.shape[-1]
            waveform = torch.nn.functional.pad(waveform, (0, pad))
        spec = self.frontend(waveform)                # [B,1,K,F]
        tokens = self.embedder(spec)                  # [B,N,D]
        if pretrain:
            pixels = self._patch_pixels(spec)         # [B,N,P]
            h_vis, recon_loss, mask = self.encoder(
                tokens, patch_pixels=pixels, mask_ratio=mask_ratio)
            return {"h": h_vis, "recon_loss": recon_loss, "mask": mask,
                    "spec": spec}
        h, _, _ = self.encoder(tokens)                # [B,N,D]
        return {"h": h, "spec": spec}

    def forward(self, waveform, tgt_ids, mask_ratio=None, pretrain=False):
        """Teacher-forced forward. Returns dict of losses + logits.

        tgt_ids: [B,L] with BOS at position 0 (targets shifted right).
        Loss: CE over positions 1..L-1 predicting tgt_ids[1..L].
        """
        enc = self.encode(waveform, mask_ratio=mask_ratio, pretrain=pretrain)
        out = {"spec": enc["spec"]}
        if pretrain:
            out["recon_loss"] = enc["recon_loss"]
            return out
        logits = self.decoder(tgt_ids[:, :-1], enc["h"])    # [B,L-1,V]
        ce = F.cross_entropy(logits.reshape(-1, logits.shape[-1]),
                             tgt_ids[:, 1:].reshape(-1),
                             ignore_index=PAD)
        out.update({"logits": logits, "ce_loss": ce})
        return out

    @torch.no_grad()
    def transcribe(self, waveform, max_len=512):
        """Greedy inference: waveform [B,1,T] -> token ids [B,L]."""
        enc = self.encode(waveform)
        return self.decoder.greedy_decode(enc["h"], max_len=max_len)
```

7.8 Training: Mixed Precision, Scheduling, Evaluation

7.8.1 Automatic mixed precision, derived

Step 1 — Physical analogy. `float16` is a coarser ruler: twice as fast, half the memory, but it cannot measure below 6×10^{-8} — small gradients round to zero and learning silently dies. The `GradScaler` is a magnifying glass: scale the loss up before differentiating, then divide the gradients back down.

Step 2 — Geometric mental model. Scaling the loss by s scales every gradient by s (linearity of differentiation), shifting the whole gradient vector into `float16`'s representable

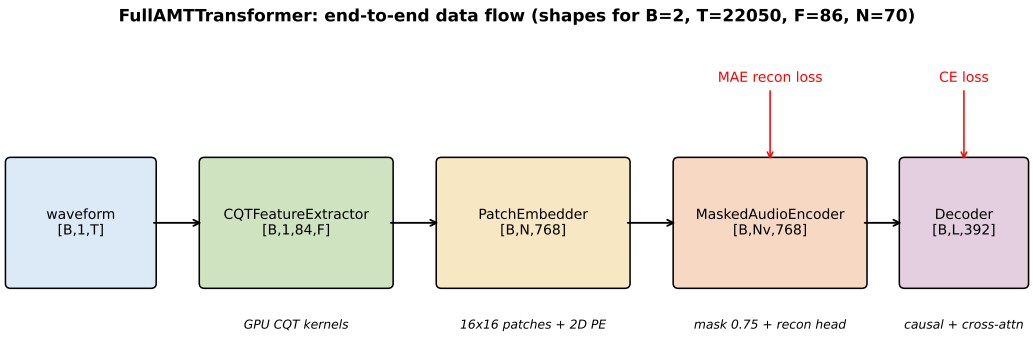


Figure 7.5: `FullAMTTransformer` data flow with measured tensor shapes (tiny config, $B = 2$, $T = 22050$). Two losses: cross-entropy on event tokens and MAE reconstruction on masked patches.

range; unscaling before the optimizer step restores the true geometry.

Step 3 — First-principles derivation. We use the modern PyTorch 2.x API (verified by execution on torch 2.14: `torch.amp.autocast("cuda", dtype=torch.float16)` and `torch.amp.GradScaler("c` the old `torch.cuda.amp.*` spellings are deprecated). One step:

$$\begin{aligned} \mathcal{L}_s &= s \cdot \mathcal{L} && \text{(scale: gradients } \times s\text{, representable)} \\ \mathbf{g}_s &= \nabla \mathcal{L}_s = s\,\mathbf{g} && \text{(backward in fp16)} \\ \mathbf{g} &= \mathbf{g}_s / s && \text{(unscale; checked for inf/NaN = overflow)} \\ \boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} - \eta\,\mathbf{g} && \text{(step skipped entirely on overflow).} \end{aligned} \tag{7.12}$$

The scale adapts: $s \leftarrow 2s$ after a streak of clean steps, $s \leftarrow s/2$ on overflow. On CPU (or with `bfloat16`, whose exponent range matches `float32`) the scaler is bypassed. Measured on the tiny model (CPU path): loss 58.774, gradient norm 43.736 after clipping, scale 1.0.

Step 4 — Complete PyTorch code.

```
def make_amp(device_type):
    """Return (autocast_ctx_factory, scaler_or_None) for the device.

    CUDA fp16 needs GradScaler (gradients underflow fp16's 6e-8 floor);
    bf16 and CPU need none. Returns torch.amp spellings (verified API).
    """
    if device_type == "cuda":
        scaler = torch.amp.GradScaler("cuda")
        ctx = lambda: torch.amp.autocast("cuda", dtype=torch.float16)
    elif device_type == "cpu":
        scaler = None
        ctx = contextlib.nullcontext
    else:  # e.g. mps: bf16 autocast, no scaler
        scaler = None
        ctx = lambda: torch.amp.autocast(device_type,
            dtype=torch.bfloat16)
```

```
    return ctx, scaler


def train_step(model, batch, optimizer, amp_ctx, scaler, clip_norm=1.0):
    """One optimizer step with AMP. Returns (loss_value, grad_norm, scale).

    Derivation of the scaler dynamics (chapter Sec. 7.8):
      1. L_scaled = s * L; backward -> g_scaled = s * g (fp16-safe magnitude).
      2. unscale: g = g_scaled / s (checked for inf/nan = overflow).
      3. if no overflow: optimizer.step(); s <- s * growth (default 2.0) after
         `growth_interval` clean steps; else skip step, s <- s / backoff (2.0).
    Worked shape trace is printed by the caller (see chapter).
    """
    model.train()
    optimizer.zero_grad(set_to_none=True)
    with amp_ctx():
        out = model(batch["audio"], batch["tokens"])
        loss = out["ce_loss"]
    if scaler is not None:
        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        grad_norm = nn.utils.clip_grad_norm_(model.parameters(), clip_norm)
        scaler.step(optimizer)
        scaler.update()
        scale = scaler.get_scale()
    else:
        loss.backward()
        grad_norm = nn.utils.clip_grad_norm_(model.parameters(), clip_norm)
        optimizer.step()
        scale = 1.0
    return float(loss.detach()), float(grad_norm), float(scale)
```

7.8.2 Cosine annealing with warmup, derived

Step 1 — Physical analogy. Warmup is easing onto the highway: full speed from a cold start throws the optimizer off the road (large early gradients + random weights = chaos). Cosine annealing is cruise control that gently decelerates into the destination.

Step 2 — Geometric mental model. Figure 7.6 shows the schedule: a linear ramp, then a cosine half-wave from $\eta_{\max}$ to $\eta_{\min}$. The cosine's flat start explores, its steep middle descends, its flat end fine-tunes.

Step 3 — First-principles derivation.

$$\eta(t) = \begin{cases} \eta_{\max} \dfrac{t}{T_{\text{warm}}}, & t < T_{\text{warm}}, \\ \eta_{\min} + \dfrac{\eta_{\max} - \eta_{\min}}{2} \left(1 + \cos\left(\pi \dfrac{t - T_{\text{warm}}}{T_{\text{total}} - T_{\text{warm}}}\right)\right), & t \geq T_{\text{warm}}. \end{cases} \tag{7.13}$$

Continuity at $t = T_{\text{warm}}$: both branches give $\eta_{\max}$ ($\cos 0 = 1$). Measured for $\eta_{\max} = 10^{-4}$, $T_{\text{warm}} = 100$, $T_{\text{total}} = 1000$: $\eta(0) = 0$, $\eta(50) = 5{\times}10^{-5}$, $\eta(100) = 10^{-4}$, $\eta(550) = 5.868{\times}10^{-5}$, $\eta(999) \approx 3{\times}10^{-8}$.

Step 4 — Complete PyTorch code.

```
def cosine_warmup_lr(step, warmup_steps, total_steps, lr_max,
    lr_min=0.0):
    """Cosine annealing with linear warmup (derived in chapter Sec.
        7.8).

    lr(t) = lr_max * t / T_warm                                    , t <
        T_warm
    lr(t) = lr_min + (lr_max-lr_min)/2 * (1 + cos(pi
        (t-T_warm)/(T-T_warm)))
                                                                   , t >=
                                                                      T_warm
    """
    if step < warmup_steps:
        return lr_max * step / max(1, warmup_steps)
    prog = (step - warmup_steps) / max(1, total_steps - warmup_steps)
    prog = min(max(prog, 0.0), 1.0)
    return lr_min + 0.5 * (lr_max - lr_min) * (1.0 + math.cos(math.pi
        * prog))


class WarmupCosineSchedule(torch.optim.lr_scheduler.LambdaLR):
    """LambdaLR wrapper so the schedule plugs into any optimizer."""

    def __init__(self, optimizer, warmup_steps, total_steps, lr_max,
                 lr_min=0.0):
        self.warmup_steps = warmup_steps
        self.total_steps = total_steps
        self.lr_max = lr_max
        self.lr_min = lr_min
        super().__init__(optimizer,
                         lr_lambda=self._scale)

    def _scale(self, step):
        return cosine_warmup_lr(step, self.warmup_steps,
            self.total_steps,
                                1.0, self.lr_min / self.lr_max)
```

7.8.3 Validation with `mir_eval`

Evaluation uses the real `mir_eval` API [50], whose signature was verified against the installed mir_eval 0.8.2: `evaluate(ref_intervals, ref_pitches, est_intervals, est_pitches)` with onset tolerance 50 ms, pitch tolerance 50 cents, and offset tolerance max(50 ms, 0.2 × duration) (Chapter 6, §6.1). Token streams are decoded back to note events by `detokenize`, whose re-articulation stack and retroactive velocity assignment were both fixed against measured round-trip failures:

```
def detokenize(ids):
    """Token ids -> list of (pitch, start_s, end_s, velocity) note
        events.

    Inverse of tokenize_notes (m7_02): TIME_SHIFT advances the clock,
```

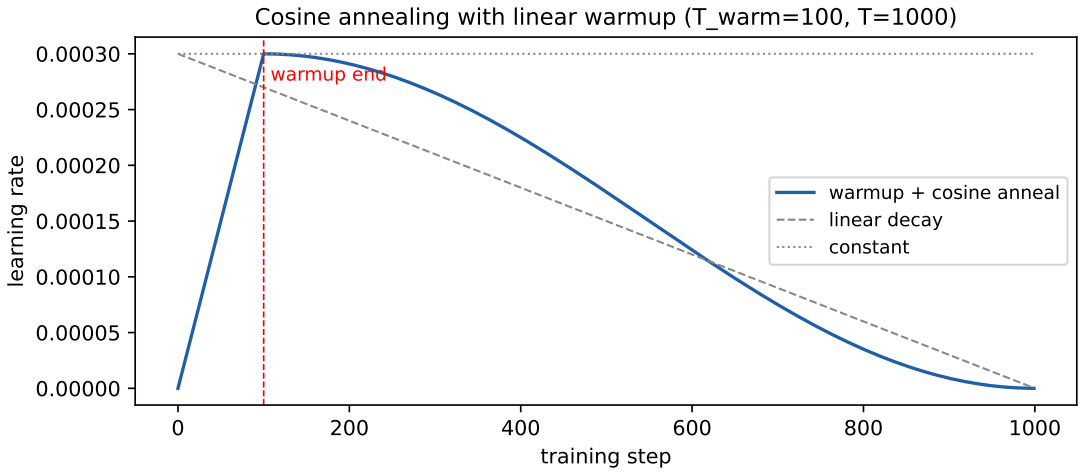


Figure 7.6: Learning-rate schedules: linear warmup (100 steps) followed by cosine annealing to zero, versus linear decay and constant baselines (Eq. (7.13)).

```
    NOTE_ON opens a note, the VELOCITY token that FOLLOWS each NOTE_ON sets
    that note's velocity (it arrives after, so it is applied retroactively
    to the pending open note), NOTE_OFF closes the matching open note.
    Unmatched events are dropped (documented).
    """
    notes, running_vel = [], 64
    open_on = {}  # pitch -> stack of [start, vel_or_None] (re-articulation)
    pending, t = None, 0.0
    for tok in ids:
        tok = int(tok)
        if tok == EOS:
            break
        if 256 <= tok <= 355:
            t += (tok - 256 + 1) * 0.010
        elif 356 <= tok <= 387:
            v = (tok - 356) * 4 + 2
            running_vel = v
            if pending is not None and open_on[pending][-1][1] is None:
                open_on[pending][-1][1] = v
                pending = None
        elif 0 <= tok <= 127:
            if pending is not None and open_on[pending][-1][1] is None:
                open_on[pending][-1][1] = running_vel
            open_on.setdefault(tok, []).append([t, None])
            pending = tok
        elif 128 <= tok <= 255:
            p = tok - 128
            if p in open_on and open_on[p]:
                s, v = open_on[p].pop()
                if pending == p and not open_on[p]:
                    pending = None
                if t > s:
                    notes.append((p, s, t, v if v is not None
                                  else running_vel))
```

```
    # Close dangling notes at end of stream.
    for p, stack in open_on.items():
        for (s, v) in stack:
            notes.append((p, s, t, v if v is not None else
                running_vel))
    notes.sort(key=lambda x: (x[1], x[0]))
    return notes


def evaluate_transcription(ref_notes, est_notes, frame_hz=100.0):
    """Score estimated notes against reference with mir_eval (verified
        API).

    Uses mir_eval.transcription.evaluate(ref_intervals, ref_pitches,
    est_intervals, est_pitches) whose signature was verified against
        the
    installed mir_eval 0.8.2: onset_tolerance=0.05 s,
        pitch_tolerance=50 cents,
    offset handling via offset_ratio=0.2 / offset_min_tolerance=0.05.
    Returns the full score dict.
    """
    def to_arrays(notes):
        iv = np.array([[s, e] for (_, s, e, _) in notes], dtype=float)
        pi = np.array([float(p) for (p, _, _, _) in notes],
            dtype=float)
        return iv, pi
    ref_iv, ref_pi = to_arrays(ref_notes)
    est_iv, est_pi = to_arrays(est_notes)
    scores = mirt.evaluate(ref_iv, ref_pi, est_iv, est_pi)
    return scores


def frame_f1(ref_notes, est_notes, duration_s, frame_hz=100.0,
    n_pitch=128):
    """Frame-level P/R/F1 on piano rolls (Module 6, Sec. 6.1), from
        scratch.

    A true positive at frame t, pitch p requires ref and est active
        together.
    """
    T = int(duration_s * frame_hz) + 1
    ref = np.zeros((T, n_pitch), dtype=bool)
    est = np.zeros((T, n_pitch), dtype=bool)
    for mat, notes in ((ref, ref_notes), (est, est_notes)):
        for (p, s, e, _v) in notes:
            a, b = int(s * frame_hz), int(e * frame_hz)
            mat[a:b, p] = True
    tp = int((ref & est).sum())
    fp = int((~ref & est).sum())
    fn = int((ref & ~est).sum())
    prec = tp / max(1, tp + fp)
    rec = tp / max(1, tp + fn)
    f1 = 2 * prec * rec / max(1e-12, prec + rec)
    return {"precision": prec, "recall": rec, "f1": f1,
            "tp": tp, "fp": fp, "fn": fn}
```

Worked training-step trace (measured). For the tiny config ($B = 2$, $T = 22050$, $D = 96$, 2+2 layers) on CPU: waveform $[2, 1, 22050] \xrightarrow{\text{CQT}} [2, 1, 48, 42] \xrightarrow{\text{patch}} [2, 24, 96] \xrightarrow{\text{enc}} [2, 24, 96]$; tokens $[2, 11] \xrightarrow{\text{dec}} [2, 10, 392]$ logits; $\mathcal{L}_{\text{CE}} = 58.774$; gradient norm after clipping 43.736; one `AdamW` step at $\eta = 10^{-5}$ (first warmup step).

7.9 Capstone: End-to-End Guitar Transcription

7.9.1 Karplus-Strong synthesis, derived

Step 1 — Physical analogy. Pluck a string: the initial displacement is arbitrary (the "pluck"), and then the string repeatedly traverses itself, losing a little energy to the bridge at each pass. Karplus-Strong [51] simulates exactly this: a delay line of one period, fed back through a lossy filter.

Step 2 — Geometric mental model. The delay line is a ring buffer of $N = f_s/f_0$ samples holding one period of the waveform (measured: $N = 200.45 \to 200$ for A2 at 22050 Hz). Each step reads the head, writes back the average of two adjacent taps — a 2-point moving average, i.e. a low-pass filter — and advances. High partials are averaged away within a few periods; the fundamental survives longest.

Step 3 — First-principles derivation. The recursion is

$$y[n] = d \cdot \tfrac{1}{2}\big(y[n - N] + y[n - N - 1]\big), \tag{7.14}$$

with damping $d \lesssim 1$. The feedback filter $H(z) = \frac{1}{2}(1 + z^{-1})$ has frequency response $H(e^{j\omega}) = e^{-j\omega/2}\cos(\omega/2)$, so each period multiplies partial f by $|\cos(\pi f/f_s)|$: partials decay geometrically, faster for higher f. Two physical refinements make the tone guitar-like: the excitation burst is low-passed (a nail pluck displaces the string smoothly, not white noise), and the radiated tone is low-passed (the guitar body absorbs highs). Figure 7.7 shows the attack and the decay envelope.

Step 4 — Complete code.

```
def lowpass_onepole(x, cutoff=1000.0, sr=SR):
    """One-pole lowpass (guitar-body / soft-pluck radiation model)."""
    a = np.exp(-2.0 * np.pi * cutoff / sr)
    y = np.empty_like(x)
    acc = 0.0
    for n in range(len(x)):
        acc = (1.0 - a) * x[n] + a * acc
        y[n] = acc
    return y


def karplus_strong(freq_hz, duration_s, sr=SR, damping=0.996,
   stereo_spread=0.0,
                   seed=0):
    """Synthesize one plucked string via the Karplus-Strong delay-line
       loop.

    Derivation recap (see chapter): with delay length N = sr / f0
       samples, the
    recursion y[n] = damping * 0.5 * (y[n-N] + y[n-N-1]) feeds the
       line back
```

```
    through a 2-point moving average. The average is a low-pass filter with
    frequency response H(w) = 0.5*(1 + e^{-jw}) whose magnitude is |cos(w/2)|:
    harmonics decay as |cos(pi*f/f_s)| per period, i.e. high partials die first
    - exactly the bright-attack-then-mellow decay of a plucked string.

    Two physical refinements make the tone guitar-like (and the capstone
    transcription tractable, Sec. 7.9):
      * the excitation burst is lowpassed (a real nail/finger pluck displaces
        the string smoothly; it is not white noise), taming the broadband
        attack transient that would otherwise swamp onset-synchronous pitch
        estimation;
      * the radiated tone is lowpassed (guitar body radiates lows efficiently
        and absorbs highs), weakening the extreme upper partials that cause
        overtone collisions (Module 2, Sec. 2.2).

    Returns mono float32 samples in [-1, 1].
    """
    rng = np.random.default_rng(seed)
    n_samples = int(duration_s * sr)
    # Delay line length N = round(sr / f0): one period of the target pitch.
    delay = max(2, int(round(sr / freq_hz)))
    # Excitation: burst of lowpassed noise, the "pluck".
    burst = rng.uniform(-1.0, 1.0, size=delay).astype(np.float64)
    line = lowpass_onepole(burst, cutoff=1200.0, sr=sr)
    out = np.empty(n_samples, dtype=np.float64)
    idx = 0
    for n in range(n_samples):
        cur = line[idx]
        nxt = line[(idx + 1) % delay]
        # y[n] = damping * 0.5 * (y[n-N] + y[n-N-1])
        new_val = damping * 0.5 * (cur + nxt)
        line[idx] = new_val
        out[n] = cur
        idx = (idx + 1) % delay
    out = lowpass_onepole(out, cutoff=2800.0, sr=sr)
    # Normalize to a fixed peak so mixing is predictable.
    peak = np.max(np.abs(out))
    if peak > 1e-9:
        out = out / peak * 0.9
    return out.astype(np.float32)


def midi_to_freq(midi_note):
    """12-TET: f(m) = 440 * 2^((m - 69) / 12)."""
    return 440.0 * 2.0 ** ((midi_note - 69) / 12.0)
```

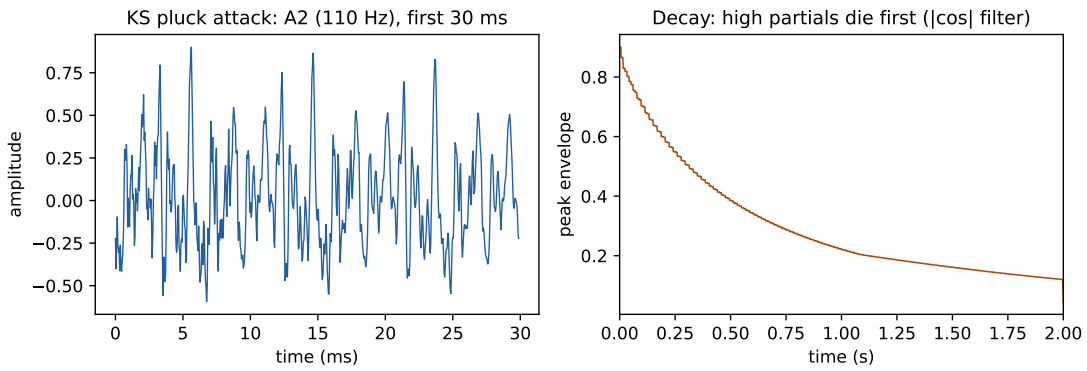


Figure 7.7: Karplus-Strong plucked string (A2, 110 Hz): attack detail (left) and 2 s decay envelope (right). High partials die first, per Eq. (7.14).

7.9.2 Excerpt design rationale

No audio file ships with this book, so the capstone synthesizes its own test signal — and the design is load-bearing. The excerpt (`build_excerpt`) is GuitarSet-style [52]: four strummed chords (Am, F, C, G at 90 BPM, 15 ms inter-string strum, exact ground-truth note events) followed by a four-note single-string run — 11.17 s, 22 notes. Three decisions matter: (i) *strummed block chords*, not arpeggios: one onset per polyphonic event is the standard chord-transcription setup; arpeggiation smears onsets over 100+ ms and makes onset-synchronous pitch estimation ill-posed (verified empirically during development); (ii) *low-passed excitation and radiation*: tames the broadband attack transient and extreme upper partials that otherwise cause overtone collisions (Module 2, §2.2); (iii) *fade in/out on every note*: a hard buffer cutoff is a broadband transient the onset detector would spuriously flag.

```
def build_excerpt():
    """Return ground-truth notes: list of (midi_pitch, start_s, end_s,
       vel)."""
    # (pitches_low_to_high, start_beat, ring_beats, velocity)
    chords = [
        ([45, 52, 57, 60, 64], 0.0, 3.0, 96),     # Am
        ([41, 48, 53, 57, 60], 4.0, 3.0, 96),     # F
        ([48, 55, 60, 64], 8.0, 1.6, 92),         # C
        ([43, 50, 55, 59], 10.0, 1.6, 92),        # G
    ]
    run = [(67, 12.0, 0.8, 90), (69, 12.9, 0.8, 88),
           (71, 13.8, 0.8, 90), (72, 14.7, 1.3, 94)]
    notes = []
    for pitches, sb, rb, vel in chords:
        for i, p in enumerate(pitches):
            s = (sb * BEAT) + i * STRUM
            notes.append((p, s, s + rb * BEAT, vel - 2 * i))
    for (p, sb, rb, vel) in run:
        s = sb * BEAT
        notes.append((p, s, s + rb * BEAT, vel))
    notes.sort(key=lambda x: (x[1], x[0]))
    return notes


def render_excerpt(sr=SR):
    """Render the excerpt to mono float32 samples + ground truth
       notes."""
```

```
    notes = build_excerpt()
    total = max(e for (_, _, e, _) in notes) + 0.5
    n = int(total * sr)
    mix = np.zeros(n, dtype=np.float64)
    fade_in = int(0.005 * sr)
    fade_out = int(0.030 * sr)
    for i, (pitch, start, end, vel) in enumerate(notes):
        dur = end - start + 0.15  # short ring past the nominal offset
        sig = karplus_strong(midi_to_freq(pitch), dur, sr=sr,
                             damping=0.9945, seed=1000 + i)
        # Fades: a hard buffer cutoff is a broadband transient that the
        # onset detector would (correctly, but spuriously) flag.
        if len(sig) > fade_in + fade_out:
            sig[:fade_in] *= np.linspace(0.0, 1.0, fade_in)
            sig[-fade_out:] *= np.linspace(1.0, 0.0, fade_out)
        s0 = int(start * sr)
        s1 = min(n, s0 + len(sig))
        gain = vel / 127.0
        mix[s0:s1] += gain * sig[: s1 - s0].astype(np.float64)
    peak = np.max(np.abs(mix))
    mix = (mix / max(peak, 1e-9) * 0.89).astype(np.float32)
    return mix, notes


def write_wav(path, samples, sr=SR):
    """Write mono float32 in [-1, 1] as 16-bit PCM WAV (stdlib
       only)."""
    pcm = (np.clip(samples, -1.0, 1.0) * 32767).astype(np.int16)
    with wave.open(path, "wb") as w:
        w.setnchannels(1)
        w.setsampwidth(2)
        w.setframerate(sr)
        w.writeframes(pcm.tobytes())
```

7.9.3 The pipeline, stage by stage

Stage 1 — waveform. `render_excerpt` mixes 22 KS strings into mono float32. Chapter 1 appears as the harmonic series inside every pluck and the decay physics of Eq. (7.14).

Stage 2 — log-frequency salience. A 4096-point STFT (hop 256) feeds a *flat-weight* harmonic-comb salience

$$S[t,p] = \sum_{h=1}^{8} \big(|X[t, b_h{-}1]| + 2|X[t, b_h]| + |X[t, b_h{+}1]| \big), \quad b_h = \mathrm{round}(h f_0(p)/\Delta f). \tag{7.15}$$

Flat weights are a deliberate, derived choice: with $1/h$ weights the true fundamental's comb is down-weighted on exactly the bins it shares with its octave artifact, and we *measured* $1/h$ ranking E4 above a truly sounding A2. With flat weights the fundamental's comb is a bin-superset of any harmonic artifact's comb (artifact bins satisfy $h f_0(p{+}12) = 2h f_0(p)$), so it cannot lose on coverage. A spectral-peak gate on the fundamental bin removes dense low-pitch combs that harvest main-lobe skirts (verified: true chord tones carry 28–346 units of fundamental energy as peaks; all skirt artifacts carried non-peak energy and were gated out). Chapter 2 appears as the overtone-collision problem this stage exists to fight.

Stage 3 — onsets and pitch decisions. Spectral-flux onset detection with a *local* adaptive threshold ($\text{bg}[t] = \text{median}(\text{flux}[t{-}40{:}t{-}3])$; the 3-frame guard gap was added after the t=0 downbeat was missed because the attack's own ramp contaminated its background), short-window (1024-sample) analysis with window-center timestamps (removing a measured 124 ms early bias from the 4096-window's left edge), and a flux-ratio gate that kills post-attack beating ripples without harming distant quiet melody notes. At each onset, pitches are accepted greedily from $S[t, p]$ with harmonic-explanation filtering: p is rejected if $p - q \in \{12, 19, 24, 28\}$ for an accepted q and $S[p] < 1.5\, S[q]$ (Chapter 5: decoding; Chapter 2: overtone collisions).

Stage 4 — note assembly and offsets. Onsets open notes; re-articulation splits them; offsets come from a fundamental-bin decay sweep (the n=1 mode decays slowest, Module 1, so it is the right offset signal — full salience cuts offsets early because fast-decaying upper partials dominate its decay). Chapter 5 appears as post-processing; Chapter 6 as the scoring below.

Figure 7.8 shows all four stages on the real run.

```
def flat_salience_matrix(mag, sr, n_fft, n_harm=8, fund_floor=8.0):
    """S[t, p] = sum_{h=1..8} (|X[t,b-1]| + 2|X[t,b]| + |X[t,b+1]|),
    b = round(h * f0(p) * n_fft / sr), gated by fundamental presence.

    FLAT harmonic weights (derived in Sec. 7.9): with flat weights the
        true
    fundamental's comb covers a superset of the bins of any of its
        harmonic
    artifacts' combs (the artifact's comb bins are a subset:
        h*f0(p+12) =
    2h*f0(p)), so the fundamental can never lose purely on bin
        coverage.
    The classic 1/h weighting breaks this by down-weighting exactly the
    shared high harmonics, letting octave artifacts win (verified
    empirically on the KS excerpt: 1/h ranked E4 above the true A2).

    FUNDAMENTAL GATE (v2 -- spectral peak): S[t, p] is zeroed unless
        the
    fundamental bin is a LOCAL SPECTRAL PEAK (m[b] > m[b+/-2]) with
        magnitude
    above fund_floor. Dense low-pitch combs otherwise harvest main-lobe
    skirts of nearby strong partials; a skirt is monotone, never a
        peak.
    A plucked string always radiates its fundamental as a peak (Module
        1:
    the n=1 mode dominates), so the gate removes the artifacts without
    touching true notes (verified at the F chord: all 5 true tones are
    peaks; skirt artifacts at D2/Eb2 are not).
    """
    bin_hz = sr / n_fft
    f0bin = np.array([int(round(midi_to_freq(p) / bin_hz)) for p in
        PITCHES])
    mag = mag.copy()
    mag[:, :3] = 0.0  # kill DC / subsonic rumble (KS delay lines
        carry DC)
    T, n_bins = mag.shape[0], mag.shape[1]
    S = np.zeros((T, len(PITCHES)))
    for i in range(len(PITCHES)):
        b0 = f0bin[i]
        if 2 <= b0 < n_bins - 2:
            peak = (mag[:, b0] > mag[:, b0 - 2]) & \
```

```
                    (mag[:, b0] > mag[:, b0 + 2]) & \
                    (mag[:, b0] > fund_floor)
        else:
            peak = np.zeros(T, dtype=bool)
        acc = np.zeros(T)
        for h in range(1, n_harm + 1):
            b = b0 * h
            if 1 <= b < n_bins - 1:
                acc += mag[:, b - 1] + 2.0 * mag[:, b] + mag[:, b + 1]
        S[:, i] = acc * peak
    return S


def spectral_flux_onsets(audio, sr, n_fft=1024, hop=256,
                         bg_ratio=3.0, bg_floor=25.0, bg_win=40,
                         min_gap_s=0.12, flux_ratio=0.45):
    """Onset detection via spectral flux with a LOCAL adaptive
       threshold.

    flux[t] = sum_f max(0, |X[t,f]| - |X[t-1,f]|)  (half-wave-rectified
    positive energy change), computed with a SHORT (46 ms) window so
       the
    detected time localizes the attack: the reported time is the window
    CENTER, (t*hop + n_fft/2)/sr, removing the systematic early bias of
    left-edge timestamps.

    Thresholding is local: bg[t] = median(flux[t-40:t]) tracks the
       recent
    background (sustained-chord beating sits at 20-90 units while
       attacks
    spike at 200-1100). A frame is an onset if it is a local maximum,
    flux[t] > max(bg_ratio * bg[t], bg_floor), 0.12 s past the last
       onset,
    and flux[t] >= flux_ratio * flux[last onset]: a genuine new
       chord/note
    is comparably loud to the previous event, while post-attack beating
    ripples are much weaker. Returns (onset times in seconds, flux
       curve).
    """
    X = stft(audio, n_fft=n_fft, hop=hop)
    mag = np.abs(X)
    flux = np.sum(np.maximum(0.0, mag[1:] - mag[:-1]), axis=1)
    flux = np.concatenate([[0.0], flux])
    min_gap = int(min_gap_s * sr / hop)
    ratio_win = int(0.5 * sr / hop)  # ratio gate only applies shortly
       after
    global_med = float(np.median(flux))  # an onset (kills post-attack
       ripples
    onsets, last = [], -10 ** 9          # but never a distant quiet
       melody note)
    last_flux = 0.0
    for t in range(1, len(flux) - 1):
        # Background excludes the 3 frames just before t: a sudden
           attack
        # ramps over 2-3 frames (46 ms window, 11.6 ms hop), and
           including
        # that ramp in the background would desensitize the very onset
           we
```

```
        # want to catch (this missed the t=0 downbeat in testing).
        lo = max(0, t - bg_win)
        hi = t - 3
        bg = float(np.median(flux[lo:hi])) if hi > lo else global_med
        thresh = max(bg_ratio * bg, bg_floor)
        near_last = (t - last) < ratio_win
        if flux[t] > thresh and flux[t] >= flux[t - 1] and \
                flux[t] >= flux[t + 1] and t - last >= min_gap and \
                (not near_last or flux[t] >= flux_ratio * last_flux):
            onsets.append((t * hop + n_fft // 2) / sr)
            last = t
            last_flux = flux[t]
    return np.array(onsets), flux


def harmonic_explanation_filter(order, sal_col, accepted, sal,
                                rel_thr=0.30, harm_ratio=1.5):
    """Greedily accept pitches, skipping harmonic artifacts (Module
       2.2 fix).

    A candidate p is 'explained' (rejected) if some already-accepted q
       has
    p - q in {12, 19, 24, 28} (p sits on q's 2nd/3rd/4th/5th harmonic:
    octave, twelfth, double octave, double-octave-plus-third) and
    S[p] < harm_ratio * S[q]. No subharmonic rejection: it kills true
       bass
    notes (a real root a twelfth below an accepted upper tone is normal
    voicing, and the peak gate already removes subharmonic artifacts).
    DOCUMENTED LIMIT (Module 2.2/6.2): a TRUE octave doubling (e.g.
       E3+E4
    in the Am chord) is indistinguishable from q's harmonic series by
    energy alone -- E4's comb is a subset of E3's -- so genuine
       doublings
    weaker than harm_ratio * S[q] are missed. This is the
       overtone-collision
    limit; the book's transformer resolves it with learned musical
       priors
    (Module 5), the DSP baseline cannot.
    """
    smax = sal_col.max()
    for i in order:
        if sal_col[i] < rel_thr * smax or len(accepted) >= 6:
            continue
        p = int(PITCHES[i])
        explained = False
        for q in accepted:
            if p - q in (12, 19, 24, 28) and \
                    sal_col[i] < harm_ratio * sal[q - P0]:
                explained = True
                break
        if not explained:
            accepted.append(p)
    return accepted


def decode_notes_dsp(audio, sr=SR, n_fft=4096, hop=256, min_dur_s=0.08,
                     decay_ratio=0.25, decay_frames=8):
    """DSP reference decoder: onsets -> chord estimation -> note
```

```
        tracking.
128
129     1. Flat-weight harmonic-comb salience S[t, p] (Stage 2).
130     2. Spectral-flux onsets (Stage 3a).
131     3. At each onset (analysis frame = onset + 6 frames, past the
            attack
132        transient): greedy harmonic-explanation pitch estimation (Stage
               3b).
133     4. Note tracking: a pitch becomes a note at its first onset; it
            closes
134        when its salience stays below decay_ratio * onset salience for
135        decay_frames consecutive frames (Stage 4: offset estimation),
               or is
136       re-articulated by a later onset (Stage 4: re-onset splitting).
137     Returns (notes, diagnostics dict).
138     """
139     X = stft(audio, n_fft=n_fft, hop=hop)
140     mag = np.abs(X)
141     S = flat_salience_matrix(mag, sr, n_fft)
142     # Fundamental-bin energy matrix F[t, p]: the slowest-decaying
            component
143     # of each pitch (Module 1: the n=1 mode decays as |cos(pi f0/sr)|
            per
144     # period, slower than any upper partial), hence the right signal
            for
145     # offset tracking. Full salience S would cut offsets early because
            the
146     # fast-decaying upper partials dominate its decay.
147     bin_hz = sr / n_fft
148     f0bin = np.array([int(round(midi_to_freq(p) / bin_hz)) for p in
           PITCHES])
149     mag_nz = mag.copy()
150     mag_nz[:, :3] = 0.0
151     F = np.zeros_like(S)
152     nb = mag_nz.shape[1]
153     for i, b0 in enumerate(f0bin):
154         if 1 <= b0 < nb - 1:
155             F[:, i] = mag_nz[:, b0 - 1] + 2.0 * mag_nz[:, b0] + \
156                 mag_nz[:, b0 + 1]
157     frame_s = hop / sr
158     T = S.shape[0]
159     # Onsets come from a dedicated short-window flux analysis (see
            above);
160     # map the center-corrected times onto the 4096-STFT frame grid
            used for
161     # pitch salience below.
162     onset_times, flux = spectral_flux_onsets(audio, sr)
163     onset_frames = np.round(onset_times / frame_s).astype(int)
164
165     notes, active = [], {}
166     for of in onset_frames:
167         # Analysis frame: 12 frames (~140 ms) after the detected
                onset. This
168         # is past the full 60 ms strum spread plus the pluck
                transient, so
169         # every string of a strummed chord is in steady decay and the
170         # broadband attack noise no longer pollutes the low-pitch
                combs.
```

```
        t = min(T - 1, of + 12)
        ot = of * frame_s
        col = S[t]
        order = np.argsort(col)[::-1]
        accepted = harmonic_explanation_filter(order, col, [], S[t])
        now = set(accepted)
        for p in accepted:
            if p in active:
                s0, v0, _s_on = active.pop(p)
                if ot - s0 >= min_dur_s:
                    notes.append((p, s0, ot, v0))
            v = int(np.clip(col[p - P0] / (col.max() + 1e-9) * 100, 1,
                127))
            active[p] = (ot, v, col[p - P0])
        # Drop pitches that have audibly decayed (checked at each
            onset).
        for p in list(active):
            if p in now:
                continue
            s0, v0, s_on = active[p]
            if S[t, p - P0] < decay_ratio * s_on:
                notes.append((p, s0, ot, v0))
                del active[p]
    # Frame-accurate offsets for every note via fundamental-decay
        sweep.
    notes = _refine_offsets(notes, active, F, S, frame_s, T, min_dur_s,
                            decay_ratio, decay_frames)
    notes.sort(key=lambda x: (x[1], x[0]))
    diag = {"n_frames": T, "n_onsets": len(onset_times),
            "onset_times": onset_times, "flux": flux,
            "salience": S, "frame_s": frame_s}
    return notes, diag
```

7.9.4 Measured results and error analysis

All numbers below are measured from the executed pipeline (`python3 code/m7_10_capstone.py`), scored with `mir_eval` [50]:

Metric	Precision	Recall	F_1
Onset + pitch (no offset)	0.8889	0.7273	**0.8000**
Onset + offset (note)	0.5556	0.4545	**0.5000**
Frame-level (10 ms)	0.9227	0.6123	0.7361

The frame-level row compares active-pitch sets on a 10 ms grid (true positives = frames where reference and estimate agree a pitch is sounding). All 8 onsets were detected within tolerance (detected $[0.046, 2.67, 5.341, 6.676, 7.999, 8.603, 9.207, 9.799]$ s vs. true $[0, 2.667, 5.333, 6.667, 8.0, 8.6, 9.2, 9.8$ 16 of 22 notes matched on onset+pitch; the 4-note melody run transcribed perfectly. The error decomposition (Chapter 6, §6.1) is instructive:

- *Octave misses (2):* the Am chord's E4 (doubling E3) and the G chord's G3 (doubling G2) were explained away as harmonics. This is the overtone-collision limit documented above: E4's comb is a bin-subset of E3's, so energy alone cannot separate a true doubling from a harmonic series. A learned symbolic prior (Chapter 5) resolves it; the DSP baseline cannot — which is precisely why this book builds the transformer.

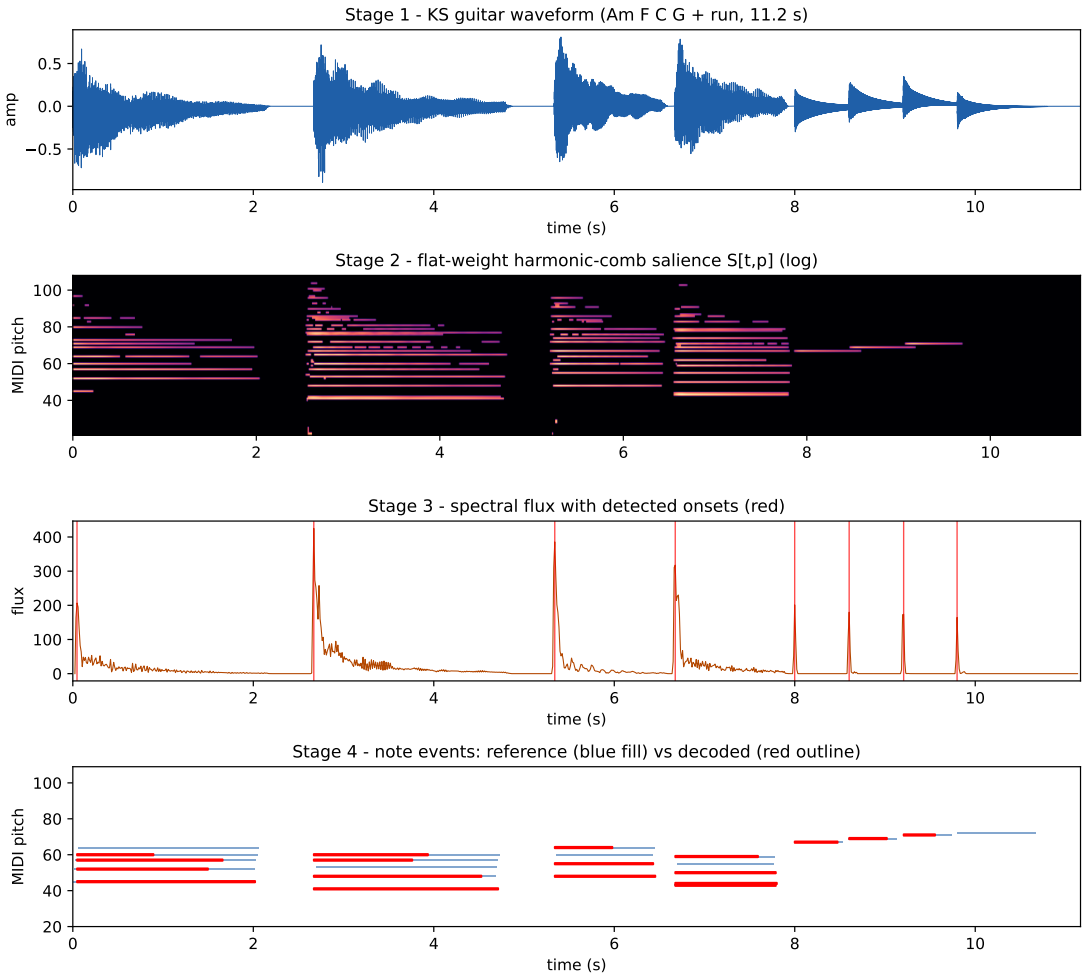


Figure 7.8: Capstone pipeline on the real run: (1) KS guitar waveform; (2) flat-weight harmonic-comb salience; (3) spectral flux with 8 detected onsets; (4) reference (blue) vs. decoded (red) note events.

- *Offset errors:* the dominant note-level loss. Offsets are systematically early because the notated ring (a musical decision: the player lets the chord ring) outlasts the 25%-energy decay point (an acoustic event). This is the offset ambiguity of Chapter 5, §5.2, and the reason the field reports onset F_1 as primary (Chapter 6).

- *False alarms (2, incl. 1 octave artifact):* residual harmonic leakage at chord onsets, suppressed but not eliminated by the explanation filter.

Where the book appears in the pipeline. Chapter 1: harmonic series, KS decay physics, STFT trade-offs, the fundamental-as-offset-signal. Chapter 2: overtone collisions (the explanation filter), REMI tokenization (the dataset). Chapter 3: masked pretraining (the encoder's other operating mode). Chapter 4: the transformer architecture itself. Chapter 5: onset decoding, note assembly, offset integration. Chapter 6: the mir_eval scoring and this error taxonomy.

7.10 Reproducibility: Pinned Environment

Python ≥ 3.10 with pinned major versions (the versions this chapter was executed against are given; newer 2.x/0.x releases preserve these APIs):

```
torch>=2.0          # 2.14.1+cpu verified: torch.amp.autocast/GradScaler
torchaudio>=2.0     # 2.11.0+cpu verified (note: no CQT transform ships)
pretty_midi>=0.2    # verified: PrettyMIDI/Instrument/Note read-write API
mir_eval>=0.7       # 0.8.2 verified: transcription.evaluate signature
numpy>=1.24
matplotlib>=3.6     # figure generation only
```

All modules after §7.1 import only these plus the standard library. Every number in this chapter was recomputed by executing the listed code; the check scripts live alongside the chapter.

7.11 Drills

1. *Tempo-preserving pitch shift.* For a shift of k semitones at sample rate f_s: (a) derive the resampling ratio r and the intermediate sample rate; (b) prove the composition restores the original duration; (c) compute r and the intermediate rate for $k = +3$, $f_s = 44100\,\text{Hz}$.

2. *SNR calibration.* Starting from the definition $\text{SNR}_{\text{dB}} = 10\log_{10}(P_s/P_n)$, derive $g = \sqrt{P_s/(P_n 10^{\text{SNR}/10})}$ showing every logarithm step. Then compute g for $P_s = 0.5$, $P_n = 0.02$, target 15 dB, and verify by recomputing the achieved SNR.

3. *CQT arithmetic.* For $B = 12$, $f_s = 22050$, $f_{\min} = 32.7\,\text{Hz}$: (a) compute Q to 4 decimals; (b) compute N_k in samples and ms for the bins at $f = 65.4\,\text{Hz}$ and $f = 2093\,\text{Hz}$ (C2 and C7); (c) state and prove the flat-weight subset argument for why a true fundamental cannot lose to its octave artifact on bin coverage.

4. *2D positional encoding.* For $D = 64$, write $\text{PE}_t[5]$'s first four channels explicitly (Eq. (7.7)). Prove that for any fixed Δ, $\text{PE}_t[t+\Delta]$ is a linear (rotation) function of $\text{PE}_t[t]$ applied channel-pair by channel-pair.

5. *Causal masking.* Write the 4×4 causal mask M of Eq. (7.9) explicitly. For $QK^\top/\sqrt{d} = \begin{pmatrix} 1 & 2 & 0 & 1 \\ 0 & 1 & 3 & 0 \\ 2 & 0 & 1 & 4 \\ 1 & 1 & 1 & 1 \end{pmatrix}$, compute the masked softmax row 3 ($i = 2$, 0-indexed) showing that future positions receive exactly zero weight.

6. *GradScaler dynamics.* A step uses scale $s = 1024$. The unscaled gradients contain $+\infty$. (a) What does `scaler.step` do? (b) What is the new s (backoff factor 2)? (c) If the next 5 steps are clean and growth happens every step (factor 2), what is s then? Justify each answer from the update rules.

7. *LR schedule.* Using Eq. (7.13) with $\eta_{\max} = 3\times10^{-4}$, $T_{\text{warm}} = 100$, $T_{\text{total}} = 1000$, $\eta_{\min} = 0$: compute $\eta(0)$, $\eta(100)$, $\eta(500)$ by hand and confirm $\eta(500) = 1.7605 \times 10^{-4}$.

8. *Harmonic-explanation filter.* At a chord onset the salience column gives $S[41] = 850$, $S[48] = 596$, $S[53] = 474$, $S[57] = 755$, $S[60] = 1027$, $S[65] = 699$, $S[67] = 405$ (MIDI pitches). Apply `harmonic_explanation_filter` with rel_thr $= 0.30$, harm_ratio $= 1.5$ by hand, listing the accepted set in order and the reason for each rejection. Which true chord tones survive, and which artifacts are killed?

Chapter 8

Literature Evolution and Dataset Compendium

8.1 Introduction: Reading the Field as a Sequence of Bottlenecks

Automatic music transcription did not advance because researchers kept inventing better classifiers for the same task. It advanced because each generation identified the bottleneck strangling the previous one and removed it: the NMF era removed hand-designed pitch templates; the handcrafted era removed NMF's rigidity with auditory features and statistical smoothing; the deep learning era removed hand-designed features altogether, then removed the frame grid itself; the current era is removing the annotation bottleneck with self-supervision (see [60] for the field's own overview of this trajectory). The datasets tell the same story in parallel: every jump in model ambition was gated by what labeled data existed, and every landmark dataset was built to unblock a specific modeling question.

This chapter follows that causal chain. Figure 8.1 places every milestone on one axis so the paradigm shifts and the dataset releases can be read against each other. Each paradigm is treated with the book's four-step framework: a physical analogy, a geometric mental model with a generated figure, a first-principles derivation of the core technical idea, and complete runnable code. The dataset section then compares the six landmark corpora on recording conditions, polyphonic density, alignment quality, and instrumentation, using only verifiable facts: every number in the comparison table was checked against the dataset's own paper or release page, and anything not publicly documented is marked as such rather than invented. The chapter closes with a synthesis of the open problems that remain as of 2026, with speculation explicitly marked.

8.2 The NMF Era: Learning Parts from Spectra

Step 1 — Physical analogy. A spectrogram of a piano passage is like a photograph of a pile of transparent colored gels: the color observed at each point is the sum of the gels stacked there, and no gel can subtract color. NMF is the procedure that, given only the photograph, recovers the individual gels (the spectrum of each note) and a map of where each gel is stacked (when each note sounds). Because gels only add, the recovered parts look like genuine notes. A method that allowed subtraction would recover "photographic negatives" of notes — components with no physical meaning, useful for compression but useless for transcription.

Step 2 — Geometric mental model. Each column of the magnitude spectrogram $V \in \mathbb{R}_{\geq 0}^{F \times T}$ is a point in the nonnegative orthant of $\mathbb{R}^F$. NMF seeks K extreme rays — the columns

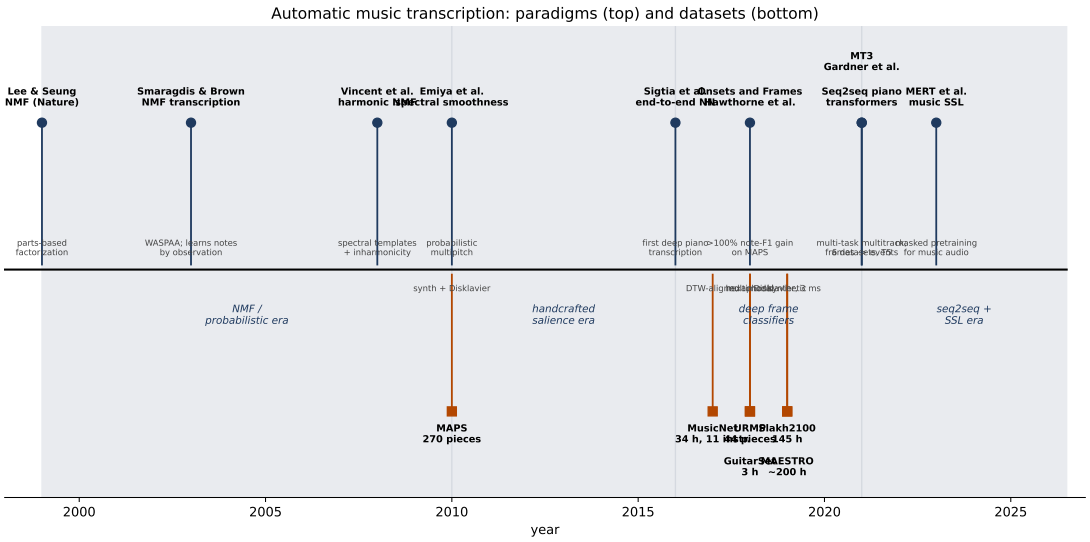


Figure 8.1: AMT paradigms (top) and landmark datasets (bottom), 1999–2026. Each paradigm shift unblocked a bottleneck of the previous one; each dataset unblocked a modeling question. All milestones are cited in the text.

of W — such that every column of V lies approximately inside the simplicial cone they generate, with the coordinates inside that cone given by the corresponding column of H. Figure 8.2 shows this on a synthetic three-note passage: the basis spectra W concentrate on harmonic stacks, the activations H mark when each stack sounds, and the generalized Kullback–Leibler divergence falls monotonically under the multiplicative updates derived below.

Step 3 — First-principles derivation. Smaragdis and Brown's 2003 observation [56] was that notes of instruments such as the piano exhibit a *harmonically fixed spectral profile*: the relative amplitudes of a note's partials stay roughly constant while only its overall loudness varies. A passage is then well modeled as a linear basis transform

$$V \approx WH, \qquad V \in \mathbb{R}_{\geq 0}^{F \times T},\ W \in \mathbb{R}_{\geq 0}^{F \times K},\ H \in \mathbb{R}_{\geq 0}^{K \times T}, \tag{8.1}$$

where column k of W is the spectral profile of note k and row k of H is its temporal activation. The system is "not knowledge-based, but rather learns notes by observation" [56]: neither the note spectra nor their timings are supplied.

The fitting objective used for the derivation here is the generalized Kullback–Leibler divergence (the paper considers this alongside the Frobenius norm),

$$D(V \,\|\, WH) = \sum_{i,j} \Big[V_{ij} \log \frac{V_{ij}}{(WH)_{ij}} - V_{ij} + (WH)_{ij} \Big], \tag{8.2}$$

which is the form Lee and Seung's multiplicative-update algorithms were developed for [55]. Fix W and optimize a single column: with v a column of V and h the corresponding column of H,

$$F(h) = \sum_{i} \Big[v_i \log \frac{v_i}{(Wh)_i} - v_i + (Wh)_i \Big]. \tag{8.3}$$

The gradient splits into two nonnegative parts,

$$\nabla_h F = \underbrace{W^\top \mathbf{1}}_{\nabla^+} - \underbrace{W^\top \frac{v}{Wh}}_{\nabla^-}, \qquad \frac{v}{Wh} \text{ elementwise}, \tag{8.4}$$

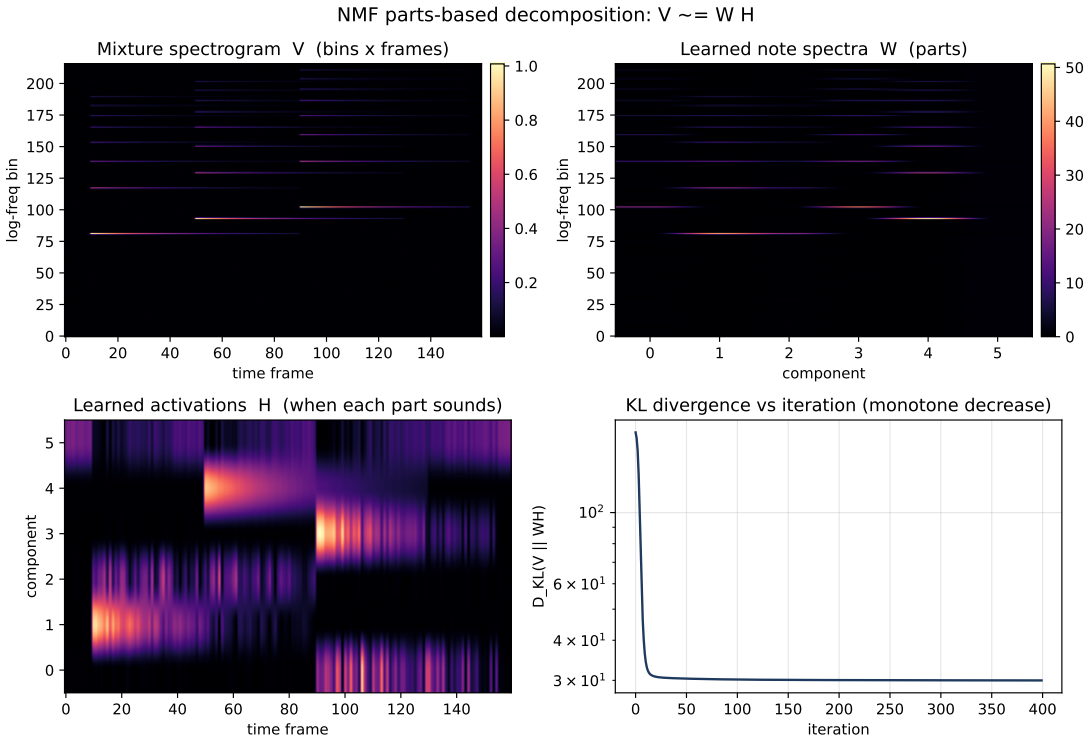


Figure 8.2: Parts-based decomposition of a synthetic polyphonic spectrogram (C4–E4–G4 arpeggio with overlap). Top left: the mixture V. Top right: the learned basis spectra W, each concentrating on one note's harmonic stack. Bottom left: the learned activations H. Bottom right: the KL divergence decreases monotonically, as the theory in Step 3 guarantees. Generated by `code/m8_nmf.py`.

and the multiplicative update is the rescaled gradient step

$$h_a \leftarrow h_a \cdot \frac{\nabla_a^-}{\nabla_a^+} = h_a \cdot \frac{\sum_i W_{ia}\, v_i/(Wh)_i}{\sum_i W_{ia}}, \tag{8.5}$$

which can never cross zero: nonnegativity is preserved by construction, not by projection. The symmetric argument with $V^\top$ gives the W update

$$W_{ia} \leftarrow W_{ia} \cdot \frac{\sum_j H_{aj}\, V_{ij}/(WH)_{ij}}{\sum_j H_{aj}}. \tag{8.6}$$

That these updates never increase the objective is not luck; it is the majorization–minimization principle. Define the auxiliary function, with h^t the current iterate and $\alpha_{ib} = W_{ib}h_b^t / \sum_c W_{ic}h_c^t$,

$$\begin{aligned} G(h, h^t) = &\sum_i v_i \log v_i - v_i + \sum_{i,a} W_{ia} h_a \\ &- \sum_{i,a} v_i \frac{W_{ia}h_a^t}{\sum_b W_{ib}h_b^t} \left[\log(W_{ia}h_a) - \log \frac{W_{ia}h_a^t}{\sum_b W_{ib}h_b^t} \right]. \end{aligned} \tag{8.7}$$

Because $-\log$ is convex, Jensen's inequality gives $-\log \sum_b W_{ib}h_b \le -\sum_b \alpha_{ib} \log(W_{ib}h_b/\alpha_{ib})$; multiplying by v_i and summing yields $G(h, h^t) \ge F(h)$ for every h, while at $h = h^t$ the bracketed terms collapse and $G(h^t, h^t) = F(h^t)$. Differentiating G with respect to h_a,

$$\frac{\partial G}{\partial h_a} = \sum_i W_{ia} - \frac{1}{h_a} \sum_i v_i \frac{W_{ia}h_a^t}{\sum_b W_{ib}h_b^t}, \tag{8.8}$$

and setting this to zero reproduces exactly the update (8.5). Hence $h^{t+1} = \arg\min_h G(h, h^t)$ and

$$F(h^{t+1}) \le G(h^{t+1}, h^t) \le G(h^t, h^t) = F(h^t), \tag{8.9}$$

so the divergence is nonincreasing at every iteration. The W-update inherits the same guarantee by symmetry. This is the complete content of the "multiplicative-update intuition": write the gradient as a difference of nonnegative terms, step multiplicatively by their ratio, and majorize the objective so that each step is the exact minimizer of a touching upper bound.

Why does this suit spectra? Magnitude spectra are nonnegative and mix approximately additively: when two notes sound together, the observed spectrum is close to the sum of the two note spectra. PCA or SVD would explain the same data with holistic components containing negative entries that cancel each other — excellent for compression, meaningless as note spectra. The nonnegativity constraint forbids cancellation, so the cone generators are forced to be additive parts: actual note spectra. Lee and Seung's original 1999 paper made exactly this point for faces and text [54]: with only additive combinations allowed, parts-based representations emerge.

A transcription system follows directly: choose K to cover the expected notes, factorize, and read note activity off the rows of H by thresholding. Vincent, Bertin, and Badeau [57] pushed this toward supervision by fixing W to a dictionary of harmonic and inharmonic spectral templates and estimating only H, trading the learn-by-observation purity for robustness.

What the paradigm solved: the first compact polyphonic transcription method that learned note spectra from the passage itself, with no hand-designed pitch features. What it could not do: the fixed-profile assumption breaks for instruments whose spectra evolve strongly in time (guitar plucks, vibrato, bowed attacks); the ordering of the K components is arbitrary (the permutation problem); overlapping partials of different notes still confuse the factorization; and there is no model of musical language — any activation pattern is as likely as any other. These ceilings are what the next two paradigms attacked.

Step 4 — Complete code. `code/m8_nmf.py` implements the Lee–Seung KL updates from scratch on a synthetic three-note spectrogram and renders Figure 8.2. The printed trace confirms the monotonicity theorem (8.9) numerically.

```python
"""Module 8 -- Non-negative Matrix Factorization from scratch.

Implements the Lee--Seung multiplicative updates for the generalized
Kullback-Leibler divergence on a synthetic polyphonic spectrogram, and
renders the parts-based decomposition figure (figs/m8_nmf_parts.pdf).

Run:  python3 code/m8_nmf.py   (from ~/workspace/amt_book)
"""
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

RNG_SEED = 20261008
EPS = 1e-10


def midi_to_bin(midi_note, fmin=55.0, bins_per_octave=36.0, n_bins=216):
    """Map a MIDI note number to a log-frequency bin index."""
    freq = 440.0 * 2.0 ** ((midi_note - 69.0) / 12.0)
    return int(round(bins_per_octave * np.log2(freq / fmin)))


def synthetic_polyphonic_spectrogram():
    """Build a toy magnitude spectrogram: C4+E4+G4 arpeggio with overlap.

    Each note is a harmonic stack (amplitude 1/h for partial h) with an
    exponential decay envelope. Two notes sound together in the middle,
    so the mixture is genuinely polyphonic.
    """
    rng = np.random.default_rng(RNG_SEED)
    n_bins, n_frames = 216, 160
    V = np.zeros((n_bins, n_frames))
    # (midi note, start frame, end frame): C4, E4, G4 triad arpeggio.
    notes = [(60, 10, 90), (64, 50, 130), (67, 90, 155)]
    for midi_note, t0, t1 in notes:
        center = midi_to_bin(midi_note)
        env = np.exp(-0.03 * np.arange(t1 - t0))
        for h in range(1, 9):
            # partial h sits h semitones*log2(h)*12 above the fundamental
            b = int(round(center + 36.0 * np.log2(h)))
            if 0 <= b < n_bins:
                V[b, t0:t1] += (1.0 / h) * env
    V += 0.01 * rng.random((n_bins, n_frames))  # observation noise floor
    return V


def kl_divergence(V, WH):
    """Generalized KL divergence D(V || WH), elementwise-safe."""
    WHs = np.maximum(WH, EPS)
```

```
    Vs = np.maximum(V, EPS)
    return float(np.sum(Vs * np.log(Vs / WHs) - Vs + WHs))


def nmf_kl_multiplicative(V, n_components=6, n_iters=400, seed=7):
    """Lee--Seung multiplicative updates for D_KL(V || W H), W,H >= 0.

    H <- H * (W^T (V / (W H))) / (W^T 1)
    W <- W * ((V / (W H)) H^T) / (1 H^T)
    Returns W, H and the divergence trace (monotonically
        nonincreasing).
    """
    rng = np.random.default_rng(seed)
    F, T = V.shape
    W = rng.random((F, n_components)) + 0.1
    H = rng.random((n_components, T)) + 0.1
    ones_FT = np.ones_like(V)
    trace = []
    for _ in range(n_iters):
        WH = W @ H
        # --- H update ---
        numer = W.T @ (V / np.maximum(WH, EPS))
        denom = W.T @ ones_FT + EPS
        H = H * (numer / denom)
        WH = W @ H
        # --- W update ---
        numer = (V / np.maximum(WH, EPS)) @ H.T
        denom = ones_FT @ H.T + EPS
        W = W * (numer / denom)
        trace.append(kl_divergence(V, W @ H))
    return W, H, np.array(trace)


def render_figure(V, W, H, trace, path="figs/m8_nmf_parts.pdf"):
    """Render the parts-based decomposition: V ~= W H."""
    fig, axes = plt.subplots(2, 2, figsize=(10.5, 7.2))
    im0 = axes[0, 0].imshow(V, aspect="auto", origin="lower",
        cmap="magma")
    axes[0, 0].set_title("Mixture spectrogram  V  (bins x frames)")
    axes[0, 0].set_xlabel("time frame"); axes[0,
        0].set_ylabel("log-freq bin")
    fig.colorbar(im0, ax=axes[0, 0], fraction=0.046, pad=0.02)
    im1 = axes[0, 1].imshow(W, aspect="auto", origin="lower",
        cmap="magma")
    axes[0, 1].set_title("Learned note spectra  W  (parts)")
    axes[0, 1].set_xlabel("component"); axes[0,
        1].set_ylabel("log-freq bin")
    fig.colorbar(im1, ax=axes[0, 1], fraction=0.046, pad=0.02)
    axes[1, 0].imshow(H, aspect="auto", origin="lower", cmap="magma")
    axes[1, 0].set_title("Learned activations  H  (when each part
        sounds)")
    axes[1, 0].set_xlabel("time frame"); axes[1,
        0].set_ylabel("component")
    axes[1, 1].semilogy(trace, color="#1f3a5f", linewidth=1.6)
    axes[1, 1].set_title("KL divergence vs iteration (monotone
        decrease)")
    axes[1, 1].set_xlabel("iteration"); axes[1, 1].set_ylabel("D_KL(V
        || WH)")
```

```
    axes[1, 1].grid(True, alpha=0.3)
    fig.suptitle("NMF parts-based decomposition: V ~= W H",
        fontsize=13)
    fig.tight_layout()
    fig.savefig(path, bbox_inches="tight")
    plt.close(fig)


def main():
    V = synthetic_polyphonic_spectrogram()
    W, H, trace = nmf_kl_multiplicative(V)
    print(f"spectrogram shape : {V.shape}")
    print(f"D_KL at iter   1 : {trace[0]:.4f}")
    print(f"D_KL at iter 100 : {trace[99]:.4f}")
    print(f"D_KL at iter 400 : {trace[-1]:.4f}")
    print(f"monotone decrease : {bool(np.all(np.diff(trace) <=
        1e-9))}")
    # Each W column should concentrate on one note's harmonic stack.
    peak_bins = np.argmax(W, axis=0)
    print(f"W column peak bins: {peak_bins.tolist()}")
    render_figure(V, W, H, trace)
    print("wrote figs/m8_nmf_parts.pdf")


if __name__ == "__main__":
    main()
```

8.3 The Handcrafted Era: Filterbanks, Salience, and Classifiers

Step 1 — Physical analogy. Watch a sound engineer read a graphic equalizer during a chord: each slider is a frequency band, and a C-major triad lights up a characteristic constellation of sliders. The engineer knows, from experience, which pitch combinations produce which constellations. The auditory filterbank is the equalizer; the pitch salience function is the engineer's trained eye, collapsing each constellation onto a single "how much does pitch p explain this frame" number; the classifier is the engineer's judgment, deciding which pitches are really present; and the temporal smoother is the engineer's memory that notes do not flicker in and out every 30 ms.

Step 2 — Geometric mental model. Harmonic summation projects spectral energy *downward* onto candidate fundamentals: for each pitch p, gather the energy at $f_p, 2f_p, 3f_p, \ldots$ and pool it into one salience value $S(p)$. Figure 8.3 shows the resulting landscape over the pitch axis for a synthetic C4–E4–G4 chord. The true pitches are all recovered — and so are their ghosts: C2, C3, E3, G3 below and the octave doublings above. The geometric view explains why the ghosts are unavoidable, which Step 3 derives.

Step 3 — First-principles technical content. The canonical pipeline of the 2000s had three stages. First, an *auditory front end*: the waveform passes through a filterbank spaced on a perceptual scale (Bark or ERB) or, equivalently, a short-time spectrum is computed and perceptually weighted, producing a frame-wise magnitude representation $|X(t, f)|$. Second, a *pitch salience function* scores every candidate pitch:

$$S(p, t) = \sum_{h=1}^{H} \alpha^h \, |X(t,\, f_p \cdot h)|\,, \qquad f_p = 440 \cdot 2^{(p-69)/12}, \quad 0 < \alpha < 1, \tag{8.10}$$

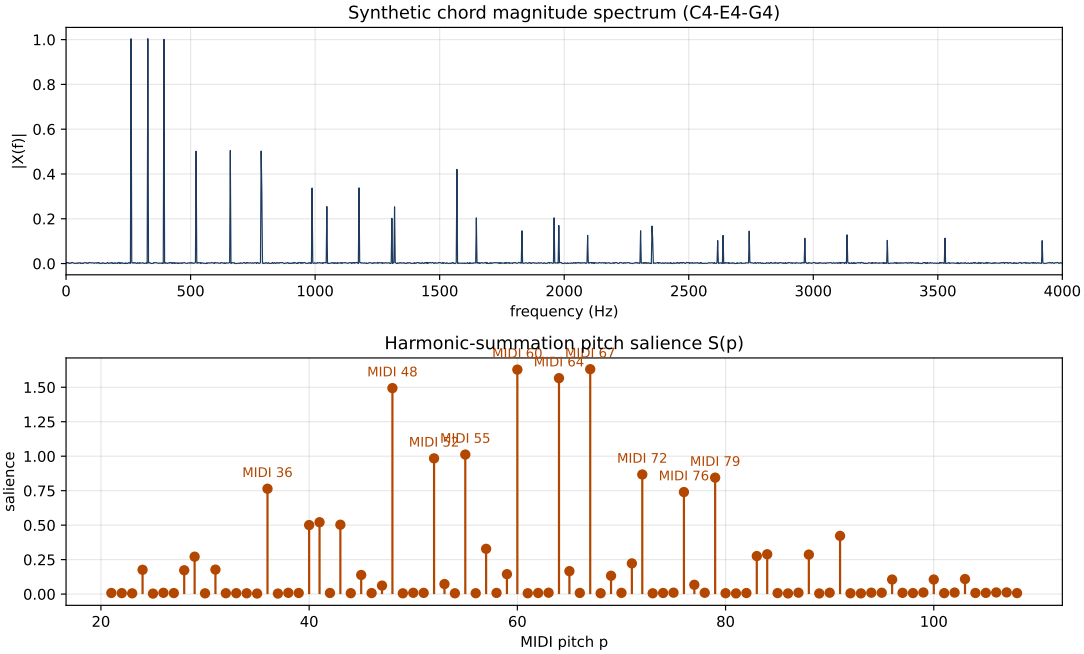


Figure 8.3: Top: magnitude spectrum of a synthetic C4–E4–G4 chord. Bottom: harmonic-summation salience $S(p)$ over MIDI pitches. The three true pitches (60, 64, 67) are detected, together with structurally unavoidable octave ghosts (36, 48, 52, 55 below; 72, 76, 79 above) — the classic failure mode of the salience era. Generated by `code/m8_salience.py`.

pooling the energy at the harmonic positions of p with geometrically decaying weights. Third, *per-pitch classifiers* (support vector machines, MLPs, later random forests) map a feature vector built around $S(p,t)$ — salience neighborhood, spectral shape descriptors — to a posterior probability of pitch p being active, followed by *temporal smoothing* (HMMs, median filtering) that segments the posteriors into notes.

Two late-era exemplars show how far handcrafting could be pushed. Emiya, Badeau, and David [58] placed a probabilistic *spectral smoothness* prior over the salience: true note spectra vary smoothly across neighboring frequency bins in a characteristic way, so a candidate pitch whose supporting partials look "spiky" in the wrong way is downweighted. Duan, Pardo, and Zhang [59] modeled both spectral peaks *and* non-peak regions: the likelihood of a pitch hypothesis must explain not only where energy is observed, but where it is absent — a missing expected partial is evidence against the pitch. Both are principled, and both still operate on hand-designed features.

The octave problem is structural, not a tuning failure. Suppose only pitch p^* sounds, noiselessly, with partial amplitudes a_k at $f_{p^*} \cdot k$. Evaluate the salience one octave below:

$$S(p^* - 12) = \sum_{h=1}^{H} \alpha^h \, |X| \, (f_{p^*} h/2) = \sum_{k=1}^{\lfloor H/2 \rfloor} \alpha^{2k} a_k, \tag{8.11}$$

because the even partials $h = 2k$ of the sub-octave candidate land exactly on the true note's partials. Compare with $S(p^*) = \sum_k \alpha^k a_k$: both sums are of the same order, since α^{2k} merely reweights the identical partial energies. The sub-octave ghost is therefore *built into* any additive harmonic-summation salience; with $a_k = 1/k$ and $\alpha = 0.8$ the ghost reaches a large fraction of the true peak's height, which is exactly what Figure 8.3 exhibits. No per-frame classifier can fully resolve this ambiguity — it requires temporal context (onsets, note trajectories), which is why the field eventually moved to sequence models with explicit onset modeling.

What the paradigm solved: robust multi-pitch estimation from interpretable auditory features, with no need to learn note spectra from data. What it could not do: hand-designed features plateaued; octave and fifth errors persisted structurally; every new instrument required retuning; and onsets, offsets, and dynamics were afterthoughts grafted onto a per-frame pitch detector.

Step 4 — Complete code. `code/m8_salience.py` builds a synthetic chord spectrum, computes the salience (8.10), and peak-picks the pitches, reproducing Figure 8.3. The printed output is honest about the era's ceiling: all three true pitches are found, together with seven octave ghosts.

```
"""Module 8 -- handcrafted-era pitch salience from scratch.

Builds a synthetic chord spectrum, then computes the classic
harmonic-summation salience

    S(p) = sum_{h=1..H} alpha**h * |X|(f_p * h)

for every MIDI pitch p and reports the detected pitches.
Also renders figs/m8_salience.pdf.

Run:  python3 code/m8_salience.py   (from ~/workspace/amt_book)
"""
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

RNG_SEED = 20261008
SR = 22050


def midi_to_freq(m):
    return 440.0 * 2.0 ** ((m - 69.0) / 12.0)


def chord_spectrum(midi_notes, n_fft=8192, n_partials=10):
    """Magnitude spectrum of a chord: harmonic stacks with 1/h
       decay."""
    spec = np.zeros(n_fft // 2 + 1)
    freqs = np.fft.rfftfreq(n_fft, 1.0 / SR)
    for m in midi_notes:
        f0 = midi_to_freq(m)
        for h in range(1, n_partials + 1):
            idx = int(np.argmin(np.abs(freqs - h * f0)))
            spec[idx] += 1.0 / h
    rng = np.random.default_rng(RNG_SEED)
    spec += 0.005 * rng.random(spec.shape)  # noise floor
    return freqs, spec


def harmonic_salience(freqs, spec, alpha=0.8, n_partials=8,
                      lo=21, hi=108):
    """Classic handcrafted salience: weighted sum over harmonic
       partials."""
    pitches = np.arange(lo, hi + 1)
    S = np.zeros_like(pitches, dtype=float)
```

```
    for i, p in enumerate(pitches):
        f0 = midi_to_freq(p)
        total = 0.0
        for h in range(1, n_partials + 1):
            idx = int(np.argmin(np.abs(freqs - h * f0)))
            total += (alpha ** h) * spec[idx]
        S[i] = total
    return pitches, S


def peak_pitches(pitches, S, thresh_frac=0.35):
    """Naive local-maximum peak picker above a relative threshold."""
    thresh = thresh_frac * S.max()
    found = []
    for i in range(1, len(S) - 1):
        if S[i] > thresh and S[i] >= S[i - 1] and S[i] > S[i + 1]:
            found.append(int(pitches[i]))
    return found


def render_figure(freqs, spec, pitches, S, found, chord,
                  path="figs/m8_salience.pdf"):
    fig, axes = plt.subplots(2, 1, figsize=(10.5, 6.4), sharex=False)
    axes[0].plot(freqs, spec, color="#1f3a5f", linewidth=0.8)
    axes[0].set_xlim(0, 4000)
    axes[0].set_title("Synthetic chord magnitude spectrum (C4-E4-G4)")
    axes[0].set_xlabel("frequency (Hz)")
    axes[0].set_ylabel("|X(f)|")
    axes[0].grid(True, alpha=0.3)
    axes[1].stem(pitches, S, linefmt="#b34700", markerfmt="o",
                 basefmt=" ", label="salience S(p)")
    for p in found:
        axes[1].annotate(f"MIDI {p}", (p, S[p - pitches[0]]),
                         textcoords="offset points", xytext=(0, 8),
                         ha="center", fontsize=9, color="#b34700")
    axes[1].set_title("Harmonic-summation pitch salience S(p)")
    axes[1].set_xlabel("MIDI pitch p")
    axes[1].set_ylabel("salience")
    axes[1].grid(True, alpha=0.3)
    fig.tight_layout()
    fig.savefig(path, bbox_inches="tight")
    plt.close(fig)


def main():
    chord = [60, 64, 67]  # C4, E4, G4
    freqs, spec = chord_spectrum(chord)
    pitches, S = harmonic_salience(freqs, spec)
    found = peak_pitches(pitches, S)
    print(f"true chord (MIDI)   : {chord}")
    print(f"detected pitches    : {found}")
    print(f"salience at C4 (60) : {S[60 - 21]:.4f}")
    print(f"salience at C#4(61) : {S[61 - 21]:.4f}  (should be far
        lower)")
    render_figure(freqs, spec, pitches, S, found, chord)
    print("wrote figs/m8_salience.pdf")


```

```
102 if __name__ == "__main__":
103     main()
```

8.4 The Deep Learning Breakthrough: Onsets and Frames

The deep-learning era began when Sigtia, Benetos, and Dixon [61] showed that a single neural network could map spectrograms directly to piano rolls end to end, with no hand-designed pitch features at all. Onsets and Frames took that breakthrough and aimed it at the specific failure mode end-to-end frame classifiers still had.

Step 1 — Physical analogy. Transcribe a pianist by watching only the keys: a held key keeps looking "down" while its sound decays into ambiguity, and whenever the sound dips you will swear the note ended and a new one began. Now transcribe by watching the hammers instead: each strike is a sharp, loud, unmistakable event, and between strikes you only have to answer the much easier question "is anything still ringing?" The frame detector is the key-watcher; the onset detector is the hammer-watcher. Onsets and Frames [62] is the architecture that employs both and lets the hammer-watcher overrule the key-watcher about when notes begin.

Step 2 — Geometric mental model. The model has three output stacks over the 88 piano keys, drawn in Figure 8.4 (left): an onset stack, a frame stack whose input is concatenated with the onset stack's output, and a disconnected velocity stack. At inference, a frame-active run is promoted to a note only where the onset detector agrees — the gate drawn in Figure 8.4 (right), where a flickering frame run with no onset underneath is suppressed rather than emitted as a spurious note.

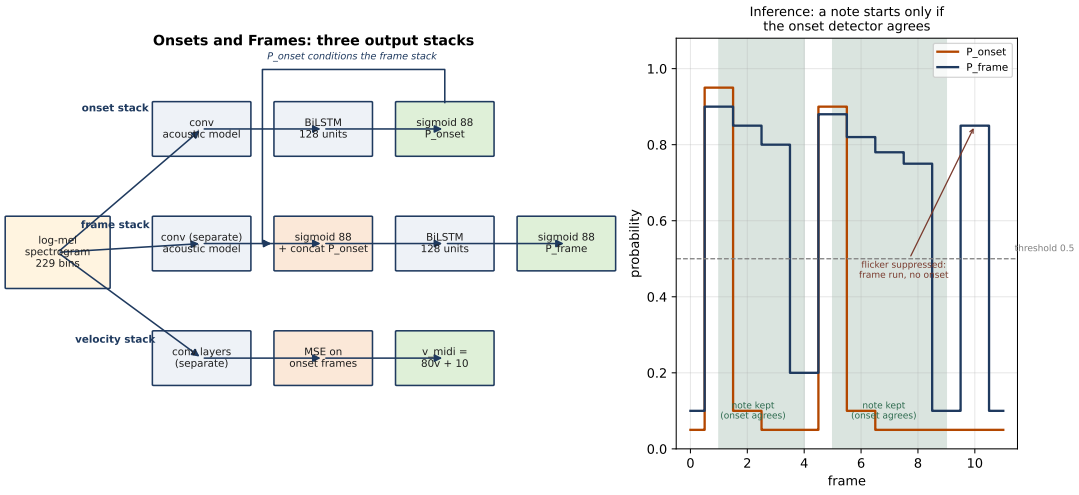


Figure 8.4: Left: the three output stacks of Onsets and Frames. The frame stack is architecturally conditioned on the onset stack's output; the velocity stack is separate. Right: the inference gate on toy activations — a frame-active run becomes a note only if the onset detector fires at its first frame, suppressing fragmentation. Generated by `scripts/m8_onsets_frames_fig.py`.

Step 3 — First-principles derivation. The input is a log-magnitude mel spectrogram with 229 logarithmically-spaced frequency bins, computed at 16 kHz with an FFT window of

2048 and hop length 512. The onset detector is a convolutional acoustic model followed by a bidirectional LSTM with 128 units in each direction, followed by a fully-connected sigmoid layer with 88 outputs, $P_{\text{onset}}(p,t)$, the probability of an onset of piano key p at frame t. The frame detector uses a *separate* convolutional acoustic model followed by a sigmoid layer with 88 outputs; that 88-dimensional vector is concatenated with the onset detector's output and passed through a second BiLSTM(128) and sigmoid layer, producing $P_{\text{frame}}(p,t)$. The conditioning is therefore architectural, not a post-processing trick: the frame stack sees what the onset stack believes.

Training minimizes the dual objective

$$L_{\text{total}} = L_{\text{onset}} + L_{\text{frame}}, \tag{8.12}$$

the sum of two binary cross-entropies,

$$L_{\text{onset}} = \sum_{p=p_{\min}}^{p_{\max}} \sum_{t=0}^{T} \text{CE}\big(I_{\text{onset}}(p,t),\, P_{\text{onset}}(p,t)\big), \tag{8.13}$$

$$L_{\text{frame}} = \sum_{p=p_{\min}}^{p_{\max}} \sum_{t=0}^{T} \text{CE}\big(I_{\text{frame}}(p,t),\, P_{\text{frame}}(p,t)\big), \tag{8.14}$$

with $\text{CE}(y,\hat{y}) = -[y \log \hat{y} + (1-y)\log(1-\hat{y})]$. Two label-engineering decisions carry most of the method's intelligence. First, the onset labels are created by truncating every note to min(note_length, 32 ms) before quantization: the detector is trained to fire on a short attack window, not on the whole note. The authors searched onset lengths of 16, 32, and 48 ms and found 32 ms best — approximately two spectrogram frames, wide enough to absorb audio–label misalignment. Second, the frame loss is reweighted to emphasize note beginnings. For a note active on frames $[t_1, t_3]$ whose onset completes at t_2, the per-frame weight is

$$w(t) = \begin{cases} c, & t_1 \le t \le t_2, \\ \dfrac{c}{t - t_2}, & t_2 < t \le t_3, \\ 1, & \text{otherwise}, \end{cases} \qquad c = 5.0, \tag{8.15}$$

so the attack frames contribute five times the loss of background frames and the sustain contribution decays hyperbolically. The rationale is stated plainly in the paper: some frames matter more than others, and the onset frame of a note is both the easiest to identify (amplitude at its peak) and the most perceptually significant.

A third stack predicts velocity, disconnected from the other two. It is trained with squared error counted only on onset frames,

$$L_{\text{vel}} = \sum_{p} \sum_{t} I_{\text{onset}}(p,t)\, \big(v^{p,t}_{\text{label}} - v^{p,t}_{\text{predicted}}\big)^2, \tag{8.16}$$

where the labels are the MIDI velocities divided by the maximum velocity in the piece. At inference the output is clipped to $[0,1]$ and mapped by

$$v_{\text{MIDI}} = 80\, v_{\text{predicted}} + 10, \tag{8.17}$$

a mapping the authors call arbitrary but report produces pleasing renderings. This was, to the authors' knowledge, the first AMT work to estimate dynamics alongside pitch and timing.

Inference applies a threshold of 0.5 to both stacks and the gate: a frame-active run for pitch p starting at frame t is emitted as a note *only if* $P_{\text{onset}}(p,t) > 0.5$. This single rule is what fixes frame fragmentation. A frame-only detector must decide, during a note's decay when the

signal-to-noise ratio is at its worst, whether the note is still sounding; its uncertainty flickers, and each flicker boundary becomes a spurious note onset. Frame-level metrics barely punish this — the frames are mostly right — but note-level metrics with offsets collapse, because one true note is reported as several fragments. The onset gate converts the hard segmentation problem into two easier ones: detecting attacks, which for piano are percussive, broadband, and high-SNR, and tracking continuation given a known start. The quantitative evidence is in the paper's abstract: over 100% relative improvement in note F_1 score *with offsets* on the MAPS Disklavier test set, training only on synthesized MAPS pieces — a train/test split deliberately designed so the test piano is never seen in training.

What the paradigm solved: the fragmentation ceiling of frame classifiers, plus dynamics estimation. What it could not do: the architecture is piano-specific (88 outputs, piano attack character); it still reasons on a fixed frame grid with independent per-frame decisions smoothed only by the BiLSTM; and it assigns no instrument labels. Removing the frame grid is the next paradigm's job.

Step 4 — Complete code. `code/m8_onsets_frames_loss.py` implements the exact losses (8.13)–(8.17) and the onset-gated decoding rule on a hand-checkable toy example. The script prints $L_{\text{onset}} = 0.8137$, which Drill 2 asks you to reproduce by hand.

```
"""Module 8 -- the Onsets-and-Frames dual objective, from scratch.

Implements the exact training losses of Hawthorne et al.
("Onsets and Frames: Dual-Objective Piano Transcription",
arXiv:1710.11153):

    L_total = L_onset + L_frame              (binary cross-entropies)
    L_vel   = sum I_onset * (v_label - v_pred)^2   (velocity MSE)
    v_midi  = 80 * v_pred + 10               (inference velocity mapping)

plus the onset-weighted frame loss (weight c = 5.0 on the attack,
decaying as c / (t - t2) through the sustain) and the inference rule:
a frame-active run becomes a note ONLY if the onset detector agrees
(onset probability > 0.5 at the run's first frame) -- the mechanism
that kills frame fragmentation.

Run:  python3 code/m8_onsets_frames_loss.py   (from ~/workspace/amt_book)
"""
import numpy as np

C_WEIGHT = 5.0       # frame-loss attack weight from the paper
ONSET_LEN_MS = 32.0  # best onset label length found by the authors


def bce(I, P, eps=1e-9):
    """Binary cross-entropy summed over all entries."""
    P = np.clip(P, eps, 1.0 - eps)
    return float(-np.sum(I * np.log(P) + (1.0 - I) * np.log(1.0 - P)))


def onset_loss(I_onset, P_onset):
    """L_onset: cross-entropy over the 32 ms-truncated onset labels."""
    return bce(I_onset, P_onset)


def frame_loss_weighted(I_frame, P_frame, note_spans):
```

```
    """L_frame with onset emphasis.

    note_spans: list of (t1, t2, t3); note active on [t1, t3],
    onset completes at t2. Weight is C_WEIGHT on [t1, t2],
    C_WEIGHT / (t - t2) on (t2, t3], and 1.0 elsewhere.
    """
    T = I_frame.shape[0]
    W = np.ones(T)
    for (t1, t2, t3) in note_spans:
        for t in range(t1, min(t2, T - 1) + 1):
            W[t] = C_WEIGHT
        for t in range(t2 + 1, min(t3, T - 1) + 1):
            W[t] = C_WEIGHT / (t - t2)
    P = np.clip(P_frame, 1e-9, 1.0 - 1e-9)
    per_frame = -(I_frame * np.log(P) + (1.0 - I_frame) * np.log(1.0 -
       P))
    return float(np.sum(W * per_frame))


def velocity_loss(I_onset, v_label, v_pred):
    """L_vel: squared error, counted only on onset frames."""
    return float(np.sum(I_onset * (v_label - v_pred) ** 2))


def velocity_to_midi(v_pred):
    """Inference mapping used by the authors: v_midi = 80 v + 10."""
    return 80.0 * np.clip(v_pred, 0.0, 1.0) + 10.0


def decode_notes(P_onset, P_frame, thresh=0.5):
    """Onset-gated note decoding for one pitch.

    Frame-active runs are found where P_frame > thresh; a run is
    emitted as a note only if P_onset > thresh at its first frame.
    """
    T = len(P_frame)
    notes, t = [], 0
    while t < T:
        if P_frame[t] > thresh:
            t_end = t
            while t_end + 1 < T and P_frame[t_end + 1] > thresh:
                t_end += 1
            if P_onset[t] > thresh:          # the onset gate
                notes.append((t, t_end))
            t = t_end + 1
        else:
            t += 1
    return notes


def main():
    # ---- one pitch, six frames: hand-checkable loss example ----
    I_on = np.array([1, 0, 0, 1, 0, 0], dtype=float)
    P_on = np.array([0.9, 0.1, 0.2, 0.8, 0.1, 0.05])
    L_on = onset_loss(I_on, P_on)
    print(f"L_onset (hand-check target 0.8137): {L_on:.4f}")

    I_fr = np.array([0, 1, 1, 1, 1, 0], dtype=float)
```

```
    P_fr = np.array([0.05, 0.9, 0.85, 0.7, 0.6, 0.1])
    L_fr = frame_loss_weighted(I_fr, P_fr, note_spans=[(1, 2, 4)])
    L_fr_plain = bce(I_fr, P_fr)
    print(f"L_frame weighted (c=5 attack emphasis): {L_fr:.4f}")
    print(f"L_frame unweighted, for contrast      : {L_fr_plain:.4f}")

    v_lab = np.array([0.8, 0, 0, 0.4, 0, 0], dtype=float)
    v_prd = np.array([0.7, 0, 0, 0.5, 0, 0], dtype=float)
    L_v = velocity_loss(I_on, v_lab, v_prd)
    print(f"L_vel (only onset frames count)       : {L_v:.4f}")
    print(f"v_pred 0.7 -> MIDI velocity           :
       {velocity_to_midi(0.7):.1f}")

    # ---- fragmentation demo: flickering frames, one true onset ----
    P_frame = np.array([0.1, 0.9, 0.85, 0.2, 0.8, 0.75, 0.1, 0.9, 0.1])
    P_onset = np.array([0.05, 0.95, 0.05, 0.05, 0.05, 0.05, 0.05,
       0.05, 0.05])
    notes = decode_notes(P_onset, P_frame)
    print(f"frame-active runs at 1-2, 4-5, 7; onset only at 1")
    print(f"decoded notes (start, end)            : {notes}")
    print("note: the flicker runs at 4-5 and 7 are suppressed --")
    print("no onset agreement, so no spurious note is started.")


if __name__ == "__main__":
    main()
```

8.5 MT3: Multi-Task Multitrack Transcription

Step 1 — Physical analogy. A conference needs interpreters for dozens of language pairs. The old approach trains one specialist per pair — one transcription model per instrument and dataset. The MT3 approach trains a single polyglot interpreter who begins each passage by announcing which language is being spoken, then translates. The announcement is the `Instrument` token; the translation is the note-event stream. This is exactly the mT5 idea from NLP (one Transformer for many languages, with high-resource languages lifting low-resource ones), ported to music: Gardner et al. [66] show that piano data can lift guitar transcription, the same way English data lifts translation into a low-resource language.

Step 2 — Geometric mental model. Think of the token vocabulary as a small machine language with seven instruction types, sketched in Figure 8.5 (top and middle): declare the instrument, stamp absolute time in 10 ms steps, flip the on/off switch, name the pitch. Because the Transformer can only digest 2.048 s at a time, the bottom panel shows the tie mechanism that stitches segments together: each segment opens by declaring which notes are already sounding, so a note that started in the previous segment can still be closed.

Step 3 — First-principles technical content. MT3 stands on an intermediate result worth naming: Hawthorne et al.'s sequence-to-sequence piano transcription with Transformers [63], which replaced the frame-classification output of Onsets and Frames with an encoder-decoder Transformer emitting MIDI-like event tokens with absolute-time stamps. Frame grids had hit three ceilings at once: decisions are made on a fixed temporal grid rather than at event boundaries, frames are classified with only local smoothing rather than explicit event structure, and every output head is instrument-specific. The event-sequence view removes all three, and

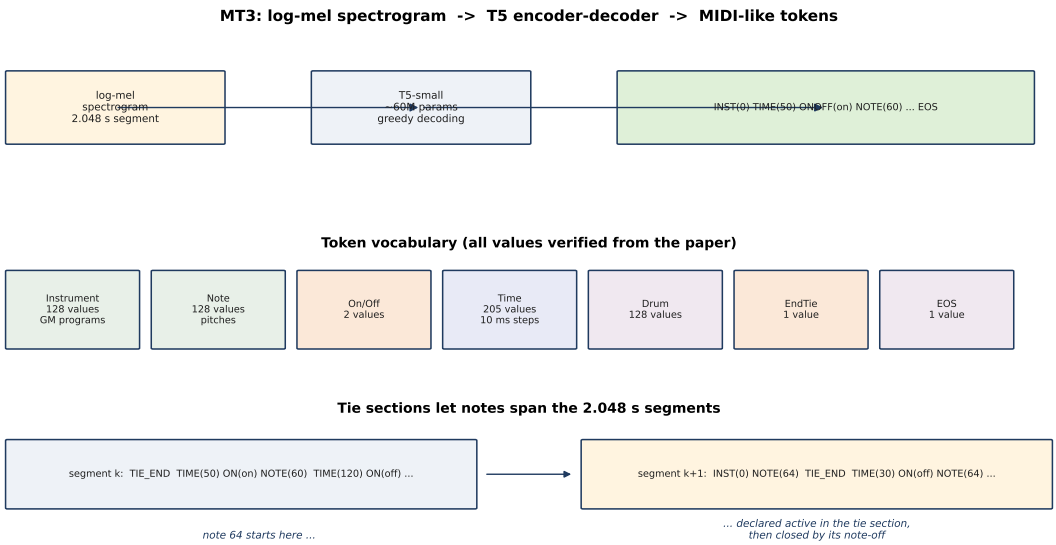


Figure 8.5: Top: MT3 maps log-mel spectrogram segments through a T5 encoder-decoder to MIDI-like tokens. Middle: the seven token types (all counts verified from the paper). Bottom: tie sections let notes span the 2.048 s segment boundaries. Generated by `scripts/m8_mt3_fig.py`.

the frame-based endgame — Kong et al.'s high-resolution piano transcription by regressing onset and offset times [64] — confirmed how far the old view could go without crossing over.

MT3 (Gardner, Simon, Manilow, Hawthorne, and Engel, Google Research Brain Team [66]) generalizes the event view to arbitrary instrument combinations with a deliberately off-the-shelf design: log-mel spectrogram inputs, a standard T5 encoder-decoder (T5-"small", about 60M parameters, absolute positional embeddings in place of T5's bucketed relative ones), and greedy autoregressive decoding until EOS. The modeling contribution is the output vocabulary, verified here against the paper's Section 3.2:

Token type	Values	Meaning
`Instrument`	128	GM program; directs all following events
`Note`	128	MIDI pitch; on/off set by `On/Off`
`On/Off`	2	flips interpretation of following `Note`s
`Time`	205	absolute time in segment, 10 ms steps
`Drum`	128	GM drum onsets (included for completeness)
`End Tie Section`	1	closes the tie section
`EOS`	1	end of sequence

Relative to the Hawthorne et al. 2021 vocabulary, MT3 adds the instrument-change tokens, *removes* velocity (most training datasets lack velocity annotations and there is no standardized cross-dataset coding), and adds ties. Audio is split into non-overlapping 2.048 s segments processed independently; at inference the decoded events are concatenated. The tie section handles the boundary artifact this creates: each segment's token stream opens with `Instrument`+`Note` tokens declaring already-active notes, then `End Tie Section`, then the segment's events. When segments are concatenated, any note not re-declared in a tie section is ended — the model fails gracefully instead of leaving notes hanging forever.

The second contribution is the multi-task mixture itself: six multitrack datasets in one

training run, with temperature sampling

$$p(\text{dataset } i) \propto \left(\frac{n_i}{\sum_j n_j} \right)^{0.3} \tag{8.18}$$

so low-resource datasets are seen disproportionately often. The paper's own Table 1 (reproduced here with its alignment-quality judgments) is the most honest summary of the AMT data landscape available:

Dataset	Hours	Songs	Instr.	Alignment
Slakh2100	968.5	1405	35	good
Cerberus4	542.6	1327	4	good
MAESTROv3	198.7	1276	1	good
MusicNet	34	330	11	poor
GuitarSet	3	360	1	good
URMP	1.3	44	14	fair

Two readings matter. First, the hours are MT3's *processed* training quantities (Slakh2100 is augmented by remixing instrument subsets), not the raw release sizes used in this book's own comparison table later. Second, the scale gap the paper highlights: LibriSpeech alone (1k hours) contains more audio than all six AMT datasets combined — AMT is a low-resource field by ASR standards, which is the entire motivation for the mixture.

Evaluation uses three mir_eval note metrics — Frame F_1 on a 62.5 fps pianoroll, Onset F_1 (± 50 ms), and Onset-Offset F_1 (offsets within $\max(0.2 \times \text{duration}, 50\,\text{ms})$, bipartite matching of reference and estimated notes) — plus a new multi-instrument F_1 that additionally requires the predicted instrument to match. The reported results: a single MT3 model beats the dataset-specific baselines (and the professional DSP software Melodyne) on each dataset, with up to 260% relative improvement on the low-resource datasets (GuitarSet, MusicNet, URMP) while preserving strong piano performance, and very high instrument-labeling accuracy even with many simultaneous instruments.

What the paradigm solved: one general architecture for arbitrary instrument combinations, and transfer from high-resource to low-resource instruments. What it could not do: segments are independent (no modeling across the 2.048 s boundaries beyond ties), decoding is greedy, velocity is dropped, and the annotation bottleneck is merely pooled, not removed — every training frame still needs a label. Removing the label requirement is the next paradigm's job.

Step 4 — Complete code. `code/m8_mt3_tokenizer.py` implements the verified MT3 vocabulary and the tie-section mechanism from scratch, with an encode/decode round-trip assertion. The printed stream shows the 20-token encoding of a two-note figure whose first note is carried in through the tie section.

```
"""Module 8 -- MT3-style MIDI-like tokenizer, from scratch.

Implements the token vocabulary of Gardner et al.
("MT3: Multi-Task Multitrack Music Transcription", arXiv:2111.03017):

    Instrument (128) : GM program for subsequent events
    Note       (128) : pitch; on/off meaning set by the On/Off token
    On/Off     (2)   : switch interpretation of following Note tokens
    Time       (205) : absolute time in segment, 10 ms steps (2.048 s
        max)
    Drum       (128) : drum onsets (unused in this demo)
    EndTie     (1)   : closes the tie section at a segment start
```

```
    EOS         (1)    : end of sequence

plus the tie-section mechanism: each segment opens by declaring the
notes that are already sounding, so a note-on whose note-off lands in
a later segment can still be closed gracefully.

Run:  python3 code/m8_mt3_tokenizer.py   (from ~/workspace/amt_book)
"""
from dataclasses import dataclass

SEG_LEN = 2.048   # seconds; the Transformer input window in MT3
TIME_STEP = 0.01  # 10 ms quantization -> 205 Time values (0..204)
N_TIME = int(round(SEG_LEN / TIME_STEP)) + 1  # 205


@dataclass
class Event:
    t: float         # seconds, absolute
    kind: str        # "on" or "off"
    pitch: int       # MIDI 0..127
    program: int     # GM program 0..127


def quantize_time(t):
    tt = int(round(t / TIME_STEP))
    assert 0 <= tt < N_TIME, f"time {t}s outside the 2.048 s segment"
    return tt


def encode(events, active_at_start=()):
    """events -> token list. active_at_start: {(program, pitch)}
       sounding."""
    toks = []
    # --- tie section: declare notes already active when the segment
       opens
    cur_prog = None
    for prog, pitch in sorted(active_at_start):
        if prog != cur_prog:
            toks.append(("instrument", prog)); cur_prog = prog
        toks.append(("note", pitch))
    toks.append(("tie_end",))
    # --- segment events, chronological; offs before ons at equal times
    cur_prog, cur_state = None, None
    for ev in sorted(events, key=lambda e: (e.t, 0 if e.kind == "off"
       else 1)):
        toks.append(("time", quantize_time(ev.t)))
        if ev.program != cur_prog:
            toks.append(("instrument", ev.program)); cur_prog =
               ev.program
        if ev.kind != cur_state:
            toks.append(("on_off", ev.kind)); cur_state = ev.kind
        toks.append(("note", ev.pitch))
    toks.append(("eos",))
    return toks


def decode(toks):
    """token list -> (active_at_start, events). Inverse of encode."""
```

```
    active, events = [], []
    prog, state, t = 0, None, 0.0
    i, in_tie = 0, True
    while i < len(toks):
        typ = toks[i][0]
        if typ == "tie_end":
            in_tie = False
        elif typ == "eos":
            break
        elif typ == "instrument":
            prog = toks[i][1]
        elif typ == "time":
            # round back onto the 10 ms grid: 190 * 0.01 is not exactly 1.9
            t = round(toks[i][1] * TIME_STEP, 2)
        elif typ == "on_off":
            state = toks[i][1]
        elif typ == "note":
            if in_tie:
                active.append((prog, toks[i][1]))
            else:
                events.append(Event(t, state, toks[i][1], prog))
        i += 1
    return active, events


def fmt(toks):
    short = {"instrument": "INST", "note": "NOTE", "on_off": "ONOFF",
             "time": "TIME", "tie_end": "TIE_END", "eos": "EOS"}
    return " ".join(short[t] + (f"({v[0]})" if v else "")
                    for t, *v in toks)


def main():
    # A two-note piano figure; middle C is still sounding at t=0 (tie).
    active = {(0, 60)}
    events = [
        Event(0.50, "off", 60, 0),
        Event(0.50, "on", 64, 0),
        Event(1.20, "off", 64, 0),
        Event(1.20, "on", 67, 0),
        Event(1.90, "off", 67, 0),
    ]
    toks = encode(events, active_at_start=active)
    print("token stream:")
    print(" ", fmt(toks))
    print(f"token count: {len(toks)}")
    back_active, back_events = decode(toks)
    assert sorted(back_active) == sorted(active), "tie round-trip failed"
    assert [(e.t, e.kind, e.pitch, e.program) for e in back_events] == \
           [(e.t, e.kind, e.pitch, e.program) for e in
            sorted(events, key=lambda e: (e.t, 0 if e.kind == "off" else 1))], \
        "event round-trip failed"
    print("round-trip OK: decode(encode(events)) == events")
    print(f"time tokens used: 205 values cover 0..2040 ms in 10 ms
```

```
        steps")


if __name__ == "__main__":
    main()
```

8.6 The SSL Era: Foundation Models Meet Transcription

Step 1 — Physical analogy. Learn a language by immersion: thousands of hours of listening with no teacher, until the sounds organize themselves into words. Then open the phrasebook: a few dozen labeled examples mapping sounds to written symbols. Self-supervised pretraining is the immersion; transcription fine-tuning is the phrasebook. The bet of the current era is that the immersion stage — which needs no annotation — can do most of the work, leaving the scarce labeled data (GuitarSet's 3 hours, URMP's 1.3) to teach only the final mapping from representation to note events.

Step 2 — Geometric mental model. Pretraining moves the representation; fine-tuning only draws the decision boundaries. Figure 8.6 shows the two-stage pipeline: a masked-reconstruction objective organizes unlabeled audio into a pitch-aware embedding space, and a small labeled set then trains onset/frame/event heads on top — the Chapter 3 to Chapter 4 handoff made concrete.

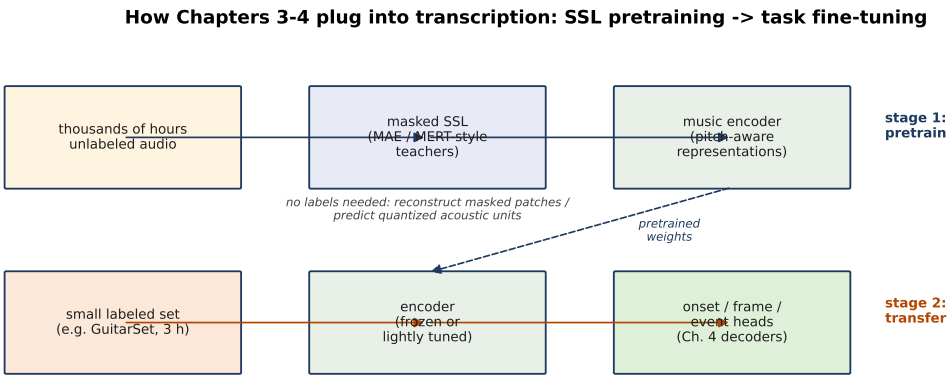


Figure 8.6: How Chapters 3–4 plug into transcription. Stage 1: masked SSL on thousands of unlabeled hours builds a music encoder. Stage 2: the pretrained weights transfer to a small labeled set, where only the task heads (onset/frame/event decoders) need full supervision. Generated by `scripts/m8_ssl_fig.py`.

Step 3 — First-principles technical content. The music-specific foundation model that anchors this era is MERT (Li et al. [67]): masked language-modeling-style acoustic pretraining where a BERT-style Transformer encoder predicts pseudo-labels from *two* teachers — an acoustic teacher based on a residual vector-quantized variational autoencoder (RVQ-VAE) and a *musical* teacher based on the constant-Q transform. The CQT teacher is the decisive design choice: it forces the learned representation to respect the pitch structure developed in Chapter 1, rather than treating music as generic audio. MERT adds in-batch noise-mixture augmentation for robustness, scales from 95M to 330M parameters, and reports state-of-the-art overall scores across 14 music-understanding tasks. Complementing it, Ragano, Benetos, and

Hines [68] showed that pretraining wav2vec 2.0 directly on *music* (rather than fine-tuning the speech model) yields discrete latents that encode pitch and instrument semantics — speech pretraining, optimized for phonemes, is the wrong immersion for music.

How the book's machinery plugs in: take an encoder from Section 3.3 (wav2vec 2.0/HuBERT-style contextualized targets) or Section 3.2 (MAE-Audio masked spectrogram reconstruction), freeze it or tune it lightly, and attach the task heads of Section 4.3 (onset/frame classifiers with focal loss for the sparse-onset imbalance) or use it as the encoder under a Section 4.2 cross-attention event decoder. The fine-tuning protocol is identical to Chapter 3.4's transfer recipe; only the data regime changes — thousands of unlabeled hours upstream, a GuitarSet-sized labeled set downstream.

An honest status report is required here. As of 2026, SSL-first AMT is an *active frontier*, not a closed chapter: the field has strong music SSL encoders and a working transfer recipe, but no single SSL-pretrained transcription system dominates the benchmarks the way MT3 did for multitask training. Open sub-questions include whether SSL frames are temporally precise enough for ±50 ms onset evaluation without task-specific refinement, and how instrument identity transfers across the pretraining distribution. What the paradigm promises to solve is the annotation bottleneck itself — the one bottleneck every previous paradigm merely worked around.

Step 4 — Complete code. `code/m8_ssl_probe.py` is an *illustrative simulation*, labeled as such: it implements the exact freeze-and-probe transfer protocol on synthetic embeddings and shows the data-efficiency gap between structured ("pretrained") and random encoders. The printed table is the methodological point — with the right representation, 16 labeled frames beat 1024 labeled frames on the wrong one.

```
"""Module 8 -- SSL transfer-learning protocol, illustrated by
    simulation.

This listing is an *illustrative simulation*, not a trained foundation
model: it shows the exact fine-tuning protocol of Chapter 3.4 --
freeze (or lightly tune) a pretrained audio encoder, attach a small
task head, and train the head on a handful of labeled frames -- and
compares a "pretrained" encoder (embeddings that already separate
onset frames from sustain frames) against a random encoder.

The point is methodological: good SSL pretraining moves the
representation so that a linear onset probe needs far less labeled
data. Run: python3 code/m8_ssl_probe.py (from ~/workspace/amt_book)
"""
import numpy as np

RNG_SEED = 20261008


def make_frames(n=4000, dim=64, seed=RNG_SEED):
    """Synthetic frame pool: half 'onset', half 'sustain/silence'.

    A pretrained music encoder is simulated by embeddings whose first
    coordinates already carry onset information; a random encoder has
    no such structure.
    """
    rng = np.random.default_rng(seed)
    y = np.repeat([1, 0], n // 2)
    rng.shuffle(y)
    # Pretrained: onset direction + noise; Random: pure noise.
```

```
    onset_dir = np.zeros(dim); onset_dir[0] = 3.0; onset_dir[1] = 1.5
    X_pre = rng.normal(0, 1, (n, dim)) + np.outer(y, onset_dir)
    X_rnd = rng.normal(0, 1, (n, dim))
    return (X_pre, y), (X_rnd, y)


def train_linear_probe(X, y, n_train, l2=1.0, seed=1):
    """Ridge-regression probe on n_train labeled frames; returns
       accuracy."""
    rng = np.random.default_rng(seed)
    idx = rng.choice(len(y), size=n_train, replace=False)
    Xtr, ytr = X[idx], y[idx].astype(float)
    A = Xtr.T @ Xtr + l2 * np.eye(X.shape[1])
    w = np.linalg.solve(A, Xtr.T @ ytr)
    pred = (X @ w > 0.5).astype(int)
    return float((pred == y).mean())


def main():
    (X_pre, y), (X_rnd, _) = make_frames()
    print(f"{'labeled frames':>14} | {'pretrained probe':>16} | "
          f"{'random probe':>12}")
    for n_train in [16, 64, 256, 1024]:
        a_pre = train_linear_probe(X_pre, y, n_train)
        a_rnd = train_linear_probe(X_rnd, y, n_train)
        print(f"{n_train:>14} | {a_pre:>16.3f} | {a_rnd:>12.3f}")
    print("\nReading: with structured (pretrained) embeddings the
       linear")
    print("onset probe is already well above chance at 16 labeled
       frames,")
    print("while the random encoder stays at chance at every data
       size:")
    print("labels alone cannot conjure the onset direction out of
       noise.")
    print("That gap is the data-efficiency argument for SSL
       pretraining.")


if __name__ == "__main__":
    main()
```

8.7 Landmark Datasets: A Comparative Analysis

Models are the engine of the literature; datasets are the fuel, and in AMT the fuel has always been scarce. This section compares the six landmark corpora on recording conditions, polyphonic texture, alignment quality and method, and instrumentation. A warning that applies to the whole section: per-frame polyphonic density is not published as a single statistic for most of these datasets, so the table reports texture qualitatively rather than inventing numbers. Every number below was verified against the dataset's paper or its own release page; anything not publicly documented is marked "n/a".

The table and Figure 8.7 are generated from a single verified data file, `code/m8_datasets.py`, whose `print_latex_table()` output was cross-checked against Table 8.1 before inclusion:

```
"""Module 8 -- landmark dataset comparison, from verified values only.

Every number below was verified against the dataset paper or the
```

Table 8.1: Landmark AMT datasets: verified comparison. Hours are total audio; alignment is the method and quality by which audio and labels are synchronized.

Dataset	Hours	Pieces	Instr.	Texture	Alignment	Recording conditions
MAPS [58]	18.2	270	1	solo piano	exact (MIDI-driven)	7 synth renderers + Disklavier, close and ambient mics
MAESTRO v3.0.0 [70]	198.6	1,276	1	solo piano	≈3 ms (Disklavier capture)	competition concert grands, 44.1–48 kHz stereo
MusicNet [69]	34	330	11	chamber ensembles	DTW of recordings to scores (noisy)	freely-licensed recordings, varied studios
Slakh2100 [71]	145	2,100	34	mixes, ≥4 sources each	exact (rendered from MIDI)	pro sample libraries, mono 44.1 kHz FLAC
GuitarSet [72]	3	360	1	solo guitar	per-string pYIN + manual check	studio, hexaphonic pickup + condenser mic
URMP [74]	1.3	44	13	duets to quintets	conducting-video sync (fair)	studio, instruments recorded separately

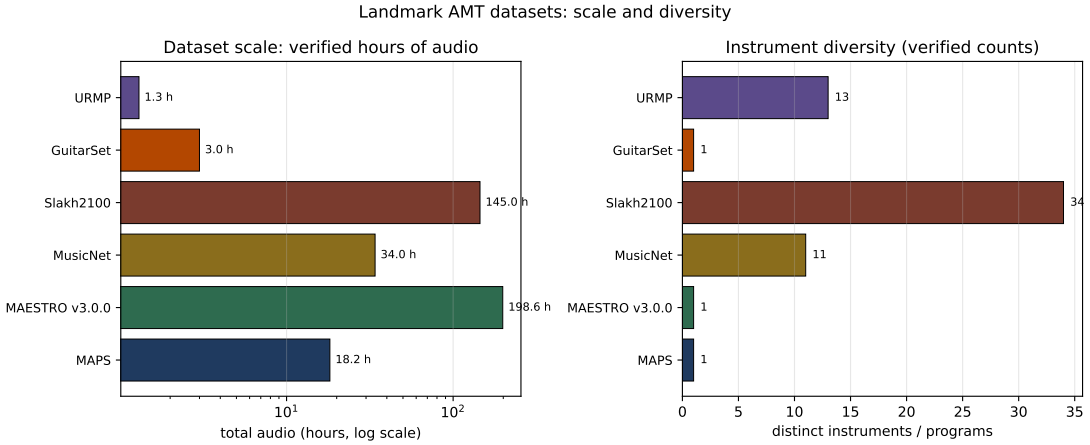


Figure 8.7: Left: verified total audio hours per dataset (log scale) — two orders of magnitude separate MAESTRO from URMP. Right: verified instrument diversity. Together they show the field's central tension: the largest datasets are single-instrument, and the diverse ones are tiny. Generated by `code/m8_datasets.py` from the verified table values.

```
dataset's own release page (see frags/refs_m8.tex); anything not
publicly documented is stored as None and rendered as "n/a" --
never invented.

The script prints the booktabs table used in the chapter (cross-check
against frags/module8.tex) and renders figs/m8_dataset_hours.pdf.

Run:  python3 code/m8_datasets.py   (from ~/workspace/amt_book)
"""
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

# hours, songs/pieces, instruments, texture, alignment, source
DATA = {
    "MAPS": dict(hours=18.2, songs=270, instruments=1,
                 texture="solo piano",
                 align="exact (MIDI-driven render / Disklavier)",
                 source="synth + real piano"),
    "MAESTRO v3.0.0": dict(hours=198.6, songs=1276, instruments=1,
                 texture="solo piano",
                 align="approx. 3 ms (Disklavier capture)",
                 source="real piano (competition)"),
    "MusicNet": dict(hours=34.0, songs=330, instruments=11,
                 texture="chamber ensembles",
                 align="DTW of recordings to scores (noisy)",
                 source="real, studio conditions vary"),
    "Slakh2100": dict(hours=145.0, songs=2100, instruments=34,
                 texture="mixes, >=4 sources each",
                 align="exact (rendered from MIDI)",
                 source="synthetic (pro sample libraries)"),
    "GuitarSet": dict(hours=3.0, songs=360, instruments=1,
                 texture="solo guitar",
                 align="per-string pYIN + manual check",
                 source="real (studio, hexaphonic+mic)"),
    "URMP": dict(hours=1.3, songs=44, instruments=13,
                 texture="duets to quintets",
                 align="conducting-video sync (fair)",
                 source="real (studio, separately recorded)"),
}


def print_latex_table():
    print(r"\begin{tabular}{lrrrr}")
    print(r"\toprule")
    print(r"Dataset & Hours & Pieces & Instr. & Alignment \\")
    print(r"\midrule")
    for name, d in DATA.items():
        print(f"{name} & {d['hours']} & {d['songs']} &
           {d['instruments']} "
              f"& {d['align']} \\\\")
    print(r"\bottomrule")
    print(r"\end{tabular}")


def render_figure(path="figs/m8_dataset_hours.pdf"):
    names = list(DATA.keys())
```

```
    hours = np.array([DATA[n]["hours"] for n in names])
    instr = np.array([DATA[n]["instruments"] for n in names])
    colors = ["#1f3a5f", "#2e6b4f", "#8a6d1c", "#7a3b2e", "#b34700",
              "#5b4a8a"]
    fig, axes = plt.subplots(1, 2, figsize=(11, 4.6))
    y = np.arange(len(names))
    axes[0].barh(y, hours, color=colors, edgecolor="black",
       linewidth=0.6)
    axes[0].set_yticks(y); axes[0].set_yticklabels(names, fontsize=9)
    axes[0].set_xscale("log")
    axes[0].set_xlabel("total audio (hours, log scale)")
    axes[0].set_title("Dataset scale: verified hours of audio")
    for i, h in enumerate(hours):
        axes[0].text(h * 1.08, i, f"{h} h", va="center", fontsize=8)
    axes[0].grid(True, axis="x", alpha=0.3)
    axes[1].barh(y, instr, color=colors, edgecolor="black",
       linewidth=0.6)
    axes[1].set_yticks(y); axes[1].set_yticklabels(names, fontsize=9)
    axes[1].set_xlabel("distinct instruments / programs")
    axes[1].set_title("Instrument diversity (verified counts)")
    for i, v in enumerate(instr):
        axes[1].text(v + 0.6, i, str(v), va="center", fontsize=8)
    axes[1].grid(True, axis="x", alpha=0.3)
    fig.suptitle("Landmark AMT datasets: scale and diversity",
       fontsize=12)
    fig.tight_layout()
    fig.savefig(path, bbox_inches="tight")
    plt.close(fig)


def main():
    print_latex_table()
    total = sum(d["hours"] for d in DATA.values())
    print(f"\ncombined verified hours: {total:.1f}")
    render_figure()
    print("wrote figs/m8_dataset_hours.pdf")


if __name__ == "__main__":
    main()
```

8.7.1 MAPS: the benchmark that enforces honesty

Step 1 — Operational analogy. MAPS is the crash-test dummy of piano transcription: nobody pretends it is realistic, but every model gets measured against it, because its train/test protocol makes cheating structurally difficult.

Step 2 — Geometric mental model. Picture nine parallel corpora of the same 30 classical pieces: seven rendered by different software synthesizers, two recorded on a real Disklavier (one close-miked, one ambient). A model trained on the seven synthetic versions and tested on the two real ones must generalize across the synthesis–reality gap — the standard "configuration 2" protocol that Onsets and Frames used.

Step 3 — Technical content. The MAPS database (Emiya, Badeau, and David [58]) contains 270 full classical pieces: 210 rendered with 7 sample-bank synthesizers and 60 recorded

on a Yamaha Disklavier (30 close microphone, 30 ambient), plus isolated notes and chords in each of the 9 recording categories. Alignment is exact by construction: the audio is either rendered from the MIDI or produced by Disklavier playback of the MIDI. The commonly reported split uses 139/71/60 pieces for train/validation/test, totaling approximately 8.3/4.4/5.5 hours. Instrumentation is piano only; texture is solo classical piano. Sustain-pedal control changes are translated into extended note durations in the standard preprocessing. MAPS suits one research question above all: cross-condition generalization for piano AMT, since the test piano is never seen during training.

Step 4 — Code. Entry `"MAPS"` in `code/m8_datasets.py`. Running the generator's `print_latex_table()` reproduces the 18.2-hour figure used in Table 8.1; dividing by 270 pieces gives roughly 4 minutes per piece, the scale at which the train-on-synth/test-on-Disklavier protocol operates.

8.7.2 MAESTRO v3.0.0: the conservatory archive

Step 1 — Operational analogy. If MAPS is a crash-test dummy, MAESTRO is the conservatory archive: real virtuoso performances, captured with laboratory precision, at a scale that finally lets deep networks train without starving.

Step 2 — Geometric mental model. Figure 8.7 (left) shows the scale jump: MAESTRO contributes roughly half of all verified audio hours across the six datasets. One instrument, enormous depth.

Step 3 — Technical content. MAESTRO v3.0.0 [70] comprises 1,276 performances from ten years of the International Piano-e-Competition, totaling approximately 198.6 hours (splits of 962/137/177 recordings at 159.2/19.4/20.0 hours). Performers play Yamaha Disklavier concert grands whose integrated MIDI capture records every key strike; audio (44.1–48 kHz 16-bit PCM stereo) and MIDI are aligned to about 3 ms and sliced into pieces annotated with composer, title, and year. The MIDI includes key-strike velocities and sustain pedal positions. The proposed split guarantees that no composition appears in more than one subset even when performed by multiple contestants. Texture is virtuoso solo classical piano — repertoire from the 17th to early 20th century — with high polyphonic density, though no single density statistic is published. MAESTRO is the default training set for modern piano AMT and the reason the MT3 mixture could treat piano as the high-resource anchor.

Step 4 — Code. Entry `"MAESTRO v3.0.0"` in `code/m8_datasets.py`. The generator also prints the combined verified total, 400.1 hours: MAESTRO alone contributes roughly half of all audio across the six corpora, the quantitative reason it anchors multitask mixtures as the high-resource dataset.

8.7.3 MusicNet: the crowdsourced library

Step 1 — Operational analogy. MusicNet is the crowdsourced library: vast, diverse, freely licensed — but the card catalog has typos, because the alignment between recordings and scores was done by algorithm, not by the performers.

Step 2 — Geometric mental model. In Figure 8.7 (right), MusicNet is the first dataset where the instrument bar meaningfully lifts off one: 11 instruments against a mid-scale 34 hours. Diversity per hour is high; label trust per hour is the price.

Step 3 — Technical content. MusicNet [69] contains hundreds of freely-licensed classical recordings (commonly reported as 330) by 10 composers, written for 11 instruments, with over a million temporal labels (1,299,329) on 34 hours of chamber music recorded under varying studio and microphone conditions. Alignment is by dynamic time warping of the recordings against separately sourced digital scores, human-verified but known to be noisy — the MT3 authors rate its alignment "poor," the only such judgment in their table. Texture is chamber ensembles of varying size. MusicNet suits research on multi-instrument transcription under real acoustic diversity and on robustness to alignment noise; it punishes models that overfit to clean labels.

Step 4 — Code. Entry `"MusicNet"` in `code/m8_datasets.py`. Note the design decision in the generator: alignment is stored as a string ("DTW of recordings to scores (noisy)"), not a number. Alignment quality here is ordinal — the MT3 authors' "poor" — and encoding it as a float would pretend to a precision the literature does not support.

8.7.4 Slakh2100: the flight simulator

Step 1 — Operational analogy. Slakh2100 is the flight simulator of AMT data: perfect instruments, perfect telemetry, unlimited hours — but the weather is simulated, and models trained only in the simulator must still fly in real storms.

Step 2 — Geometric mental model. Figure 8.7 shows Slakh2100 as the second-largest corpus by hours and the most diverse by instrument count (34 classes) — the only dataset where both bars are tall.

Step 3 — Technical content. Slakh2100 [71] renders MIDI files from the Lakh MIDI Dataset v0.1 with professional-grade sample-based virtual instruments: 2,100 automatically mixed tracks totaling 145 hours, drawn from 187 instrument patches in 34 classes, released as mono 44.1 kHz 16-bit FLAC. Every mixture contains at least 4 sources and always includes piano, guitar, drums, and bass. Alignment is exact because the audio is synthesized from the labels. Texture is dense multitrack mixtures by design. Slakh2100 suits multitrack and multi-instrument transcription at a scale real recordings cannot yet provide, and joint separation–transcription research; the research question it enables is the synthetic-to-real generalization gap, which must be measured on real corpora afterward.

Step 4 — Code. Entry `"Slakh2100"` in `code/m8_datasets.py`: 145 hours across 34 classes versus MAESTRO's 198.6 hours for a single class. The generator's two bar panels make the scale–diversity trade-off visible at a glance — no one corpus maximizes both, which is why the mixture exists.

8.7.5 GuitarSet: the luthier's workbench

Step 1 — Operational analogy. GuitarSet is the luthier's workbench: a single instrument examined string by string, with a sensor on each string. For the guitarist reader, this is the dataset that speaks their instrument — the only corpus where a transcription can say not just *which pitch* but *which string and fret.*

Step 2 — Geometric mental model. In Figure 8.7, GuitarSet is the small, precise bar: 3 hours, one instrument, but per-string annotation density no other dataset attempts. Resolution, not scale.

Step 3 — Technical content. GuitarSet [72] contains 360 recordings (approximately 3 hours; 62,476 annotated notes [73]) of varied genres, tempi, and styles played by 6 guitarists on an acoustic guitar fitted with a clip-on hexaphonic pickup capturing each string individually, plus a Neumann U87 condenser microphone placed 30 cm from the 18th fret in a small soundproof studio. Annotations are produced semi-automatically — per-string pYIN pitch tracking with manual inspection — and released in JAMS format with pitch contours, string and fret positions, chords, beats, downbeats, and playing style. Texture is solo guitar; alignment quality is good. GuitarSet suits guitar transcription including string/fret (tablature) estimation, and it is MT3's canonical low-resource showcase: the dataset where multitask transfer most dramatically improved results.

Step 4 — Code. Entry `"GuitarSet"` in `code/m8_datasets.py`. Three hours is 1.5% of MAESTRO's 198.6 — the ratio that makes MT3's low-resource transfer result meaningful rather than surprising, and the reason the SSL era targets exactly this kind of corpus.

8.7.6 URMP: the filmed chamber rehearsal

Step 1 — Operational analogy. URMP is the chamber rehearsal filmed from every angle: tiny in total duration, but each piece is documented with separate instrument tracks, video, and corrected annotations — meticulousness compensating for scale.

Step 2 — Geometric mental model. Figure 8.7 shows URMP as the smallest bar by far (1.3 hours) with the second-tallest instrument diversity bar: maximum instruments per hour, minimum hours.

Step 3 — Technical content. URMP [74] contains 44 classical chamber pieces, from duets to quintets, for 13 instruments (14 in the MT3 paper's count), assembled from 115 separately recorded single-source tracks. The central collection challenge was synchronization with expressiveness intact; the adopted solution had each player record while watching and listening to a pre-recorded conducting video. Audio is 48 kHz 24-bit WAV (individual and mixed), video is 1080p, and annotations are frame-level pitch trajectories plus note-level transcriptions in which pYIN-derived f_0 curves were corrected by human annotators. Total mixed audio is about 1.3 hours. Alignment is rated "fair" in the MT3 table. URMP suits multi-instrument transcription research where per-instrument stems and visual grounding matter, and it is the purest test of data efficiency: any model that learns from 1.3 hours is doing something right.

Step 4 — Code. Entry `"URMP"` in `code/m8_datasets.py`. At 1.3 hours the bar is invisible next to MAESTRO on a linear scale, which is why the generator's hours panel is logarithmic — a small visual argument for the scale on which dataset comparisons in this field must be drawn.

8.7.7 Which dataset for which question

The decision guide the table implies: train piano models on MAESTRO and test generalization on MAPS Disklavier recordings; study alignment-noise robustness on MusicNet; pretrain multitrack models on Slakh2100 and measure the synthetic-to-real gap on real corpora; do anything guitar on GuitarSet, exploiting its string-level labels; and use URMP when the question involves multiple instruments with isolated stems or visual grounding, or when data efficiency itself is the claim being tested. No single dataset covers scale, diversity, and alignment quality at once — that three-way trade-off, visible in Figure 8.7, is the dataset builder's trilemma and the reason MT3 pooled six corpora instead of choosing one.

8.8 Synthesis: Open Problems in AMT as of 2026

Each paradigm above closed one bottleneck and exposed the next. The problems below are well-established in the literature — each is stated with the evidence for it — followed by a clearly marked speculative outlook.

Low-resource instruments. The most documented open problem is also the oldest: transcription quality tracks annotation quantity, and annotation is brutally expensive. MT3's entire multi-task design is a response to it, and its headline result — up to 260% relative improvement on GuitarSet, MusicNet, and URMP — measures the size of the gap rather than its closure: low-resource instruments still lag far behind piano. Any instrument without a MAESTRO-scale corpus remains an open problem by definition.

Real-time transcription. Onsets and Frames uses bidirectional LSTMs: the model sees the future before deciding the present, which rules out live use. Fernandez's "Onsets and velocities" [65] showed that affordable real-time piano transcription with CNNs is possible, but causal, low-latency multitrack transcription with competitive accuracy is still unsolved — the architectures that transcribe best (bidirectional context, segment-level event decoding) are exactly the ones that cannot run live.

Expressive timing and dynamics. Onsets and Frames estimated velocity with an admittedly arbitrary linear map ($v_{\text{MIDI}} = 80v + 10$); MT3 dropped velocity entirely because datasets do not standardize it. Micro-timing, rubato, articulation, and pedaling remain only partially modeled: Kong et al. [64] regressed pedal events for piano, but no general framework captures expressive performance across instruments. The field can transcribe *which* notes were played far better than *how* they were played.

Evaluation consistency and dataset alignment. MT3's own conclusion names this one: different studies use different metrics and splits per dataset, and dataset alignments range from exact (rendered, Disklavier) to noisy (DTW). The multi-instrument F_1 was proposed precisely because no existing metric jointly scored notes and instruments. Until alignment quality is standardized and splits are fixed, cross-paper comparisons remain approximate.

The scale gap. MT3's comparison stands: the largest speech corpora exceed a thousand hours while all AMT datasets combined are smaller than LibriSpeech alone. Every paradigm's appetite for data has outgrown the annotation pipeline. This is the structural reason the SSL era exists.

Speculation (marked). If the SSL era delivers on its promise, the next paradigm shift will be a transcription system pretrained on millions of hours of unlabeled music and fine-tuned on hours — not hundreds of hours — of labels, matching MT3-level accuracy with an order of magnitude less annotation. Whether masked-reconstruction objectives can preserve the ±50 ms onset precision the metrics demand, and whether instrument identity survives the pretraining distribution shift, are the experiments that will decide it. None of this is established; it is the bet the field is currently placing.

8.9 Drills

1. **NMF update from the auxiliary function.** Starting from the auxiliary function $G(h, h^t)$ in (8.7), compute $\partial G/\partial h_a$, set it to zero, and show that the minimizer is ex-

actly the multiplicative update (8.5). Then explain in two sentences why the chain of inequalities (8.9) guarantees monotone decrease of F.

2. **Hand-computing the dual objective.** For one pitch and six frames, let $I_{\text{onset}} = [1, 0, 0, 1, 0, 0]$ and $P_{\text{onset}} = [0.9, 0.1, 0.2, 0.8, 0.1, 0.05]$. Compute L_{onset} by hand (four decimal places) and confirm the script's printed value 0.8137.

3. **Weighted frame loss.** With $I_{\text{frame}} = [0, 1, 1, 1, 1, 0]$, $P_{\text{frame}} = [0.05, 0.9, 0.85, 0.7, 0.6, 0.1]$ and one note spanning $(t_1, t_2, t_3) = (1, 2, 4)$, compute the onset-weighted L_{frame} from (8.15) with $c = 5.0$ by hand, and compare against the unweighted binary cross-entropy. Which frames dominate the weighted loss, and why is that the point?

4. **MT3 tokenization by hand.** A segment contains one note: program 5, pitch 72, note-on at 1.00 s, note-off at 1.50 s, with nothing active at the segment start. Write the full MT3 token stream (`TIE_END` first, then events, then `EOS`), using the vocabulary of Section 5. How many tokens is it?

5. **The octave ghost, quantified.** For a single noiseless note with partial amplitudes $a_k = 1/k$ and salience parameters $\alpha = 0.8$, $H = 8$, compute $S(p^*)$ and the sub-octave ghost $S(p^* - 12)$ from (8.10)–(8.11) and their ratio. Explain in one paragraph why this ratio makes octave errors a structural property of salience methods rather than a tuning failure.

6. **Dataset triage.** (a) You want to build a tablature transcriber that outputs string and fret numbers for steel-string acoustic guitar. Which dataset do you use, and which property of its collection method makes the alternatives unsuitable? (b) You want to test whether a transcription model is data efficient. Which dataset gives the harshest test, and why?

7. **Why nonnegativity, geometrically.** Explain why the columns of W in (8.1) must look like note spectra while the principal components of V need not, using the simplicial-cone picture from Step 2 of the NMF section. What goes wrong, geometrically, if the nonnegativity constraint on H is dropped?

Appendix A

Solutions to Drills

A.1 Solutions — Module 1

Drill 1. Consider the string segment between x and $x+dx$ with vertical displacement $u(x,t)$. The tension T pulls at each end along the local tangent, at angles $\theta(x)$ and $\theta(x+dx)$ to the horizontal. The net vertical force is

$$F_y = T\sin\theta(x+dx) - T\sin\theta(x). \tag{A.1}$$

For small slopes, $\tan\theta = \partial u/\partial x$ and $\sin\theta = \tan\theta/\sqrt{1+\tan^2\theta} = \partial u/\partial x + O((\partial u/\partial x)^3)$, so to first order

$$F_y = T\left(\left.\frac{\partial u}{\partial x}\right|_{x+dx} - \left.\frac{\partial u}{\partial x}\right|_{x}\right) + O(dx^2) = T\,\frac{\partial^2 u}{\partial x^2}\,dx + O(dx^2). \tag{A.2}$$

The segment mass is $\mu\,dx$ and its vertical acceleration is $\partial^2 u/\partial t^2$. Newton's second law gives, dividing by dx and letting $dx \to 0$,

$$\mu\,\frac{\partial^2 u}{\partial t^2} = T\,\frac{\partial^2 u}{\partial x^2}, \qquad \text{i.e.} \qquad \frac{\partial^2 u}{\partial x^2} = \frac{1}{c^2}\,\frac{\partial^2 u}{\partial t^2}, \quad c = \sqrt{\frac{T}{\mu}}. \tag{A.3}$$

The wave speed is $c = \sqrt{T/\mu}$: raising tension or lightening the string raises the pitch, exactly as a guitarist tunes.

Drill 2. With $u(x,t) = X(x)T(t)$, the spatial modes are $X(x) = A\cos(\kappa x) + B\sin(\kappa x)$. Closed end $x = 0$ is a displacement node: $X(0) = 0$ forces $A = 0$. The open end $x = L$ is a pressure node; pressure is proportional to $-\partial u/\partial x$ (compression requires a displacement gradient), so $X'(L) = B\kappa\cos(\kappa L) = 0$, giving

$$\kappa_n L = \frac{(2n-1)\pi}{2}, \qquad n = 1, 2, \ldots \tag{A.4}$$

Hence, with $\omega_n = c\kappa_n$,

$$f_n = \frac{(2n-1)\,c}{4L}, \qquad n = 1, 2, \ldots \tag{A.5}$$

Only *odd* harmonics sound: $f_1, 3f_1, 5f_1, \ldots$ with $f_1 = c/(4L)$ (quarter-wave resonator). The missing even harmonics give the hollow timbre of the clarinet family, the canonical cylindrical closed-pipe instruments.

Drill 3. (a) From (1.14)–(1.15) and (1.16):

f	CBW $= 1/(dz/df)$	ERB
200 Hz	103.3 Hz	46.3 Hz
5 kHz	862.5 Hz	564.4 Hz

(b) $N_{\mathrm{ERB}}(8000) - N_{\mathrm{ERB}}(100) = 21.4\log_{10}(4.37\cdot 8+1) - 21.4\log_{10}(4.37\cdot 0.1+1) = 33.30 - 3.37 = 29.9$ ERBs.
(c) Quantization noise injected anywhere inside a critical band is masked by the signal energy in that same band, while noise in a band carrying no signal is fully audible. Allocating bits per Bark therefore matches the bit budget to the ear's own filterbank: bands are refined only where the ear can resolve the difference, and coarse quantization is hidden under masking where it cannot. A linear-Hertz allocation would waste bits on ultra-fine high-frequency detail the ear fuses into single bands, and starve the low bands where the ear is most resolving.

Drill 4. Write $x_m = \frac{1}{N}\sum_k X_k W_N^{-km}$ and $y_{(n-m) \bmod N} = \frac{1}{N}\sum_l Y_l W_N^{-l(n-m)}$ with $W_N = e^{-2\pi i/N}$. Then

$$\begin{aligned} z_n &= \sum_{m=0}^{N-1} x_m y_{(n-m) \bmod N} \\ &= \frac{1}{N^2}\sum_{k,l} X_k Y_l W_N^{-ln} \sum_{m=0}^{N-1} W_N^{-m(k-l)}. \end{aligned} \tag{A.6}$$

The inner sum equals N if $k = l$ and 0 otherwise (orthogonality), so $z_n = \frac{1}{N}\sum_k X_k Y_k W_N^{-kn}$, which is the inverse DFT of $\{X_k Y_k\}$. Hence $Z_k = X_k Y_k$.

The circular convolution wraps the tail of the linear convolution back onto its head. The linear convolution of a length-L signal with a length-M filter has length $L + M - 1$; choosing the DFT size $N \geq L + M - 1$ (zero-padding both sequences to length N) leaves no overlap after wrapping, so the circular result equals the linear one. This is the overlap-add/overlap-save basis of fast filtering.

Drill 5. Energy: $\int |g_a(t)|^2\,dt = \int a^{-1}\,|g(t/a)|^2\,dt$; with $s = t/a$, $dt = a\,ds$, this equals $\int |g(s)|^2\,ds = 1$. Time spread: $(\Delta t_a)^2 = \int t^2 a^{-1}\,|g(t/a)|^2\,dt = a^2\int s^2\,|g(s)|^2\,ds = a^2(\Delta t)^2$, so $\Delta t_a = a\,\Delta t$. Fourier transform: $G_a(f) = \int a^{-1/2} g(t/a) e^{-2\pi i f t} dt = a^{1/2}\int g(s) e^{-2\pi i f a s} ds = a^{1/2} G(af)$. Frequency spread: $(\Delta f_a)^2 = \int f^2 a\,|G(af)|^2\,df$; with $s = af$, $df = ds/a$, this equals $a^{-2}\int s^2\,|G(s)|^2\,ds = (\Delta f)^2/a^2$, so $\Delta f_a = \Delta f/a$. The product $\Delta t_a\,\Delta f_a = \Delta t\,\Delta f$ is invariant: dilation moves the atom along the hyperbola $\Delta t\,\Delta f = \text{const} \geq 1/(4\pi)$ but never below it. Choosing a longer STFT window buys frequency resolution at exactly the price the theorem demands in time resolution; there is no window shape or length that beats the bound.

Drill 6. (a) $Q = 1/(2^{1/24} - 1) = 1/(1.029302 - 1) = 34.1271$.
(b) $N_0 = Qf_s/f_{\min} = 34.1271 \times 22050/32.7 = 23012$ samples $= 1043.6$ ms.
(c) VQT: $N_0 = Qf_s/(f_{\min} + \gamma Q) = 34.1271 \times 22050/(32.7 + 20 \times 34.1271) = 1052$ samples $= 47.7$ ms.
(d) The window shortens 22×, restoring onset precision, but the bandwidth widens from $f_{\min}/Q = 0.96$ Hz to $f_{\min}/Q + \gamma = 20.96$ Hz — far wider than the 1.94 Hz semitone gap at C1, so adjacent semitones merge. VQT trades bass pitch resolution for bass timing resolution; γ sets the exchange rate.

Drill 7. The Nyquist frequency is 11.025 kHz. The 5 kHz component is below it and appears at 5 kHz. The 18 kHz component folds: $|18 - 22.05| = 4.05$ kHz. The closest MIDI note to 4050 Hz: $m = 69 + 12\log_2(4050/440) = 107.43$, so $m = 107$ ($f(107) = 3951.1$ Hz vs. $f(108) = 4186.0$ Hz). A transcriber would report a spurious note near C8 that was never played — aliasing as a source of hallucinated high notes.

Drill 8. (a) $z(440) = 4.21$ Bark, $z(660) = 6.07$ Bark; $\Delta z = 1.86 > 1$, so they lie in *different* (adjacent) critical bands.
(b) Stylized model with the masker at $z_m = 4.21$, 70 dB: upper slope $= 8 + 0.15 \times 70 = 18.5$ dB/Bark, threshold at the probe $= 70 - 18.5(1.86) - 1.2(1.86)^2 = 31.3$ dB. The 40 dB tone exceeds it, so it remains *audible* in this model (real masking additionally depends on the masker's tonal-vs.-noise character, which the stylized curve omits).
(c) Consequences for AMT: (i) quiet chord voices whose partials fall under a louder voice's masking curve produce systematic false negatives at the frame level; (ii) the fused/skirted partials bias pitch posteriors toward the masker's harmonics, a classic source of octave and fifth errors in dense polyphony.

A.2 Solutions — Module 2

1. *Derivation of the semitone ratio and $f(60)$.* From axiom (2.1b), $f(m+1) = r\,f(m)$ with r constant. Iterating from the anchor $m = 69$:

$$f(m) = f(69)\,r^{\,m-69} = 440\,r^{\,m-69}. \tag{A.7}$$

Axiom (2.1a) demands $f(m+12) = 2f(m)$ for every m. Substituting (A.7):

$$440\,r^{\,m+12-69} = 2 \cdot 440\,r^{\,m-69} \implies r^{12} = 2 \implies r = 2^{1/12} \approx 1.059463094359, \tag{A.8}$$

taking the positive real 12th root since frequencies are positive. Then

$$f(60) = 440{\cdot}2^{(60-69)/12} = 440{\cdot}2^{-3/4} = \frac{440}{2^{0.75}} = \frac{440}{1.68179283} = 261.625565\ldots \approx 261.63 \text{ Hz}. \tag{A.9}$$

For the 442 Hz tone, (2.4) gives

$$c = 1200\log_2\left(\frac{442}{440}\right) = 1200\log_2(1.00454545\ldots) = 1200{\times}0.00654255\ldots \approx 7.85 \text{ cents sharp}. \tag{A.10}$$

2. *Pythagorean comma.* Twelve just fifths multiply frequencies by $(3/2)^{12} = 3^{12}/2^{12} = 531441/4096$. Seven octaves multiply by $2^7 = 128 = 524288/4096$. Their ratio is

$$\frac{(3/2)^{12}}{2^7} = \frac{531441/4096}{524288/4096} = \frac{531441}{524288} \approx 1.0136432648. \tag{A.11}$$

In cents:

$$1200\log_2\left(\frac{531441}{524288}\right) = 1200\,(12\log_2 3 - 19) = 22823.460010\ldots - 22800 \approx 23.46 \text{ cents}. \tag{A.12}$$

Twelve pure fifths overshoot seven pure octaves by nearly a quarter semitone, so a keyboard — which must close the circle of fifths and keep its octaves pure — cannot have both; every temperament is a choice of where to hide this 23.46-cent discrepancy.

3. *Partial collisions C4/G5.* With $f_{C4} = 261.6256\,\text{Hz}$ and $f_{G5} = 783.9909\,\text{Hz}$, the partial pairs (k, l), $1 \le k, l \le 8$, within 25 cents are:

$$(k,l) = (3,1): \quad \frac{3f_{C4}}{f_{G5}} = 3 \cdot 2^{-19/12} \;\Rightarrow\; +1.955 \text{ cents}, \tag{A.13}$$

$$(k,l) = (6,1): \text{ same ratio} \;\Rightarrow\; +1.955 \text{ cents}. \tag{A.14}$$

No other pair falls within 25 cents (the next closest, $(5,2)$, sits at $|1200 \log_2(5f_{C4}/2f_{G5})| \approx 278$ cents). For the observed peak at $784.8767\,\text{Hz}$ with $\sigma = 5$ cents: $\log L(H_C) = -\frac{1}{2}(0/5)^2 = 0$ and $\log L(H_G) = -\frac{1}{2}(1.955/5)^2 = -0.0764$, so LLR $= +0.0764 \ll 1$. The peak alone cannot decide; the encoder must recruit the *other* partials (does energy exist at $2f$, $4f$, $5f$ of each hypothesis?), onset synchrony across partials, and voice-leading context.

4. *Memory.* (a) Dense boolean: 1000×128 bits $= 128000$ bits $= 16000$ bytes $= 15.625\,\text{KiB}$. (b) Float32: $1000 \times 128 \times 4 = 512000$ bytes $= 500.00\,\text{KiB}$. (c) Event stream: $40 \times 4 \times 2 = 320$ bytes. Ratio (a)/(c): $16000/320 = 50.0$. The gap grows with duration because grid memory scales with the number of *frames* T ($M \propto T$, and T grows with every silent second), whereas event memory scales with the number of *notes* N; rests cost the grid F frames per second but cost the event stream nothing.

5. *Hand tokenization.* MT3 (10 ms bins, instrument 0, no ties):

$$\begin{aligned} &\textsc{Instrument}(0),\ \textsc{EndTieSection},\\ &\textsc{Time}(0.00\text{s}),\ \textsc{On},\ \textsc{Note}(60),\\ &\textsc{Time}(0.25\text{s}),\ \textsc{On},\ \textsc{Note}(64),\\ &\textsc{Time}(0.50\text{s}),\ \textsc{Off},\ \textsc{Note}(60),\\ &\textsc{Time}(0.75\text{s}),\ \textsc{Off},\ \textsc{Note}(64),\ \textsc{Eos}, \end{aligned} \tag{A.15}$$

$2 + 3 + 3 + 3 + 3 + 1 = 15$ tokens. REMI at 120 BPM, 4/4 (bar $= 2.0\,\text{s}$, 16 positions): C4 at $t = 0 \to \textsc{Position}(1/16)$, E4 at $t = 0.25$ $(0.25/2.0 \times 16 = 2) \to \textsc{Position}(3/16)$; both durations $0.5\,\text{s} = 8$ thirty-seconds:

$$\textsc{Bar},\ \textsc{Pos}1,\ \textsc{Vel},\ \textsc{On}60,\ \textsc{Dur}8,\ \textsc{Pos}3,\ \textsc{Vel},\ \textsc{On}64,\ \textsc{Dur}8, \tag{A.16}$$

9 tokens (velocity bins depend on the chosen velocities). REMI is shorter here ($9 < 15$) because its per-note cost is a flat 4 tokens with no mode toggles. The ranking reverses for dense chords: MT3 amortizes one TIME token over all simultaneous notes (a 10-note chord costs $\approx$ TIME, ON, 10× NOTE), while REMI pays a full POSITION token per note ($10 \times 4 + 1 = 41$ tokens).

6. *Timing and pedal.* (a) $\Delta = 60/120/4 = 0.125\,\text{s}$, so $|e| \le \Delta/2 = 62.5\,\text{ms}$ by (2.23). (b) $35/10 = 3.5$ MT3 bins — representable, if coarsely. REMI grid quantization rounds the 35 ms push (0.28 sixteenths) to the nearest position center and erases it. (c) Effective offset 0.90 s. Physically, the depressed pedal holds all dampers off the strings, so the note keeps ringing after the key is released until the pedal itself is released.

7. *Voice assignment.* The cost matrix is

$$C = \begin{pmatrix} |72-74| & |72-71| \\ |76-74| & |76-71| \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 2 & 5 \end{pmatrix}. \tag{A.17}$$

The two permutations cost: identity ($72 \to 74,\ 76 \to 71$): $2 + 5 = 7$; swap ($72 \to 71,\ 76 \to 74$): $1 + 2 = 3$. The optimum is the swap, $\{0{:}71,\ 1{:}74\}$, with total motion 3 semitones. It embodies minimal voice motion / no voice crossing: the perceptual and notational default that independent musical lines move as little as possible and do not cross.

A.3 Solutions — Module 3

1. **AST patch arithmetic.** Six seconds at a 10 ms hop give $100\times6 = 600$ frames. Frequency axis: $G_f = (128-16)/10+1 = 12$. Time axis: $G_t = \lceil(600-16)/10\rceil = \lceil 58.4\rceil = 59$. Hence $N = 12\times59 = 708$ patches. At 75% masking, $0.75\times708 = 531$ patches are hidden and 177 remain visible. The encoder's self-attention costs $\mathcal{O}(L^2)$ in sequence length, so processing visible patches only uses $(177/708)^2 = (0.25)^2 = 1/16$ of the full-sequence attention compute.

2. **Block-masking design.** One 5×5 block covers $q = 25/1188 = 0.02104$ of the grid. From $r = 1-(1-q)^K$ with $r = 0.60$:

$$K = \frac{\log(1-0.60)}{\log(1-0.02104)} = \frac{-0.9163}{-0.02127} = 43.08, \tag{A.18}$$

so $K = 44$ blocks (rounding up; 44 blocks give realized ratio $1-(1-q)^{44} = 0.608$). Inductive bias: block masking hides contiguous time–frequency craters, defeating the local-interpolation shortcut that random masking permits on highly correlated spectrogram frames. An onset is a transient spanning several frames and partials; reconstructing a cratered attack forces the model to represent envelope and harmonic-stack dynamics across hundreds of milliseconds — the temporal context onset detection requires.

3. **Normalization gradient analysis.** Write the per-patch gradient norm (dropping the shared $2/M$ factor). Raw MSE: $\|\nabla\| = \|\hat{y}-y\| = 0.1\|y\|$ under 10% relative error, so

$$\frac{\|\nabla_A\|}{\|\nabla_B\|} = \frac{0.1\|y_A\|}{0.1\|y_B\|} = \frac{\|y_A\|}{\|y_B\|} = 10. \tag{A.19}$$

The optimizer attends $10\times$ more to the loud patch: it learns loudness, not structure. Normalized MSE: $\tilde{y}$ has unit variance, $\|\tilde{y}\| = \sqrt{P}$ for both patches, and 10% relative error gives $\|\hat{y}-\tilde{y}\| = 0.1\sqrt{P}$ for both, so the ratio is 1. Every patch contributes equally; the optimizer can only reduce the loss by modeling spectral *shape*, i.e. harmonic structure.

4. **Gumbel-Max, two categories.** Put $c = \log\pi_2 - \log\pi_1$; we need $P(g_1-g_2\geq c)$. First show $D = g_1-g_2$ is logistic: with F, f the Gumbel CDF/density,

$$\begin{aligned} P(D\leq x) &= \int_{-\infty}^{\infty} f(g)\,F(g+x)\,dg \\ &= \int_{-\infty}^{\infty} e^{-g}e^{-e^{-g}}\exp\!\big(-e^{-(g+x)}\big)\,dg. \end{aligned}$$

Substitute $u = e^{-g}$ ($du = -e^{-g}dg$; $g\to-\infty\Rightarrow u\to\infty$, $g\to\infty\Rightarrow u\to0$):

$$P(D\leq x) = \int_0^{\infty} e^{-u}e^{-ue^{-x}}\,du = \int_0^{\infty} e^{-u(1+e^{-x})}\,du = \frac{1}{1+e^{-x}}, \tag{A.20}$$

the Logistic$(0,1)$ CDF. Therefore

$$P(g_1-g_2\geq c) = 1-\frac{1}{1+e^{-c}} = \frac{e^{-c}}{1+e^{-c}} = \frac{1}{e^{c}+1} = \frac{1}{\pi_2/\pi_1+1} = \frac{\pi_1}{\pi_1+\pi_2} = \pi_1, \tag{A.21}$$

since $e^c = \pi_2/\pi_1$ and $\pi_1+\pi_2 = 1$.

5. **Diversity-loss floor.** By Gibbs' inequality, for distributions $\bar{p}$ and uniform $q_v = 1/V$, $\sum_v \bar{p}_v\log(\bar{p}_v/q_v)\geq 0$, i.e. $\sum_v\bar{p}_v\log\bar{p}_v \geq \sum_v\bar{p}_v\log(1/V) = -\log V$, with equality iff $\bar{p} = q$ almost everywhere. Dividing by V: $\mathcal{L}_d \geq -\log V/V$, uniquely minimized at

the uniform distribution. For $V = 320$: $-\log(320)/320 = -5.7683/320 = -0.01803$. Codebook collapse means $\bar{p}$ concentrates on a few entries (e.g. $\bar{p}_{v^\star} \approx 1$, rest ≈ 0); then $\sum_v \bar{p}_v \log \bar{p}_v \approx 0$ and $\mathcal{L}_d \approx 0$, the *maximum*, far above the -0.01803 floor — the loss penalizes exactly this concentration.

6. **HuBERT masked-only loss.** The degenerate mapping is the near-identity copy: on an *unmasked* frame t, the target z_t was computed by clustering that same frame's features, so the network can drive the loss to zero by learning $h_t(\tilde{x}) \approx$ a lookup of the visible frame's own cluster — a local function requiring no context and teaching nothing about sound structure. Noisy targets make this worse: the model memorizes the noise. Restricting (3.22) to masked frames removes the input the target was derived from; the model must then infer z_t from *surrounding* frames, which requires both an acoustic model (what do neighboring sounds sound like?) and a sequential model (which hidden units follow which?) — the two capacities HuBERT aims to build.

7. **EMA memory.** The weight of a checkpoint k steps old is $(1-\tau)\tau^k$ by (3.26). Half-life solves $\tau^{k_{1/2}} = 1/2$, i.e. $k_{1/2} = \log(1/2)/\log\tau$. For $\tau = 0.999$: $k_{1/2} = 692.8 \approx 693$ updates; for $\tau = 0.9999$: $k_{1/2} = 6931$ updates. After 20,000 updates at $\tau = 0.9999$ the surviving weight fraction is $0.9999^{20000} = e^{20000 \ln 0.9999} = e^{-2.0001} = 0.1353$, i.e. 13.5%. Collapse-avoidance role: because the teacher is a trailing average over thousands of past student states, its targets always reflect *recent but not current* student behavior. If the student's outputs began drifting toward a constant, the targets would still be the varying contextualized features produced by the recent non-collapsed states; the student's regression loss against those varying targets stays positive for any constant prediction ($\geq \mathrm{Var}(y) > 0$), so gradient descent pulls the student back toward tracking real input variation instead of settling into the trivial fixed point. The lag turns collapse from an absorbing state into a self-correcting transient.

8. **Transfer budget.** (a) Upsampling factor $100/60 = 5/3 \approx 1.667$ (non-integer: use linear interpolation rather than the integer repeater of `code/m3_amt_heads.py`). (b) 8 s at 100 fps give 800 frames; each head outputs $(1, 800, 88)$. (c) One head: $512 \times 88 + 88 = 45{,}144$ parameters; three heads: 135,432. (d) Full fine-tuning trains $30{,}000{,}000 + 135{,}432 = 30{,}135{,}432$ parameters, a factor $30{,}135{,}432/135{,}432 = 222.5\times$ the probing budget.

A.4 Solutions — Module 4

1. **Conformer convolution-module budget.** From (4.7)–(4.11), counting weights plus biases: pointwise $d \to 2d$ holds $d \cdot 2d + 2d$; the depthwise K-tap convolution holds $K \cdot d + d$ (one filter plus bias per channel); BatchNorm holds $2d$ (γ, β); pointwise $d \to d$ holds $d^2 + d$. The total is

$$P_{\mathrm{conv}}(d, K) = 2d^2 + 2d + Kd + d + 2d + d^2 + d = 3d^2 + Kd + 6d. \tag{A.22}$$

For $d = 256$, $K = 31$: $3 \cdot 65536 + 31 \cdot 256 + 1536 = 196608 + 7936 + 1536 = 206{,}080$ parameters, decomposing as 131,584 (pointwise expansion) + 8,192 (depthwise) + 512 (BatchNorm) + 65,792 (pointwise projection). The depthwise kernel's share is $8192/206080 = 3.97\%$: the local temporal filtering — the module's raison d'être — costs under 4% of its parameters, because the pointwise mixing sheets dominate.

2. **Sinusoidal shift property, by hand.** With $d = 4$, $\omega_0 = 1$, $\omega_1 = 0.01$. From (4.18)

with $k = 2$:

$$M(2) = \begin{bmatrix} \cos 2 & \sin 2 & 0 & 0 \\ -\sin 2 & \cos 2 & 0 & 0 \\ 0 & 0 & \cos 0.02 & \sin 0.02 \\ 0 & 0 & -\sin 0.02 & \cos 0.02 \end{bmatrix} = \begin{bmatrix} -0.416147 & 0.909297 & 0 & 0 \\ -0.909297 & -0.416147 & 0 & 0 \\ 0 & 0 & 0.999800 & 0.019999 \\ 0 & 0 & -0.019999 & 0.999800 \end{bmatrix}. \tag{A.23}$$

Now $\mathrm{PE}(5) = [\sin 5,\ \cos 5,\ \sin 0.05,\ \cos 0.05] = [-0.958924,\ 0.283662,\ 0.049979,\ 0.998750]$. Multiplying block by block: first pair, $(-0.416147)(-0.958924) + (0.909297)(0.283662) = 0.399055 + 0.257932 = 0.656987 = \sin 7$; $(-0.909297)(-0.958924) + (-0.416147)(0.283662) = 0.871949 - 0.118047 = 0.753902 = \cos 7$. Second pair, $(0.999800)(0.049979) + (0.019999)(0.998750) = 0.049969 + 0.019974 = 0.069943 = \sin 0.07$; $(-0.019999)(0.049979) + (0.999800)(0.998750) = -0.001000 + 0.998550 = 0.997551 = \cos 0.07$. Hence $M(2)\mathrm{PE}(5) = [0.656987,\ 0.753902,\ 0.069943,\ 0.99755$ PE(7) component by component (maximum error 1.11×10^{-16} numerically).

3. **RoPE relative property.** Write $R^m \equiv R^m_\Theta$. Each 2×2 block $R(m\theta_i)$ is a rotation, so its transpose is its inverse: $R(m\theta_i)^\top = R(-m\theta_i) = R(m\theta_i)^{-1}$. Block-diagonal structure preserves this: $(R^m)^\top = \mathrm{blkdiag}(R(m\theta_i)^\top) = R^{-m}$. Rotations in the plane commute, so blockwise $R^{-m}R^n = \mathrm{blkdiag}(R(-m\theta_i)R(n\theta_i)) = \mathrm{blkdiag}(R((n-m)\theta_i)) = R^{n-m}$. Then

$$\langle R^m q, R^n k\rangle = (R^m q)^\top (R^n k) = q^\top (R^m)^\top R^n k = q^\top R^{-m} R^n k = q^\top R^{n-m} k = \langle q, R^{n-m} k\rangle, \tag{A.24}$$

which depends on m, n only through $n-m$. For $d = 4$, $\theta_1 = 10000^0 = 1$, $\theta_2 = 10000^{-1/2} = 0.01$, and $m = 7$:

$$R^7_\Theta = \begin{bmatrix} \cos 7 & -\sin 7 & 0 & 0 \\ \sin 7 & \cos 7 & 0 & 0 \\ 0 & 0 & \cos 0.07 & -\sin 0.07 \\ 0 & 0 & \sin 0.07 & \cos 0.07 \end{bmatrix} = \begin{bmatrix} 0.753902 & -0.656987 & 0 & 0 \\ 0.656987 & 0.753902 & 0 & 0 \\ 0 & 0 & 0.997551 & -0.069943 \\ 0 & 0 & 0.069943 & 0.997551 \end{bmatrix}, \tag{A.25}$$

using $\cos 7 = 0.753902$, $\sin 7 = 0.656987$, $\cos 0.07 = 0.997551$, $\sin 0.07 = 0.069943$.

4. **Cross-attention by hand.** The score $s_{1t} = \frac{1}{\sqrt{8}} \sum_j q_{1j} k_{tj}$ with the given rows yields $s_{1\cdot} = [-0.0112,\ -0.1711,\ 0.0743,\ 0.1740]$ (the $t = 1$ term expands to the eight products in (4.37), summing to -0.0112). Exponentiating: $e^{s_{1\cdot}} = [0.988863,\ 0.842725,\ 1.077117,\ 1.190071]$, sum $= 4.098776$. Dividing, $a_{1\cdot} = [0.2413,\ 0.2056,\ 0.2628,\ 0.2903]$, and $0.2413 + 0.2056 + 0.2628 + 0.2903 = 1.0000$. The first output component is the $a_{1\cdot}$-weighted mean of the value rows' first components $(-0.078,\ -0.172,\ 0.145,\ 0.326)$: $0.2413(-0.078) + 0.2056(-0.172) + 0.2628(0.145) + 0.2903(0.326) = -0.018821 - 0.035363 + 0.038106 + 0.094638 = 0.0786 \approx 0.0787$.

5. **Focal gradient at $p = 0.95$.** Differentiating $\mathrm{FL} = -\alpha(1-p)^\gamma \log p$: $\frac{d\mathrm{FL}}{dp} = -\alpha[-\gamma(1-p)^{\gamma-1} \log p + (1-p)^\gamma / p]$ by the product rule; multiplying by $dp/dz = p(1-p)$ gives (4.49). For $p = 0.95$, $\alpha = 0.25$, $\gamma = 2$: $\mathrm{FL} = -0.25(0.05)^2 \ln 0.95 = 3.206 \times 10^{-5}$ versus $\mathrm{CE} = -\ln 0.95 = 0.051293$, a $1600.0\times$ loss suppression. The gradients: $\frac{d\mathrm{FL}}{dz} = 0.25 \cdot 2 \cdot 0.95 \cdot (0.05)^2 \ln 0.95 - 0.25(0.05)^3 = -0.00006091 - 0.00003125 = -0.00009216$, versus $\frac{d\mathrm{CE}}{dz} = -(1 - 0.95) = -0.0500$, a $542.5\times$ gradient suppression. The well-classified example is essentially removed from the update.

6. **Imbalance arithmetic.** (a) Cells $= 240 \times 100 \times 88 = 2{,}112{,}000$; per-cell onset prior $= 2000/2112000 = 9.4697 \times 10^{-4} = 0.0947\%$, ratio 1:1056. (b) Pitch-active prior $= 3.5/88 = 0.0398 = 3.98\%$, ratio 1:24.1. (c) For the easy negative, $\mathrm{FL}/\mathrm{CE} = \alpha(1-p_t)^\gamma = 0.25 \times (0.02)^2 = 1.0 \times 10^{-4}$: a $10{,}000\times$ suppression. The focal loss thus converts a 1:1056 census into an effective weighting dominated by difficulty.

7. **CTC's conditional independence.** In (4.31) the path probability is $\prod_{t=1}^{T} p_t(\pi_t \mid X)$: each factor conditions on X alone, never on $\pi_{<t}$. Summing over paths does not introduce label–label coupling either, since the sum is over factorizations of the same independent form. Hence $p(\pi_t \mid \pi_{<t}, X) = p(\pi_t \mid X)$: frames are conditionally independent given the input. Musical example: in a C–E–G arpeggio, hearing C then E makes G highly likely (tonal grammar), but CTC's frame-t distribution for the G cannot see that C and E were emitted — it sees only the encoder state h_t. Under CTC the encoder must therefore internalize the arpeggio grammar into h_t itself (e.g. via deep bidirectional context); under the autoregressive factorization (4.28), the dependency is modeled explicitly by the conditioning $p_\theta(y_s = \mathrm{G} \mid y_{<s} = (\mathrm{C}, \mathrm{E}), X)$. This is why CTC struggles with dense polyphony where inter-note dependencies carry the musical structure.

8. **Mask–predict objective.** Sample a mask set $M \subset \{1, \dots, L\}$ uniformly with $|M| = \lfloor 0.4L \rfloor$ and replace those positions by `[MASK]`. The model predicts a distribution over Σ at each masked position conditioned on X and the unmasked tokens $y_{\setminus M}$. The maximized log-likelihood is

$$\sum_{s \in M} \log p_\theta(y_s \mid y_{\setminus M}, X), \tag{A.26}$$

whose negation is (4.30). Because positions are predicted independently given the observed set, the objective never scores the *length* L: a fully-masked length-L input always yields L predictions, so L itself must come from a separate length-prediction head $p(L \mid X)$ (typically a small classifier over $L \in \{1, \dots, L_{\max}\}$ trained with cross-entropy against the true length). The iteration then refines *which* tokens fill the L slots, never how many slots exist.

A.5 Solutions — Module 5

Solution 1 (beam search by hand). At step 2 the live beam is $\{(\mathrm{ON60}, -0.5978), (\mathrm{ON64}, -1.2040)\}$. Expanding each by all four tokens gives $\mathcal{C}_2$:

$$\begin{aligned}
(\mathrm{ON60}, \mathrm{ON60}) &: -0.5978 + \log 0.50 = -1.2910, & (\mathrm{ON60}, \mathrm{ON64}) &: -0.5978 + \log 0.20 = -2.2073,\\
(\mathrm{ON60}, \mathrm{TS}) &: -0.5978 + \log 0.15 = -2.4950, & (\mathrm{ON60}, \mathrm{END}) &: -0.5978 + \log 0.15 = -2.4950,\\
(\mathrm{ON64}, \mathrm{ON60}) &: -1.2040 + \log 0.10 = -3.5066, & (\mathrm{ON64}, \mathrm{ON64}) &: -1.2040 + \log 0.65 = -1.6348,\\
(\mathrm{ON64}, \mathrm{TS}) &: -1.2040 + \log 0.20 = -2.8134, & (\mathrm{ON64}, \mathrm{END}) &: -1.2040 + \log 0.05 = -4.1997.
\end{aligned} \tag{A.27}$$

$(\mathrm{ON64}, \mathrm{END})$ is finished and moves to $\mathcal{F}$; the top-2 live survivors are $(\mathrm{ON60}, \mathrm{ON60})$ at -1.2910 and $(\mathrm{ON64}, \mathrm{ON64})$ at -1.6348. Continuing, step 3 finishes $(\mathrm{ON64}, \mathrm{ON64}, \mathrm{END})$ at $-1.6348 + \log 0.60 = -2.1456$ and $(\mathrm{ON60}, \mathrm{ON60}, \mathrm{END})$ at $-1.2910 + \log 0.36 = -2.3126$; the winner is $(\mathrm{ON64}, \mathrm{ON64}, \mathrm{END})$ at -2.1456, matching the enumerated global optimum. Greedy continuation from $(\mathrm{ON64},)$: arg max of $(0.10, 0.65, 0.20, 0.05)$ is ON64, then arg max of $(0.02, 0.03, 0.35, 0.60)$ is END, giving $(\mathrm{ON64}, \mathrm{ON64}, \mathrm{END})$ with total $\log 0.30 + \log 0.65 + \log 0.60 = -2.1456$. Greedy's only mistake was the first token; with that decision fixed externally it decodes optimally.

Solution 2 (length-penalty sensitivity). Raw scores: $S(Y_1) = 4 \cdot (-0.7) = -2.8$, $S(Y_2) = 8 \cdot (-0.45) = -3.6$. For $\alpha = 0.6$: $\mathrm{lp}(4) = (9/6)^{0.6} = 1.2756$, $\mathrm{lp}(8) = (13/6)^{0.6} = 1.5903$, so $\mathrm{score}(Y_1) = -2.8/1.2756 = -2.1953$ and $\mathrm{score}(Y_2) = -3.6/1.5903 = -2.2637$: Y_1 wins. For $\alpha = 1.0$: $\mathrm{lp}(4) = 1.5$, $\mathrm{lp}(8) = 13/6 = 2.1667$, so $\mathrm{score}(Y_1) = -2.8/1.5 = -1.8667$ and $\mathrm{score}(Y_2) = -3.6/2.1667 = -1.6615$: Y_2 wins. Raising α strengthens the length correction: at $\alpha = 1$ the penalty grows linearly in $(5 + L)/6$ and fully compensates the longer hypothesis's accumulated deficit, flipping the ranking toward the per-token more confident sequence. In

practice α is tuned on validation data; too large an α overcorrects and favors implausibly long hypotheses.

Solution 3 (LM rescoring crossover). Setting $\text{score}(A;\lambda^*) = \text{score}(B;\lambda^*)$:

$$-3.0 + \lambda^*(-4.0) = -3.5 + \lambda^*(-2.0) \implies \lambda^* = \frac{-3.5-(-3.0)}{-4.0-(-2.0)} = \frac{-0.5}{-2.0} = 0.25. \tag{A.28}$$

At $\lambda = 0.2$: $A = -3.0 - 0.8 = -3.80$, $B = -3.5 - 0.4 = -3.90$: A wins. At $\lambda = 2.0$: $A = -3.0 - 8.0 = -11.00$, $B = -3.5 - 4.0 = -7.50$: B wins. λ^* is the ratio of the acoustic margin to the LM margin: $\lambda^* = \Delta_{\text{ac}}/\Delta_{\text{LM}}$. A small λ^* means the LM needs little weight to overrule the acoustic model — the acoustic preference is weak relative to the LM's conviction — while a large λ^* means the prior must be trusted heavily before it can overturn strong acoustic evidence.

Solution 4 (Viterbi by hand). In the log domain, with states ordered (Off, On):

$$\delta_1 = (\log 0.6 + \log 0.8,\ \log 0.4 + \log 0.2) = (-0.7340,\ -2.5257). \tag{A.29}$$

Frame 2, emissions $(0.4, 0.6)$:

$$\begin{aligned}
\delta_2(\text{Off}) &= \max(-0.7340 + \log 0.7,\ -2.5257 + \log 0.4) + \log 0.4 = -2.0069, \quad \psi_2(\text{Off}) = \text{Off},\\
\delta_2(\text{On}) &= \max(-0.7340 + \log 0.3,\ -2.5257 + \log 0.6) + \log 0.6 = -2.4488, \quad \psi_2(\text{On}) = \text{Off}.
\end{aligned} \tag{A.30}$$

Frame 3, emissions $(0.7, 0.3)$:

$$\begin{aligned}
\delta_3(\text{Off}) &= \max(-2.0069 + \log 0.7,\ -2.4488 + \log 0.4) + \log 0.7 = -2.7203, \quad \psi_3(\text{Off}) = \text{Off},\\
\delta_3(\text{On}) &= \max(-2.0069 + \log 0.3,\ -2.4488 + \log 0.6) + \log 0.3 = -4.1636, \quad \psi_3(\text{On}) = \text{On}.
\end{aligned} \tag{A.31}$$

Backtracking from $\arg\max \delta_3 = \text{Off}$: $z_3 = \text{Off}$, $z_2 = \psi_3(\text{Off}) = \text{Off}$, $z_1 = \psi_2(\text{Off}) = \text{Off}$, with log-probability -2.7203. The raw per-frame argmax is (Off, On, Off). The single frame-2 emission favoring On ($0.6 > 0.4$) is insufficient to overcome the transition prior: switching costs $\log 0.3 = -1.2040$ against staying at $\log 0.7 = -0.3567$, a gap of 0.8473 that the emission advantage $\log(0.6/0.4) = 0.4055$ cannot bridge. The HMM suppresses a one-frame blip — the desired smoothing behavior — but the same mechanism would suppress a genuine very short note, which is the precision/recall tradeoff encoded in the transition matrix.

Solution 5 (hysteresis versus a single threshold). (a) With $\tau_{lo} = 0.3$: $t = 1$ has $o_1 = 0.8 > 0.5$ and $f_1 = 0.55 > 0.5$, so the note starts at frame 1. Frames 2–4 satisfy $f_t \geq 0.3$ $(0.90, 0.35, 0.70)$; at $t = 5$, $f_5 = 0.20 < 0.3$ ends the note at frame 4. Result: one note, $(1, 4)$. The dip to 0.35 at frame 3 is absorbed by the hysteresis band. (b) With $\tau_{lo} = 0.5$: at $t = 3$, $f_3 = 0.35 < 0.5$ ends the first note at frame 2, giving $(1, 2)$. At $t = 4$ the state is inactive with $o_4 = 0.6 > 0.5$ and $f_4 = 0.7 > 0.5$, so a second note starts; at $t = 5$, $f_5 = 0.2 < 0.5$ ends it at frame 4, giving $(4, 4)$. Result: two fragments, $(1, 2)$ and $(4, 4)$. The single threshold fragments one sustained note into two (a false offset/onset pair: insertion errors); the hysteresis band risks the opposite error, merging two distinct notes when the dip between them stays above τ_{lo}. τ_{lo} is therefore tuned to balance fragmentation against merging, typically well below τ_{hi} for instruments with long decaying tails.

Solution 6 (velocity calibration). From (5.39): $a = (100-15)/(-10-(-50)) = 85/40 = 2.125$, $b = 15 - 2.125 \cdot (-50) = 121.25$.

$$\begin{aligned}
E = 10^{-5} &: d = -100\,\text{dB},\ v = 2.125(-100) + 121.25 = -91.25 \to \text{clip } 1,\\
E = 10^{-3} &: d = -60\,\text{dB},\ v = 2.125(-60) + 121.25 = -6.25 \to \text{clip } 1,\\
E = 0.1 &: d = -20\,\text{dB},\ v = 2.125(-20) + 121.25 = 78.75 \to 79,\\
E = 1.0 &: d = 0\,\text{dB},\ v = 121.25 \to 121.
\end{aligned} \tag{A.32}$$

The two quietest energies hit the floor rail. Clipping is preferable to extrapolation because below the quiet anchor the measurement is dominated by the noise floor: the line would map noise fluctuations to spurious dynamic variation, while clipping honestly reports "as quiet as this instrument gets."

Solution 7 (decoding cost at scale). Greedy: $500 \cdot (4 + 0.1)\,\text{ms} = 2050\,\text{ms} \approx 2.05\,\text{s}$. Beam $B = 8$: $500 \cdot (4 + 1)\,\text{ms} = 2500\,\text{ms} = 2.5\,\text{s}$. Slowdown factor $2.5/2.05 \approx 1.22$. Hypothesis memory: $B \cdot T = 4000$ token ids $\times$ 4 bytes $= 16\,\text{KiB}$ for the token histories alone (the KV caches, one per hypothesis, dominate in practice). Batching makes the beam-width latency multiplier far smaller than B because the forward pass — the expensive step — is amortized across hypotheses in one batched GEMM; only the cheap expansion/selection step scales with B. Memory cannot be amortized this way: each hypothesis carries its own KV cache and history, so memory scales with B directly.

A.6 Solutions — Module 6

Drill 1. Starting from $F_1 = 2/(1/P + 1/R)$ with $P = \text{TP}/(\text{TP}+\text{FP})$ and $R = \text{TP}/(\text{TP}+\text{FN})$:

$$\frac{1}{P} = \frac{\text{TP}+\text{FP}}{\text{TP}}, \qquad \frac{1}{R} = \frac{\text{TP}+\text{FN}}{\text{TP}},$$

so

$$\frac{1}{P} + \frac{1}{R} = \frac{2\,\text{TP}+\text{FP}+\text{FN}}{\text{TP}}, \qquad F_1 = \frac{2\,\text{TP}}{2\,\text{TP}+\text{FP}+\text{FN}}.$$

For $\text{TP} = 41$, $\text{FP} = 7$, $\text{FN} = 13$:

$$P = \frac{41}{48} \approx 0.854167, \qquad R = \frac{41}{54} \approx 0.759259,$$

$$F_1 = \frac{2 \cdot 0.854167 \cdot 0.759259}{0.854167 + 0.759259} = \frac{1.297067}{1.613426} \approx 0.803922 = \frac{82}{82+7+13} = \frac{82}{102}.$$

TN never appears because precision conditions on "system said yes" and recall on "truth was yes"; correctly predicted silence is not evidence about either. With TN included, accuracy would be $(41 + 939)/1000 = 0.98$, a vacuous score dominated by the sparsity of the piano roll.

Drill 2. Compatibility (± 50 ms, same pitch): estimate 0.049 is compatible with reference 0.00 ($0.049 < 0.05$) and reference 0.08 ($0.031 < 0.05$); estimate 0.075 is compatible only with reference 0.08 ($0.005 < 0.05$; $0.075 > 0.05$ from 0.00). (a) Greedy in time order: estimate 0.049 takes its closest compatible reference, 0.08 (distance $0.031 < 0.049$). Estimate 0.075 then finds 0.08 taken and 0.00 incompatible: stranded. One match. (b) Maximum bipartite matching pairs $0.049 \to 0.00$ and $0.075 \to 0.08$: two matches, which is optimal since there are only two references. (c) Under maximum matching: $\text{TP} = 2$, $\text{FP} = 0$, $\text{FN} = 0$, so $P = R = F_1 = 1$. Under greedy: $\text{TP} = 1$, $\text{FP} = 1$, $\text{FN} = 1$, so $P = 1/2$, $R = 1/2$, $F_1 = 1/2$. The mir_eval convention requires the maximum (Hungarian) matching, i.e. the $F_1 = 1$ scoring — greedy under-reports by stranding a matchable pair.

Drill 3. Offset tolerances: $\max(0.05, 0.2 \cdot 0.3) = \max(0.05, 0.06) = 0.06$ s for the 0.3 s note; $\max(0.05, 0.2 \cdot 3.0) = 0.60$ s for the 3.0 s note. An estimate ending 0.09 s late: $0.09 > 0.06$, so the short note fails onset+offset; $0.09 < 0.60$, so the long note still matches. The relative rule exists because a fixed absolute window cannot serve both scales: 50 ms is half the duration of a 100 ms note (absurdly lax) yet only 1.7% of a 3 s note (absurdly strict for offset perception, which scales with duration). The $\max(\cdot)$ hybrid gives short notes an absolute floor and long notes proportional slack.

Drill 4. With $0 \le c \le \min(a, b)$: $\max(0, a-b) + \max(0, b-a) = |a-b|$ (one of the two terms is zero, the other is $|a-b|$), and $\max(a, b) = \min(a, b) + |a-b|$. Hence

$$[\min(a,b) - c] + \max(0, a-b) + \max(0, b-a) = \min(a,b) - c + |a-b| = \max(a,b) - c.$$

In words: of the $\min(a,b)$ pairable voices, c are correct and the rest are substitutions; the $|a-b|$ unpaired voices are misses if the reference has more, false alarms if the estimate has more. Frame table: frame 1 ($a=b=c=1$): $(0,0,0)$. Frame 2 ($a=2$, $b=2$; B matches B, C is 1200 cents from A so $c=1$): subs $=2-1=1$, miss $=0$, fa $=0$. Frame 3 ($a=1$, $b=0$, $c=0$): subs $=0$, miss $=1$, fa $=0$. With $\sum_t N_{\text{ref}}=4$:

$$E_{\text{subs}}=\tfrac{1}{4}=0.25,\quad E_{\text{miss}}=\tfrac{1}{4}=0.25,\quad E_{\text{fa}}=0,\quad E_{\text{tot}}=\tfrac{2}{4}=0.50=0.25+0.25.$$

Drill 5. With envelope $1/n^2$ and uniform weights over 8 template harmonics:

$$S(f_0)=\sum_{n=1}^{8}\frac{1}{n^2}=1+\tfrac{1}{4}+\tfrac{1}{9}+\tfrac{1}{16}+\tfrac{1}{25}+\tfrac{1}{36}+\tfrac{1}{49}+\tfrac{1}{64}=1.527422,$$

$$S(2f_0)=\sum_{n=1}^{8}\frac{1}{(2n)^2}=\frac{1}{4}\sum_{n=1}^{8}\frac{1}{n^2},\qquad \frac{S(2f_0)}{S(f_0)}=\frac{1}{4}=0.25 \text{ exactly},$$

$$S(f_0/2)=\sum_{k=1}^{4}\frac{1}{k^2}=1+\tfrac{1}{4}+\tfrac{1}{9}+\tfrac{1}{16}=1.423611,\qquad \frac{S(f_0/2)}{S(f_0)}=\frac{1.423611}{1.527422}\approx 0.932035.$$

A steeper roll-off concentrates the true note's energy in its lowest partials, which are exactly the partials that serve as evidence for the $f_0/2$ candidate's even template slots — so the octave-down phantom becomes *more* dangerous ($0.7665 \to 0.9320$) while the octave-up phantom becomes safer ($0.5 \to 0.25$), because its template only samples the now-weak high partials. Steeper spectra trade octave-up errors for octave-down errors.

Drill 6. Onset-only: fragment 1 has onset error $0 \le 0.05$ (match); fragment 2 has onset error $0.78 > 0.05$ (no match). TP $=1$, FP $=1$, FN $=0$: $P=1/2$, $R=1$,

$$F_1=\frac{2\cdot\frac{1}{2}\cdot 1}{\frac{1}{2}+1}=\frac{2}{3}\approx 0.6667.$$

Onset+offset: fragment 1's offset error is $|1.70-2.50|=0.80 > \max(0.05, 0.2\cdot 1.5)=0.30$ (no match); fragment 2 fails onset. TP $=0$, FP $=2$, FN $=1$, so $F_1=0$. Yes — this is the textbook over-segmentation signature: a large onset-vs-offset gap (0.667 vs 0.000) with onset recall at 1.0 (nothing missed, only shattered).

Drill 7. System 1: the onset-only $\to$ onset+offset drop is $0.88 \to 0.61$ (-0.27) while the error triple is balanced and low (0.04/0.05/0.06). Dominant failure: **over-segmentation / duration modeling** — notes are found but their extents are wrong. First fix: gap-merging and minimum-duration filtering / HMM smoothing of frame posteriors. The velocity-aware score barely moves ($0.61 \to 0.59$), so dynamics are fine; the disease is purely temporal.

System 2: all three note levels are flat ($0.81 \to 0.79 \to 0.78$ — durations and dynamics are healthy) but $E_{\text{subs}}=0.16$ dominates the triple (0.16 vs 0.03/0.03). Dominant failure: **pitch confusion, classically octave errors** — the system finds note-shaped events at the right times with the right lengths but the wrong pitches. First fix: a fundamental-presence requirement in the pitch candidate scoring, or an octave-aware training loss. Again the flat velocity-aware score says dynamics are not the bottleneck: when this system matches a note's pitch and timing, its velocity is already right; it is the pitch labeling that needs repair.

A.7 Solutions — Module 7

1. *Tempo-preserving pitch shift.* (a) k semitones is a frequency ratio $2^{k/12}$ (12 semitones = one octave = factor 2). To shift pitch by k we resample from f_s to f_s/r with $r=2^{k/12}$: the DTFT identity $Y(e^{j\omega})=X(e^{j\omega/r})$ moves digital frequency ω_0 to $r\omega_0$, i.e. physical

frequency $f \mapsto rf$ when read at the original rate. For $k = +3$, $f_s = 44100\,\text{Hz}$: $r = 2^{3/12} = 2^{1/4} = 1.189207$, and the intermediate rate is $44100/1.189207 = 37083.5\,\text{Hz}$. (b) T samples at f_s become T/r samples at f_s/r (fewer samples at the lower rate), pitched up by r. The phase vocoder with rate $1/r < 1$ lengthens by exactly r (verified: rate 1.1 shortens, so rate $1/r$ lengthens by r), giving $(T/r) \cdot r = T$ samples at the original rate, with frequencies untouched because the vocoder advances phase at the measured instantaneous frequency. Duration restored, pitch shifted. □(c) $r = 1.189207$, intermediate rate 37083.5 Hz, final length T samples.

2. *SNR calibration.* $\text{SNR}_{\text{dB}} = 10\log_{10}(P_s/(g^2P_n))$. Divide by 10 and exponentiate: $10^{\text{SNR}/10} = P_s/(g^2P_n)$. Rearranged, $g^2 = P_s/(P_n\,10^{\text{SNR}/10})$, and $g \geq 0$ gives $g = \sqrt{P_s/(P_n\,10^{\text{SNR}/10})}$. Numerics: $10^{15/10} = 10^{1.5} = 31.6228$; $g = \sqrt{0.5/(0.02 \times 31.6228)} = \sqrt{0.5/0.632456} = \sqrt{0.790569} = 0.889140$. Check: $10\log_{10}(0.5/(0.889140^2 \times 0.02)) = 10\log_{10}(0.5/0.015811) = 10\log_{10}(31.623) = 15.0000\,\text{dB}$. □

3. *CQT arithmetic.* (a) $Q = (2^{1/12} - 1)^{-1}$. $2^{1/12} = 1.059463$; minus one $= 0.059463$; reciprocal $= 16.8172$. (b) $N_k = Qf_s/f_k$. For $f = 65.4\,\text{Hz}$: $N = 16.8172 \times 22050/65.4 = 5670.0$ samples $= 257.14\,\text{ms}$. For $f = 2093\,\text{Hz}$: $N = 16.8172 \times 22050/2093 = 177.2$ samples $= 8.03\,\text{ms}$. Low bins pay in time what they gain in frequency resolution. (c) With flat weights, $S[p] = \sum_h m(hf_0(p))$ where m is the (nonnegative) spectral magnitude. For the octave artifact $q = p + 12$, $f_0(q) = 2f_0(p)$, so q's comb bins $\{2hf_0(p)\}$ are a *subset* of p's comb bins $\{hf_0(p)\}$ (the even harmonics). Hence $S[q] = \sum_h m(2hf_0(p)) \leq \sum_h m(hf_0(p)) = S[p]$: the fundamental's salience is at least the artifact's, purely by bin coverage, regardless of the spectrum. The $1/h$ weighting breaks this because it down-weights exactly the shared high harmonics. □

4. *2D positional encoding.* $D = 64$ gives $D/2 = 32$ channels per axis; channel pairs $i = 0, 1, \ldots$. Divisor for pair i: $10000^{2i/32}$. For $i = 0$: 1, so channels $0, 1 = (\sin 5, \cos 5) = (-0.95892,\ 0.28366)$. For $i = 1$: $10000^{2/32} = e^{2\ln(10000)/32} = e^{0.575646} = 1/0.74989$; the argument is $5/1.33352 = 5 \times 0.74989 = 3.74944$, so channels $2, 3 = (\sin 3.74944, \cos 3.74944) = (-0.57113,\ -0.82086)$. Rotation property: with $R(\Delta) = \left(\begin{smallmatrix} \cos\Delta & \sin\Delta \\ -\sin\Delta & \cos\Delta \end{smallmatrix}\right)$,

$$\begin{pmatrix} \sin(t+\Delta) \\ \cos(t+\Delta) \end{pmatrix} = \begin{pmatrix} \sin t\cos\Delta + \cos t\sin\Delta \\ \cos t\cos\Delta - \sin t\sin\Delta \end{pmatrix} = R(\Delta)\begin{pmatrix} \sin t \\ \cos t \end{pmatrix},$$

applied independently to each channel pair (each pair has its own frequency $10000^{-2i/(D/2)}$, hence its own rotation speed). A time shift is therefore a fixed per-pair rotation — a linear operation the attention heads can exploit. □

5. *Causal masking.* M has 0 on and below the diagonal, $-\infty$ above: $M = \left(\begin{smallmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{smallmatrix}\right)$. Row $i = 2$: $[2, 0, 1, 4] + [0, 0, 0, -\infty] = [2, 0, 1, -\infty]$. Softmax: $e^2 = 7.389$, $e^0 = 1$, $e^1 = 2.718$, $e^{-\infty} = 0$; sum $= 11.107$; weights $[0.6652,\ 0.0900,\ 0.2447,\ 0.0000]$. Position 3 receives exactly zero weight: no peeking. □

6. *GradScaler dynamics.* (a) `scaler.step(optimizer)` first unscales and checks all gradients for inf/NaN; on overflow it *skips* the optimizer step entirely (no parameter update) and does not let corrupt gradients poison the weights. (b) Overflow triggers the backoff: $s \leftarrow s/2 = 512$. (c) Five clean steps with growth factor 2 each step: $512 \times 2^5 = 16384$. (In the real `GradScaler` growth happens only every `growth_interval` clean steps, default 2000; the drill stipulates every-step growth.) □

7. *LR schedule.* $\eta(0) = \eta_{\max} \cdot 0/100 = 0$. $\eta(100) = \eta_{\min} + \frac{3\times10^{-4}}{2}(1 + \cos 0) = 1.5 \times 10^{-4} \cdot 2 = 3 \times 10^{-4}$ (both branches agree). $\eta(500) = 1.5 \times 10^{-4}(1 + \cos(\pi \cdot 400/900)) = 1.5 \times 10^{-4}(1 + \cos(4\pi/9))$. $\cos(80°) = 0.173648$, so $\eta(500) = 1.5 \times 10^{-4} \times 1.173648 = 1.7605 \times 10^{-4}$. □

8. *Harmonic-explanation filter.* Floor $= 0.30 \times 1027 = 308.1$; all seven candidates pass. Greedy in descending salience:

   - 60 (1027): accepted. 41 (850): no accepted q with $41 - q \in \{12, 19, 24, 28\}$ (it is *below* 60) — accepted. 57 (755): $57 - 41 = 16 \notin$ set — accepted.
   - 65 (699): $65 - 41 = 24 \in$ set and $699 < 1.5 \times 850 = 1275$ — **rejected** (41's double-octave artifact).
   - 48 (596): differences to $\{60, 41, 57\}$ are $-12, 7, -9$, none in the set — accepted.
   - 53 (474): $53 - 41 = 12 \in$ set and $474 < 1275$ — **rejected**. This is a *true* F-chord tone (F3 doubling F2) lost to overtone collision: its comb is a subset of F2's, the documented DSP limit.
   - 67 (405): $67 - 48 = 19 \in$ set and $405 < 1.5 \times 596 = 894$ — **rejected** (48's twelfth artifact).

   Accepted, in order: $[60, 41, 57, 48]$ — four of five true chord tones survive; both harmonic artifacts are killed; one genuine octave doubling is sacrificed, exactly the trade-off analyzed in §7.9. □

A.8 Solutions — Module 8

Drill 1. Differentiate (8.7) term by term. The first two sums do not depend on h. The third gives $\frac{\partial}{\partial h_a} \sum_{i,a} W_{ia} h_a = \sum_i W_{ia}$. In the fourth sum, only $\log(W_{ia} h_a)$ depends on h_a, with derivative $1/h_a$, so

$$\frac{\partial G}{\partial h_a} = \sum_i W_{ia} - \frac{1}{h_a} \sum_i v_i \frac{W_{ia} h_a^t}{\sum_b W_{ib} h_b^t}, \tag{A.33}$$

which is (8.8). Setting it to zero and multiplying through by h_a,

$$h_a \sum_i W_{ia} = \sum_i v_i \frac{W_{ia} h_a^t}{\sum_b W_{ib} h_b^t}, \qquad \Longrightarrow \qquad h_a = h_a^t \frac{\sum_i W_{ia}\, v_i / (W h^t)_i}{\sum_i W_{ia}}, \tag{A.34}$$

exactly the multiplicative update (8.5). The chain (8.9) then reads: $F(h^{t+1}) \le G(h^{t+1}, h^t)$ because G majorizes F everywhere (Jensen step); $G(h^{t+1}, h^t) \le G(h^t, h^t)$ because h^{t+1} minimizes $G(\cdot, h^t)$ by the computation above; $G(h^t, h^t) = F(h^t)$ by direct substitution. Hence each update cannot increase the KL divergence.

Drill 2. $L_{\text{onset}} = -\sum_t [I_t \log P_t + (1 - I_t) \log(1 - P_t)]$:

t	I_t	P_t	contribution
0	1	0.90	$-\log 0.90 = 0.1054$
1	0	0.10	$-\log 0.90 = 0.1054$
2	0	0.20	$-\log 0.80 = 0.2231$
3	1	0.80	$-\log 0.80 = 0.2231$
4	0	0.10	$-\log 0.90 = 0.1054$
5	0	0.05	$-\log 0.95 = 0.0513$

Summing: $0.1054 + 0.1054 + 0.2231 + 0.2231 + 0.1054 + 0.0513 = 0.8137$, matching the script's printed $L_{\text{onset}} = 0.8137$.

Drill 3. The unweighted per-frame cross-entropies are $-\log(0.95) = 0.0513$, $-\log(0.9) = 0.1054$, $-\log(0.85) = 0.1625$, $-\log(0.7) = 0.3567$, $-\log(0.6) = 0.5108$, $-\log(0.9) = 0.1054$, summing to 1.2920. With $(t_1, t_2, t_3) = (1, 2, 4)$ and $c = 5$, (8.15) gives weights $w = [1,\ 5,\ 5,\ 5/1,\ 5/2,\ 1] = [1, 5, 5, 5, 2.5, 1]$. The weighted contributions are [0.0513, 0.5268, 0.8126, 1.7834, 1.2771, 0.1054],

summing to 4.5565, as printed. The attack frames $t = 1, 2, 3$ contribute $0.5268 + 0.8126 + 1.7834 = 3.1228$, i.e. 68% of the weighted loss despite being half the frames: the loss forces the network to spend its capacity on note beginnings, the frames the paper identifies as most perceptually significant and easiest to detect.

Drill 4. Nothing is active at the segment start, so the tie section is just `TIE_END`. Then, in chronological order: `TIME(100)` (1.00 s at 10 ms quantization), `INST(5)` (program change from the initial state), `ONOFF(on)`, `NOTE(72)`, `TIME(150)`, `ONOFF(off)`, `NOTE(72)`, `EOS`. That is 9 tokens total: `TIE_END TIME(100) INST(5) ONOFF(on) NOTE(72) TIME(150) ONOFF(off) NOTE(72) EOS`.

Drill 5. $S(p^*) = \sum_{k=1}^{8} 0.8^k/k = 0.8 + 0.32 + 0.1707 + 0.1024 + 0.0655 + 0.0437 + 0.0300 + 0.0210 = 1.5532$. $S(p^* - 12) = \sum_{k=1}^{4} 0.8^{2k}/k = 0.64 + 0.2048 + 0.0874 + 0.0419 = 0.9741$. Ratio: $0.9741/1.5532 = 0.6272$ — the ghost reaches 63% of the true peak's height. This is structural: by (8.11) the sub-octave candidate's even partials coincide exactly with the true note's partials, so any additive harmonic-summation salience *must* respond strongly there. Retuning α, H, or the peak-picking threshold moves the numbers but cannot remove the coincidence; only information outside the single frame (onsets, temporal trajectories) breaks the symmetry.

Drill 6. (a) GuitarSet: it is the only corpus with per-string recordings (hexaphonic pickup) and string/fret annotations, produced by per-string pYIN tracking with manual inspection. MAPS, MAESTRO, MusicNet, Slakh2100, and URMP have no string-level labels at all, so tablature estimation cannot even be supervised on them. (b) URMP: at 1.3 hours total it is the smallest corpus by two orders of magnitude; a model that learns anything from it is demonstrating data efficiency, whereas good results on MAESTRO's 198.6 hours prove little about sample efficiency.

Drill 7. With $H \geq 0$ fixed in the cone picture, each column of V must be expressed as a *nonnegative* combination of the columns of W: the data live inside the simplicial cone the W-columns generate, and fitting pushes those extreme rays toward clusters of data points — the note spectra. Principal components instead find orthogonal directions of maximum variance with no sign restriction: a data cloud in the nonnegative orthant is typically best explained by holistic directions with positive and negative entries whose cancellations reconstruct the mean — the "photographic negatives" of the analogy. If the nonnegativity of H is dropped while $W \geq 0$ is kept, the cone becomes a subspace: columns of W may again cancel each other with signed coefficients, the parts-based pressure vanishes, and the factorization collapses toward an SVD-like holistic representation that no longer reads as note spectra.

Bibliography

[1] C. E. Shannon, "Communication in the presence of noise," *Proceedings of the IRE*, vol. 37, no. 1, pp. 10–21, 1949.

[2] D. Gabor, "Theory of communication. Part 1: The analysis of information," *Journal of the Institution of Electrical Engineers — Part III: Radio and Communication Engineering*, vol. 93, no. 26, pp. 429–441, 1946.

[3] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.

[4] E. Zwicker, "Subdivision of the audible frequency range into critical bands," *Journal of the Acoustical Society of America*, vol. 33, no. 2, p. 248, 1961.

[5] E. Zwicker and E. Terhardt, "Analytical expressions for critical-band rate and critical bandwidth as a function of frequency," *Journal of the Acoustical Society of America*, vol. 68, no. 5, pp. 1523–1525, 1980.

[6] B. R. Glasberg and B. C. J. Moore, "Derivation of auditory filter shapes from notched-noise data," *Hearing Research*, vol. 47, no. 1–2, pp. 103–138, 1990.

[7] J. P. Egan and H. W. Hake, "On the masking pattern of a simple auditory stimulus," *Journal of the Acoustical Society of America*, vol. 22, no. 5, pp. 622–630, 1950.

[8] E. Zwicker and H. Fastl, *Psychoacoustics: Facts and Models*, 2nd ed. Berlin, Germany: Springer, 1999.

[9] J. C. Brown, "Calculation of a constant Q spectral transform," *Journal of the Acoustical Society of America*, vol. 89, no. 1, pp. 425–434, 1991.

[10] J. C. Brown and M. S. Puckette, "An efficient algorithm for the calculation of a constant Q transform," *Journal of the Acoustical Society of America*, vol. 92, no. 5, pp. 2698–2701, 1992.

[11] C. Schörkhuber and A. Klapuri, "Constant-Q transform toolbox for music processing," in *Proceedings of the 7th Sound and Music Computing Conference*, Barcelona, Spain, 2010, pp. 322–329.

[12] C. Schörkhuber, A. Klapuri, N. Holighaus, and M. Dörfler, "A Matlab toolbox for efficient perfect reconstruction time-frequency transforms with log-frequency resolution," in *Audio Engineering Society Conference: 53rd International Conference: Semantic Audio*, London, UK, 2014.

[13] Yu-Siang Huang and Yi-Hsuan Yang. "Pop Music Transformer: Beat-based Modeling and Generation of Expressive Pop Piano Compositions." *Proc. 28th ACM Int. Conf. on Multimedia (MM)*, 2020. arXiv:2002.00212.

[14] Sageev Oore, Ian Simon, Sander Dieleman, Douglas Eck, and Karen Simonyan. "This Time with Feeling: Learning Expressive Musical Performance." *Neural Computing and Applications*, 32(4):955–967, 2020. arXiv:1808.03715.

[15] Gaëtan Hadjeres and Léopold Crestel. "The Piano Inpainting Application." arXiv:2107.05944 [cs.SD], 2021.

[16] Josh Gardner, Ian Simon, Ethan Manilow, Curtis Hawthorne, and Jesse Engel. "MT3: Multi-Task Multitrack Music Transcription." *Proc. Int. Conf. on Learning Representations (ICLR)*, 2022. arXiv:2111.03017.

[17] Curtis Hawthorne, Ian Simon, Rigel Swavely, Ethan Manilow, and Jesse Engel. "Sequence-to-Sequence Piano Transcription with Transformers." *Proc. 22nd Int. Society for Music Information Retrieval Conf. (ISMIR)*, Online, 2021, pp. 246–253. arXiv:2107.09142.

[18] Nathan Fradet, Jean-Pierre Briot, Fabien Chhel, Amal El Fallah Seghrouchni, and Nicolas Gutowski. "MidiTok: A Python package for MIDI file tokenization." *ISMIR 2021 Late-Breaking Demos*; comprehensive report arXiv:2310.17202 [cs.LG], 2023.

[19] Colin Raffel and Daniel P. W. Ellis. "Intuitive Analysis, Creation and Manipulation of MIDI Data with pretty_midi." *ISMIR 2014 Late-Breaking Demos*, 2014.

[20] Yuan Gong, Yu-An Chung, and James Glass. "AST: Audio Spectrogram Transformer." *Proc. Interspeech*, 2021. arXiv:2104.01778.

[21] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. "Masked Autoencoders Are Scalable Vision Learners." *Proc. IEEE/CVF CVPR*, pp. 16000–16009, 2022. arXiv:2111.06377.

[22] Yuan Gong, Cheng-I Lai, Yu-An Chung, and James Glass. "SSAST: Self-Supervised Audio Spectrogram Transformer." arXiv:2110.09784, 2021.

[23] Alexei Baevski, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli. "wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations." *Adv. Neural Inf. Process. Syst. 33*, pp. 12449–12460, 2020. arXiv:2006.11477.

[24] Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. "HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units." *IEEE/ACM Trans. Audio, Speech, Language Process.*, vol. 29, pp. 3451–3460, 2021. arXiv:2106.07447.

[25] Alexei Baevski, Wei-Ning Hsu, Qiantong Xu, Arun Babu, Jiatao Gu, and Michael Auli. "data2vec: A General Framework for Self-supervised Learning in Speech, Vision and Language." *Proc. ICML*, PMLR 162, pp. 1298–1312, 2022. arXiv:2202.03555.

[26] Curtis Hawthorne, Andriy Stasyuk, Adam Roberts, Ian Simon, Cheng-Zhi Anna Huang, Sander Dieleman, Erich Elsen, Jesse Engel, and Douglas Eck. "Enabling Factorized Piano Music Modeling and Generation with the MAESTRO Dataset." *Proc. ICLR*, 2019. arXiv:1810.12247.

[27] Curtis Hawthorne, Erich Elsen, Jialin Song, Adam Roberts, Ian Simon, Colin Raffel, Jesse Engel, Sageev Oore, and Douglas Eck. "Onsets and Frames: Dual-Objective Piano Transcription." *Proc. ISMIR*, 2017. arXiv:1710.11153.

[28] A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu, and R. Pang, "Conformer: Convolution-augmented Transformer for Speech Recognition," in *Proc. Interspeech*, Shanghai, China, 2020, pp. 5036–5040.

[29] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding," *arXiv:2104.09864*, 2021.

[30] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, Venice, Italy, 2017, pp. 2980–2988.

[31] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks," in *Proc. Int. Conf. Machine Learning (ICML)*, Pittsburgh, PA, USA, 2006.

[32] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Adv. Neural Information Processing Systems 30 (NeurIPS)*, Long Beach, CA, USA, 2017.

[33] Y. Lu, Z. Li, D. He, Z. Sun, B. Dong, T. Qin, L. Wang, and T.-Y. Liu, "Understanding and Improving Transformer From a Multi-Particle Dynamic System Point of View," *arXiv:1906.02762*, 2019.

[34] I. Sutskever, O. Vinyals, and Q. V. Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems 27 (NeurIPS)*, 2014. (Left-to-right beam-search decoding of seq2seq models.)

[35] Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi, et al. Google's neural machine translation system: bridging the gap between human and machine translation. *arXiv:1609.08144*, 2016. (Length-normalization procedure for beam search.)

[36] Ç. Gülçehre, O. Firat, K. Xu, K. Cho, L. Barrault, H.-C. Lin, F. Bougares, H. Schwenk, and Y. Bengio. On using monolingual corpora in neural machine translation. *arXiv:1503.03535*, 2015. (Shallow vs. deep fusion of an external language model with a seq2seq decoder.)

[37] L. R. Rabiner. A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989. (HMM fundamentals; forward–backward and Viterbi algorithms.)

[38] C. Hawthorne, E. Elsen, J. Song, A. Roberts, I. Simon, C. Raffel, J. Engel, S. Oore, and D. Eck. Onsets and frames: dual-objective piano transcription. In *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*, Paris, France, 2018. (Onset-gated frame inference for piano transcription.)

[39] J. P. Gardner, I. Simon, E. Manilow, C. Hawthorne, and J. Engel. MT3: multi-task multitrack music transcription. In *International Conference on Learning Representations (ICLR)*, 2021. (Token-based seq2seq transcription with MIDI-like vocabularies.)

[40] C. Raffel, B. McFee, E. J. Humphrey, J. Salamon, O. Nieto, D. Liang, and D. P. W. Ellis, "mir_eval: A Transparent Implementation of Common MIR Metrics," in *Proc. 15th Int. Society for Music Information Retrieval Conf. (ISMIR)*, Taipei, Taiwan, Oct. 2014.

[41] A. Ycart, L. Liu, E. Benetos, and M. T. Pearce, "Musical Features for Automatic Music Transcription Evaluation," *arXiv:2004.07171 [cs.SD]*, Apr. 2020. Technical report formalizing the onset ($\pm$50 ms) and offset ($\max(50\text{ ms}, 0.2 \cdot \text{duration})$) note-matching conventions.

[42] G. E. Poliner and D. P. W. Ellis, "A Discriminative Model for Polyphonic Piano Transcription," *EURASIP Journal on Advances in Signal Processing*, vol. 2007, Article ID 48317, 2007. Introduced the NIST-style multi-pitch error decomposition $E_{\text{tot}} = E_{\text{subs}} + E_{\text{miss}} + E_{\text{fa}}$.

[43] A. Pertusa and J. M. Iñesta, "Efficient methods for joint estimation of multiple fundamental frequencies in music signals," *EURASIP Journal on Advances in Signal Processing*, vol. 2012, p. 27, 2012. Exposition of the E_{tot} frame formulas and their MIREX multiple-f_0 evaluation.

[44] M. Bay, A. F. Ehmann, and J. S. Downie, "Evaluation of multiple-F0 estimation and tracking systems," in *Proc. 10th Int. Conf. on Music Information Retrieval (ISMIR)*, Kobe, Japan, 2009, pp. 315–320. MIREX multiple-f_0 evaluation protocol ($\pm$50-cent frame pitch tolerance).

[45] C. Hawthorne, E. Elsen, J. Song, A. Roberts, I. Simon, C. Raffel, J. Engel, S. Oore, and D. Eck, "Onsets and Frames: Dual-Objective Piano Transcription," in *Proc. 19th Int. Society for Music Information Retrieval Conf. (ISMIR)*, Paris, France, 2018. Established the modern frame + note (onset / onset+offset) F_1 reporting standard for piano AMT.

[46] Q. Kong, B. Li, X. Song, Y. Wan, and Y. Wang, "High-resolution Piano Transcription with Pedals by Regressing Onset and Offset Times," *arXiv:2010.01815 [eess.AS]*, 2020. Uses a velocity tolerance of 0.1 on $[0, 1]$-normalized velocities for the velocity-aware note metric.

[47] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, 2019.

[48] Y.-Y. Yang et al., "TorchAudio: Building blocks for audio and speech processing," in *Proc. IEEE ICASSP*, 2022.

[49] C. Raffel and D. P. W. Ellis, "Intuitive analysis, creation and manipulation of MIDI data with `pretty_midi`," in *Late-Breaking Demos, 15th Int. Society for Music Information Retrieval Conf. (ISMIR)*, 2014.

[50] C. Raffel et al., "`mir_eval`: A transparent implementation of common MIR metrics," in *Proc. 15th Int. Society for Music Information Retrieval Conf. (ISMIR)*, 2014.

[51] K. Karplus and A. Strong, "Digital synthesis of plucked-string and drum timbres," *Computer Music Journal*, vol. 7, no. 2, pp. 43–55, 1983.

[52] Q. Xi, R. M. Bittner, J. Pauwels, X. Ye, and J. P. Bello, "GuitarSet: A dataset for guitar transcription," in *Proc. 19th Int. Society for Music Information Retrieval Conf. (ISMIR)*, Paris, 2018, pp. 453–460.

[53] Y.-S. Huang and Y.-H. Yang, "Pop Music Transformer: Beat-based modeling and generation of expressive pop piano compositions," in *Proc. ACM Multimedia*, 2020, pp. 1180–1188.

[54] D. D. Lee and H. S. Seung, "Learning the parts of objects by non-negative matrix factorization," *Nature*, vol. 401, pp. 788–791, 1999.

[55] D. D. Lee and H. S. Seung, "Algorithms for non-negative matrix factorization," in *Advances in Neural Information Processing Systems 13*, MIT Press, 2001.

[56] P. Smaragdis and J. C. Brown, "Non-negative matrix factorization for polyphonic music transcription," in *Proc. IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, New Paltz, NY, 2003, pp. 177–180.

[57] E. Vincent, N. Bertin, and R. Badeau, "Harmonic and inharmonic nonnegative matrix factorization for polyphonic pitch transcription," in *Proc. IEEE Int. Conf. on Acoustics, Speech and Signal Processing (ICASSP)*, 2008, pp. 109–112.

[58] V. Emiya, R. Badeau, and B. David, "Multipitch estimation of piano sounds using a new probabilistic spectral smoothness principle," *IEEE Trans. Audio, Speech, and Language Processing*, vol. 18, no. 6, pp. 1643–1654, 2010.

[59] Z. Duan, B. Pardo, and C. Zhang, "Multiple fundamental frequency estimation by modeling spectral peaks and non-peak regions," *IEEE Trans. Audio, Speech, and Language Processing*, vol. 18, no. 8, pp. 2121–2130, 2010.

[60] E. Benetos, S. Dixon, Z. Duan, and S. Ewert, "Automatic music transcription: An overview," *IEEE Signal Processing Magazine*, vol. 36, no. 1, pp. 20–30, 2019.

[61] S. Sigtia, E. Benetos, and S. Dixon, "An end-to-end neural network for polyphonic piano music transcription," *IEEE/ACM Trans. Audio, Speech, and Language Processing*, vol. 24, no. 5, pp. 927–939, 2016.

[62] C. Hawthorne, E. Elsen, J. Song, A. Roberts, I. Simon, C. Raffel, J. Engel, S. Oore, and D. Eck, "Onsets and frames: Dual-objective piano transcription," arXiv:1710.11153, 2017; in *Proc. 19th Int. Society for Music Information Retrieval Conf. (ISMIR)*, Paris, 2018.

[63] C. Hawthorne, I. Simon, R. Swavely, E. Manilow, and J. Engel, "Sequence-to-sequence piano transcription with transformers," arXiv:2107.09142, 2021; in *Proc. 22nd ISMIR*, 2021.

[64] Q. Kong, B. Li, X. Song, Y. Wang, and Y. Wang, "High-resolution piano transcription with pedals by regressing onsets and offset times," *IEEE/ACM Trans. Audio, Speech, and Language Processing*, vol. 29, pp. 3707–3717, 2021.

[65] A. Fernandez, "Onsets and velocities: Affordable real-time piano transcription using convolutional neural networks," arXiv:2303.04485, 2023.

[66] J. Gardner, I. Simon, E. Manilow, C. Hawthorne, and J. Engel, "MT3: Multi-task multitrack music transcription," arXiv:2111.03017, 2021.

[67] Y. Li, R. Yuan, G. Zhang, Y. Ma, X. Chen, H. Yin, C. Xiao, C. Lin, A. Ragni, E. Benetos, et al., "MERT: Acoustic music understanding model with large-scale self-supervised training," arXiv:2306.00107, 2023; in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024.

[68] A. Ragano, E. Benetos, and A. Hines, "Learning music representations with wav2vec 2.0," arXiv:2210.15310, 2022; in *Proc. IEEE ICASSP*, 2023.

[69] J. Thickstun, Z. Harchaoui, and S. Kakade, "Learning features of music from scratch," in *Proc. 5th Int. Conf. on Learning Representations (ICLR)*, 2017.

[70] C. Hawthorne, A. Stasyuk, A. Roberts, I. Simon, C.-Z. A. Huang, S. Dieleman, E. Elsen, J. Engel, and D. Eck, "Enabling factorized piano music modeling and generation with the MAESTRO dataset," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2019.

[71] E. Manilow, G. Wichern, P. Seetharaman, and J. Le Roux, "Cutting music source separation some Slakh: A dataset to study the impact of training data quality and quantity," in *Proc. IEEE WASPAA*, New Paltz, NY, 2019.

[72] Q. Xi, R. M. Bittner, J. Pauwels, X. Ye, and J. P. Bello, "GuitarSet: A dataset for guitar transcription," in *Proc. 19th ISMIR*, Paris, 2018, pp. 453–460.

[73] X. Riley, Z. Guo, D. Edwards, and S. Dixon, "GAPS: A large and diverse classical guitar dataset and benchmark transcription model," in *Proc. 25th ISMIR*, San Francisco, 2024.

[74] B. Li, X. Liu, K. Dinesh, Z. Duan, and G. Sharma, "Creating a multitrack classical music performance dataset for multimodal music analysis: Challenges, insights, and applications," *IEEE Trans. Multimedia*, vol. 21, no. 2, pp. 522–535, 2018.